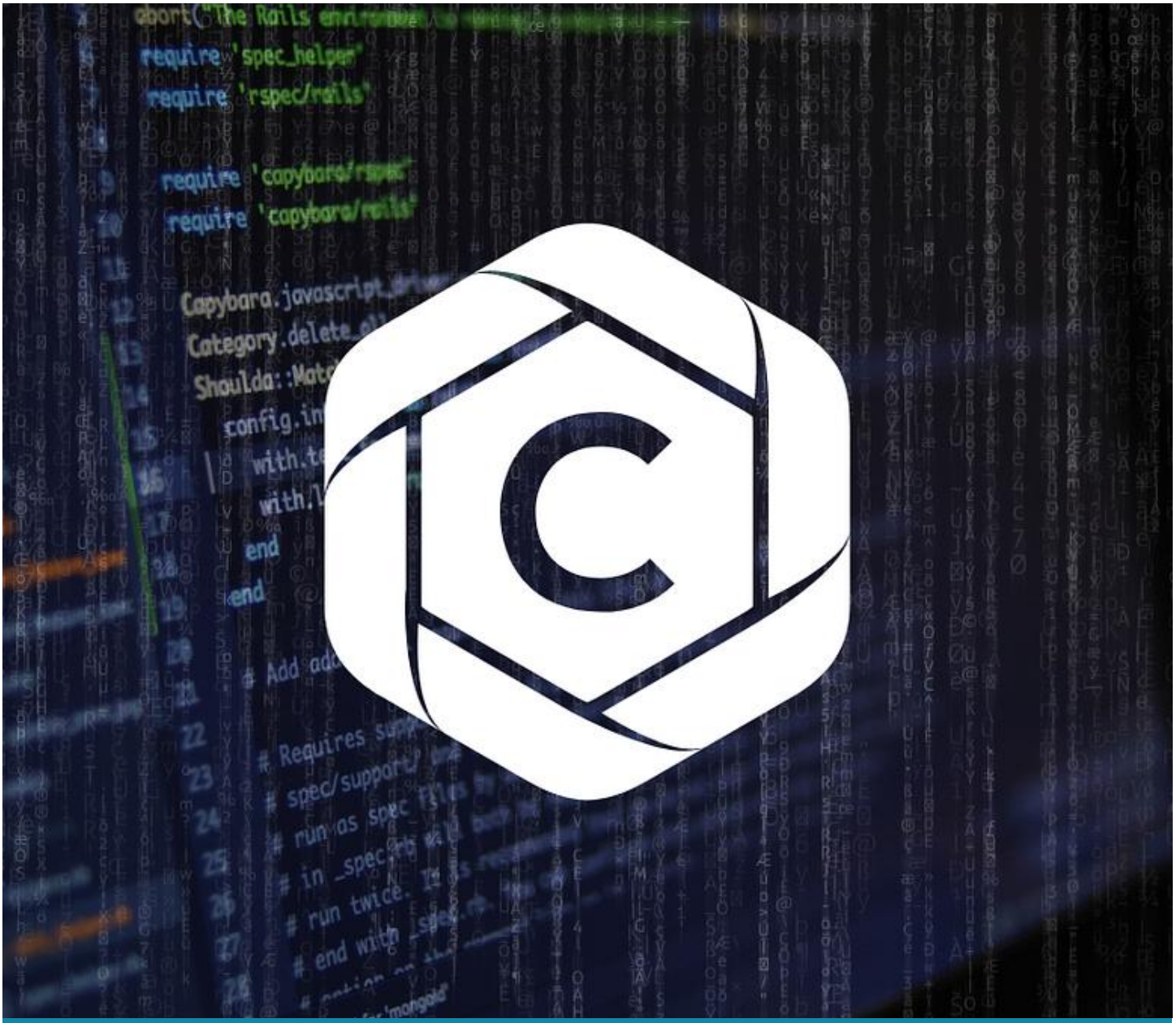


Embedded C Notes

Embedded Systems

Aschref Ben Thabet

8/4/25

Learning Series from scratch to expert

Contents

Chapter 1: C Fundamentals Refresher 3
Master C Syntax and Flow Control 4
Understand Pointers and Arrays 9
Master Bit Manipulation 14
Master Structs, Unions, and Enums 18
Consolidate Week 1 Concepts 23
Chapter 2: Embedded C Concepts 29
Understand volatile, const, and static 30
Learn Memory-Mapped I/O 36
Understand ISR Basics and Startup Code 41
Consolidate Week 2 Concepts 50
Chapter 3: Microcontroller Fundamentals 55
Master GPIO and UART 56
Understand Timers and PWM 62
Learn Interrupts and NVIC 68
Use HAL and Direct Registers 75
Consolidate Week 3 Concepts 80
Chapter 4: Intermediate Peripheral Work 86
Master SPI and I2C Drivers 87
Implement FSM for Task Management 94
Learn ADC and Sensor Handling 100
Use Watchdog Timer 109
Consolidate Week 4 Concepts 115
Chapter 5: Advanced Firmware Design 123
Learn DMA Integration 124
Understand Bootloader Basics 134
Optimize Stack/Heap 143
Learn MISRA and Static Analysis 151
Consolidate Week 5 Concepts 159
Chapter 6: RTOS Intro & Final Project 169
Learn FreeRTOS Basics 170
Understand RTOS Timers 179
Learn Resource Protection 188
Design RTOS System 196

Build Final Project 206
Chapter 7: Secure Boot, FOTA, and Crypto 215
Understand Secure Boot Principles 216
Implement Cryptographic Secure Boot 223
Learn FOTA Basics 230
Implement FOTA Update Process 238
Integrate Crypto into FOTA 248
Chapter 8: Secure Boot and FOTA Optimization 260
Enhance Secure Boot with Key Management 261
Optimize FOTA for Reliability 269
Add Secure Communication to FOTA 283
Test Secure Boot and FOTA Integration 298
Consolidate, Debug, and Document Secure Boot and FOTA 315
Chapter 9: Low Power Modes and Power Profiling 335
Understand Low-Power Modes 336
Implement Stop Mode 344
Learn Power Profiling 351
Optimize Low-Power Design 362
Consolidate Week 9 Concepts 372
Chapter 10: Unit Testing and CI/CD for Embedded 384
Understand Unit Testing for Embedded 385
Mock Hardware for Testing 394
Set Up CI/CD Pipeline 402
Integrate Tests in CI/CD 409
Consolidate Week 10 Concepts 420
Appendix: MISRA C 433

Chapter 1: C Fundamentals Refresher

- **Master C syntax and flow control**
- **Understand pointers and arrays**
- **Learn bit manipulation**
- **Master structs, unions, enums**
- **Consolidate Week 1 concepts**

Master C Syntax and Flow Control

Goal

Master C syntax and flow control to build a strong foundation for embedded programming, understanding how variables, data types, operators, loops, conditionals, and functions work in memory-constrained environments.

Tasks

- **Study**: Learn C variables, data types, operators, loops, conditionals, and functions.
- **Practice**: Write programs to compute factorial and check prime numbers.
- **Project**: Develop a CLI program to compute the sum of squares from 1 to N.
- **Tools**: GCC compiler, OnlineGDB (online C compiler for quick testing).
- **Resources**: K&R "The C Programming Language" (Chapters 1-2), YouTube: Neso Academy C tutorials.

Detailed Notes

1. Variables and Data Types

- **Definition**: Variables store data in memory, and data types define the size and format (e.g., integer, float).
- **Common Types for Embedded**:
  - int (2 or 4 bytes, platform-dependent)
  - char (1 byte, signed or unsigned)
  - unsigned int, long, short (control size explicitly for embedded systems)
  - float, double (avoid in resource-constrained systems due to FPU dependency)
- **Embedded Tip**: Use fixed-width types from <stdint.h> (e.g., uint8_t, int32_t) for portability across microcontrollers.
- **Example**:

```
#include <stdio.h>
#include <stdint.h ;^p^pint >

int main() {
    uint8_t sensor_value = 255;  // 8-bit unsigned integer
    int32_t temperature = -10;   // 32-bit signed integer
    printf("Sensor: %u, Temp: %d\n", sensor_value, temperature);
    return 0;
}
```

2. Operators

- **Types**:
  - Arithmetic: +, -, *, /, % (modulus)
  - Relational: ==, !=, >, <, >=, <=
  - Logical: &&, ||, !
  - Assignment: =, +=, -=, *=, /=
- **Embedded Tip**: Avoid division (/) and modulus (%) on microcontrollers without hardware support, as they are computationally expensive.

- **Example**:

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;
    printf("Sum: %d, Mod: %d\n", a + b, a % b);
    if (a > b && b != 0) {
        printf("Division: %d\n", a / b);
    }
    return 0;
}
```

3. Loops

- **Types**:
  - for: Known iteration count.
  - while: Condition-based.
  - do-while: Executes at least once.
- **Embedded Tip**: Avoid infinite loops without escape conditions in embedded systems to prevent watchdog timer resets.
- **Example**:

```
#include <stdio.h>

int main() {
    // Print numbers 1 to 5 using for loop
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");
    // While loop to count down
    int j = 5;
    while (j > 0) {
        printf("%d ", j--);
    }
    return 0;
}
```

4. Conditionals

- **Types**:
  - if, else if, else
  - switch-case: Efficient for multiple discrete values.
- **Embedded Tip**: Use switch-case for state machines in embedded systems to improve readability and performance.
- **Example**:

```
#include <stdio.h>

int main() {
    int value = 2;
    switch (value) {
        case 1: printf("Low\n"); break;
        case 2: printf("Medium\n"); break;
        case 3: printf("High\n"); break;
        default: printf("Invalid\n");
    }
```

```
    return 0;
}
```

5. Functions

- **Definition**: Modularize code for reusability and readability.
- **Syntax**: return_type function_name(parameters) { body }
- **Embedded Tip**: Use static functions for internal linkage to reduce memory footprint and avoid namespace pollution.
- **Example**:

```
#include <stdio.h>

static int square(int x) {
    return x * x;
}

int main() {
    int num = 4;
    printf("Square of %d is %d\n", num, square(num));
    return 0;
}
```

6. Practice Programs

a. Factorial Program

- Computes n! (e.g., 5! = 5 × 4 × 3 × 2 × 1 = 120).
- **Code**:

```
#include <stdio.h>

unsigned long factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
    }

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 0) {
        printf("Factorial not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d is %lu\n", n, factorial(n));
    }
    return 0;
}
```

- **Note**: Use unsigned long for larger factorials, but beware of overflow (e.g., 13! exceeds 32-bit range).

b. Prime Number Checker

- Checks if a number is prime (divisible only by 1 and itself).
- **Code**:

```
#include <stdio.h>

int is_prime(int n) {
    if (n <= 1) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (is_prime(n)) {
        printf("%d is prime.\n", n);
    } else {
        printf("%d is not prime.\n", n);
    }
    return 0;
}
```

- **Optimization**: Check divisors only up to sqrt(n) (i.e., i * i <= n).

Project: CLI Program to Compute Sum of Squares (1 to N)

- **Objective**: Write a program that takes an integer N as input and computes the sum of squares from 1 to N (e.g., for N=3, sum = $1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$).
- **Requirements**:
  - Handle invalid inputs (e.g., negative numbers).
  - Use a function to compute the sum.
  - Print the result clearly.
- **Code**:

```
#include <stdio.h>

unsigned long sum_of_squares(int n) {
    unsigned long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i * i;
    }
    return sum;
}

int main() {
    int n;
    printf("Enter a positive integer N: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return 1;
    }
    printf("Sum of squares from 1 to %d is %lu\n", n, sum_of_squares(n));
    return 0;
}
```

- **Embedded Tip**: Use unsigned long to handle larger sums, but check for overflow in real embedded systems (e.g., use 64-bit uint64_t if needed).

Tools Setup

- **GCC**: Install GCC (e.g., sudo apt install gcc on Ubuntu) or use a pre-installed version on your system.
  - Compile: gcc program.c -o program
  - Run: ./program
- **OnlineGDB**: Access via browser (https://www.onlinegdb.com/). Paste code, select C, and run.
- **Debugging**: Use printf for now; GDB will be introduced later.

Resources

- **K&R "The C Programming Language" (Chapters 1-2)**: Covers syntax, data types, operators, loops, conditionals, and functions.
- **YouTube: Neso Academy (C Programming Playlist)**: Visual explanations of C basics (search for "Neso Academy C Programming").
- **Online**: GCC documentation (https://gcc.gnu.org/onlinedocs/), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run each code snippet using GCC or OnlineGDB to verify output.
- **Experiment**: Modify programs (e.g., change loop bounds, add error checks).
- **Optimize**: Refactor factorial to use iteration instead of recursion for embedded efficiency.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Understand Pointers and Arrays

Goal

Understand pointers and arrays, mastering pointer arithmetic, dereferencing, and dynamic memory management, with a focus on their application in memory-constrained embedded systems.

Tasks

- **Study**: Learn pointer arithmetic, dereferencing, arrays, and memory models.
- **Practice**: Write programs to swap numbers using pointers and reverse an array.
- **Project**: Develop a dynamic array resizer using malloc and free.
- **Tools**: GCC compiler, OnlineGDB (online C compiler for quick testing).
- **Resources**: K&R "The C Programming Language" (Chapter 5), YouTube: Pointers tutorials.

Detailed Notes

1. Pointers

- **Definition**: A pointer is a variable that stores a memory address. It allows direct memory access, critical for embedded systems.
- **Declaration**: type *pointer_name; (e.g., int *ptr;)
- **Operators**:
  - &: Address-of operator (gets memory address of a variable).
  - *: Dereference operator (accesses value at the address).
- **Embedded Tip**: Always initialize pointers to NULL to avoid undefined behavior in embedded systems.
- **Example**:

```c
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr = &x;  // ptr holds address of x
    printf("Value of x: %d\n", x);
    printf("Address of x: %p\n", (void*)&x);
    printf("Value via ptr: %d\n", *ptr);
    *ptr = 20;  // Modify x via pointer
    printf("New value of x: %d\n", x);
    return 0;
}
```

- **Note**: Use (void*) for printing addresses with %p to ensure portability.

2. Pointer Arithmetic

- **Definition**: Performing arithmetic on pointers (e.g., increment, decrement) adjusts the address based on the size of the data type.

- **Rules**:
  - ptr + n moves the pointer n * sizeof(type) bytes forward.
  - ptr - n moves backward similarly.
- **Embedded Tip**: Use pointer arithmetic cautiously in embedded systems to avoid accessing invalid memory regions.
- **Example**:

```
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr;  // Points to arr[0]
    printf("First element: %d\n", *ptr);
    ptr++;  // Move to next integer (4 bytes forward)
    printf("Second element: %d\n", *ptr);
    return 0;
}
```

3. Arrays

- **Definition**: Contiguous memory blocks storing elements of the same type.
- **Syntax**: type array_name[size];
- **Relation to Pointers**: Array name acts as a pointer to the first element (e.g., arr == &arr[0]).
- **Embedded Tip**: Define array sizes explicitly in embedded systems to avoid stack overflow on microcontrollers with limited memory.
- **Example**:

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    printf("First element: %d\n", arr[0]);
    printf("Via pointer: %d\n", *arr);
    printf("Third element: %d\n", *(arr + 2));  // Pointer arithmetic
    return 0;
}
```

4. Dynamic Memory Allocation

- **Functions** (from <stdlib.h>):
  - malloc(size): Allocates size bytes, returns pointer to start.
  - free(ptr): Deallocates memory to prevent leaks.
  - calloc(n, size): Allocates memory for n elements of size bytes, initializes to zero.
  - realloc(ptr, new_size): Resizes allocated memory.
- **Embedded Tip**: Avoid dynamic allocation in real-time embedded systems due to fragmentation and non-deterministic behavior. Use static arrays when possible.
- **Example**:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int*)malloc(3 * sizeof(int));
    if (ptr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
```

```
    }
    ptr[0] = 10; ptr[1] = 20; ptr[2] = 30;

    for (int i = 0; i < 3; i++) {
        printf("Value %d: %d\n", i, ptr[i]);
    }
    free(ptr);  // Release memory
    ptr = NULL;  // Prevent dangling pointer
    return 0;
}
```

5. Practice Programs

a. Swap Numbers Using Pointers

- Swaps two integers using pointers without a temporary variable (or with one for clarity).
- **Code**:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

- **Note**: Pass addresses (&x, &y) to modify original variables.

b. Reverse an Array

- Reverses elements of an array using pointers.
- **Code**:

```
#include <stdio.h>

void reverse_array(int *arr, int size) {
    int *start = arr;
    int *end = arr + size - 1;
    while (start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    printf("Original array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
```

```
    printf("\n");
    reverse_array(arr, 5);
    printf("Reversed array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");
    return 0;
}
```

- **Optimization**: Use pointer arithmetic to avoid indexing overhead.

Project: Dynamic Array Resizer Using malloc and free

- **Objective**: Write a program that creates an array dynamically, allows user input for elements, resizes it to a new size, and copies elements using realloc.
- **Requirements**:
  - Prompt for initial size and elements.
  - Resize array to a new user-specified size.
  - Handle memory allocation failures.
  - Free memory properly.
- **Code**:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size, new_size;
    printf("Enter initial array size: ");
    scanf("%d", &size);
    if (size <= 0) {
        printf("Invalid size!\n");
        return 1;
    }
    // Allocate memory
    int *arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    // Input elements
    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Original array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    // Resize array
    printf("Enter new array size: ");
    scanf("%d", &new_size);
    if (new_size <= 0) {
        printf("Invalid size!\n");
        free(arr);
        return 1;
    }
```

```
    int *temp = (int*)realloc(arr, new_size * sizeof(int));
    if (temp == NULL) {
        printf("Memory reallocation failed!\n");
        free(arr);
        return 1;
    }

    arr = temp;
    // Initialize new elements to 0 if new_size > size
    if (new_size > size) {
        for (int i = size; i < new_size; i++) {
            arr[i] = 0;
        }
    }
    printf("Resized array: ");

    for (int i = 0; i < new_size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    free(arr);  // Release memory
    arr = NULL;
    return 0;
}
```

- **Embedded Tip**: In embedded systems, prefer static arrays over dynamic allocation. This project is for learning purposes only.

Tools Setup

- **GCC**: Compile with gcc program.c -o program and run ./program.
- **OnlineGDB**: Use https://www.onlinegdb.com/ for quick testing. Paste code, select C, and run.
- **Debugging**: Add printf statements to track pointer values or array contents.

Resources

- **K&R "The C Programming Language" (Chapter 5)**: Covers pointers, arrays, and memory management.
- **YouTube: Pointers Tutorials**: Search for "C pointers tutorial" (e.g., Neso Academy, freeCodeCamp).
- **Online**: C reference (https://en.cppreference.com/w/c), GCC docs (https://gcc.gnu.org/onlinedocs/).

Practice Tips

- **Compile and Test**: Run each code snippet to verify output.
- **Experiment**: Modify programs (e.g., add bounds checking, print addresses).
- **Optimize**: Rewrite array reverse using indexing instead of pointers to compare performance.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Master Bit Manipulation

Goal

Master bit manipulation techniques, including bitwise operators (AND, OR, XOR, shift), and their application in embedded systems for hardware register manipulation and optimization.

Tasks

- **Study**: Learn bitwise operators (AND, OR, XOR, shift) and their use in embedded programming.
- **Practice**: Write macros for setting, clearing, toggling, and testing bits.
- **Project**: Develop a bitfield encoder/decoder for a 32-bit status register.
- **Tools**: GCC compiler, OnlineGDB (online C compiler for quick testing).
- **Resources**: "Bit Twiddling Hacks" (online), K&R "The C Programming Language" (Chapter 2).

Detailed Notes

1. Bitwise Operators

- **Definition**: Operate on individual bits of integers, essential for low-level hardware control in embedded systems.
- **Operators**:
  - **AND (&)**: Sets bit to 1 if both bits are 1 (e.g., 5 & 3 → 0101 & 0011 = 0001 → 1).
  - **OR (|)**: Sets bit to 1 if either bit is 1 (e.g., 5 | 3 → 0101 | 0011 = 0111 → 7).
  - **XOR (^)**: Sets bit to 1 if bits differ (e.g., 5 ^ 3 → 0101 ^ 0011 = 0110 → 6).
  - **NOT (~)**: Inverts all bits (e.g., ~5 → ~0101 = 1010 for 4-bit, but depends on integer size).
  - **Left Shift (<<)**: Shifts bits left, fills with zeros (e.g., 5 << 1 → 0101 << 1 = 1010 → 10).
  - **Right Shift (>>)**: Shifts bits right, fills with sign bit for signed types or zeros for unsigned (e.g., 5 >> 1 → 0101 >> 1 = 0010 → 2).
- **Embedded Tip**: Use unsigned types (e.g., uint32_t) for predictable shift behavior in embedded systems.
- **Example**:

```c
#include <stdio.h>
#include <stdint.h>

int main() {
    uint8_t a = 5;  // 0101
    uint8_t b = 3;  // 0011
    printf("AND: %u\n", a & b);  // 0001 = 1
    printf("OR: %u\n", a | b);   // 0111 = 7
    printf("XOR: %u\n", a ^ b);  // 0110 = 6
    printf("NOT a: %u\n", ~a);   // 11111010 (8-bit) = 250
    printf("Left shift: %u\n", a << 1);  // 1010 = 10
    printf("Right shift: %u\n", a >> 1); // 0010 = 2
    return 0;
}
```

2. Bit Manipulation in Embedded Systems

- **Use Cases**:
  - **Register Manipulation**: Set/clear/toggle bits in hardware registers (e.g., enable a peripheral).
  - **Flags**: Use bits to represent states (e.g., error flags).
  - **Optimization**: Replace arithmetic with shifts for faster computation (e.g., x * 2 → x << 1).
- **Embedded Tip**: Always use bitmasks with clear names to improve code readability and avoid magic numbers.
- **Example** (Set a bit in a register):

```
#include <stdio.h>
#include <stdint.h>

#define LED_ENABLE 0x04  // Bit 2 (00000100)

int main() {
    uint8_t reg = 0;  // Register initial state
    reg |= LED_ENABLE;  // Set bit 2
    printf("Register: 0x%02X\n", reg);  // 0x04
    return 0;
}
```

3. Macros for Bit Manipulation

- **Purpose**: Encapsulate common bit operations for reusability and clarity.
- **Common Macros**:
  - Set bit: reg |= (1 << n)
  - Clear bit: reg &= ~(1 << n)
  - Toggle bit: reg ^= (1 << n)
  - Test bit: (reg & (1 << n)) != 0
- **Example**:

```
#include <stdio.h>
#include <stdint.h>

#define SET_BIT(reg, bit) ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1U << (bit)))
#define TEST_BIT(reg, bit) (((reg) & (1U << (bit))) != 0)

int main() {
    uint8_t reg = 0;
    SET_BIT(reg, 3);  // Set bit 3
    printf("After set: 0x%02X\n", reg);  // 0x08
    TOGGLE_BIT(reg, 3);  // Toggle bit 3
    printf("After toggle: 0x%02X\n", reg);  // 0x00
    SET_BIT(reg, 2);
    printf("Bit 2 is %s\n", TEST_BIT(reg, 2) ? "set" : "clear");  // set
    CLEAR_BIT(reg, 2);
    printf("Bit 2 is %s\n", TEST_BIT(reg, 2) ? "set" : "clear");  // clear
    return 0;
}
```

- **Note**: Use 1U to ensure unsigned operations, avoiding sign-extension issues.

4. Practice Programs

a. Set/Clear/Toggle/Test Bit Macros

- Write a program to demonstrate the bit manipulation macros.
- **Code** (Extended from above):

```
#include <stdio.h>
#include <stdint.h>

#define SET_BIT(reg, bit) ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1U << (bit)))
#define TEST_BIT(reg, bit) (((reg) & (1U << (bit))) != 0)

int main() {
    uint8_t reg = 0;
    printf("Initial: 0x%02X\n", reg);
    SET_BIT(reg, 4);  // Set bit 4
    printf("Set bit 4: 0x%02X\n", reg);  // 0x10
    CLEAR_BIT(reg, 4);  // Clear bit 4
    printf("Clear bit 4: 0x%02X\n", reg);  // 0x00
    TOGGLE_BIT(reg, 5);  // Toggle bit 5
    printf("Toggle bit 5: 0x%02X\n", reg);  // 0x20
    printf("Bit 5 is %s\n", TEST_BIT(reg, 5) ? "set" : "clear");  // set
    return 0;
}
```

5. Project: Bitfield Encoder/Decoder for 32-bit Status Register

- **Objective**: Create a program to encode and decode fields in a 32-bit status register, simulating a hardware register (e.g., for a sensor or peripheral).
- **Requirements**:
  - Define a 32-bit register with fields: error (2 bits), status (3 bits), mode (4 bits), and data (8 bits).
  - Write functions to encode (set fields) and decode (read fields).
  - Use macros for bit operations.
  - Print register state in binary for debugging.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

#define SET_FIELD(reg, value, shift, mask) ((reg) = ((reg) & ~(mask)) | (((value) & (mask >> shift))
<< (shift)))
#define GET_FIELD(reg, shift, mask) (((reg) >> (shift)) & (mask >> (shift)))
// Register fields
#define ERROR_SHIFT 0
#define ERROR_MASK 0x00000003  // 2 bits
#define STATUS_SHIFT 2
#define STATUS_MASK 0x0000001C  // 3 bits
#define MODE_SHIFT 5
#define MODE_MASK 0x000001E0   // 4 bits
#define DATA_SHIFT 9
#define DATA_MASK 0x0001FE00   // 8 bits

void print_binary(uint32_t reg) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (reg >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }
```

```
    printf("\n");
}

int main() {
    uint32_t reg = 0;  // Initialize register
    // Encode values
    SET_FIELD(reg, 2, ERROR_SHIFT, ERROR_MASK);   // Error: 2 (10b)
    SET_FIELD(reg, 5, STATUS_SHIFT, STATUS_MASK); // Status: 5 (101b)
    SET_FIELD(reg, 10, MODE_SHIFT, MODE_MASK);    // Mode: 10 (1010b)
    SET_FIELD(reg, 100, DATA_SHIFT, DATA_MASK);   // Data: 100 (01100100b)
    printf("Encoded register: 0x%08X\n", reg);
    printf("Binary: ");
    print_binary(reg);
    // Decode values
    uint32_t error = GET_FIELD(reg, ERROR_SHIFT, ERROR_MASK);
    uint32_t status = GET_FIELD(reg, STATUS_SHIFT, STATUS_MASK);
    uint32_t mode = GET_FIELD(reg, MODE_SHIFT, MODE_MASK);
    uint32_t data = GET_FIELD(reg, DATA_SHIFT, DATA_MASK);
    printf("Decoded: Error=%u, Status=%u, Mode=%u, Data=%u\n", error, status, mode, data);
    return 0;
}
```

- **Explanation**:
  - SET_FIELD: Clears the field's bits using mask, then sets new value shifted to position.
  - GET_FIELD: Extracts field by shifting and masking.
  - print_binary: Displays register in binary for debugging.
  - Fields are packed into a 32-bit uint32_t to simulate a hardware register.
- **Embedded Tip**: Ensure masks align with field sizes to prevent overflow. Test edge cases (e.g., max values).

Tools Setup

- **GCC**: Compile with gcc program.c -o program and run ./program.
- **OnlineGDB**: Use https://www.onlinegdb.com/ for quick testing. Paste code, select C, and run.
- **Debugging**: Use printf to print binary representation or intermediate values.

Resources

- **"Bit Twiddling Hacks"**: Online resource (https://graphics.stanford.edu/~seander/bithacks.html) for bit manipulation techniques.
- **K&R "The C Programming Language" (Chapter 2)**: Covers bitwise operators and their use.
- **Online**: C reference (https://en.cppreference.com/w/c), GCC docs (https://gcc.gnu.org/onlinedocs/).

Practice Tips

- **Compile and Test**: Run each code snippet to verify output.
- **Experiment**: Modify macros to handle 16-bit registers or add error checking for invalid bit positions.
- **Optimize**: Replace print_binary with a more efficient version (e.g., use lookup table for hex-to-binary).
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Master Structs, Unions, and Enums

Goal

Master structs, unions, and enums, understanding their use in organizing data, managing memory efficiently, and handling hardware registers in embedded systems, with a focus on padding and alignment.

Tasks

- **Study**: Learn structs, unions, enums, padding, and alignment.
- **Practice**: Write a program to store sensor data using structs.
- **Project**: Develop a CLI calculator using function pointers.
- **Tools**: GCC compiler, OnlineGDB (online C compiler for quick testing).
- **Resources**: K&R "The C Programming Language" (Chapter 6), embedded.com articles on structs.

Detailed Notes

1. Structs

- **Definition**: A composite data type that groups variables of different types under a single name.
- **Syntax**:

```
struct name {
    type member1;
    type member2;
};
```

- **Usage in Embedded**: Organizes related data (e.g., sensor readings, configuration settings).
- **Padding and Alignment**: Compilers add padding bytes to align members to memory boundaries (e.g., 4-byte alignment for 32-bit systems).
- **Embedded Tip**: Use __attribute__((packed)) to eliminate padding in embedded systems, but beware of performance penalties due to unaligned access.
- **Example**:

```
#include <stdio.h>
#include <stdint.h>

struct Sensor {
    uint8_t id;    // 1 byte
    uint16_t value; // 2 bytes
    uint32_t timestamp; // 4 bytes
};

int main() {
    struct Sensor s = {1, 100, 123456};
    printf("Size: %zu bytes\n", sizeof(struct Sensor)); // May be 8 or more due to padding
    printf("ID: %u, Value: %u, Timestamp: %u\n", s.id, s.value, s.timestamp);
    return 0;
}
```

- **Note**: sizeof(struct Sensor) may be larger than expected (e.g., 12 bytes instead of 7) due to padding for alignment.

2. Packed Structs

- **Purpose**: Eliminate padding to save memory in embedded systems.
- **Syntax**: Use compiler-specific attributes (e.g., GCC's __attribute__((packed))).
- **Example**:

```
#include <stdio.h>
#include <stdint.h>

struct SensorPacked {
    uint8_t id;
    uint16_t value;
    uint32_t timestamp;
} __attribute__((packed));

int main() {
    struct SensorPacked s = {1, 100, 123456};
    printf("Size: %zu bytes\n", sizeof(struct SensorPacked)); // Exactly 7 bytes
    printf("ID: %u, Value: %u, Timestamp: %u\n", s.id, s.value, s.timestamp);
    return 0;
}
```

- **Embedded Tip**: Use packed structs for communication protocols (e.g., UART packets), but avoid for frequently accessed data due to slower unaligned access.

3. Unions

- **Definition**: A data type where all members share the same memory location, allowing different interpretations of the same data.
- **Syntax**:

```
union name {
    type member1;
    type member2;
};
```

- **Usage in Embedded**: Access hardware registers as different types (e.g., 32-bit value or individual bytes).
- **Example**:

```
#include <stdio.h>
#include <stdint.h>

union Register {
    uint32_t word;  // 4 bytes
    uint8_t bytes[4]; // Same memory as word
};

int main() {
    union Register reg = {0x12345678};
    printf("Word: 0x%08X\n", reg.word);
    printf("Bytes: %02X %02X %02X %02X\n", reg.bytes[0], reg.bytes[1], reg.bytes[2], reg.bytes[3]);
    reg.bytes[0] = 0xFF; // Modify first byte
    printf("Modified word: 0x%08X\n", reg.word); // Reflects change
    return 0;
}
```

- **Note**: Union size is the size of the largest member (e.g., 4 bytes here).

4. Enums

- **Definition**: A type for defining named constants, improving code readability.
- **Syntax**:

```
enum name { VALUE1, VALUE2, ... };
```

- **Usage in Embedded**: Represent states, modes, or error codes.
- **Embedded Tip**: Explicitly assign values to enums for consistent behavior across compilers.
- **Example**:

```
#include <stdio.h>

enum State { OFF = 0, ON = 1, ERROR = -1 };

int main() {
    enum State s = ON;
    printf("State: %d\n", s); // Prints 1
    if (s == ON) {
        printf("System is ON\n");
    }
    return 0;
}
```

5. Practice Program: Store Sensor Data

- **Objective**: Create a program to store and display sensor data using a struct.
- **Requirements**:
  - Define a struct for sensor data (ID, temperature, humidity).
  - Store multiple sensor readings in an array.
  - Display readings.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

struct SensorData {
    uint8_t id;      // Sensor ID
    int16_t temp;    // Temperature in Celsius
    uint8_t humidity; // Humidity percentage
} __attribute__((packed));

int main() {
    struct SensorData sensors[3] = {
        {1, 25, 60},
        {2, 22, 55},
        {3, -5, 70}
    };
    printf("Sensor Data:\n");
    for (int i = 0; i < 3; i++) {
        printf("Sensor %u: Temp=%d°C, Humidity=%u%%\n",
               sensors[i].id, sensors[i].temp, sensors[i].humidity);
    }
    printf("Size of struct: %zu bytes\n", sizeof(struct SensorData)); // 4 bytes
    return 0;
}
```

- **Note**: Packed struct ensures compact memory usage, suitable for embedded systems.

Project: CLI Calculator Using Function Pointers

- **Objective**: Build a CLI calculator that performs arithmetic operations using function pointers for modularity.
- **Requirements**:
  - Support addition, subtraction, multiplication, and division.
  - Use a struct to store operation and function pointer.
  - Handle user input for operation and numbers.
  - Check for division by zero.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

// Function prototypes
int32_t add(int32_t a, int32_t b) { return a + b; }

int32_t subtract(int32_t a, int32_t b) { return a - b; }

int32_t multiply(int32_t a, int32_t b) { return a * b; }

int32_t divide(int32_t a, int32_t b) { return b != 0 ? a / b : 0; }

// Struct for operation
struct Operation {
    char op;                    // Operator (+, -, *, /)
    int32_t (*func)(int32_t, int32_t); // Function pointer
};

int main() {
    // Array of operations
    struct Operation ops[] = {
        {'+', add},
        {'-', subtract},
        {'*', multiply},
        {'/', divide}
    };
    int num_ops = sizeof(ops) / sizeof(ops[0]);
    char op;
    int32_t a, b;
    printf("Enter operation (+, -, *, /): ");
    scanf(" %c", &op);
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    // Find and execute operation
    for (int i = 0; i < num_ops; i++) {
        if (ops[i].op == op) {
            if (op == '/' && b == 0) {
                printf("Error: Division by zero!\n");
                return 1;
            }
            printf("Result: %d\n", ops[i].func(a, b));
            return 0;
        }
    }
    printf("Invalid operation!\n");
    return 1;
}
```

- **Embedded Tip**: Function pointers are useful for state machines or event-driven systems in embedded programming, but avoid excessive use to keep code simple.

Tools Setup

- **GCC**: Compile with gcc program.c -o program and run ./program.
- **OnlineGDB**: Use https://www.onlinegdb.com/ for quick testing. Paste code, select C, and run.
- **Debugging**: Use printf to inspect struct sizes or function pointer results.

Resources

- **K&R "The C Programming Language" (Chapter 6)**: Covers structs, unions, and enums.
- **embedded.com Articles**: Search for “structs in embedded C” on https://www.embedded.com/ for practical examples.
- **Online**: C reference (https://en.cppreference.com/w/c), GCC docs (https://gcc.gnu.org/onlinedocs/).

Practice Tips

- **Compile and Test**: Run each code snippet to verify output.
- **Experiment**: Add a new field to the sensor struct or a new operation to the calculator.
- **Optimize**: Rearrange struct members to minimize padding without __attribute__((packed)).
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Consolidate Week 1 Concepts

Goal

Consolidate Week 1 concepts (C syntax, pointers, bit manipulation, structs) by reviewing notes, fixing bugs in prior projects, and optimizing the CLI calculator, ensuring readiness for embedded programming challenges.

Tasks

- **Study**: Review C syntax, pointers, bitwise operations, structs, unions, and enums.
- **Practice**: Fix bugs in prior projects (e.g., factorial, array resizer, bitfield encoder).
- **Project**: Optimize the CLI calculator from Day 4 and combine it with the bitfield encoder from Day 3.
- **Tools**: GCC compiler, GDB (debugger), OnlineGDB (online C compiler for quick testing).
- **Resources**: GCC documentation, GDB basics tutorials, K&R "The C Programming Language" (Chapters 1-2, 5-6).

Detailed Notes

1. Review of Week 1 Concepts

- **C Syntax and Flow Control (Day 1)**:
  - Variables, data types (uint8_t, int32_t), operators, loops (for, while), conditionals (if, switch), functions.
  - Key Embedded Tip: Use fixed-width types from <stdint.h> for portability.
- **Pointers and Arrays (Day 2)**:
  - Pointer arithmetic, dereferencing, arrays, dynamic memory (malloc, free, realloc).
  - Key Embedded Tip: Avoid dynamic allocation in real-time systems; initialize pointers to NULL.
- **Bit Manipulation (Day 3)**:
  - Bitwise operators (&, |, ^, ~, <<, >>), macros for set/clear/toggle/test bits.
  - Key Embedded Tip: Use unsigned types for shifts and clear bitmasks for register operations.
- **Structs, Unions, Enums (Day 4)**:
  - Structs (padding, __attribute__((packed))), unions (shared memory), enums (named constants).
  - Key Embedded Tip: Use packed structs for communication protocols, function pointers for modularity.

2. Debugging Basics with GDB

- **Purpose**: Identify and fix bugs in C programs, critical for embedded systems where errors can crash hardware.
- **Basic GDB Commands**:
  - gdb ./program: Start GDB with compiled program.
  - break main: Set breakpoint at main function.
  - run: Execute program until breakpoint or end.
  - print variable: Display variable value.
  - next: Step to next line.

- quit: Exit GDB.

- **Compilation for Debugging**: Use gcc -g program.c -o program to include debug symbols.
- **Embedded Tip**: Debugging on microcontrollers often uses hardware debuggers (e.g., JTAG), but GDB is great for initial testing.
- **Example** (Debugging factorial from Day 1):

```
gcc -g factorial.c -o factorial
gdb ./factorial
(gdb) break main
(gdb) run
(gdb) print n
(gdb) next
(gdb) quit
```

3. Bug Fixing in Prior Projects

a. Factorial Program (Day 1)

- **Potential Bug**: Recursive factorial overflows for large inputs (e.g., n > 12 for 32-bit unsigned long).
- **Fix**: Add input validation and switch to iterative version for efficiency.
- **Revised Code**:

```
#include <stdio.h>
#include <stdint.h>

uint32_t factorial(uint8_t n) {
    if (n > 12) return 0; // Prevent overflow
    uint32_t result = 1;
    for (uint8_t i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    uint8_t n;
    printf("Enter a number (0-12): ");
    scanf("%hhu", &n);
    if (n > 12) {
        printf("Input too large!\n");
    } else {
        printf("Factorial of %u is %u\n", n, factorial(n));
    }
    return 0;
}
```

- **Note**: Use uint8_t for input to limit range and uint32_t for result to handle up to 12!.

b. Dynamic Array Resizer (Day 2)

- **Potential Bug**: No check for realloc failure, potential memory leak if reallocation fails.
- **Fix**: Add error handling and ensure proper memory cleanup.

- **Revised Snippet**:

```
int *temp = (int*)realloc(arr, new_size * sizeof(int));

if (temp == NULL) {
    printf("Memory reallocation failed!\n");
    free(arr);
    arr = NULL;
    return 1;
}
arr = temp;
```

c. Bitfield Encoder/Decoder (Day 3)

- **Potential Bug**: No validation for field values exceeding mask size.
- **Fix**: Add checks in SET_FIELD macro.
- **Revised Macro**:

```
#define SET_FIELD(reg, value, shift, mask) do { \
    if ((value) <= (mask >> shift)) { \
        (reg) = ((reg) & ~(mask)) | (((value) & (mask >> shift)) << (shift)); \
    } else { \
        printf("Value 0x%X too large for field!\n", (unsigned)(value)); \
    } \
} while (0)
```

- **Note**: Use do-while(0) to ensure macro works correctly in all contexts.

4. Project: Optimize CLI Calculator and Combine with Bitfield Encoder

- **Objective**: Enhance the Day 4 CLI calculator for efficiency and integrate it with the Day 3 bitfield encoder to simulate a hardware control system.
- **Optimizations for CLI Calculator**:
  - Add input validation for numbers.
  - Use const for operation array to optimize memory.
  - Handle edge cases (e.g., overflow in multiplication).
- **Integration with Bitfield Encoder**:
  - Use a 32-bit register to store calculator state (e.g., operation code, error flags).
  - Encode/decode state using bitfield macros.
- **Requirements**:
  - Support add, subtract, multiply, divide.
  - Store operation and error status in a register.
  - Display register state in binary.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

// Bitfield macros
#define SET_FIELD(reg, value, shift, mask) do { \
    if ((value) <= (mask >> shift)) { \
        (reg) = ((reg) & ~(mask)) | (((value) & (mask >> shift)) << (shift)); \
    } else { \
        printf("Value 0x%X too large for field!\n", (unsigned)(value)); \
    } \
} while (0)
```

```c
#define GET_FIELD(reg, shift, mask) (((reg) >> (shift)) & (mask >> (shift)))

// Register fields
#define OP_SHIFT 0
#define OP_MASK 0x00000003    // 2 bits (0=add, 1=sub, 2=mul, 3=div)
#define ERROR_SHIFT 2
#define ERROR_MASK 0x00000004  // 1 bit (1=error)
#define RESULT_SHIFT 3
#define RESULT_MASK 0x0007FFF8 // 16 bits (signed result)

// Calculator functions
int32_t add(int32_t a, int32_t b) { return a + b; }

int32_t subtract(int32_t a, int32_t b) { return a - b; }

int32_t multiply(int32_t a, int32_t b) { return a * b; }

int32_t divide(int32_t a, int32_t b) { return b != 0 ? a / b : 0; }

// Operation struct
struct Operation {
    char op;
    uint8_t op_code; // Maps to register OP field
    int32_t (*func)(int32_t, int32_t);
};

// Print register in binary
void print_binary(uint32_t reg) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (reg >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }
    printf("\n");
}

int main() {
    // Operation table
    const struct Operation ops[] = {
        {'+', 0, add},
        {'-', 1, subtract},
        {'*', 2, multiply},
        {'/', 3, divide}
    };
    const int num_ops = sizeof(ops) / sizeof(ops[0]);
    char op;
    int32_t a, b;
    uint32_t reg = 0; // Status register
    // Input
    printf("Enter operation (+, -, *, /): ");
    scanf(" %c", &op);
    printf("Enter two integers (-32768 to 32767): ");
    if (scanf("%d %d", &a, &b) != 2 || a < -32768 || a > 32767 || b < -32768 || b > 32767) {
        printf("Invalid input!\n");
        SET_FIELD(reg, 1, ERROR_SHIFT, ERROR_MASK);
        printf("Register: 0x%08X\n", reg);
        printf("Binary: ");
        print_binary(reg);
        return 1;
    }
    // Find operation
    for (int i = 0; i < num_ops; i++) {
        if (ops[i].op == op) {
            // Check for division by zero
            if (op == '/' && b == 0) {
                printf("Error: Division by zero!\n");
                SET_FIELD(reg, 1, ERROR_SHIFT, ERROR_MASK);
                SET_FIELD(reg, ops[i].op_code, OP_SHIFT, OP_MASK);
```

```
                printf("Register: 0x%08X\n", reg);
                printf("Binary: ");
                print_binary(reg);
                return 1;
            }
            // Compute result
            int32_t result = ops[i].func(a, b);
            if (result < -32768 || result > 32767) {
                printf("Error: Result overflow!\n");
                SET_FIELD(reg, 1, ERROR_SHIFT, ERROR_MASK);
                SET_FIELD(reg, ops[i].op_code, OP_SHIFT, OP_MASK);
            } else {
                SET_FIELD(reg, ops[i].op_code, OP_SHIFT, OP_MASK);
                SET_FIELD(reg, result, RESULT_SHIFT, RESULT_MASK);
                printf("Result: %d\n", result);
            }

            printf("Register: 0x%08X\n", reg);
            printf("Binary: ");
            print_binary(reg);
            // Decode register
            uint32_t op_code = GET_FIELD(reg, OP_SHIFT, OP_MASK);
            uint32_t error = GET_FIELD(reg, ERROR_SHIFT, ERROR_MASK);
            int32_t decoded_result = (int16_t)GET_FIELD(reg, RESULT_SHIFT, RESULT_MASK); // Sign-
extend
            printf("Decoded: Op=%u, Error=%u, Result=%d\n", op_code, error, decoded_result);
            return 0;
        }
    }
    printf("Invalid operation!\n");
    SET_FIELD(reg, 1, ERROR_SHIFT, ERROR_MASK);
    printf("Register: 0x%08X\n", reg);
    printf("Binary: ");
    print_binary(reg);
    return 1;
}
```

- **Optimizations**:
  - Added input range checks (-32768 to 32767) to fit 16-bit result field.
  - Used const for operation array to optimize memory placement.
  - Improved error handling for overflow and invalid inputs.
- **Integration**:
  - Stores operation code (2 bits), error flag (1 bit), and result (16 bits) in a 32-bit register.
  - Uses bitfield macros to encode/decode state.
  - Prints register in binary for debugging.
- **Embedded Tip**: This simulates a hardware control register, where operation state and results are stored compactly, common in microcontroller programming.

Tools Setup

- **GCC**: Compile with gcc -g program.c -o program (include -g for GDB debugging).
- **GDB**: Run gdb ./program, set breakpoints, and inspect variables.
- **OnlineGDB**: Use https://www.onlinegdb.com/ for quick testing.
- **Debugging**: Use GDB to step through calculator code if results are incorrect.

Resources

- **K&R "The C Programming Language" (Chapters 1-2, 5-6)**: Review syntax, pointers, structs.
- **GCC Documentation**: https://gcc.gnu.org/onlinedocs/ for compiler options.
- **GDB Basics Tutorials**: Search for “GDB tutorial” (e.g., https://www.tutorialspoint.com/gdb/).
- **Online**: C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Review**: Spend ~1 hour rereading Days 1-4 notes and code.
- **Debug**: Use GDB to fix at least one bug in a prior project (e.g., factorial overflow).
- **Optimize**: Test calculator with edge cases (e.g., max inputs, division by zero).
- **Time Management**: Spend ~1 hour on review, ~1 hour on bug fixing, ~1-1.5 hours on the project.

Chapter 2: Embedded C Concepts

- **Understand volatile, const, static**
- **Learn memory-mapped I/O**
- **Understand ISR basics and startup code**
- **Explore linker scripts**
- **Consolidate Week 2 concepts**

Understand volatile, const, and static

Goal

Understand the volatile, const, and static keywords in the context of embedded systems, focusing on their roles in hardware interaction, memory optimization, and code reliability.

Tasks

- **Study**: Learn volatile, const, and static keywords, their behavior, and their importance in embedded programming.
- **Practice**: Write programs using these keywords to simulate hardware interaction.
- **Project**: Develop a volatile counter for a simulated hardware register.
- **Tools**: GCC compiler, STM32CubeIDE (for embedded development environment setup).
- **Resources**: embedded.com articles on volatile, STM32 reference manuals.

Detailed Notes

1. volatile Keyword

- **Definition**: Informs the compiler that a variable's value may change unexpectedly (e.g., by hardware or another process), preventing optimization that assumes the value is stable.
- **Use Cases in Embedded**:
  - Accessing hardware registers that change due to external events (e.g., timer or interrupt flags).
  - Variables modified in interrupt service routines (ISRs).
- **Behavior**: Forces the compiler to read/write the variable from/to memory every time, avoiding register caching.
- **Embedded Tip**: Always use volatile for variables tied to hardware or shared between ISRs and main code to prevent bugs from compiler optimizations.
- **Example**:

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const timer_reg = (volatile uint32_t *)0x40000000; // Simulated timer register

int main() {
    uint32_t count = 0;
    while (1) {
        if (*timer_reg != 0) { // Read register every loop
            count++;
            printf("Timer tick: %u\n", count);
            *timer_reg = 0; // Reset register
        }
    }
    return 0;  }
```

- **Note**: In a real system, timer_reg would point to a hardware register (e.g., STM32 TIMx_CNT). This example simulates checking a timer.

2. const Keyword

- **Definition**: Indicates a variable's value cannot be modified after initialization, enabling compiler optimizations and improving code safety.
- **Use Cases in Embedded**:
  - Define constant configuration data (e.g., pin assignments, lookup tables).
  - Protect hardware register addresses from modification.
- **Behavior**: const variables are often placed in read-only memory (e.g., flash) on microcontrollers.
- **Embedded Tip**: Combine const with pointers for register addresses (e.g., volatile uint32_t *const) to ensure the pointer itself is fixed but the data it points to is mutable.
- **Example**:

```
#include <stdio.h>
#include <stdint.h>

const uint8_t SENSOR_MAX = 100; // Constant limit
const uint32_t *const gpio_base = (uint32_t *)0x40010000; // Fixed register address

int main() {
    uint8_t sensor_value = 50;
    if (sensor_value <= SENSOR_MAX) {
        printf("Sensor value: %u\n", sensor_value);
    }
    // *gpio_base = 0x1; // Error: cannot modify volatile data via const pointer
    return 0;
}
```

- **Note**: const ensures SENSOR_MAX and gpio_base cannot be altered, suitable for fixed configurations.

3. static Keyword

- **Definition**: Controls variable/function scope and lifetime:
  - **Variables**: Retains value between function calls, stored in static memory (not stack).
  - **Functions**: Limits scope to the current file (internal linkage).
- **Use Cases in Embedded**:
  - Persistent state variables (e.g., counters in functions).
  - Restrict function visibility to avoid namespace conflicts in large projects.
- **Embedded Tip**: Use static variables sparingly in embedded systems due to limited static memory (e.g., SRAM on microcontrollers).
- **Example**:

```
#include <stdio.h>

static uint32_t counter = 0; // Persistent across function calls
static void increment_counter(void) { // Visible only in this file
    counter++;
    printf("Counter: %u\n", counter);
}

int main() {
    increment_counter(); // Counter: 1
    increment_counter(); // Counter: 2
```

```
    return 0;
}
```

- **Note**: static ensures counter retains its value and increment_counter is not exposed to other files.

4. Combining Keywords

- **Common Combinations**:
  - volatile const: Read-only hardware register (e.g., status register).
  - static volatile: Persistent variable modified by hardware/ISRs.
  - static const: Fixed configuration data private to a file.
- **Example** (Simulated status register):

```
#include <stdio.h>
#include <stdint.h>

volatile const uint32_t *const status_reg = (volatile const uint32_t *)0x40020000;

int main() {
    printf("Status: 0x%08X\n", *status_reg); // Read-only access
    // *status_reg = 0; // Error: cannot modify const
    return 0;
}
```

5. Practice Programs

a. Volatile Counter

- Simulate a counter incremented by a hardware event (e.g., timer interrupt).
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t hw_counter = 0; // Simulated hardware counter

int main() {
    uint32_t local_count = 0;
    for (int i = 0; i < 5; i++) {
        hw_counter++; // Simulate hardware increment
        local_count = hw_counter; // Read volatile variable
        printf("Count: %u\n", local_count);
    }
    return 0;
}
```

- **Note**: Without volatile, the compiler might optimize hw_counter reads, missing updates.

b. Const Configuration

- Define a const array for sensor thresholds.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

const uint8_t thresholds[] = {10, 50, 100}; // Fixed thresholds

int main() {
    uint8_t sensor_value = 75;
    for (int i = 0; i < 3; i++) {
        if (sensor_value <= thresholds[i]) {
            printf("Sensor below threshold %u: %u\n", i, thresholds[i]);
            break;
        }
    }
    // thresholds[0] = 20; // Error: cannot modify const
    return 0;
}
```

c. Static State Machine

- Use a static variable to track state in a function.
- **Code**:

```
#include <stdio.h>

static void toggle_led(void) {
    static uint8_t state = 0; // Persistent state
    state = !state;
    printf("LED state: %s\n", state ? "ON" : "OFF");
}

int main() {
    toggle_led(); // LED state: ON
    toggle_led(); // LED state: OFF
    toggle_led(); // LED state: ON
    return 0;
}
```

Project: Volatile Counter for Simulated Hardware Register

- **Objective**: Simulate a hardware register that increments periodically (e.g., a timer or event counter) using volatile, and track its value with error checking.
- **Requirements**:
  - Use volatile for the register to ensure proper reads.
  - Use const for fixed register address and error threshold.
  - Use static for a persistent error counter.
  - Simulate register updates and detect overflow.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>
```

```c
// Simulated hardware register
volatile uint32_t *const event_reg = (volatile uint32_t *)0x40030000;

const uint32_t MAX_COUNT = 1000; // Threshold for error
static uint32_t error_count = 0; // Persistent error counter

void check_register(void) {
    static uint32_t last_count = 0; // Track last value
    uint32_t current = *event_reg; // Read volatile register
    if (current >= MAX_COUNT) {
        error_count++;
        printf("Error: Count overflow (%u), Errors: %u\n", current, error_count);
    } else if (current < last_count) {
        error_count++;
        printf("Error: Counter reset detected (%u), Errors: %u\n", current, error_count);
    } else {
        printf("Count: %u\n", current);
    }
    last_count = current;
}

int main() {
    // Simulate register updates
    for (int i = 0; i < 5; i++) {
        *event_reg = i * 200; // Simulate hardware increment
        check_register();
    }
    *event_reg = 0; // Simulate reset
    check_register();
    *event_reg = 1001; // Simulate overflow
    check_register();
    return 0;      }
```

- **Explanation**:
  - event_reg is a volatile pointer to a simulated hardware register, ensuring every read/write accesses memory.
  - MAX_COUNT is const for a fixed threshold.
  - error_count and last_count are static to persist across function calls.
  - Program simulates register updates, detects overflow, and tracks errors.
- **Embedded Tip**: In real systems, event_reg would map to a microcontroller's peripheral register (e.g., STM32 TIMx_CNT).

Tools Setup

- **GCC**: Compile with gcc program.c -o program and run ./program.
- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Create a new project for a generic C program (no hardware needed for simulation).
- **Debugging**: Use printf for now; GDB will be used on Day 10.
- **Note**: This code runs on a PC for simulation. For STM32 hardware, replace event_reg with actual register addresses from the STM32 reference manual.

Resources

- **embedded.com Articles**: Search for "volatile in embedded C" on https://www.embedded.com/ for practical examples.
- **STM32 Reference Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1 series) for register details.

- **Online**: C reference (https://en.cppreference.com/w/c), GCC docs (https://gcc.gnu.org/onlinedocs/).

Practice Tips

- **Compile and Test**: Run each code snippet using GCC or STM32CubeIDE to verify output.
- **Experiment**: Modify the volatile counter to track different conditions (e.g., underflow).
- **Optimize**: Add a const lookup table for error messages in the project.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Learn Memory-Mapped I/O

Goal

Understand memory-mapped I/O, learning how to manipulate hardware registers directly to control peripherals in embedded systems, with a focus on practical application like LED control.

Tasks

- **Study**: Learn memory-mapped registers and their role in embedded systems.
- **Practice**: Write code to manipulate registers directly.
- **Project**: Develop a program to blink an LED using a delay loop (simulated for Arduino or STM32).
- **Tools**: Arduino IDE, STM32CubeIDE (for embedded development environment).
- **Resources**: Arduino documentation, STM32 reference manuals.

Detailed Notes

1. Memory-Mapped I/O

- **Definition**: A technique where hardware peripherals (e.g., GPIO, timers) are accessed via specific memory addresses, allowing the CPU to read/write registers as if they were memory.
- **How It Works**:
  - Each peripheral has a set of registers (e.g., control, status, data) mapped to predefined addresses in the microcontroller's memory space.
  - Reading/writing these addresses controls the hardware.
- **Embedded Tip**: Use volatile for register pointers to prevent compiler optimizations from skipping reads/writes, as hardware can change register values unexpectedly.
- **Example** (Simulated GPIO register):

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_OUT = (volatile uint32_t *)0x40010000; // Simulated output register

int main() {
    *GPIO_OUT = 0x1; // Set bit 0 (e.g., turn on LED)
    printf("GPIO_OUT: 0x%08X\n", *GPIO_OUT);
    *GPIO_OUT = 0x0; // Clear bit 0 (e.g., turn off LED)
    printf("GPIO_OUT: 0x%08X\n", *GPIO_OUT);
    return 0;
}
```

- **Note**: In real systems, GPIO_OUT would map to a microcontroller's GPIO register (e.g., STM32 GPIOx_ODR).

2. GPIO Registers

- **Common Registers**:
  - **Mode/Configuration (e.g., GPIO_MODER)**: Sets pin as input, output, or alternate function.
  - **Output Data (e.g., GPIO_ODR)**: Sets output pin states (high/low).
  - **Input Data (e.g., GPIO_IDR)**: Reads input pin states.
  - **Set/Reset (e.g., GPIO_BSRR)**: Atomically sets or clears pins.
- **Embedded Tip**: Use bit set/reset registers (e.g., STM32's GPIO_BSRR) for atomic operations to avoid read-modify-write issues in ISRs.
- **Example Structure** (STM32 GPIO, simplified):

```
typedef struct {
    volatile uint32_t MODER;   // Mode register
    volatile uint32_t ODR;     // Output data register
    volatile uint32_t IDR;     // Input data register
    volatile uint32_t BSRR;    // Bit set/reset register
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *)0x40020000) // STM32 GPIOA base address
```

3. Delay Loops

- **Purpose**: Create timing delays in software (e.g., for LED blinking) when precise hardware timers are not used.
- **Implementation**: Use a loop with a known number of iterations to approximate a delay.
- **Embedded Tip**: Delay loops are simple but imprecise; prefer hardware timers for real applications due to variability in CPU speed.
- **Example**:

```
#include <stdint.h>

void delay_ms(uint32_t ms) {
    // Approximate 1ms delay at 16MHz (adjust for CPU speed)
    for (uint32_t i = 0; i < ms * 16000; i++) {
        __asm volatile("nop"); // No operation (prevents loop optimization)
    }
}
```

4. Practice Programs

a. Simulated GPIO Toggle

- Toggle a bit in a simulated GPIO output register.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_OUT = (volatile uint32_t *)0x40010000;

#define LED_PIN 0 // Bit 0

void toggle_led(void) {
    *GPIO_OUT ^= (1U << LED_PIN); // Toggle bit
    printf("GPIO_OUT: 0x%08X\n", *GPIO_OUT); }
```

```
int main() {
    *GPIO_OUT = 0; // Initialize
    toggle_led(); // Set bit 0
    toggle_led(); // Clear bit 0
    return 0;
}
```

b. Configure GPIO Mode

- Simulate configuring a GPIO pin as output.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;

#define PIN_0_MODE (3U << 0) // 2 bits per pin, 01=output

int main() {
    *GPIO_MODER &= ~(3U << 0); // Clear mode bits for pin 0
    *GPIO_MODER |= (1U << 0);  // Set pin 0 as output (01)
    printf("GPIO_MODER: 0x%08X\n", *GPIO_MODER);
    return 0;
}
```

- **Note**: STM32 uses 2 bits per pin in MODER (00=input, 01=output, 10=alternate, 11=analog).

Project: Blink LED Using Delay Loop

- **Objective**: Write a program to blink an LED by manipulating a simulated GPIO register using a delay loop, adaptable for Arduino or STM32.
- **Requirements**:
  - Toggle a GPIO pin to simulate LED on/off.
  - Use a delay loop for timing (e.g., 500ms on, 500ms off).
  - Use volatile for register access and const for fixed addresses.
  - Simulate hardware for learning; provide notes for real hardware.
- **Code** (Simulated for PC, with Arduino/STM32 notes):

```
#include <stdio.h>
#include <stdint.h>

// Simulated GPIO registers

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_ODR = (volatile uint32_t *)0x40010004;

#define LED_PIN 5 // Pin 5 (bit 5)
#define OUTPUT_MODE (1U << (LED_PIN * 2)) // 01 for output

void delay_ms(uint32_t ms) {
    for (uint32_t i = 0; i < ms * 16000; i++) { // Simulate 1ms at 16MHz
        __asm volatile("nop");
    }
}
```

```
void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2)); // Clear mode bits
    *GPIO_MODER |= OUTPUT_MODE; // Set pin 5 as output
}

void toggle_led(void) {
    *GPIO_ODR ^= (1U << LED_PIN); // Toggle LED
    printf("LED state: %s (GPIO_ODR: 0x%08X)\n",
           (*GPIO_ODR & (1U << LED_PIN)) ? "ON" : "OFF", *GPIO_ODR);
}

int main() {
    init_gpio();
    *GPIO_ODR = 0; // Initialize output
    while (1) {
        toggle_led();
        delay_ms(500); // 500ms delay
        toggle_led();
        delay_ms(500);
    }
    return 0;
}
```

- **Arduino Adaptation**:
  - Replace register accesses with Arduino functions:

```
#define LED_PIN 13 // Arduino Uno LED pin

void init_gpio(void) { pinMode(LED_PIN, OUTPUT); }

void toggle_led(void) { digitalWrite(LED_PIN, !digitalRead(LED_PIN)); }
```

  - Compile using Arduino IDE (https://www.arduino.cc/en/software).
- **STM32 Adaptation**:
  - Use real GPIO registers (e.g., for STM32F103):

```
#define GPIOA ((volatile GPIO_TypeDef *)0x40010800) // STM32F1 GPIOA

typedef struct {
    volatile uint32_t MODER;
    volatile uint32_t ODR;
    // Other registers
} GPIO_TypeDef;

#define LED_PIN 5 // PA5
```

  - Enable GPIO clock in RCC register before use (see STM32 reference manual).
  - Use STM32CubeIDE to configure and compile.
- **Embedded Tip**: Use hardware timers instead of delay loops for precise timing in production code.

Tools Setup

- **Arduino IDE**: Download from https://www.arduino.cc/en/software. Use for Arduino simulation or real hardware (e.g., Arduino Uno).
- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Create a new project for STM32 or use for simulation.
- **GCC**: Compile simulation code with gcc program.c -o program and run ./program.

- **Debugging**: Use printf for simulation; use STM32CubeIDE's debugger for real hardware.

Resources

- **Arduino Documentation**: https://www.arduino.cc/reference/en/ for Arduino functions.
- **STM32 Reference Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1) for register details.
- **Online**: C reference (https://en.cppreference.com/w/c), GCC docs (https://gcc.gnu.org/onlinedocs/).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or test Arduino code in Arduino IDE.
- **Experiment**: Modify delay_ms to change blink frequency or add multiple LEDs.
- **Optimize**: Use bit set/reset register (e.g., STM32 GPIO_BSRR) instead of ODR for atomic updates.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Understand ISR Basics and Startup Code

Goal

Understand Interrupt Service Routines (ISRs) and startup code structure, learning how to handle hardware interrupts and initialize a microcontroller, with a focus on practical application for GPIO toggling.

Tasks

- **Study**: Learn Interrupt Service Routines (ISRs), their implementation, and startup code structure in embedded systems.
- **Practice**: Write a simple ISR to handle an interrupt.
- **Project**: Develop a program to toggle a GPIO pin using direct register access triggered by an ISR (simulated for Arduino or STM32).
- **Tools**: STM32CubeIDE, Arduino IDE.
- **Resources**: ARM Cortex-M manuals, YouTube: ISR basics tutorials.

Detailed Notes

1. Interrupt Service Routines (ISRs)

- **Definition**: Special functions executed by the CPU in response to hardware events (e.g., timer overflow, button press).
- **Key Features**:
  - **Fast Execution**: ISRs must be short to avoid delaying other tasks.
  - **No Parameters/Return**: Declared as void isr(void).
  - **Volatile Variables**: Use volatile for variables shared with main code to prevent optimization issues.
- **Components**:
  - **Interrupt Vector Table**: Maps interrupt sources to ISR addresses.
  - **Enable/Disable**: Controlled via peripheral registers and NVIC (Nested Vectored Interrupt Controller in ARM).
  - **Clear Flag**: ISRs must clear the interrupt flag to prevent repeated triggering.
- **Embedded Tip**: Minimize ISR code; set flags for main code to handle complex logic.
- **Example** (Simulated ISR):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const INT_FLAG = (volatile uint32_t *)0x40020000; // Simulated interrupt flag
volatile uint32_t event_count = 0;

void EXTI0_IRQHandler(void) { // Simulated ISR
    if (*INT_FLAG & (1U << 0)) { // Check interrupt flag
        event_count++;
        *INT_FLAG &= ~(1U << 0); // Clear flag    }                }
```

```
int main() {
    *INT_FLAG |= (1U << 0); // Simulate interrupt
    EXTI0_IRQHandler();
    printf("Event count: %u\n", event_count); // 1
    return 0;
}
```

2. Startup Code

- **Definition**: Code executed before main() to initialize the microcontroller (e.g., set up stack, copy data to RAM, clear BSS).
- **Components**:
  - **Reset Handler**: Entry point after reset, calls startup code and jumps to main.
  - **Vector Table**: Array of function pointers for ISRs and reset handler, stored in flash (e.g., at 0x08000000 for STM32).
  - **Data Initialization**: Copies .data section from flash to RAM, zeros .bss section.
- **Embedded Tip**: Startup code is compiler-specific (e.g., provided by STM32CubeIDE or Arduino). Understand it to debug boot issues.
- **Example** (Simplified startup code):

```
#include <stdint.h>

void Reset_Handler(void);

void EXTI0_IRQHandler(void) {}
// Vector table (simplified)
uint32_t *vector_table[] __attribute__((section(".isr_vector"))) = {
    (uint32_t *)0x2001C000, // Stack pointer
    (uint32_t *)Reset_Handler, // Reset handler
    (uint32_t *)EXTI0_IRQHandler // Interrupt handler
};

void Reset_Handler(void) {
    // Copy .data, zero .bss (simplified)
    main();
}

int main() {
    printf("System started\n");
    return 0;
}
```

- **Note**: Real startup code (e.g., startup_stm32f103xx.s) includes assembly for low-level initialization.

3. Practice Programs

a. Simple ISR for Button Press

- Simulate an ISR triggered by a button press (sets a flag).
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const BUTTON_FLAG = (volatile uint32_t *)0x40020000;
volatile uint8_t button_pressed = 0;
```

```
void EXTI0_IRQHandler(void) {
    if (*BUTTON_FLAG & (1U << 0)) {
        button_pressed = 1;
        *BUTTON_FLAG &= ~(1U << 0); // Clear flag
    }
}

int main() {
    *BUTTON_FLAG |= (1U << 0); // Simulate button press
    EXTI0_IRQHandler();
    printf("Button pressed: %u\n", button_pressed);
    return 0;
}
```

b. ISR with Counter

- Count interrupts using a volatile variable.
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const TIMER_FLAG = (volatile uint32_t *)0x40030000;
volatile uint32_t tick_count = 0;

void TIM2_IRQHandler(void) {
    if (*TIMER_FLAG & (1U << 0)) {
        tick_count++;
        *TIMER_FLAG &= ~(1U << 0); // Clear flag
    }
}

int main() {
    for (int i = 0; i < 3; i++) {
        *TIMER_FLAG |= (1U << 0); // Simulate timer interrupt
        TIM2_IRQHandler();
        printf("Tick count: %u\n", tick_count);
    }
    return 0;
}
```

Project: GPIO Toggle Using Direct Register Access via ISR

- **Objective**: Toggle a GPIO pin (simulating an LED) using an ISR triggered by a simulated interrupt (e.g., timer or button).
- **Requirements**:
  - Configure GPIO pin as output.
  - Use ISR to toggle the pin when an interrupt occurs.
  - Simulate interrupt triggers for testing.
  - Use volatile for registers and shared variables.
- **Code** (Simulated for PC, with Arduino/STM32 notes):

```
#include <stdio.h>
#include <stdint.h>

// Simulated registers
volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_ODR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const INT_FLAG = (volatile uint32_t *)0x40020000;
```

```
#define LED_PIN 5 // Pin 5
#define OUTPUT_MODE (1U << (LED_PIN * 2)) // 01 for output

volatile uint8_t led_state = 0;

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2)); // Clear mode bits
    *GPIO_MODER |= OUTPUT_MODE; // Set pin 5 as output
    *GPIO_ODR &= ~(1U << LED_PIN); // LED off
}

void EXTI0_IRQHandler(void) {
    if (*INT_FLAG & (1U << 0)) {
        led_state = !led_state;
        if (led_state) {
            *GPIO_ODR |= (1U << LED_PIN); // LED on
        } else {
            *GPIO_ODR &= ~(1U << LED_PIN); // LED off
        }
        *INT_FLAG &= ~(1U << 0); // Clear flag
    }
}

int main() {
    init_gpio();
    printf("Initial LED state: %s\n", led_state ? "ON" : "OFF");
    for (int i = 0; i < 3; i++) {
        *INT_FLAG |= (1U << 0); // Simulate interrupt
        EXTI0_IRQHandler();
        printf("LED state: %s (GPIO_ODR: 0x%08X)\n", led_state ? "ON" : "OFF", *GPIO_ODR);
    }
    return 0;
}
```

- **Arduino Adaptation**:
  - Use Arduino's interrupt system:

```
#define LED_PIN 13

volatile uint8_t led_state = 0;

void setup() {
    pinMode(LED_PIN, OUTPUT);
    attachInterrupt(digitalPinToInterrupt(2), isr, RISING); // Pin 2 for interrupt
}

void isr() {
    led_state = !led_state;
    digitalWrite(LED_PIN, led_state);
}

void loop() {}
```

  - Compile with Arduino IDE (https://www.arduino.cc/en/software).
- **STM32 Adaptation**:
  - Use STM32's EXTI for external interrupts (e.g., button on PA0):

```
#define GPIOA ((volatile GPIO_TypeDef *)0x40010800)
#define EXTI ((volatile EXTI_TypeDef *)0x40010400)
#define RCC_APB2ENR (*(volatile uint32_t *)0x40021018)
```

```
typedef struct {
    volatile uint32_t MODER, ODR; // Simplified
} GPIO_TypeDef;

typedef struct {
    volatile uint32_t IMR, PR; // Interrupt mask, pending
} EXTI_TypeDef;

void init_gpio_and_exti(void) {
    RCC_APB2ENR |= (1U << 2); // Enable GPIOA clock
    GPIOA->MODER &= ~(3U << (5 * 2)); // PA5 output
    GPIOA->MODER |= (1U << (5 * 2));
    RCC_APB2ENR |= (1U << 0); // Enable AFIO clock
    EXTI->IMR |= (1U << 0); // Enable EXTI0
}

void EXTI0_IRQHandler(void) {
    if (EXTI->PR & (1U << 0)) {
        led_state = !led_state;
        GPIOA->ODR = (GPIOA->ODR & ~(1U << 5)) | (led_state << 5);
        EXTI->PR |= (1U << 0); // Clear pending
    }
}
```

  - Configure in STM32CubeIDE; enable NVIC for EXTI0.
- **Embedded Tip**: Ensure ISRs are short; use volatile for led_state to ensure main code sees updates.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Create a project for STM32 (e.g., STM32F103) or simulate.
- **Arduino IDE**: Download from https://www.arduino.cc/en/software for Arduino simulation.
- **GCC**: Compile simulation code with gcc program.c -o program.
- **Debugging**: Use STM32CubeIDE's debugger or Arduino Serial Monitor.

Resources

- **ARM Cortex-M Manuals**: Download from https://developer.arm.com (e.g., Cortex-M3 for STM32F1).
- **YouTube: ISR Basics**: Search for "embedded C interrupts" (e.g., Neso Academy, STMicroelectronics).
- **Online**: C reference (https://en.cppreference.com/w/c), STM32CubeIDE docs (https://www.st.com).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or test Arduino/STM32 code in respective IDEs.
- **Experiment**: Add multiple interrupt sources (e.g., timer and button).
- **Optimize**: Move LED toggle logic to main loop, using ISR to set a flag.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Explore Linker Scripts

Goal

Understand the anatomy of linker scripts, their role in defining memory layout, and how to modify them for embedded systems, with practical application in a project to control an LED blink.

Tasks

- **Study**: Learn linker script structure, including sections and memory layout.
- **Practice**: Modify a basic linker script to customize memory allocation.
- **Project**: Implement an LED blink program with a custom linker script for memory placement.
- **Tools**: STM32CubeIDE, GCC.
- **Resources**: GNU LD documentation, embedded.com linker script tutorials.

Detailed Notes

1. Linker Scripts Overview

- **Definition**: A linker script is a file that instructs the linker (e.g., GNU ld) on how to organize code, data, and memory sections in the final executable for a microcontroller.
- **Purpose in Embedded**:
  - Define memory regions (e.g., flash, RAM).
  - Place code (.text), initialized data (.data), uninitialized data (.bss), and stack/heap.
  - Ensure interrupt vector table is placed at the correct address (e.g., 0x08000000 for STM32).
- **Key Components**:
  - **MEMORY**: Defines memory regions (e.g., flash, RAM) with addresses and sizes.
  - **SECTIONS**: Specifies how input sections from object files are mapped to output sections.
  - **Entry Point**: Defines the program's starting point (e.g., Reset_Handler).
- **Embedded Tip**: Always verify memory sizes against the microcontroller's datasheet to avoid overflows.

2. Linker Script Anatomy

- **MEMORY Section**:

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

  - rx: Read/execute (for flash); rwx: Read/write/execute (for RAM).
  - ORIGIN: Start address; LENGTH: Size of region.

- **SECTIONS Section**:

```
SECTIONS
{
    .text : { *(.text) } > FLASH /* Code in flash */
    .data : { *(.data) } > RAM AT > FLASH /* Initialized data in RAM, stored in flash */
    .bss : { *(.bss) } > RAM /* Uninitialized data in RAM */
}
```

  - Maps sections to memory regions; AT > FLASH means .data is stored in flash but copied to RAM at startup.
- **Embedded Tip**: Use __attribute__((section("name"))) in C to place variables/functions in custom sections.

3. Practice Program: Custom Linker Script

- **Objective**: Create a simple linker script and place a variable in a custom memory section.
- **Code** (C and linker script):

```
// main.c
#include <stdio.h>
#include <stdint.h>

uint32_t my_var __attribute__((section(".custom_section"))) = 0x1234;

int main() {
    printf("my_var at address %p: 0x%08X\n", (void*)&my_var, my_var);
    return 0;
}
```

```
/* custom.ld */
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
    CUSTOM (rwx) : ORIGIN = 0x20005000, LENGTH = 4K /* Custom region */
}
SECTIONS
{
    .text : { *(.text) } > FLASH
    .data : { *(.data) } > RAM AT > FLASH
    .bss : { *(.bss) } > RAM
    .custom_section : { *(.custom_section) } > CUSTOM
}
```

- **Compile**: gcc -T custom.ld main.c -o program
- **Note**: CUSTOM region is simulated; adjust addresses for real hardware (e.g., STM32F103).

4. Project: LED Blink with Custom Linker Script

- **Objective**: Implement an LED blink program with a custom linker script to place the vector table and code in specific memory regions, simulating an STM32 environment.
- **Requirements**:
  - Toggle a GPIO pin (simulated LED) using direct register access.
  - Define a custom linker script to place code in flash and a status variable in a specific RAM section.
  - Use volatile for register access.

- **Code** (Simulated for PC, with STM32 notes):

```c
// main.c
#include <stdio.h>
#include <stdint.h>

// Simulated registers
volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_ODR = (volatile uint32_t *)0x40010004;

#define LED_PIN 5
#define OUTPUT_MODE (1U << (LED_PIN * 2))

uint32_t blink_count __attribute__((section(".blink_data"))) = 0;

void delay_ms(uint32_t ms) {
    for (uint32_t i = 0; i < ms * 16000; i++) {
        __asm volatile("nop");
    }
}

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= OUTPUT_MODE;
    *GPIO_ODR &= ~(1U << LED_PIN);
}

void toggle_led(void) {
    *GPIO_ODR ^= (1U << LED_PIN);
    blink_count++;
    printf("LED state: %s, Blink count: %u\n",
           (*GPIO_ODR & (1U << LED_PIN)) ? "ON" : "OFF", blink_count);
}

void Reset_Handler(void) {
    main();
}

int main() {
    init_gpio();
    for (int i = 0; i < 3; i++) {
        toggle_led();
        delay_ms(500);
        toggle_led();
        delay_ms(500);
    }
    return 0;
}
```

```c
/* blink.ld */
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
    BLINK_RAM (rwx) : ORIGIN = 0x20005000, LENGTH = 4K /* Custom RAM section */
}
SECTIONS
{
    .isr_vector : { *(.isr_vector) } > FLASH /* Vector table */
    .text : { *(.text) } > FLASH /* Code */
    .data : { *(.data) } > RAM AT > FLASH /* Initialized data */
    .bss : { *(.bss) } > RAM /* Uninitialized data */
    .blink_data : { *(.blink_data) } > BLINK_RAM /* Custom section */
}
ENTRY(Reset_Handler)
```

- **Compile**: gcc -T blink.ld main.c -o program
- **STM32 Adaptation**:
    - Use STM32CubeIDE to generate a linker script (e.g., STM32F103C8TX_FLASH.ld).
    - Modify to add BLINK_RAM section.
    - Replace simulated registers with real STM32 GPIO registers:

```
#define GPIOA ((volatile GPIO_TypeDef *)0x40010800)

typedef struct {
    volatile uint32_t MODER, ODR;
} GPIO_TypeDef;
```

    - Enable GPIOA clock: RCC->APB2ENR |= (1U << 2);.
- **Embedded Tip**: Ensure section addresses align with microcontroller memory map (check datasheet).

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Create a project and modify the default linker script.
- **GCC**: Compile with gcc -T blink.ld main.c -o program for simulation.
- **Debugging**: Use STM32CubeIDE's debugger to verify section placement (check memory view).

Resources

- **GNU LD Documentation**: https://sourceware.org/binutils/docs/ld/ for linker script syntax.
- **embedded.com Linker Tutorials**: Search for "linker scripts embedded" on https://www.embedded.com/.
- **Online**: STM32CubeIDE docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or test STM32 code in STM32CubeIDE.
- **Experiment**: Add a new section for another variable (e.g., LED state).
- **Optimize**: Align sections to 4-byte boundaries for better performance.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Consolidate Week 2 Concepts

Goal

Consolidate Week 2 concepts (volatile, memory-mapped I/O, ISRs, linker scripts) by reviewing notes, debugging prior projects, and optimizing register access for efficiency in embedded systems.

Tasks

- **Study**: Review volatile, ISRs, memory-mapped I/O, and linker scripts.
- **Practice**: Debug issues in prior projects (e.g., volatile counter, LED blink, ISR toggle).
- **Project**: Optimize the LED blink program from Days 7-9 with minimal register writes, integrating ISR and custom linker script.
- **Tools**: STM32CubeIDE, GDB (debugger), GCC.
- **Resources**: STM32CubeIDE documentation, GDB tutorials, ARM Cortex-M manuals.

Detailed Notes

1. Review of Week 2 Concepts

- **Volatile (Day 6)**:
  - Ensures variables (e.g., hardware registers) are read/written from memory, preventing compiler optimizations.
  - Use for registers and ISR-shared variables.
  - Example: volatile uint32_t *const reg = (volatile uint32_t *)0x40010000;.
- **Memory-Mapped I/O (Day 7)**:
  - Access peripherals via memory addresses (e.g., GPIO_MODER, GPIO_ODR).
  - Use bit set/reset registers (e.g., GPIO_BSRR) for atomic updates.
  - Key: Use volatile for register pointers.
- **ISRs and Startup Code (Day 8)**:
  - ISRs handle hardware events; keep them short and clear interrupt flags.
  - Startup code initializes stack, data, and jumps to main.
  - Example: void EXTI0_IRQHandler(void) { /* Clear flag, toggle pin */ }.
- **Linker Scripts (Day 9)**:
  - Define memory layout (flash, RAM) and section placement (.text, .data, .bss).
  - Custom sections for specific variables (e.g., .blink_data).
  - Example: SECTIONS { .blink_data : { *(.blink_data) } > RAM }.

2. Debugging with GDB

- **Purpose**: Identify and fix issues in embedded code (e.g., incorrect register writes, ISR failures).
- **Key GDB Commands**:
  - gdb ./program: Start debugger.
  - break function: Set breakpoint (e.g., break EXTI0_IRQHandler).
  - run: Execute until breakpoint.

- print reg: Inspect variable/register (e.g., print *GPIO_ODR).
- next/step: Step through code.
- info registers: View CPU registers.
- quit: Exit.

- **Compilation**: Use gcc -g program.c -o program for debug symbols.
- **Embedded Tip**: For STM32, use STM32CubeIDE's debugger with hardware breakpoints for real-time debugging.
- **Example** (Debugging LED blink):

```
gcc -g blink.c -o blink
gdb ./blink
(gdb) break toggle_led
(gdb) run
(gdb) print *GPIO_ODR
(gdb) next
(gdb) quit
```

3. Debugging Prior Projects

a. Volatile Counter (Day 6)

- **Potential Bug**: Missing overflow check for hw_counter.
- **Fix**: Add validation to prevent wraparound errors.
- **Revised Code Snippet**:

```
void check_register(void) {
    static uint32_t last_count = 0;
    uint32_t current = *event_reg;

    if (current < last_count && current != 0) {
        error_count++;
        printf("Error: Unexpected counter reset (%u), Errors: %u\n", current, error_count);
    } else if (current >= MAX_COUNT) {
        error_count++;
        printf("Error: Count overflow (%u), Errors: %u\n", current, error_count);
    } else {
        printf("Count: %u\n", current);
    }
    last_count = current;
}
```

b. LED Blink with Delay Loop (Day 7)

- **Potential Bug**: Imprecise delay loop due to compiler optimization.
- **Fix**: Ensure nop assembly prevents loop removal.
- **Revised Delay**:

```
void delay_ms(uint32_t ms) {
    volatile uint32_t i; // Prevent optimization
    for (i = 0; i < ms * 16000; i++) {
        __asm volatile("nop");
    }
}
```

c. ISR GPIO Toggle (Day 8)

- **Potential Bug**: Missing NVIC enable for STM32 or incorrect flag clearing.
- **Fix**: Add NVIC configuration (simulated here).
- **Revised Snippet**:

```
void init_gpio_and_exti(void) {
    // Simulated NVIC enable
    volatile uint32_t *const NVIC_ISER = (volatile uint32_t *)0xE000E100;
    *NVIC_ISER |= (1U << 6); // Enable EXTI0 interrupt (IRQ6)
    EXTI->IMR |= (1U << 0); // Enable EXTI0
}
```

4. Project: Optimize LED Blink with Minimal Register Writes

- **Objective**: Enhance the LED blink program from Days 7-9 to use minimal register writes, integrate ISR triggering, and apply a custom linker script for memory placement.
- **Optimizations**:
  - Use GPIO_BSRR for atomic set/reset to reduce register writes.
  - Trigger blinks via ISR (e.g., simulated timer interrupt).
  - Place blink counter in a custom RAM section via linker script.
- **Requirements**:
  - Toggle LED using ISR with minimal register access.
  - Store blink count in a custom section.
  - Simulate timer interrupts for testing.
- **Code** (Simulated for PC, with STM32 notes):

```
// main.c
#include <stdio.h>
#include <stdint.h>

// Simulated registers
volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004; // Bit set/reset
volatile uint32_t *const TIM_FLAG = (volatile uint32_t *)0x40020000; // Timer flag

#define LED_PIN 5
#define OUTPUT_MODE (1U << (LED_PIN * 2))
#define SET_PIN (1U << LED_PIN)
#define RESET_PIN (1U << (LED_PIN + 16)) // BSRR reset bit

volatile uint32_t blink_count __attribute__((section(".blink_data"))) = 0;
volatile uint8_t led_state = 0;

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= OUTPUT_MODE;
    *GPIO_BSRR = RESET_PIN; // LED off
}

void TIM2_IRQHandler(void) {
    if (*TIM_FLAG & (1U << 0)) {
        led_state = !led_state;
        *GPIO_BSRR = led_state ? SET_PIN : RESET_PIN; // Single write
        blink_count++;
        *TIM_FLAG &= ~(1U << 0); // Clear flag
        printf("LED: %s, Blinks: %u (BSRR: 0x%08X)\n",
               led_state ? "ON" : "OFF", blink_count, *GPIO_BSRR);
    }     }
```

```
void Reset_Handler(void) {
    main();
}

int main() {
    init_gpio();
    printf("Initial LED: %s, Blinks: %u\n", led_state ? "ON" : "OFF", blink_count);
    for (int i = 0; i < 4; i++) {
        *TIM_FLAG |= (1U << 0); // Simulate timer interrupt
        TIM2_IRQHandler();
    }
    return 0;
}
```

```
/* blink_opt.ld */
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
    BLINK_RAM (rwx) : ORIGIN = 0x20005000, LENGTH = 4K
}
SECTIONS
{
    .isr_vector : { *(.isr_vector) } > FLASH
    .text : { *(.text) } > FLASH
    .data : { *(.data) } > RAM AT > FLASH
    .bss : { *(.bss) } > RAM
    .blink_data : { *(.blink_data) } > BLINK_RAM
}
ENTRY(Reset_Handler)
```

- **Compile**: gcc -g -T blink_opt.ld main.c -o program
- **Optimizations**:
  - Uses GPIO_BSRR for single-write toggle, avoiding read-modify-write.
  - ISR handles toggle to simulate timer-driven blinks.
  - blink_count in custom .blink_data section for organized memory.
- **STM32 Adaptation**:
  - Replace registers with STM32 equivalents:

```
#define GPIOA ((volatile GPIO_TypeDef *)0x40010800)
#define TIM2 ((volatile TIM_TypeDef *)0x40000000)

typedef struct {
    volatile uint32_t MODER, ODR, BSRR;
} GPIO_TypeDef;

typedef struct {
    volatile uint32_t SR; // Status register
} TIM_TypeDef;
```

  - Enable clocks: RCC->APB2ENR |= (1U << 2); RCC->APB1ENR |= (1U << 0);.
  - Configure TIM2 in STM32CubeIDE for periodic interrupts.
- **Embedded Tip**: Minimize register writes in ISRs to reduce latency; verify linker script addresses against datasheet.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use for STM32 projects or simulation.
- **GCC**: Compile with gcc -g -T blink_opt.ld main.c -o program.
- **GDB**: Debug with gdb ./program to inspect registers and memory sections.
- **Debugging**: Use STM32CubeIDE's memory view to verify .blink_data placement.

Resources

- **STM32CubeIDE Documentation**: https://www.st.com for IDE usage.
- **GDB Tutorials**: Search for "GDB embedded debugging" (e.g., https://www.tutorialspoint.com/gdb/).
- **ARM Cortex-M Manuals**: https://developer.arm.com for interrupt and register details.
- **Online**: C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Review**: Spend ~1 hour rereading Days 6-9 notes and code.
- **Debug**: Use GDB to fix one bug (e.g., incorrect BSRR write).
- **Optimize**: Test with multiple interrupts; reduce ISR code size.
- **Time Management**: Spend ~1 hour on review, ~1 hour on debugging, ~1-1.5 hours on the project.

Chapter 3: Microcontroller Fundamentals

- **Master GPIO and UART**
- **Understand timers and PWM**
- **Learn interrupts and NVIC**
- **Use HAL and direct registers**
- **Consolidate Week 3 concepts**

Master GPIO and UART

Goal

Master GPIO configuration and UART communication on a microcontroller, learning to set up pins and implement serial communication for embedded systems.

Tasks

- **Study**: Learn GPIO setup (input/output modes) and UART configuration (baud rate, data format).
- **Practice**: Configure GPIO pins and UART for communication.
- **Project**: Develop a UART echo program to receive and retransmit characters.
- **Tools**: STM32CubeIDE, Arduino IDE, STM32CubeMX (for configuration).
- **Resources**: STM32 HAL documentation, Arduino UART tutorials.

Detailed Notes

1. GPIO Configuration

- **Definition**: General Purpose Input/Output (GPIO) pins control or read external signals (e.g., LEDs, buttons).
- **Key Registers (STM32)**:
  - **MODER**: Sets pin mode (input, output, alternate function, analog).
  - **ODR**: Output Data Register (sets pin high/low).
  - **IDR**: Input Data Register (reads pin state).
  - **BSRR**: Bit Set/Reset Register (atomic updates).
  - **AFR**: Alternate Function Register (for UART, SPI, etc.).
- **Modes**:
  - Input: Read external signals (e.g., button state).
  - Output: Drive signals (e.g., LED on/off).
  - Alternate Function: Connect pin to peripheral (e.g., UART TX/RX).
- **Embedded Tip**: Use pull-up/pull-down resistors for inputs to avoid floating states; use BSRR for atomic operations.
- **Example** (STM32 GPIO setup, simplified):

```c
#define GPIOA ((GPIO_TypeDef *)0x40010800)

typedef struct {
    volatile uint32_t MODER, ODR, IDR, BSRR;
} GPIO_TypeDef;

void init_gpio(void) {
    volatile uint32_t *RCC_APB2ENR = (volatile uint32_t *)0x40021018;
    *RCC_APB2ENR |= (1U << 2); // Enable GPIOA clock
    GPIOA->MODER &= ~(3U << (5 * 2)); // Clear PA5 mode
    GPIOA->MODER |= (1U << (5 * 2)); // PA5 output
    GPIOA->BSRR = (1U << (5 + 16)); // Clear PA5 (LED off)
}
```

2. UART Communication

- **Definition**: Universal Asynchronous Receiver/Transmitter (UART) enables serial communication for debugging, sensor data, or inter-device communication.
- **Key Parameters**:
  - **Baud Rate**: Bits per second (e.g., 9600, 115200).
  - **Data Bits**: Typically 8.
  - **Parity**: None, even, or odd.
  - **Stop Bits**: 1 or 2.
- **STM32 UART Registers** (e.g., USART1):
  - **CR1**: Control register (enable UART, TX/RX).
  - **BRR**: Baud rate register.
  - **DR**: Data register (transmit/receive data).
  - **SR**: Status register (TX empty, RX not empty).
- **Embedded Tip**: Use a logic analyzer or USB-to-serial adapter to debug UART; ensure matching baud rates between devices.
- **Example** (Simplified STM32 UART setup):

```c
#define USART1 ((USART_TypeDef *)0x40013800)

typedef struct {
    volatile uint32_t SR, DR, BRR, CR1;
} USART_TypeDef;

void init_uart(uint32_t baud) {
    volatile uint32_t *RCC_APB2ENR = (volatile uint32_t *)0x40021018;
    *RCC_APB2ENR |= (1U << 14); // Enable USART1 clock
    GPIOA->MODER &= ~(3U << (9 * 2)); // PA9 (TX) alternate function
    GPIOA->MODER |= (2U << (9 * 2));
    GPIOA->AFR[1] &= ~(0xF << 4); // AF7 for PA9
    GPIOA->AFR[1] |= (7U << 4);
    USART1->BRR = 16000000 / baud; // Assuming 16MHz clock
    USART1->CR1 = (1U << 13) | (1U << 3) | (1U << 2); // Enable UART, TX, RX
}
```

3. Practice Programs

a. GPIO LED Control

- Configure a GPIO pin to toggle an LED.
- **Code** (STM32 simulation):

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;

#define LED_PIN 5

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= (1U << (LED_PIN * 2)); // Output
    *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
}
```

```
void toggle_led(void) {
    *GPIO_BSRR = (1U << LED_PIN); // Set
    *GPIO_BSRR = (1U << (LED_PIN + 16)); // Reset
    printf("LED toggled\n");
}

int main() {
    init_gpio();
    toggle_led();
    return 0;
}
```

- **Arduino Version**:

```
#define LED_PIN 13

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);
    delay(500);
    digitalWrite(LED_PIN, LOW);
    delay(500);
}
```

b. UART Transmit

- Send a fixed string via UART.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

void uart_transmit_char(char c) {
    while (!(*USART_SR & (1U << 7))); // Wait for TXE
    *USART_DR = c;
}

void uart_transmit_string(const char *str) {
    while (*str) {
        uart_transmit_char(*str++);
    }
}

int main() {
    uart_transmit_string("Hello, UART!\n");
    return 0;
}
```

- **Arduino Version**:

```
void setup() {
    Serial.begin(9600);
}
void loop() {
    Serial.println("Hello, UART!");
    delay(1000);   }
```

Project: UART Echo Program

- **Objective**: Implement a UART echo program that receives a character and retransmits it, simulating a microcontroller's serial communication.
- **Requirements**:
  - Configure GPIO for UART TX/RX pins.
  - Set up UART with 115200 baud, 8 data bits, no parity, 1 stop bit.
  - Echo received characters back to the sender.
  - Use STM32 HAL or Arduino libraries for simplicity.
- **Code** (STM32 with HAL, simulated registers for PC):

```
// Simulated HAL-like code for PC

#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_AFRH = (volatile uint32_t *)0x40010008;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const USART_BRR = (volatile uint32_t *)0x40013808;
volatile uint32_t *const USART_CR1 = (volatile uint32_t *)0x4001380C;

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (9 * 2)); // PA9 (TX) alternate function
    *GPIO_MODER |= (2U << (9 * 2));
    *GPIO_MODER &= ~(3U << (10 * 2)); // PA10 (RX) alternate function
    *GPIO_MODER |= (2U << (10 * 2));
    *GPIO_AFRH &= ~(0xF << 4); // AF7 for PA9
    *GPIO_AFRH |= (7U << 4);
    *GPIO_AFRH &= ~(0xF << 8); // AF7 for PA10
    *GPIO_AFRH |= (7U << 8);
}

void init_uart(uint32_t baud) {
    *USART_BRR = 16000000 / baud; // 16MHz clock
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2); // Enable UART, TX, RX
}

uint8_t uart_receive(void) {
    static uint8_t sim_data[] = {'A', 'B', 'C', 0}; // Simulated input
    static uint32_t idx = 0;
    if (sim_data[idx] == 0) return 0;
    *USART_SR |= (1U << 5); // Set RXNE flag
    *USART_DR = sim_data[idx++]; // Simulate received data
    while (!(*USART_SR & (1U << 5))); // Wait for RXNE
    return (uint8_t)(*USART_DR);
}

void uart_transmit(uint8_t data) {
    while (!(*USART_SR & (1U << 7))); // Wait for TXE
    *USART_DR = data;
    printf("Sent: %c\n", data);
}

int main() {
    init_gpio();
    init_uart(115200);
    while (1) {
        uint8_t rx_data = uart_receive();
        if (rx_data) {
            uart_transmit(rx_data); // Echo back
```

```
        }
    }
    return 0;
}
```

- **STM32 HAL Version** (Real hardware, STM32CubeIDE):

```
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;

void init_gpio_and_uart(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    HAL_UART_Init(&huart1);
}

int main(void) {
    HAL_Init();
    init_gpio_and_uart();
    uint8_t data;
    while (1) {
        if (HAL_UART_Receive(&huart1, &data, 1, 10) == HAL_OK) {
            HAL_UART_Transmit(&huart1, &data, 1, 10);
        }
    }
}
```

- **Arduino Version**:

```
void setup() {
    Serial.begin(115200);
}

void loop() {
    if (Serial.available()) {
        char c = Serial.read();
        Serial.write(c);
    }
}
```

- **Setup for STM32**:
    - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX), 115200 baud.
    - Generate code in STM32CubeIDE.
    - Connect USB-to-serial adapter (e.g., FTDI) to PA9/PA10; use a terminal (e.g., PuTTY) to test.
- **Embedded Tip**: Add error handling for UART overruns or framing errors in production code.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Create a project with CubeMX integration.
- **STM32CubeMX**: Integrated in STM32CubeIDE; use to configure GPIO and UART.
- **Arduino IDE**: Download from https://www.arduino.cc/en/software for Arduino projects.
- **Debugging**: Use STM32CubeIDE's debugger or Arduino Serial Monitor; simulate on PC with GCC (gcc program.c -o program).

Resources

- **STM32 HAL Documentation**: Included in STM32CubeIDE or at https://www.st.com.
- **Arduino UART Tutorials**: https://www.arduino.cc/reference/en/language/functions/communication/serial/.
- **Online**: STM32 reference manuals (e.g., RM0008 for STM32F1), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC, Arduino code in Arduino IDE, or STM32 code in STM32CubeIDE.
- **Experiment**: Add a GPIO-controlled LED that toggles on UART receive.
- **Optimize**: Buffer multiple characters before transmitting in the echo program.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Understand Timers and PWM

Goal

Understand microcontroller timers and PWM generation, learning to configure timers for precise timing and control PWM to adjust output signals, with a focus on practical application like LED dimming.

Tasks

- **Study**: Learn timer configuration (e.g., prescaler, auto-reload) and PWM generation principles.
- **Practice**: Write timer-based code to control timing and generate PWM signals.
- **Project**: Develop a PWM LED dimmer using a timer.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration), Arduino IDE.
- **Resources**: STM32 timer manuals, YouTube PWM tutorials.

Detailed Notes

1. Timers in Microcontrollers

- **Definition**: Hardware timers are peripherals that count clock cycles to measure time, generate events, or produce signals (e.g., PWM).
- **Key Components (STM32)**:
  - **Counter (CNT)**: Tracks current count.
  - **Prescaler (PSC)**: Divides system clock to slow timer ticks (e.g., PSC = 15 for 16MHz clock → 1MHz ticks).
  - **Auto-Reload Register (ARR)**: Sets period (counter resets when reaching ARR).
  - **Control Registers (CR1, CR2)**: Enable timer, set mode.
  - **Status Register (SR)**: Flags events (e.g., update event).
- **Modes**:
  - **Up/Down Counting**: Increment or decrement CNT.
  - **One-Shot/Periodic**: Single event or continuous.
  - **PWM**: Generate pulse-width modulated signals.
- **Embedded Tip**: Calculate timer frequency as f_timer = f_clock / ((PSC + 1) * (ARR + 1)) to match desired timing.
- **Example** (Simplified STM32 timer setup):

```
#define TIM2 ((TIM_TypeDef *)0x40000000)

typedef struct {
    volatile uint32_t CR1, PSC, ARR, SR;
} TIM_TypeDef;

void init_timer(uint32_t freq_hz) {
    volatile uint32_t *RCC_APB1ENR = (volatile uint32_t *)0x4002101C;
    *RCC_APB1ENR |= (1U << 0); // Enable TIM2 clock
    TIM2->PSC = 15999; // 16MHz / (15999+1) = 1kHz
    TIM2->ARR = (1000000 / freq_hz) - 1; // Period for desired freq
    TIM2->CR1 = (1U << 0); // Enable timer
}
```

2. Pulse Width Modulation (PWM)

- **Definition**: A technique to generate a square wave with variable duty cycle (on-time vs. total period) to control power delivery (e.g., LED brightness, motor speed).
- **Key Parameters**:
  - **Period**: Time for one cycle (set by ARR).
  - **Duty Cycle**: Percentage of time signal is high (set by Compare register, CCRx).
  - **Frequency**: f_pwm = f_timer / (ARR + 1).
- **STM32 PWM Setup**:
  - Configure timer in PWM mode.
  - Map timer channel to GPIO pin via alternate function.
  - Set CCRx to control duty cycle.
- **Embedded Tip**: Ensure PWM frequency is suitable for the application (e.g., 1kHz for LEDs, 20kHz for motors to avoid audible noise).
- **Example** (Simplified PWM setup):

```
#define TIM2 ((TIM_TypeDef *)0x40000000)

typedef struct {
    volatile uint32_t CR1, PSC, ARR, CCR1, CCMR1;
} TIM_TypeDef;

void init_pwm(uint32_t freq_hz, uint8_t duty_percent) {
    volatile uint32_t *RCC_APB1ENR = (volatile uint32_t *)0x4002101C;
    *RCC_APB1ENR |= (1U << 0); // Enable TIM2
    TIM2->PSC = 15999; // 1kHz base
    TIM2->ARR = (1000000 / freq_hz) - 1; // PWM period
    TIM2->CCMR1 = (6U << 4); // PWM mode 1, channel 1
    TIM2->CCR1 = (duty_percent * (TIM2->ARR + 1)) / 100; // Duty cycle
    TIM2->CR1 = (1U << 0); // Enable timer
}
```

3. Practice Programs

a. Timer-Based Delay

- Use a timer to create a precise delay instead of a loop.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const TIM_SR = (volatile uint32_t *)0x40000010;
volatile uint32_t *const TIM_CNT = (volatile uint32_t *)0x40000024;

void init_timer(uint32_t freq_hz) {
    volatile uint32_t *RCC_APB1ENR = (volatile uint32_t *)0x4002101C;
    *RCC_APB1ENR |= (1U << 0);
    *TIM_CNT = 0;
    *TIM_SR = 0;
    *TIM_PSC = 15999; // 1kHz
    *TIM_ARR = (1000000 / freq_hz) - 1;
    *TIM_CR1 = (1U << 0);
}
```

```
void delay_ms(uint32_t ms) {
    *TIM_CNT = 0; // Reset counter
    *TIM_SR = 0; // Clear update flag
    while (!(*TIM_SR & 1U)); // Wait for update
}

int main() {
    init_timer(1000); // 1ms period
    printf("Delaying...\n");
    delay_ms(1000);
    printf("Done\n");
    return 0;
}
```

- **Arduino Version**:

```
void setup() {}

void loop() {
    Serial.println("Delaying...");
    delay(1000); // Arduino's timer-based delay
    Serial.println("Done");
}
```

b. PWM Signal Generation

- Generate a PWM signal with 50% duty cycle.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const TIM_CCMR1 = (volatile uint32_t *)0x40000018;
volatile uint32_t *const TIM_CCR1 = (volatile uint32_t *)0x40000034;

void init_pwm(uint32_t freq_hz, uint8_t duty_percent) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 alternate function
    *GPIO_MODER |= (2U << (0 * 2));
    *TIM_PSC = 15999; // 1kHz base
    *TIM_ARR = (1000000 / freq_hz) - 1;
    *TIM_CCMR1 = (6U << 4); // PWM mode 1
    *TIM_CCR1 = (duty_percent * (*TIM_ARR + 1)) / 100;
    *TIM_CR1 = (1U << 0);
    printf("PWM: %uHz, %u%%\n", freq_hz, duty_percent);
}

int main() {
    init_pwm(1000, 50); // 1kHz, 50% duty
    return 0;
}
```

Project: PWM LED Dimmer with Timer

- **Objective**: Implement a PWM-based LED dimmer using a timer to control brightness, simulating or targeting STM32/Arduino hardware.
- **Requirements**:
  - Configure timer for PWM output on a GPIO pin.
  - Gradually increase/decrease duty cycle to fade LED.

- Use 1kHz PWM frequency.
- Simulate for PC or use STM32 HAL/Arduino libraries.

- **Code** (STM32 with HAL, simulated registers for PC):

```
// Simulated HAL-like code for PC

#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_AFR = (volatile uint32_t *)0x40010008;
volatile uint32_t *const TIM_PSC = (volatile uint32_t *)0x40000028;
volatile uint32_t *const TIM_ARR = (volatile uint32_t *)0x4000002C;
volatile uint32_t *const TIM_CCR1 = (volatile uint32_t *)0x40000034;
volatile uint32_t *const TIM_CCMR1 = (volatile uint32_t *)0x40000018;
volatile uint32_t *const TIM_CR1 = (volatile uint32_t *)0x40000000;

void init_pwm(uint32_t freq_hz) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 alternate function
    *GPIO_MODER |= (2U << (0 * 2));
    *GPIO_AFR &= ~(0xF << (0 * 4)); // AF2 for TIM2 on PA0
    *GPIO_AFR |= (2U << (0 * 4));
    *TIM_PSC = 15999; // 1kHz base (16MHz clock)
    *TIM_ARR = (1000000 / freq_hz) - 1; // 1kHz PWM
    *TIM_CCMR1 = (6U << 4) | (1U << 3); // PWM mode 1, enable preload
    *TIM_CR1 = (1U << 0); // Enable timer
}

void set_pwm_duty(uint8_t duty_percent) {
    *TIM_CCR1 = (duty_percent * (*TIM_ARR + 1)) / 100;
    printf("Duty cycle: %u%%\n", duty_percent);
}

void delay_ms(uint32_t ms) {
    for (uint32_t i = 0; i < ms * 16000; i++) {
        __asm volatile("nop");
    }
}

int main() {
    init_pwm(1000); // 1kHz PWM
    while (1) {
        for (uint8_t duty = 0; duty <= 100; duty += 10) {
            set_pwm_duty(duty);
            delay_ms(500);
        }
        for (uint8_t duty = 100; duty > 0; duty -= 10) {
            set_pwm_duty(duty);
            delay_ms(500);
        }
    }
    return 0;
}
```

- **STM32 HAL Version** (Real hardware, STM32CubeIDE):

```
#include "stm32f1xx_hal.h"

TIM_HandleTypeDef htim2;

void init_pwm(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_TIM2_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
```

```
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz base
    htim2.Init.Period = 999; // 1kHz PWM
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    HAL_TIM_PWM_Init(&htim2);
    TIM_OC_InitTypeDef sConfigOC = {0};
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0; // Initial duty
    HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
}

void set_pwm_duty(uint8_t duty_percent) {
    uint32_t pulse = (duty_percent * (htim2.Init.Period + 1)) / 100;
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
}

int main(void) {
    HAL_Init();
    init_pwm();
    while (1) {
        for (uint8_t duty = 0; duty <= 100; duty += 10) {
            set_pwm_duty(duty);
            HAL_Delay(500);
        }
        for (uint8_t duty = 100; duty > 0; duty -= 10) {
            set_pwm_duty(duty);
            HAL_Delay(500);
        }
    }
}
```

- **Arduino Version**:

```
#define LED_PIN 9 // PWM-capable pin

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    for (int brightness = 0; brightness <= 255; brightness += 25) {
        analogWrite(LED_PIN, brightness);
        delay(500);
    }
    for (int brightness = 255; brightness >= 0; brightness -= 25) {
        analogWrite(LED_PIN, brightness);
        delay(500);
    }
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure TIM2, Channel 1 for PWM on PA0.
  - Connect an LED (with resistor) to PA0; use a scope or multimeter to verify PWM.
  - Generate code in STM32CubeIDE; test with a terminal if UART is added for debugging.
- **Embedded Tip**: Adjust PWM frequency for LED (1kHz is fine) but use higher frequencies (e.g., 20kHz) for motors to avoid noise.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for timer configuration.
- **STM32CubeMX**: Integrated in STM32CubeIDE; configure TIM2 for PWM.
- **Arduino IDE**: Download from https://www.arduino.cc/en/software for Arduino projects.
- **Debugging**: Use STM32CubeIDE's debugger or Arduino Serial Monitor; simulate on PC with GCC (gcc program.c -o program).

Resources

- **STM32 Timer Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1 timers).
- **YouTube PWM Tutorials**: Search for "STM32 PWM tutorial" or "Arduino PWM" (e.g., STMicroelectronics, DroneBot Workshop).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC, Arduino code in Arduino IDE, or STM32 code in STM32CubeIDE.
- **Experiment**: Add UART to print duty cycle changes.
- **Optimize**: Use timer interrupts instead of delay_ms for non-blocking operation.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Learn Interrupts and NVIC

Goal

Master interrupt handling and NVIC setup on a microcontroller, learning to configure and use interrupts for responsive applications, with a focus on implementing a button interrupt handler with debouncing.

Tasks

- **Study**: Learn interrupt handling, NVIC configuration, and debouncing techniques.
- **Practice**: Write interrupt-driven code to handle external events.
- **Project**: Develop a button interrupt handler with debouncing to toggle an LED.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration), Arduino IDE.
- **Resources**: ARM NVIC documentation, embedded.com interrupt articles.

Detailed Notes

1. Interrupts in Microcontrollers

- **Definition**: Interrupts are hardware-triggered events that pause the main program to execute an Interrupt Service Routine (ISR).
- **Types**:
  - **External (EXTI)**: Triggered by GPIO pins (e.g., button press).
  - **Timer**: Triggered by timer events (e.g., overflow).
  - **Peripheral**: Triggered by UART, SPI, etc.
- **Key Components**:
  - **Interrupt Vector Table**: Maps interrupts to ISR addresses (e.g., in flash at 0x08000000 for STM32).
  - **Interrupt Flags**: Indicate interrupt occurrence (cleared in ISR).
  - **NVIC**: Manages interrupt priorities and enabling/disabling.
- **Embedded Tip**: Keep ISRs short; set flags for main code to handle complex logic to avoid blocking other interrupts.

2. Nested Vectored Interrupt Controller (NVIC)

- **Definition**: ARM Cortex-M's NVIC manages interrupt priorities, enabling/disabling, and vector table access.
- **Key Registers**:
  - **ISER**: Interrupt Set-Enable Register (enables interrupts).
  - **ICER**: Interrupt Clear-Enable Register (disables interrupts).
  - **IPR**: Interrupt Priority Register (sets priority levels).
  - **ISPR/ICPR**: Set/Clear Pending Registers.
- **Priority**: Lower priority number = higher priority (e.g., 0 is highest). STM32F1 supports 4-bit priorities (0-15).
- **Setup**:
  - Enable interrupt in NVIC (e.g., NVIC->ISER[0] |= (1U << IRQn)).

    - Configure peripheral (e.g., EXTI for GPIO interrupts).
    - Clear pending flags in ISR.
- **Embedded Tip**: Use NVIC to prioritize critical interrupts (e.g., timer over UART).

3. Debouncing

- **Definition**: Technique to filter out noise from mechanical switches (e.g., buttons) that cause multiple rapid transitions.
- **Methods**:
    - **Hardware**: Add capacitor/resistor to smooth signal.
    - **Software**: Ignore interrupts within a debounce period (e.g., 50ms).
- **Embedded Tip**: Use a timer for precise debouncing or poll in main loop to avoid ISR complexity.
- **Example** (Simplified debounce logic):

```
volatile uint32_t last_interrupt_time = 0;

void debounce_delay(uint32_t ms) {
    volatile uint32_t start = *TIM_CNT; // Timer counter
    while ((*TIM_CNT - start) < ms);
}

void EXTI0_IRQHandler(void) {
    if (*EXTI_PR & (1U << 0)) {
        if ((*TIM_CNT - last_interrupt_time) > 50) { // 50ms debounce
            // Handle interrupt
            last_interrupt_time = *TIM_CNT;
        }
        *EXTI_PR |= (1U << 0); // Clear flag
    }
}
```

4. Practice Programs

a. Simple EXTI Interrupt

- Configure an EXTI interrupt for a button press.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const EXTI_IMR = (volatile uint32_t *)0x40010400;
volatile uint32_t *const EXTI_PR = (volatile uint32_t *)0x40010414;
volatile uint32_t *const NVIC_ISER = (volatile uint32_t *)0xE000E100;
volatile uint8_t button_count = 0;

void init_exti(void) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 input
    *EXTI_IMR |= (1U << 0); // Enable EXTI0
    *NVIC_ISER |= (1U << 6); // Enable EXTI0 IRQ
}
void EXTI0_IRQHandler(void) {
    if (*EXTI_PR & (1U << 0)) {
        button_count++;
        printf("Button pressed: %u\n", button_count);
        *EXTI_PR |= (1U << 0); // Clear flag
    }
}
```

```
int main() {
    init_exti();
    *EXTI_PR |= (1U << 0); // Simulate interrupt
    EXTI0_IRQHandler();
    return 0;
}
```

- **Arduino Version**:

```
volatile uint8_t button_count = 0;

void setup() {
    attachInterrupt(digitalPinToInterrupt(2), isr, RISING);
    Serial.begin(9600);
}

void isr() {
    button_count++;
    Serial.println(button_count);
}

void loop() {}
```

b. Timer Interrupt

- Use a timer to trigger periodic interrupts.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const TIM_CR1 = (volatile uint32_t *)0x40000000;
volatile uint32_t *const TIM_SR = (volatile uint32_t *)0x40000010;
volatile uint32_t *const TIM_PSC = (volatile uint32_t *)0x40000028;
volatile uint32_t *const TIM_ARR = (volatile uint32_t *)0x4000002C;
volatile uint32_t *const NVIC_ISER = (volatile uint32_t *)0xE000E100;
volatile uint32_t tick_count = 0;

void init_timer(void) {
    *TIM_PSC = 15999; // 1kHz
    *TIM_ARR = 999; // 1ms period
    *TIM_CR1 = (1U << 0); // Enable timer
    *NVIC_ISER |= (1U << 28); // Enable TIM2 IRQ
}

void TIM2_IRQHandler(void) {
    if (*TIM_SR & (1U << 0)) {
        tick_count++;
        printf("Tick: %u\n", tick_count);
        *TIM_SR &= ~(1U << 0); // Clear update flag
    }
}

int main() {
    init_timer();
    *TIM_SR |= (1U << 0); // Simulate interrupt
    TIM2_IRQHandler();
    return 0;
}
```

Project: Button Interrupt Handler with Debounce

- **Objective**: Implement an interrupt-driven button handler with software debouncing to toggle an LED on each press.
- **Requirements**:
  - Configure PA0 as input for button (EXTI0 interrupt, rising edge).
  - Configure PA5 as output for LED.
  - Debounce button with a 50ms delay using a timer.
  - Toggle LED on each valid button press.
- **Code** (STM32 with HAL, simulated registers for PC):

```
// Simulated HAL-like code for PC

#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const EXTI_IMR = (volatile uint32_t *)0x40010400;
volatile uint32_t *const EXTI_RTSR = (volatile uint32_t *)0x40010408;
volatile uint32_t *const EXTI_PR = (volatile uint32_t *)0x40010414;
volatile uint32_t *const TIM_CNT = (volatile uint32_t *)0x40000024;
volatile uint32_t *const TIM_PSC = (volatile uint32_t *)0x40000028;
volatile uint32_t *const TIM_ARR = (volatile uint32_t *)0x4000002C;
volatile uint32_t *const TIM_CR1 = (volatile uint32_t *)0x40000000;
volatile uint32_t *const NVIC_ISER = (volatile uint32_t *)0xE000E100;


#define LED_PIN 5
#define BUTTON_PIN 0

volatile uint8_t led_state = 0;
volatile uint32_t last_interrupt_time = 0;

void init_gpio_and_exti(void) {
    *GPIO_MODER &= ~(3U << (BUTTON_PIN * 2)); // PA0 input
    *GPIO_MODER &= ~(3U << (LED_PIN * 2)); // PA5 output
    *GPIO_MODER |= (1U << (LED_PIN * 2));
    *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
    *EXTI_IMR |= (1U << BUTTON_PIN); // Enable EXTI0
    *EXTI_RTSR |= (1U << BUTTON_PIN); // Rising edge
    *NVIC_ISER |= (1U << 6); // Enable EXTI0 IRQ
}

void init_timer(void) {
    *TIM_PSC = 15999; // 1kHz (1ms tick)
    *TIM_ARR = 0xFFFFFFFF; // Max period
    *TIM_CR1 = (1U << 0); // Enable timer
}

void EXTI0_IRQHandler(void) {
    if (*EXTI_PR & (1U << BUTTON_PIN)) {
        uint32_t current_time = *TIM_CNT;
        if ((current_time - last_interrupt_time) > 50) { // 50ms debounce
            led_state = !led_state;
            *GPIO_BSRR = led_state ? (1U << LED_PIN) : (1U << (LED_PIN + 16));
            printf("LED: %s\n", led_state ? "ON" : "OFF");
            last_interrupt_time = current_time;
        }
        *EXTI_PR |= (1U << BUTTON_PIN); // Clear flag
    }
}
```

```
int main() {
    init_gpio_and_exti();
    init_timer();
    for (int i = 0; i < 3; i++) {
        *TIM_CNT += 100; // Simulate time passing
        *EXTI_PR |= (1U << BUTTON_PIN); // Simulate button press
        EXTI0_IRQHandler();
    }
    return 0;
}
```

- **STM32 HAL Version** (Real hardware, STM32CubeIDE):

```
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;

GPIO_InitTypeDef GPIO_InitStruct = {0};

EXTI_ConfigTypeDef EXTI_ConfigStruct = {0};

void init_gpio_and_exti(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(EXTI0_IRQn);
}

void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    TIM_HandleTypeDef htim2;
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz
    htim2.Init.Period = 0xFFFFFFFF;
    HAL_TIM_Base_Init(&htim2);
    HAL_TIM_Base_Start(&htim2);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    static uint32_t last_interrupt_time = 0;
    uint32_t current_time = __HAL_TIM_GET_COUNTER(&htim2);
    if ((current_time - last_interrupt_time) > 50) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        char msg[16];
```

```
        snprintf(msg, sizeof(msg), "LED: %s\r\n", HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) ? "ON" : "OFF");
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        last_interrupt_time = current_time;
    }
}

int main(void) {
    HAL_Init();
    init_gpio_and_exti();
    init_timer();
    init_uart();
    while (1) {
        // Main loop empty; handled by ISR
    }
}
```

- **Arduino Version**:

```
#define BUTTON_PIN 2
#define LED_PIN 13

volatile uint32_t last_interrupt_time = 0;

void setup() {
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), isr, RISING);
    Serial.begin(115200);
}

void isr() {
    unsigned long current_time = millis();
    if ((current_time - last_interrupt_time) > 50) {
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
        Serial.println(digitalRead(LED_PIN) ? "ON" : "OFF");
        last_interrupt_time = current_time;
    }
}

void loop() {}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure PA0 (EXTI0, rising edge), PA5 (output), TIM2 (base timer), USART1 (115200 baud).
  - Connect button to PA0 (with pull-up) and LED to PA5 (with resistor).
  - Test with a terminal (e.g., PuTTY) via USB-to-serial adapter.
- **Embedded Tip**: Use pull-up/pull-down resistors for buttons; adjust debounce time based on switch characteristics.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for EXTI and timer setup.
- **STM32CubeMX**: Configure GPIO, EXTI, TIM2, and UART.
- **Arduino IDE**: Download from https://www.arduino.cc/en/software for Arduino projects.
- **Debugging**: Use STM32CubeIDE's debugger or Arduino Serial Monitor; simulate with GCC (gcc program.c -o program).

Resources

- **ARM NVIC Documentation**: https://developer.arm.com for Cortex-M NVIC details.
- **embedded.com Interrupt Articles**: Search for “interrupts in embedded C” on https://www.embedded.com/.
- **Online**: STM32 reference manuals (e.g., RM0008), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC, Arduino code in Arduino IDE, or STM32 code in STM32CubeIDE.
- **Experiment**: Add UART to log button presses with timestamps.
- **Optimize**: Move debounce logic to main loop using a flag.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Use HAL and Direct Registers

Goal

Understand the differences between Hardware Abstraction Layer (HAL) and direct register programming, learning to combine both approaches for efficient microcontroller programming, with a focus on a UART echo application.

Tasks

- **Study**: Learn HAL vs. direct register programming, their advantages, and trade-offs.
- **Practice**: Write code using HAL for GPIO/UART and direct registers for specific tasks.
- **Project**: Develop a UART echo program combining HAL for initialization and direct register access for data handling.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration).
- **Resources**: STM32CubeMX documentation, STM32 HAL tutorials.

Detailed Notes

1. HAL vs. Direct Register Programming

- **Hardware Abstraction Layer (HAL)**:
  - **Definition**: A vendor-provided library (e.g., STM32 HAL) that abstracts hardware details, providing high-level functions for peripherals (e.g., GPIO, UART).
  - **Advantages**:
    - Portable across STM32 families.
    - Simplifies configuration (e.g., HAL_UART_Init).
    - Includes error handling and timeouts.
  - **Disadvantages**:
    - Larger code size and overhead.
    - Less control over low-level operations.
  - **Use Case**: Rapid prototyping, complex peripherals (e.g., USB, Ethernet).
- **Direct Register Programming**:
  - **Definition**: Directly manipulate peripheral registers (e.g., USART->DR) for control and data access.
  - **Advantages**:
    - Smaller, faster code.
    - Fine-grained control for optimization.
  - **Disadvantages**:
    - Error-prone, requires deep hardware knowledge.
    - Less portable across devices.
  - **Use Case**: Performance-critical tasks, custom register manipulation.
- **Embedded Tip**: Use HAL for initialization (e.g., clock, GPIO, UART setup) and direct registers for time-critical operations (e.g., data transfer in ISRs).

2. Combining HAL and Direct Registers

- **Approach**:
  - Use HAL for peripheral setup (e.g., HAL_UART_Init, HAL_GPIO_Init) to ensure correct configuration.
  - Use direct register access for data operations (e.g., USART->DR for UART transmit/receive) to optimize performance.
- **Key STM32 UART Registers**:
  - **SR**: Status Register (TXE: transmit empty, RXNE: receive not empty).
  - **DR**: Data Register (read/write data).
  - **BRR**: Baud Rate Register.
  - **CR1**: Control Register 1 (enable UART, TX, RX).
- **Embedded Tip**: Check HAL source code (e.g., stm32f1xx_hal_uart.c) to understand register interactions for hybrid programming.

3. Practice Programs

a. HAL-Based GPIO Toggle

- Use HAL to toggle an LED.
- **Code** (STM32 HAL):

```
#include "stm32f1xx_hal.h"

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

int main(void) {
    HAL_Init();
    init_gpio();
    while (1) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_Delay(500);
    }
}
```

b. Direct Register UART Transmit

- Transmit a string using direct register access (simulated).
- **Code**:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

void uart_transmit_char(uint8_t c) {
    while (!(*USART_SR & (1U << 7))); // Wait for TXE
    *USART_DR = c;
    printf("Sent: %c\n", c);
}
```

```
void uart_transmit_string(const char *str) {
    while (*str) uart_transmit_char(*str++);
}

int main() {
    uart_transmit_string("Hello, UART!\n");
    return 0;
}
```

Project: UART Echo with HAL and Direct Registers

- **Objective**: Implement a UART echo program using HAL for initialization (GPIO, UART) and direct register access for data handling to optimize performance.
- **Requirements**:
  - Configure UART1 (PA9-TX, PA10-RX) at 115200 baud using HAL.
  - Use direct register access to read/write UART data.
  - Echo received characters back to the sender.
  - Include error handling for UART status.
- **Code** (STM32 with HAL and direct registers):

```
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;

void init_gpio_and_uart(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    HAL_UART_Init(&huart1);
}

uint8_t uart_receive_direct(void) {
    if (huart1.Instance->SR & USART_SR_RXNE) { // Check RX not empty
        return (uint8_t)(huart1.Instance->DR); // Read data
    }
    return 0; // No data
}

void uart_transmit_direct(uint8_t data) {
    while (!(huart1.Instance->SR & USART_SR_TXE)); // Wait for TX empty
    huart1.Instance->DR = data; // Send data
}

int main(void) {
    HAL_Init();
    init_gpio_and_uart();
    while (1) {
        uint8_t rx_data = uart_receive_direct();
        if (rx_data) {
            if (huart1.Instance->SR & USART_SR_ORE) { // Check overrun error
                huart1.Instance->SR &= ~USART_SR_ORE; // Clear error
```

```
                uart_transmit_direct('E'); // Send error indicator
            } else {
                uart_transmit_direct(rx_data); // Echo back
            }
        }
    }
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_AFRH = (volatile uint32_t *)0x40010008;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const USART_BRR = (volatile uint32_t *)0x40013808;
volatile uint32_t *const USART_CR1 = (volatile uint32_t *)0x4001380C;

void init_gpio_and_uart(void) {
    *GPIO_MODER &= ~(3U << (9 * 2)); // PA9 (TX) alternate function
    *GPIO_MODER |= (2U << (9 * 2));
    *GPIO_MODER &= ~(3U << (10 * 2)); // PA10 (RX) alternate function
    *GPIO_MODER |= (2U << (10 * 2));
    *GPIO_AFRH &= ~(0xF << 4); // AF7 for PA9
    *GPIO_AFRH |= (7U << 4);
    *GPIO_AFRH &= ~(0xF << 8); // AF7 for PA10
    *GPIO_AFRH |= (7U << 8);
    *USART_BRR = 16000000 / 115200; // 16MHz clock
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2); // Enable UART, TX, RX
}

uint8_t uart_receive_direct(void) {
    static uint8_t sim_data[] = {'A', 'B', 'C', 0};
    static uint32_t idx = 0;
    if (sim_data[idx] == 0) return 0;
    *USART_SR |= (1U << 5); // Simulate RXNE
    *USART_DR = sim_data[idx++]; // Simulate received data
    if (*USART_SR & (1U << 5)) {
        return (uint8_t)(*USART_DR);
    }
    return 0;
}

void uart_transmit_direct(uint8_t data) {
    while (!(*USART_SR & (1U << 7))); // Wait for TXE
    *USART_DR = data;
    printf("Sent: %c\n", data);
}

int main() {
    init_gpio_and_uart();
    while (1) {
        uint8_t rx_data = uart_receive_direct();
        if (rx_data) {
            if (*USART_SR & (1U << 3)) { // Check overrun error
                *USART_SR &= ~(1U << 3); // Clear error
                uart_transmit_direct('E');
            } else {
                uart_transmit_direct(rx_data);
            }
        }
    }
    return 0;
}
```

- **Arduino Version** (For comparison):

```
void setup() {
    Serial.begin(115200);
}

void loop() {
    if (Serial.available()) {
        char c = Serial.read();
        Serial.write(c);
    }
}
```

- **Setup for STM32**:
    - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX, 115200 baud).
    - Generate code in STM32CubeIDE.
    - Connect USB-to-serial adapter to PA9/PA10; test with a terminal (e.g., PuTTY).
- **Optimizations**:
    - HAL for robust initialization (clock, GPIO, UART).
    - Direct register access for SR and DR to reduce overhead in data handling.
    - Error checking for overrun ensures reliable communication.
- **Embedded Tip**: In production, add interrupt-driven UART handling for non-blocking operation (covered on Day 13).

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for UART setup.
- **STM32CubeMX**: Configure USART1 and GPIO pins.
- **Debugging**: Use STM32CubeIDE's debugger to inspect USART->SR; simulate with GCC (gcc program.c -o program).

Resources

- **STM32CubeMX Documentation**: https://www.st.com for configuration details.
- **STM32 HAL Tutorials**: Included in STM32CubeIDE or search "STM32 HAL UART" on https://www.st.com.
- **Online**: STM32 reference manuals (e.g., RM0008), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a GPIO LED that toggles on each UART receive.
- **Optimize**: Use direct register access for status checks only when needed.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Consolidate Week 3 Concepts

Goal

Consolidate Week 3 concepts (GPIO, UART, timers, interrupts) by reviewing notes, debugging prior projects, and optimizing the PWM LED dimmer project for efficiency using interrupt-driven techniques.

Tasks

- **Study**: Review GPIO setup, UART configuration, timer/PWM, and interrupt/NVIC concepts.
- **Practice**: Debug issues in prior projects (e.g., UART echo, button interrupt).
- **Project**: Optimize the Day 12 PWM LED dimmer to use interrupts for non-blocking fading.
- **Tools**: STM32CubeIDE, GDB (debugger), STM32CubeMX.
- **Resources**: STM32 reference manuals, GDB tutorials.

Detailed Notes

1. Review of Week 3 Concepts

- **GPIO (Day 11)**:
  - Configure pins as input/output/alternate function using MODER, ODR, BSRR, AFR.
  - Example: GPIOA->MODER |= (1U << (5 * 2)) for PA5 output.
  - Key: Use BSRR for atomic updates; enable clocks in RCC.
- **UART (Day 11, Day 14)**:
  - Configure baud rate, data format; use HAL for setup, direct registers for performance.
  - Example: USART1->DR for data; check SR for TXE/RXNE.
  - Key: Handle errors (e.g., overrun) in production code.
- **Timers and PWM (Day 12)**:
  - Timers count clock cycles; configure PSC, ARR for frequency.
  - PWM controls duty cycle via CCRx; map to GPIO via alternate function.
  - Formula: f_pwm = f_clock / ((PSC + 1) * (ARR + 1)).
- **Interrupts and NVIC (Day 13)**:
  - NVIC manages priorities; ISRs must clear flags.
  - Example: NVIC->ISER[0] |= (1U << 6) for EXTI0.
  - Key: Use debouncing for buttons; keep ISRs short.
- **HAL vs. Direct Registers (Day 14)**:
  - HAL simplifies setup; direct registers optimize performance.
  - Example: HAL_UART_Init for setup, USART->DR for data.

2. Debugging with GDB

- **Purpose**: Identify issues like incorrect register writes, interrupt failures, or timing errors.
- **Key Commands**:
  - gdb ./program: Start debugger.
  - break HAL_TIM_PeriodElapsedCallback: Set breakpoint in ISR.

    - run: Execute until breakpoint.
    - print huart1.Instance->SR: Inspect UART status.
    - next/step: Step through code.
    - info registers: View CPU state.
- **Compilation**: Use gcc -g program.c -o program for debug symbols.
- **Embedded Tip**: Use STM32CubeIDE's debugger for hardware breakpoints and peripheral views.

3. Debugging Prior Projects

a. UART Echo (Day 11, Day 14)

- **Potential Bug**: Missing overrun error handling causes data loss.
- **Fix**: Check and clear USART_SR_ORE before reading DR.
- **Revised Snippet**:

```
uint8_t uart_receive_direct(void) {
    if (huart1.Instance->SR & USART_SR_ORE) {
        huart1.Instance->SR &= ~USART_SR_ORE; // Clear overrun
        return 0;
    }
    if (huart1.Instance->SR & USART_SR_RXNE) {
        return (uint8_t)(huart1.Instance->DR);
    }
    return 0;
}
```

b. Button Interrupt with Debounce (Day 13)

- **Potential Bug**: Debounce period too short, causing multiple triggers.
- **Fix**: Increase debounce time to 100ms; use timer for accuracy.
- **Revised Snippet**:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    static uint32_t last_interrupt_time = 0;
    uint32_t current_time = __HAL_TIM_GET_COUNTER(&htim2);
    if ((current_time - last_interrupt_time) > 100) { // 100ms
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        last_interrupt_time = current_time;
    }
}
```

c. PWM LED Dimmer (Day 12)

- **Potential Bug**: Blocking HAL_Delay prevents other tasks.
- **Fix**: Use timer interrupts for non-blocking fading (implemented in project).

4. Project: Optimize PWM LED Dimmer for Efficiency

- **Objective**: Optimize the Day 12 PWM LED dimmer to use a timer interrupt for non-blocking duty cycle updates, reducing CPU load and improving responsiveness.
- **Optimizations**:
    - Replace HAL_Delay with timer interrupts for fading.
    - Use HAL for timer/PWM setup, direct registers for duty cycle updates.
    - Add UART to log duty cycle changes.

- **Requirements**:
  - Configure TIM2 for PWM on PA0 (1kHz, Channel 1).
  - Use TIM3 for 500ms interrupts to update duty cycle.
  - Fade LED from 0% to 100% and back, logging via UART.
- **Code** (STM32 with HAL and direct registers):

```
#include "stm32f1xx_hal.h"

TIM_HandleTypeDef htim2; // PWM timer
TIM_HandleTypeDef htim3; // Update timer

UART_HandleTypeDef huart1;

volatile uint8_t duty_percent = 0;
volatile int8_t direction = 10; // Increment/decrement step

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

void init_pwm(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz base
    htim2.Init.Period = 999; // 1kHz PWM
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    HAL_TIM_PWM_Init(&htim2);
    TIM_OC_InitTypeDef sConfigOC = {0};
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0;
    HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
}

void init_timer(void) {
    __HAL_RCC_TIM3_CLK_ENABLE();
    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 15999; // 1kHz
    htim3.Init.Period = 499; // 500ms
    HAL_TIM_Base_Init(&htim3);
    HAL_NVIC_SetPriority(TIM3_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM3_IRQn);
    HAL_TIM_Base_Start_IT(&htim3);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}
```

```
void set_pwm_duty(uint8_t duty) {
    htim2.Instance->CCR1 = (duty * (htim2.Init.Period + 1)) / 100; // Direct register
    char msg[32];
    snprintf(msg, sizeof(msg), "Duty: %u%%\r\n", duty);
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        duty_percent += direction;
        if (duty_percent >= 100) {
            duty_percent = 100;
            direction = -10;
        } else if (duty_percent <= 0) {
            duty_percent = 0;
            direction = 10;
        }
        set_pwm_duty(duty_percent);
    }
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_pwm();
    init_timer();
    init_uart();
    set_pwm_duty(0); // Start at 0%
    while (1) {
        // Main loop empty; handled by TIM3 ISR
    }
}

void TIM3_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim3);
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_AFR = (volatile uint32_t *)0x40010008;
volatile uint32_t *const TIM2_PSC = (volatile uint32_t *)0x40000028;
volatile uint32_t *const TIM2_ARR = (volatile uint32_t *)0x4000002C;
volatile uint32_t *const TIM2_CCR1 = (volatile uint32_t *)0x40000034;
volatile uint32_t *const TIM2_CCMR1 = (volatile uint32_t *)0x40000018;
volatile uint32_t *const TIM2_CR1 = (volatile uint32_t *)0x40000000;
volatile uint32_t *const TIM3_SR = (volatile uint32_t *)0x40000410;
volatile uint32_t *const TIM3_PSC = (volatile uint32_t *)0x40000428;
volatile uint32_t *const TIM3_ARR = (volatile uint32_t *)0x4000042C;
volatile uint32_t *const TIM3_CR1 = (volatile uint32_t *)0x40000400;
volatile uint8_t duty_percent = 0;
volatile int8_t direction = 10;

void init_pwm(void) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 alternate function
    *GPIO_MODER |= (2U << (0 * 2));
    *GPIO_AFR &= ~(0xF << (0 * 4)); // AF2 for TIM2
    *GPIO_AFR |= (2U << (0 * 4));
    *TIM2_PSC = 15999; // 1kHz
    *TIM2_ARR = 999; // 1kHz PWM
    *TIM2_CCMR1 = (6U << 4) | (1U << 3); // PWM mode 1, preload
    *TIM2_CR1 = (1U << 0);
}
```

```
void init_timer(void) {
    *TIM3_PSC = 15999; // 1kHz
    *TIM3_ARR = 499; // 500ms
    *TIM3_CR1 = (1U << 0);
}

void set_pwm_duty(uint8_t duty) {
    *TIM2_CCR1 = (duty * (*TIM2_ARR + 1)) / 100;
    printf("Duty: %u%%\n", duty);
}

void TIM3_IRQHandler(void) {
    if (*TIM3_SR & (1U << 0)) {
        duty_percent += direction;
        if (duty_percent >= 100) {
            duty_percent = 100;
            direction = -10;
        } else if (duty_percent <= 0) {
            duty_percent = 0;
            direction = 10;
        }
        set_pwm_duty(duty_percent);
        *TIM3_SR &= ~(1U << 0); // Clear flag
    }
}

int main() {
    init_pwm();
    init_timer();
    set_pwm_duty(0);
    for (int i = 0; i < 4; i++) {
        *TIM3_SR |= (1U << 0); // Simulate interrupt
        TIM3_IRQHandler();
    }
    return 0;
}
```

- **Arduino Version** (For comparison):

```
#define LED_PIN 9

volatile uint8_t duty_percent = 0;
volatile int8_t direction = 25;

void setup() {
    pinMode(LED_PIN, OUTPUT);
    Serial.begin(115200);
    Timer1.initialize(500000); // 500ms
    Timer1.attachInterrupt(update_pwm);
}
void update_pwm() {
    duty_percent += direction;
    if (duty_percent >= 255) {
        duty_percent = 255;
        direction = -25;
    } else if (duty_percent <= 0) {
        duty_percent = 0;
        direction = 25;
    }
    analogWrite(LED_PIN, duty_percent);
    Serial.print("Duty: ");
    Serial.print(duty_percent * 100 / 255);
    Serial.println("%");
}
void loop() {}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure TIM2 (PWM, PA0, 1kHz), TIM3 (500ms interrupts), USART1 (115200 baud).
  - Connect LED to PA0 (with resistor); use USB-to-serial adapter for UART.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY).
- **Optimizations**:
  - Non-blocking fading via TIM3 interrupts.
  - Direct CCR1 writes for duty cycle updates.
  - UART logging for debugging without blocking.
- **Embedded Tip**: Ensure NVIC priorities are set correctly to avoid conflicts between UART and timer interrupts.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for configuration.
- **STM32CubeMX**: Configure TIM2, TIM3, and USART1.
- **GDB**: Debug with gdb ./program for simulation; use STM32CubeIDE's debugger for hardware.
- **Debugging**: Inspect TIM2->CCR1 and TIM3->SR in STM32CubeIDE's peripheral view.

Resources

- **STM32 Reference Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **GDB Tutorials**: Search "GDB embedded debugging" (e.g., https://www.tutorialspoint.com/gdb/).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Review**: Spend ~1 hour rereading Days 11-14 notes and code.
- **Debug**: Use GDB or STM32CubeIDE to fix one bug (e.g., incorrect duty cycle).
- **Optimize**: Test with different fade steps (e.g., 5% vs. 10%).
- **Time Management**: Spend ~1 hour on review, ~1 hour on debugging, ~1-1.5 hours on the project.

Chapter 4: Intermediate Peripheral Work

- **Master SPI and I2C drivers**
- **Implement FSM for task management**
- **Learn ADC and sensor handling**
- **Use watchdog timer**
- **Consolidate Week 4 concepts**

Master SPI and I2C Drivers

Goal

Master SPI and I2C protocols, focusing on polling-based driver implementation for microcontroller communication, with a practical application of reading data from a temperature sensor via I2C.

Tasks

- **Study**: Learn SPI and I2C protocols, their configurations, and polling vs. interrupt-driven approaches.
- **Practice**: Write polling-based drivers for SPI and I2C.
- **Project**: Develop a polling-based I2C driver to read temperature from a sensor (e.g., LM75A).
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration).
- **Resources**: STM32 SPI/I2C manuals, I2Cdevlib.

Detailed Notes

1. SPI Protocol

- **Definition**: Serial Peripheral Interface (SPI) is a synchronous, full-duplex protocol for high-speed communication between a master (e.g., microcontroller) and peripherals (e.g., sensors, displays).
- **Key Features**:
  - **Roles**: Master (initiates communication) and slave.
  - **Signals**: SCLK (clock), MOSI (master out, slave in), MISO (master in, slave out), SS/CS (slave select/chip select).
  - **Modes**: 4 modes based on clock polarity (CPOL) and phase (CPHA).
  - **Speed**: Typically 1-10 MHz on STM32.
- **STM32 SPI Registers**:
  - **CR1**: Control register (enable SPI, set mode, baud rate).
  - **DR**: Data register (send/receive data).
  - **SR**: Status register (TXE: transmit empty, RXNE: receive not empty).
- **Embedded Tip**: Match SPI mode (CPOL/CPHA) with peripheral datasheet; use NSS pin for chip select control.
- **Example** (Simplified SPI transmit):

```c
#define SPI1 ((SPI_TypeDef *)0x40013000)

typedef struct {
    volatile uint32_t CR1, SR, DR;
} SPI_TypeDef;

void init_spi(void) {
    volatile uint32_t *RCC_APB2ENR = (volatile uint32_t *)0x40021018;
    *RCC_APB2ENR |= (1U << 12); // Enable SPI1 clock
    SPI1->CR1 = (1U << 6) | (0U << 3); // Enable SPI, master mode
}
```

```
void spi_transmit(uint8_t data) {
    while (!(SPI1->SR & (1U << 1))); // Wait for TXE
    SPI1->DR = data;
    while (SPI1->SR & (1U << 7)); // Wait for BSY clear
}
```

2. I2C Protocol

- **Definition**: Inter-Integrated Circuit (I2C) is a two-wire, half-duplex protocol for communication with multiple devices using a single bus.
- **Key Features**:
  - **Roles**: Master (controls bus) and slave (responds to address).
  - **Signals**: SDA (data), SCL (clock).
  - **Addressing**: 7-bit or 10-bit addresses; supports multiple slaves.
  - **Speed**: Standard (100 kHz), Fast (400 kHz), or higher on STM32.
- **STM32 I2C Registers**:
  - **CR1**: Control register (enable I2C).
  - **CR2**: Frequency and interrupt control.
  - **DR**: Data register.
  - **SR1/SR2**: Status registers (e.g., TXE, RXNE, ADDR).
- **Embedded Tip**: Use pull-up resistors (e.g., 4.7kΩ) on SDA/SCL; check device address in datasheet.

3. Polling vs. Interrupt-Driven

- **Polling**:
  - Continuously check status registers (e.g., SR for TXE/RXNE).
  - Simple but CPU-intensive; suitable for small data transfers.
- **Interrupt-Driven**:
  - Use ISRs to handle data transfer events.
  - Non-blocking, efficient for large data; more complex setup.
- **Embedded Tip**: Start with polling for simplicity; switch to interrupts for efficiency in production.

4. Practice Programs

a. SPI Polling Transmit

- Send a byte via SPI to a simulated peripheral.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_AFR = (volatile uint32_t *)0x40010008;
volatile uint32_t *const SPI1_CR1 = (volatile uint32_t *)0x40013000;
volatile uint32_t *const SPI1_SR = (volatile uint32_t *)0x40013008;
volatile uint32_t *const SPI1_DR = (volatile uint32_t *)0x4001300C;

void init_spi(void) {
    *GPIO_MODER &= ~(3U << (5 * 2)); // PA5 (SCLK) alternate function
    *GPIO_MODER |= (2U << (5 * 2));
    *GPIO_MODER &= ~(3U << (7 * 2)); // PA7 (MOSI)
    *GPIO_MODER |= (2U << (7 * 2));
    *GPIO_AFR &= ~(0xF << (5 * 4)); // AF5 for SPI1
```

```
    *GPIO_AFR |= (5U << (5 * 4));
    *GPIO_AFR &= ~(0xF << (7 * 4));
    *GPIO_AFR |= (5U << (7 * 4));
    *SPI1_CR1 = (1U << 6) | (0U << 3); // Enable SPI, master
}

void spi_transmit(uint8_t data) {
    while (!(*SPI1_SR & (1U << 1))); // Wait for TXE
    *SPI1_DR = data;
    while (*SPI1_SR & (1U << 7)); // Wait for BSY clear
    printf("SPI Sent: 0x%02X\n", data);
}

int main() {
    init_spi();
    spi_transmit(0x55);
    return 0;
}
```

b. I2C Polling Write

- Write a byte to an I2C slave address.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const I2C1_CR1 = (volatile uint32_t *)0x40005400;
volatile uint32_t *const I2C1_CR2 = (volatile uint32_t *)0x40005404;
volatile uint32_t *const I2C1_SR1 = (volatile uint32_t *)0x40005414;
volatile uint32_t *const I2C1_DR = (volatile uint32_t *)0x40005410;

void init_i2c(void) {
    *GPIO_MODER &= ~(3U << (6 * 2)); // PB6 (SCL) alternate function
    *GPIO_MODER |= (2U << (6 * 2));
    *GPIO_MODER &= ~(3U << (7 * 2)); // PB7 (SDA)
    *GPIO_MODER |= (2U << (7 * 2));
    *I2C1_CR2 = 16; // 16MHz peripheral clock
    *I2C1_CR1 = (1U << 0); // Enable I2C
}

void i2c_write(uint8_t addr, uint8_t data) {
    *I2C1_CR1 |= (1U << 8); // Start
    while (!(*I2C1_SR1 & (1U << 0))); // Wait for SB
    *I2C1_DR = addr << 1; // Send 7-bit address + write (0)
    while (!(*I2C1_SR1 & (1U << 1))); // Wait for ADDR
    (void)*I2C1_SR1; (void)*I2C1_SR2; // Clear ADDR
    *I2C1_DR = data; // Send data
    while (!(*I2C1_SR1 & (1U << 7))); // Wait for TXE
    *I2C1_CR1 |= (1U << 9); // Stop
    printf("I2C Wrote: 0x%02X to addr 0x%02X\n", data, addr);
}

int main() {
    init_i2c();
    i2c_write(0x48, 0x01); // LM75A address
    return 0;
}
```

Project: Temperature Sensor on I2C

- **Objective**: Implement a polling-based I2C driver to read temperature from an LM75A sensor (or similar) using STM32 HAL for initialization and direct registers for data transfer.
- **Requirements**:
  - Configure I2C1 (PB6-SCL, PB7-SDA) for 100kHz.
  - Read temperature from LM75A (address 0x48) via I2C.
  - Log temperature via UART (PA9-TX, 115200 baud).
- **Code** (STM32 with HAL and direct registers):

```
#include "stm32f1xx_hal.h"

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;

void init_i2c(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_I2C1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_6 | GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_OD; // Open-drain for I2C
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    HAL_I2C_Init(&hi2c1);
}

void init_uart(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

uint16_t i2c_read_temp(uint8_t addr) {
    // Write register address (0x00 for LM75A temperature)
    hi2c1.Instance->CR1 |= I2C_CR1_START;
    while (!(hi2c1.Instance->SR1 & I2C_SR1_SB));
    hi2c1.Instance->DR = (addr << 1); // Write address
    while (!(hi2c1.Instance->SR1 & I2C_SR1_ADDR));
    (void)hi2c1.Instance->SR1; (void)hi2c1.Instance->SR2;
    hi2c1.Instance->DR = 0x00; // Temperature register
    while (!(hi2c1.Instance->SR1 & I2C_SR1_TXE));
    hi2c1.Instance->CR1 |= I2C_CR1_STOP;
    // Read 2 bytes
    hi2c1.Instance->CR1 |= I2C_CR1_START;
    while (!(hi2c1.Instance->SR1 & I2C_SR1_SB));
    hi2c1.Instance->DR = (addr << 1) | 1; // Read address
    while (!(hi2c1.Instance->SR1 & I2C_SR1_ADDR));
    (void)hi2c1.Instance->SR1; (void)hi2c1.Instance->SR2;
```

```
    hi2c1.Instance->CR1 |= I2C_CR1_ACK;
    while (!(hi2c1.Instance->SR1 & I2C_SR1_RXNE));
    uint8_t msb = hi2c1.Instance->DR;
    hi2c1.Instance->CR1 &= ~I2C_CR1_ACK;
    hi2c1.Instance->CR1 |= I2C_CR1_STOP;
    while (!(hi2c1.Instance->SR1 & I2C_SR1_RXNE));
    uint8_t lsb = hi2c1.Instance->DR;
    return (msb << 8) | lsb; // 16-bit temperature
}

int main(void) {
    HAL_Init();
    init_i2c();
    init_uart();
    while (1) {
        uint16_t raw_temp = i2c_read_temp(0x48); // LM75A address
        int16_t temp_c = (raw_temp >> 7) * 0.5; // LM75A: 9-bit, 0.5°C resolution
        char msg[32];
        snprintf(msg, sizeof(msg), "Temp: %d C\r\n", temp_c);
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        HAL_Delay(1000);
    }
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const I2C1_CR1 = (volatile uint32_t *)0x40005400;
volatile uint32_t *const I2C1_CR2 = (volatile uint32_t *)0x40005404;
volatile uint32_t *const I2C1_SR1 = (volatile uint32_t *)0x40005414;
volatile uint32_t *const I2C1_SR2 = (volatile uint32_t *)0x40005418;
volatile uint32_t *const I2C1_DR = (volatile uint32_t *)0x40005410;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

void init_i2c(void) {
    *GPIO_MODER &= ~(3U << (6 * 2)); // PB6 (SCL) alternate function
    *GPIO_MODER |= (2U << (6 * 2));
    *GPIO_MODER &= ~(3U << (7 * 2)); // PB7 (SDA)
    *GPIO_MODER |= (2U << (7 * 2));
    *I2C1_CR2 = 16; // 16MHz clock
    *I2C1_CR1 = (1U << 0); // Enable I2C
}

void init_uart(void) {
    *USART_BRR = 16000000 / 115200;
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2);
}

uint16_t i2c_read_temp(uint8_t addr) {
    static uint8_t sim_data[] = {0x19, 0x00}; // Simulate 25°C (0x1900)
    static uint32_t idx = 0;
    *I2C1_CR1 |= (1U << 8); // Start
    while (!(*I2C1_SR1 & (1U << 0)));
    *I2C1_DR = addr << 1;
    while (!(*I2C1_SR1 & (1U << 1)));
    (void)*I2C1_SR1; (void)*I2C1_SR2;
    *I2C1_DR = 0x00;
    while (!(*I2C1_SR1 & (1U << 7)));
    *I2C1_CR1 |= (1U << 9); // Stop
    *I2C1_CR1 |= (1U << 8); // Start
    while (!(*I2C1_SR1 & (1U << 0)));
    *I2C1_DR = (addr << 1) | 1;
```

```
    while (!(*I2C1_SR1 & (1U << 1)));
    (void)*I2C1_SR1; (void)*I2C1_SR2;
    *I2C1_CR1 |= (1U << 10); // ACK
    while (!(*I2C1_SR1 & (1U << 6)));
    *I2C1_DR = sim_data[idx++ % 2]; // Simulate data
    uint8_t msb = *I2C1_DR;
    *I2C1_CR1 &= ~(1U << 10);
    *I2C1_CR1 |= (1U << 9);
    while (!(*I2C1_SR1 & (1U << 6)));
    uint8_t lsb = *I2C1_DR;
    return (msb << 8) | lsb;
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}

int main() {
    init_i2c();
    init_uart();
    while (1) {
        uint16_t raw_temp = i2c_read_temp(0x48);
        int16_t temp_c = (raw_temp >> 7) * 0.5;
        char msg[32];
        snprintf(msg, sizeof(msg), "Temp: %d C\n", temp_c);
        uart_transmit_string(msg);
        for (volatile uint32_t i = 0; i < 1000000; i++); // Simulate delay
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure I2C1 (PB6-SCL, PB7-SDA, 100kHz), USART1 (PA9-TX, 115200 baud).
  - Connect LM75A (address 0x48) to PB6/PB7 with 4.7kΩ pull-up resistors.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.
- **Optimizations**:
  - HAL for robust I2C/UART initialization.
  - Direct register access for I2C data transfer to reduce overhead.
  - Error handling for I2C bus errors (e.g., NACK) can be added.
- **Embedded Tip**: Verify LM75A address and register map in datasheet; use logic analyzer for I2C debugging.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for I2C/UART setup.
- **STM32CubeMX**: Configure I2C1 and USART1.
- **Debugging**: Use STM32CubeIDE's debugger to inspect I2C1->SR1; simulate with GCC (gcc program.c -o program).

Resources

- **STM32 SPI/I2C Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **I2Cdevlib**: https://github.com/jrowberg/i2cdevlib for reference drivers.

- **Online**: C reference (https://en.cppreference.com/w/c), STM32 HAL docs (https://www.st.com).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add error handling for I2C NACK or bus errors.
- **Optimize**: Buffer multiple temperature readings before UART transmission.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Implement FSM for Task Management

Goal

Understand Finite State Machines (FSMs) and implement them in embedded C to manage tasks effectively, with a focus on a traffic light control system using FSM principles.

Tasks

- **Study**: Learn FSM concepts, state transitions, and their use in embedded systems.
- **Practice**: Write FSM-based code to manage sequential tasks.
- **Project**: Develop a traffic light FSM to control LEDs representing red, yellow, and green states.
- **Tools**: STM32CubeIDE.
- **Resources**: embedded.com FSM articles, YouTube FSM tutorials.

Detailed Notes

1. Finite State Machines (FSMs)

- **Definition**: An FSM is a computational model with a finite number of states, where the system transitions between states based on events or conditions, ideal for managing tasks in embedded systems.
- **Components**:
  - **States**: Distinct modes (e.g., Red, Green, Yellow for a traffic light).
  - **Events**: Triggers for transitions (e.g., timer expiration, button press).
  - **Transitions**: Rules defining how to move between states.
  - **Actions**: Tasks performed in each state (e.g., turn on LED).
- **Implementation**:
  - Use an enum for states and a switch statement for state handling.
  - Store current state in a variable; update based on events.
  - Use timers for state duration in embedded systems.
- **Embedded Tip**: Keep FSM code modular; use a state transition table for complex systems to improve maintainability.

2. FSM Design for Embedded Systems

- **Steps**:
  1. Define states (e.g., RED, GREEN, YELLOW).
  2. Identify events (e.g., timer expires after 5 seconds).
  3. Specify transitions (e.g., RED → GREEN after 5s).
  4. Define actions (e.g., set GPIO pins for LEDs).
- **Advantages**:
  - Predictable behavior for sequential tasks.

- Easy to debug and extend.
- Reduces spaghetti code in embedded applications.

- **Embedded Tip**: Use a timer peripheral for precise state timing; avoid blocking delays in FSM loops.

3. Practice Program: Simple FSM

- **Objective**: Implement a basic FSM to toggle between two states (ON, OFF) for an LED.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;

#define LED_PIN 5

typedef enum { STATE_OFF, STATE_ON } State_t;

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= (1U << (LED_PIN * 2)); // PA5 output
    *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
}

void delay_ms(uint32_t ms) {
    for (volatile uint32_t i = 0; i < ms * 16000; i++); // Simulated delay
}

int main() {
    init_gpio();
    State_t state = STATE_OFF;
    while (1) {
        switch (state) {
            case STATE_OFF:
                *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
                printf("LED OFF\n");
                delay_ms(1000);
                state = STATE_ON;
                break;
            case STATE_ON:
                *GPIO_BSRR = (1U << LED_PIN); // LED on
                printf("LED ON\n");
                delay_ms(1000);
                state = STATE_OFF;
                break;
        }
    }
    return 0;
}
```

Project: Traffic Light FSM

- **Objective**: Implement an FSM to control a traffic light system with three LEDs (Red, Green, Yellow) using STM32, with state transitions driven by a timer.
- **Requirements**:
  - Configure PA5 (Red), PA6 (Green), PA7 (Yellow) as outputs.
  - Use TIM2 for timing state durations (e.g., Red: 5s, Green: 4s, Yellow: 1s).
  - Log state changes via UART (PA9-TX, 115200 baud).

- **Code** (STM32 with HAL):

```c
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;
TIM_HandleTypeDef htim2;

typedef enum { RED, GREEN, YELLOW } State_t;

volatile State_t current_state = RED;
volatile uint32_t state_timer = 0;

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // Red on
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_RESET);
}

void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz (16MHz clock)
    htim2.Init.Period = 999; // 1s period
    HAL_TIM_Base_Init(&htim2);
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

void log_state(State_t state) {
    char msg[32];
    const char *state_str = (state == RED) ? "Red" : (state == GREEN) ? "Green" : "Yellow";
    snprintf(msg, sizeof(msg), "State: %s\r\n", state_str);
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
}

void update_leds(State_t state) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_RESET);
    switch (state) {
        case RED: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); break;
        case GREEN: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); break;
        case YELLOW: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET); break;
    }
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        state_timer++;
        switch (current_state) {
            case RED:
                if (state_timer >= 5) { // 5s
                    current_state = GREEN;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
            case GREEN:
                if (state_timer >= 4) { // 4s
                    current_state = YELLOW;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
            case YELLOW:
                if (state_timer >= 1) { // 1s
                    current_state = RED;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
        }
    }
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_timer();
    init_uart();
    log_state(current_state);
    while (1) {
        // Main loop empty; FSM handled by TIM2 ISR
    }
}

void TIM2_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim2);
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const TIM2_SR = (volatile uint32_t *)0x40000010;
volatile uint32_t *const TIM2_PSC = (volatile uint32_t *)0x40000028;
volatile uint32_t *const TIM2_ARR = (volatile uint32_t *)0x4000002C;
volatile uint32_t *const TIM2_CR1 = (volatile uint32_t *)0x40000000;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

typedef enum { RED, GREEN, YELLOW } State_t;

volatile State_t current_state = RED;
volatile uint32_t state_timer = 0;
```

```c
void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (5 * 2)); // PA5, PA6, PA7 output
    *GPIO_MODER |= (1U << (5 * 2)) | (1U << (6 * 2)) | (1U << (7 * 2));
    *GPIO_BSRR = (1U << 5); // Red on
    *GPIO_BSRR = (1U << (6 + 16)) | (1U << (7 + 16)); // Green, Yellow off
}

void init_timer(void) {
    *TIM2_PSC = 15999; // 1kHz
    *TIM2_ARR = 999; // 1s
    *TIM2_CR1 = (1U << 0);
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}

void update_leds(State_t state) {
    *GPIO_BSRR = (1U << (5 + 16)) | (1U << (6 + 16)) | (1U << (7 + 16)); // All off
    switch (state) {
        case RED: *GPIO_BSRR = (1U << 5); break;
        case GREEN: *GPIO_BSRR = (1U << 6); break;
        case YELLOW: *GPIO_BSRR = (1U << 7); break;
    }
}

void log_state(State_t state) {
    const char *state_str = (state == RED) ? "Red" : (state == GREEN) ? "Green" : "Yellow";
    char msg[32];
    snprintf(msg, sizeof(msg), "State: %s\n", state_str);
    uart_transmit_string(msg);
}

void TIM2_IRQHandler(void) {
    if (*TIM2_SR & (1U << 0)) {
        state_timer++;
        switch (current_state) {
            case RED:
                if (state_timer >= 5) {
                    current_state = GREEN;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
            case GREEN:
                if (state_timer >= 4) {
                    current_state = YELLOW;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
            case YELLOW:
                if (state_timer >= 1) {
                    current_state = RED;
                    state_timer = 0;
                    update_leds(current_state);
                    log_state(current_state);
                }
                break;
        }
        *TIM2_SR &= ~(1U << 0); // Clear flag
    }
}
```

```
int main() {
    init_gpio();
    init_timer();
    log_state(current_state);
    for (int i = 0; i < 10; i++) {
        *TIM2_SR |= (1U << 0); // Simulate interrupt
        TIM2_IRQHandler();
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure PA5, PA6, PA7 (outputs), TIM2 (1s interrupts), USART1 (PA9-TX, 115200 baud).
  - Connect LEDs (with resistors) to PA5 (Red), PA6 (Green), PA7 (Yellow).
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.
- **Optimizations**:
  - Timer-driven transitions for precise timing.
  - Non-blocking FSM in ISR reduces main loop load.
  - UART logging for debugging.
- **Embedded Tip**: Use a state transition table (array) for more complex FSMs to simplify maintenance.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for GPIO, timer, and UART setup.
- **STM32CubeMX**: Configure GPIO, TIM2, and USART1.
- **Debugging**: Use STM32CubeIDE's debugger to inspect current_state and TIM2->SR.

Resources

- **embedded.com FSM Articles**: Search "finite state machines embedded" on https://www.embedded.com/.
- **YouTube FSM Tutorials**: Search "FSM embedded C" (e.g., Neso Academy, STMicroelectronics).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a button to manually advance states.
- **Optimize**: Use a state transition table for scalability.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Learn ADC and Sensor Handling

Goal

Master Analog-to-Digital Converter (ADC) configuration and sensor handling on a microcontroller, learning to read analog sensor data (e.g., temperature from an LM35 sensor) and process it for embedded applications.

Tasks

- **Study**: Learn ADC configuration, modes, and sensor interfacing principles.
- **Practice**: Write code to configure ADC and read sensor data.
- **Project**: Develop an ADC-based temperature sensor using an LM35, logging data via UART.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration).
- **Resources**: STM32 ADC manuals, LM35 datasheet.

Detailed Notes

1. Analog-to-Digital Converter (ADC)

- **Definition**: Converts analog signals (e.g., voltage from a sensor) to digital values for microcontroller processing.
- **Key Features (STM32)**:
  - **Resolution**: Typically 12-bit (0-4095 range).
  - **Channels**: Multiple input pins (e.g., PA0, PA1).
  - **Modes**: Single conversion, continuous, scan.
  - **Reference Voltage**: VREF (e.g., 3.3V on STM32).
  - **Conversion Time**: Depends on clock and sampling time.
- **Key Registers**:
  - **CR1/CR2**: Control registers (enable ADC, start conversion).
  - **SMPR**: Sampling time register.
  - **DR**: Data register (stores converted value).
  - **SQR**: Sequence register (channel selection).
- **Formula**: For a 12-bit ADC with VREF = 3.3V, Voltage = (ADC_Value / 4095) * 3.3.
- **Embedded Tip**: Calibrate ADC for accuracy; use averaging to reduce noise.

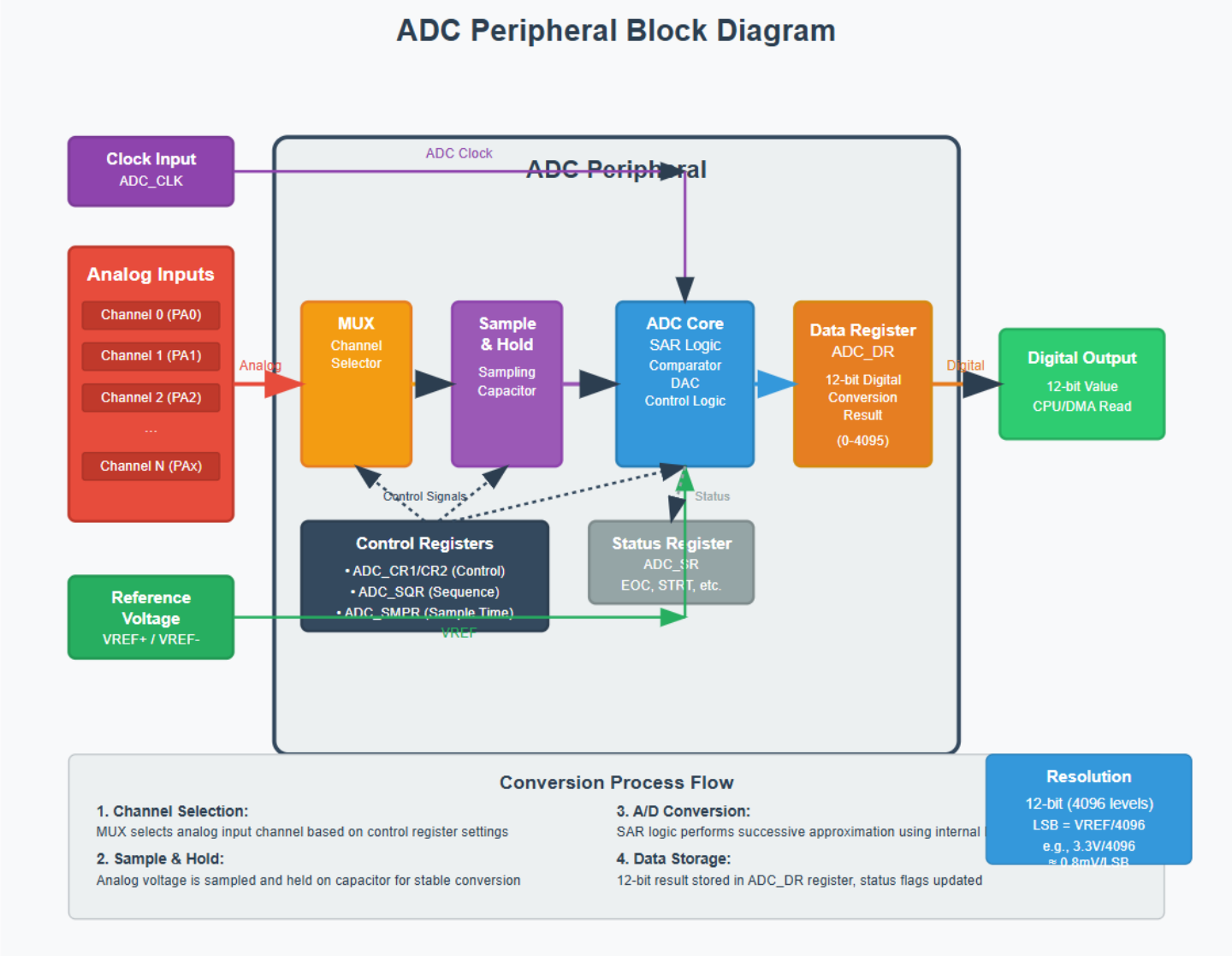


Image: ADC Block Diagram

Description: A diagram showing the ADC peripheral with inputs (analog channels), reference voltage (VREF), control registers, and output data register. Arrows indicate signal flow from analog input to digital output, with a clock input for timing.

2. Sensor Handling (LM35)

- **LM35 Overview**: A linear temperature sensor outputting 10mV/°C (e.g., 250mV at 25°C).
- **Interfacing**:
  - Connect LM35 output to an ADC pin (e.g., PA0).
  - Power LM35 with 3.3V or 5V (check datasheet).
  - Calculate temperature: Temp (°C) = (ADC_Value * VREF / 4095) * 100.
- **Embedded Tip**: Add a capacitor (e.g., 100nF) across LM35 output to ground to filter noise.

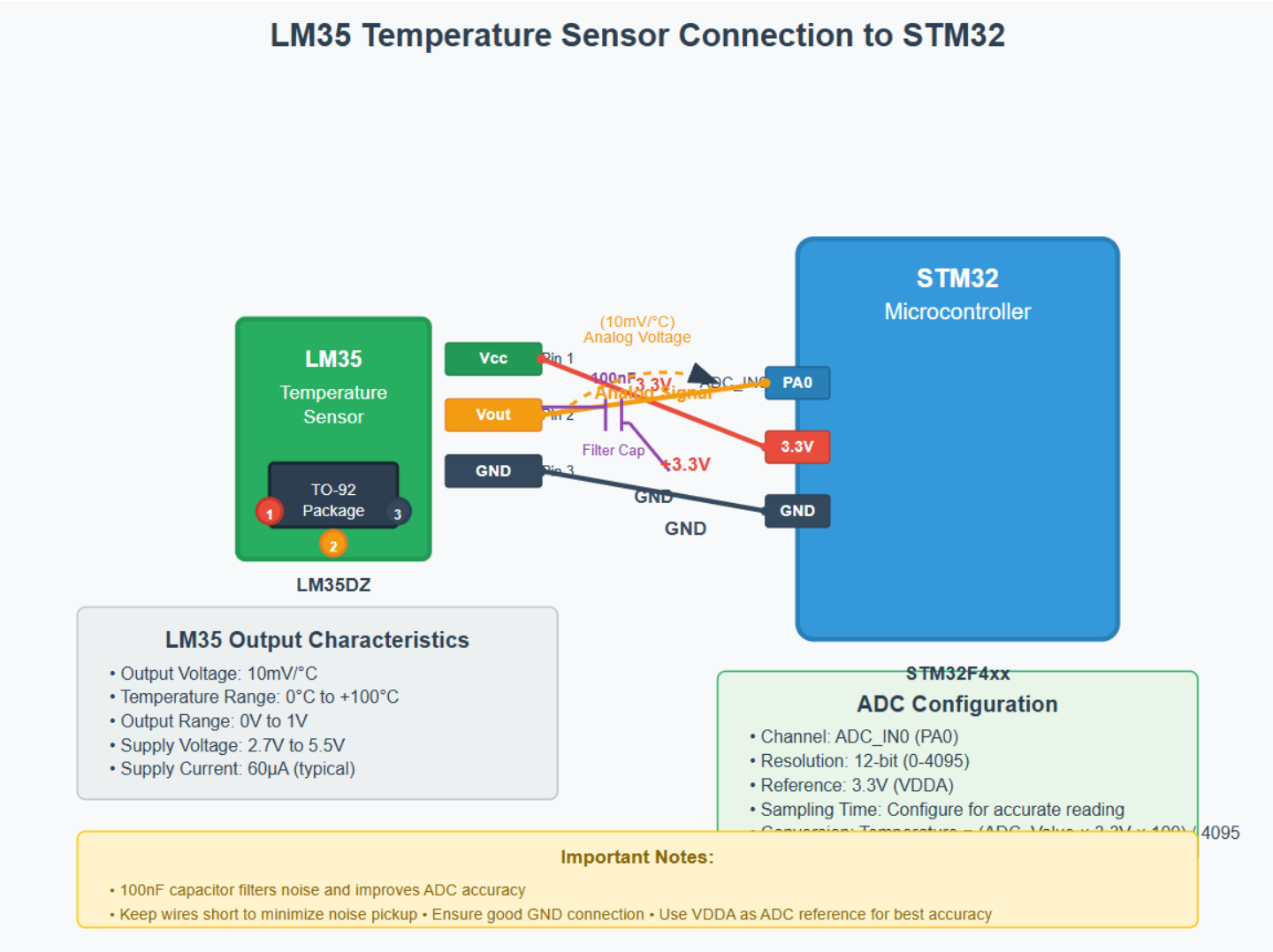


Image: LM35 Connection to STM32

Description: A schematic showing the LM35 connected to an STM32 microcontroller. The LM35 has three pins: Vcc (3.3V), GND, and Vout connected to PA0 (ADC input). A 100nF capacitor is shown between Vout and GND for noise filtering.

3. Practice Programs

a. ADC Single Conversion

- Read a single ADC value from PA0.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const ADC_CR2 = (volatile uint32_t *)0x40012408;
volatile uint32_t *const ADC_DR = (volatile uint32_t *)0x4001244C;
volatile uint32_t *const ADC_SQR3 = (volatile uint32_t *)0x4001242C;
```

```
void init_adc(void) {
    *GPIO_MODER |= (3U << (0 * 2)); // PA0 analog mode
    *ADC_SQR3 = 0; // Channel 0 (PA0)
    *ADC_CR2 = (1U << 0); // Enable ADC
}

uint16_t read_adc(void) {
    *ADC_CR2 |= (1U << 22); // Start conversion
    while (!(*ADC_CR2 & (1U << 1))); // Wait for EOC
    return *ADC_DR; // Read data
}

int main() {
    init_adc();
    uint16_t adc_value = read_adc();
    float voltage = (adc_value * 3.3f) / 4095.0f;
    printf("ADC: %u, Voltage: %.2fV\n", adc_value, voltage);
    return 0;
}
```

b. ADC with UART Output

- Read ADC and send value via UART.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const ADC_CR2 = (volatile uint32_t *)0x40012408;
volatile uint32_t *const ADC_DR = (volatile uint32_t *)0x4001244C;

void init_adc(void) {
    *ADC_CR2 = (1U << 0);
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}
uint16_t read_adc(void) {
    *ADC_CR2 |= (1U << 22);
    while (!(*ADC_CR2 & (1U << 1)));
    return *ADC_DR;
}

int main() {
    init_adc();
    uint16_t adc_value = read_adc();
    char msg[32];
    snprintf(msg, sizeof(msg), "ADC: %u\n", adc_value);
    uart_transmit_string(msg);
    return 0;
}
```

Project: ADC-Based Temperature Sensor

- **Objective**: Implement an ADC-based temperature sensor using an LM35, reading temperature via ADC and logging it via UART.

- **Requirements**:
  - Configure ADC1 (PA0) for single conversion, 12-bit resolution.
  - Connect LM35 to PA0; read temperature (10mV/°C).
  - Log temperature via UART (PA9-TX, 115200 baud) every second.
- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"

ADC_HandleTypeDef hadc1;
UART_HandleTypeDef huart1;

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

void init_adc(void) {
    __HAL_RCC_ADC1_CLK_ENABLE();
    hadc1.Instance = ADC1;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    HAL_ADC_Init(&hadc1);
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = ADC_CHANNEL_0; // PA0
    sConfig.Rank = 1;
    sConfig.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

float read_temperature(void) {
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    uint32_t adc_value = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return ((adc_value * 3.3f) / 4095.0f) * 100.0f; // LM35: 10mV/°C
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_adc();
    init_uart();
    while (1) {
        float temp_c = read_temperature();
        char msg[32];
        snprintf(msg, sizeof(msg), "Temp: %.1f C\r\n", temp_c);
```

```
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        HAL_Delay(1000);
    }
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const ADC_CR2 = (volatile uint32_t *)0x40012408;
volatile uint32_t *const ADC_DR = (volatile uint32_t *)0x4001244C;
volatile uint32_t *const ADC_SQR3 = (volatile uint32_t *)0x4001242C;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

void init_adc(void) {
    *GPIO_MODER |= (3U << (0 * 2)); // PA0 analog
    *ADC_SQR3 = 0; // Channel 0
    *ADC_CR2 = (1U << 0); // Enable ADC
}

void init_uart(void) {
    *USART_BRR = 16000000 / 115200;
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2);
}

float read_temperature(void) {
    static uint32_t sim_adc[] = {1024, 1120, 1080}; // Simulate 0.825V, 0.902V, 0.870V
    static uint32_t idx = 0;
    *ADC_CR2 |= (1U << 22);
    while (!(*ADC_CR2 & (1U << 1)));
    *ADC_DR = sim_adc[idx++ % 3];
    uint32_t adc_value = *ADC_DR;
    return ((adc_value * 3.3f) / 4095.0f) * 100.0f; // LM35: 10mV/°C
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}

int main() {
    init_adc();
    init_uart();
    for (int i = 0; i < 3; i++) {
        float temp_c = read_temperature();
        char msg[32];
        snprintf(msg, sizeof(msg), "Temp: %.1f C\n", temp_c);
        uart_transmit_string(msg);
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure ADC1 (PA0, single conversion), USART1 (PA9-TX, 115200 baud).
  - Connect LM35: Vcc to 3.3V, GND to ground, Vout to PA0 with 100nF capacitor.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.

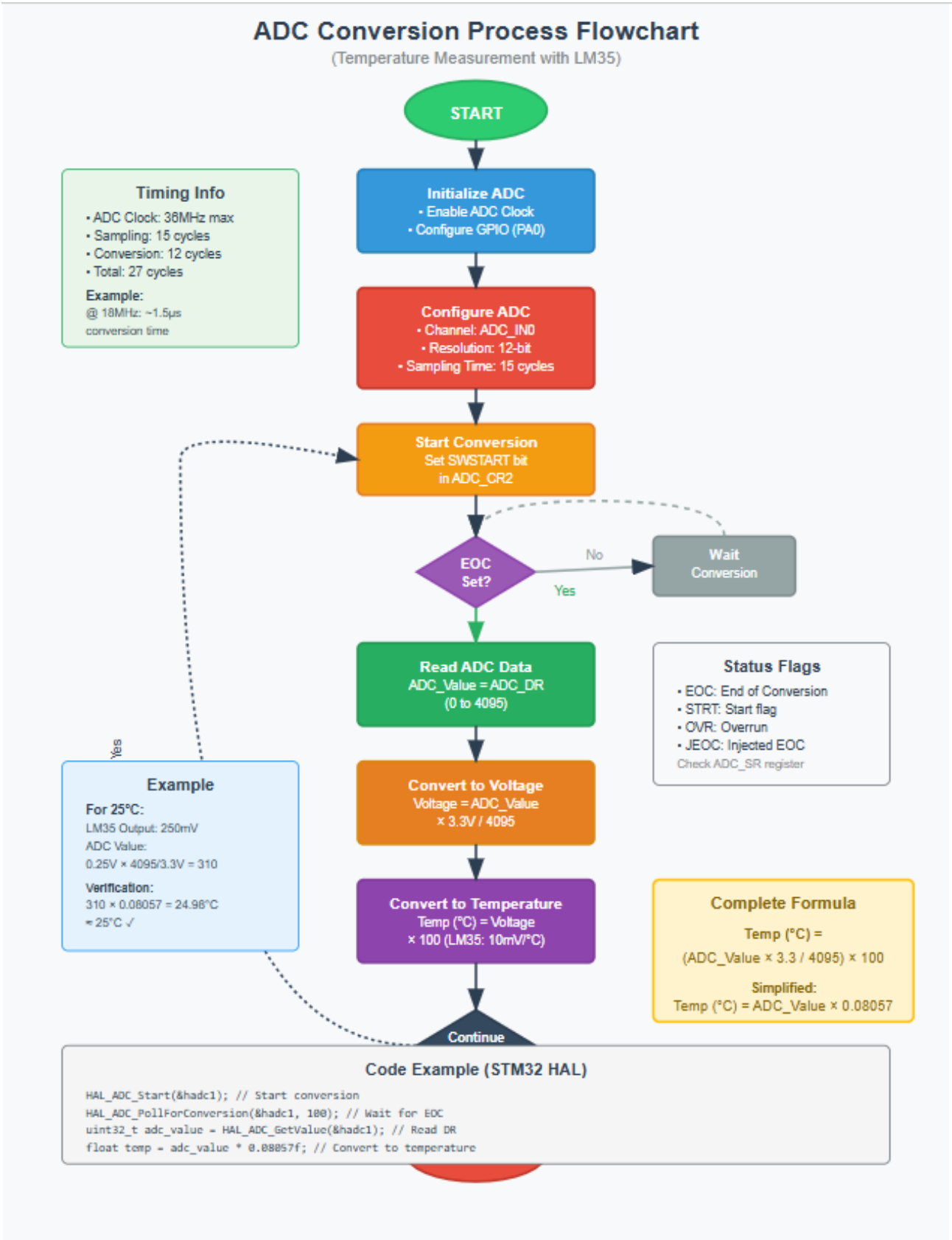


Image: ADC Conversion Process

Description: A flowchart showing the ADC conversion process: configure ADC (channel, sampling time), start conversion, wait for end-of-conversion (EOC), read data from DR, and convert to voltage/temperature. Includes a formula: Temp (°C) = (ADC_Value * 3.3 / 4095) * 100.

- **Optimizations**:
  - Use HAL for robust ADC/UART initialization.
  - Average multiple ADC readings to reduce noise (not shown, but recommended).
  - Log temperature in a human-readable format.
- **Embedded Tip**: Increase sampling time (ADC_SAMPLETIME_55CYCLES_5) for better accuracy with high-impedance sensors like LM35.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for ADC/UART setup.
- **STM32CubeMX**: Configure ADC1 and USART1.
- **Debugging**: Use STM32CubeIDE's debugger to inspect ADC1->DR; simulate with GCC (gcc program.c -o program).

Resources

- **STM32 ADC Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **LM35 Datasheet**: https://www.ti.com/product/LM35 for specifications.
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

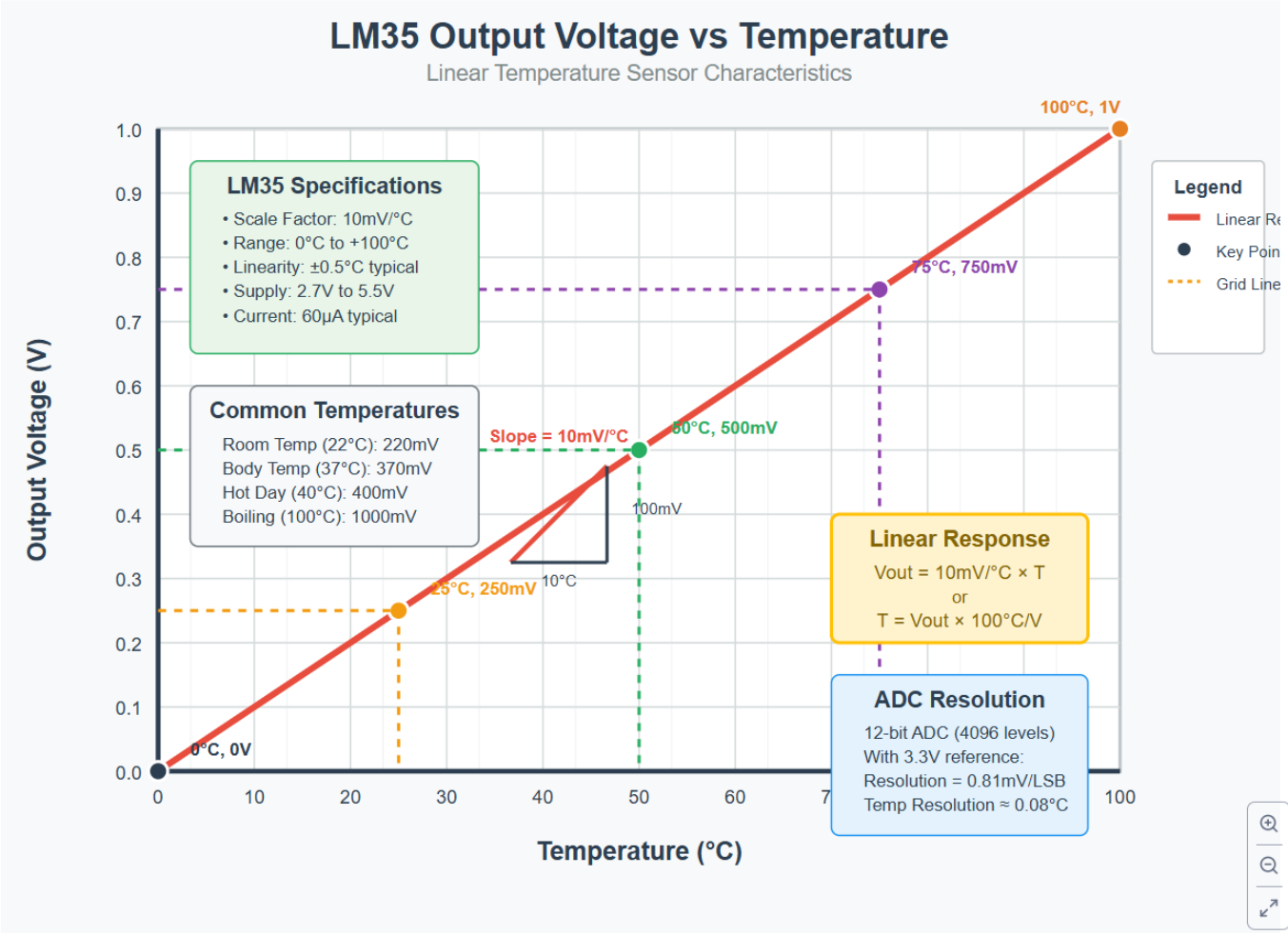


Image: LM35 Output vs. Temperature

Description: A graph plotting LM35 output voltage (y-axis, 0-1V) against temperature (x-axis, 0-100°C). A linear line shows 10mV/°C, with 250mV at 25°C and 500mV at 50°C.

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add averaging of 10 ADC samples to reduce noise.
- **Optimize**: Use DMA for continuous ADC conversions (advanced, for later).
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Use Watchdog Timer

Goal

Master the watchdog timer setup to enhance system reliability in embedded systems, ensuring automatic recovery from software faults, with a focus on implementing a watchdog-protected LED blink application.

Tasks

- **Study**: Learn watchdog timer concepts, configuration, and use cases for system reliability.
- **Practice**: Write code to configure and use the watchdog timer.
- **Project**: Develop a watchdog-protected LED blink program that resets the system on failure.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration).
- **Resources**: STM32 watchdog manuals.

Detailed Notes

1. Watchdog Timer Overview

- **Definition**: A watchdog timer (WDT) is a hardware peripheral that resets the microcontroller if not periodically refreshed, preventing system hangs due to software faults (e.g., infinite loops, crashes).
- **Types in STM32**:
  - **Independent Watchdog (IWDG)**: Uses an independent clock (e.g., LSI at 40kHz), simple to configure.
  - **Window Watchdog (WWDG)**: Uses system clock, allows reset within a specific time window.
- **Key Components (IWDG)**:
  - **Prescaler**: Divides clock to set timeout period.
  - **Reload Register (RLR)**: Sets countdown value.
  - **Key Register (KR)**: Controls enable/refresh (e.g., 0xAAAA to refresh).
  - **Status Register (SR)**: Indicates watchdog status.
- **Operation**:
  - Enable WDT, set timeout (e.g., 1s).
  - Periodically refresh (kick) the WDT in normal operation.
  - If not refreshed in time, WDT triggers a system reset.
- **Formula**: Timeout (s) = (Prescaler * Reload) / Clock_Freq (e.g., for LSI = 40kHz, Prescaler = 4, Reload = 1000, Timeout = (4 * 1000) / 40000 = 0.1s).
- **Embedded Tip**: Set timeout longer than the longest normal task; refresh in the main loop or a dedicated task.

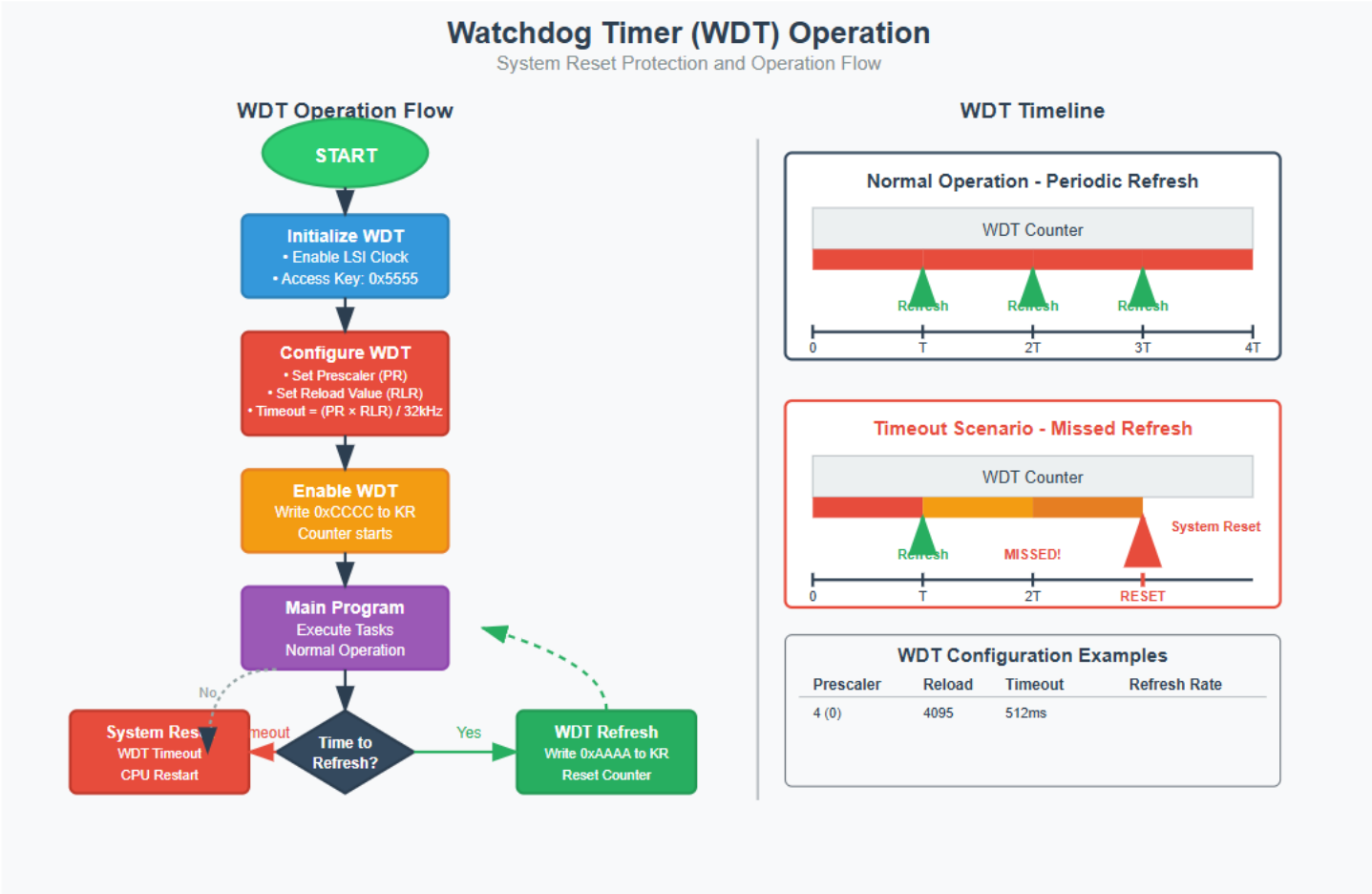


Image: Watchdog Timer Operation

Description: A flowchart showing watchdog timer operation: initialize WDT, set prescaler/reload, enable WDT, periodically refresh (write to KR), and reset if timeout occurs. Includes a timeline showing refresh intervals and reset trigger if refresh is missed.

2. STM32 IWDG Configuration

- **Steps**:
  1. Enable IWDG clock (LSI for STM32F1).
  2. Set prescaler and reload values for desired timeout.
  3. Write 0xCCCC to IWDG_KR to start WDT.
  4. Refresh with 0xAAAA to prevent reset.
- **Key Registers**:
  - **IWDG_KR**: Key register for control (0xCCCC to start, 0xAAAA to refresh).
  - **IWDG_PR**: Prescaler register (e.g., 4 for /4 divider).
  - **IWDG_RLR**: Reload register (e.g., 4095 max for STM32F1).
- **Embedded Tip**: Test WDT by intentionally skipping refresh to verify reset behavior.

3. Practice Program: Basic Watchdog Setup

- **Objective**: Configure IWDG to reset after 1s if not refreshed; blink LED to test.
- **Code** (STM32 simulation):

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const IWDG_KR = (volatile uint32_t *)0x40003000;
volatile uint32_t *const IWDG_PR = (volatile uint32_t *)0x40003004;
volatile uint32_t *const IWDG_RLR = (volatile uint32_t *)0x40003008;

#define LED_PIN 5

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= (1U << (LED_PIN * 2)); // PA5 output
    *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
}

void init_watchdog(void) {
    *IWDG_PR = 4; // Prescaler /32
    *IWDG_RLR = 1250; // Timeout = (32 * 1250) / 40000 = 1s
    *IWDG_KR = 0xCCCC; // Start IWDG
}

void refresh_watchdog(void) {
    *IWDG_KR = 0xAAAA; // Refresh
    printf("Watchdog refreshed\n");
}

void delay_ms(uint32_t ms) {
    for (volatile uint32_t i = 0; i < ms * 16000; i++); // Simulated delay
}

int main() {
    init_gpio();
    init_watchdog();
    for (int i = 0; i < 3; i++) {
        *GPIO_BSRR = (1U << LED_PIN); // LED on
        printf("LED ON\n");
        delay_ms(500);
        refresh_watchdog();
        *GPIO_BSRR = (1U << (LED_PIN + 16)); // LED off
        printf("LED OFF\n");
        delay_ms(500);
        refresh_watchdog();
    }
    printf("Skipping watchdog refresh to test reset...\n");
    while (1); // Should reset after 1s
    return 0;
}
```

Project: Watchdog-Protected LED Blink

- **Objective**: Implement a watchdog-protected LED blink program using STM32 IWDG, ensuring the system resets if the LED blink task hangs.
- **Requirements**:
  - Configure PA5 as output for LED.
  - Use IWDG with a 2s timeout.
  - Blink LED every 500ms, refreshing WDT each cycle.
  - Log state via UART (PA9-TX, 115200 baud).
  - Intentionally skip WDT refresh after 5 blinks to test reset.

- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"

IWDG_HandleTypeDef hiwdg;
UART_HandleTypeDef huart1;

volatile uint32_t blink_count = 0;

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
}

void init_watchdog(void) {
    __HAL_RCC_LSI_ENABLE();
    hiwdg.Instance = IWDG;
    hiwdg.Init.Prescaler = IWDG_PRESCALER_32;
    hiwdg.Init.Reload = 1250; // Timeout = (32 * 1250) / 40000 = 1s
    HAL_IWDG_Init(&hiwdg);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

void log_message(const char *msg) {
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_uart();
    init_watchdog();
    char msg[32];
    while (blink_count < 5) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        snprintf(msg, sizeof(msg), "Blink: %lu, LED: %s\r\n",
                blink_count, HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) ? "ON" : "OFF");
        log_message(msg);
        HAL_Delay(500);
        HAL_IWDG_Refresh(&hiwdg);
        blink_count++;
    }
    log_message("Skipping watchdog refresh to test reset...\r\n");
    while (1); // Should reset after 1s
}
```

- **Simulated Version** (For PC testing):

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const IWDG_KR = (volatile uint32_t *)0x40003000;
volatile uint32_t *const IWDG_PR = (volatile uint32_t *)0x40003004;
volatile uint32_t *const IWDG_RLR = (volatile uint32_t *)0x40003008;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

#define LED_PIN 5

volatile uint32_t blink_count = 0;

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (LED_PIN * 2));
    *GPIO_MODER |= (1U << (LED_PIN * 2));
    *GPIO_BSRR = (1U << (LED_PIN + 16));
}

void init_watchdog(void) {
    *IWDG_PR = 4; // Prescaler /32
    *IWDG_RLR = 1250; // Timeout = 1s
    *IWDG_KR = 0xCCCC; // Start
}

void refresh_watchdog(void) {
    *IWDG_KR = 0xAAAA;
}
void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}
void delay_ms(uint32_t ms) {
    for (volatile uint32_t i = 0; i < ms * 16000; i++);
}

int main() {
    init_gpio();
    init_watchdog();
    char msg[32];
    while (blink_count < 5) {
        *GPIO_BSRR = (blink_count % 2) ? (1U << LED_PIN) : (1U << (LED_PIN + 16));
        snprintf(msg, sizeof(msg), "Blink: %lu, LED: %s\n",
                blink_count, (blink_count % 2) ? "ON" : "OFF");
        uart_transmit_string(msg);
        delay_ms(500);
        refresh_watchdog();
        blink_count++;
    }
    uart_transmit_string("Skipping watchdog refresh to test reset...\n");
    while (1); // Should reset
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure PA5 (output), USART1 (PA9-TX, 115200 baud), and IWDG (32 prescaler, 1250 reload for 1s timeout).
  - Connect LED (with resistor) to PA5.

- Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.
- Verify reset by observing LED restarting after 5 blinks.

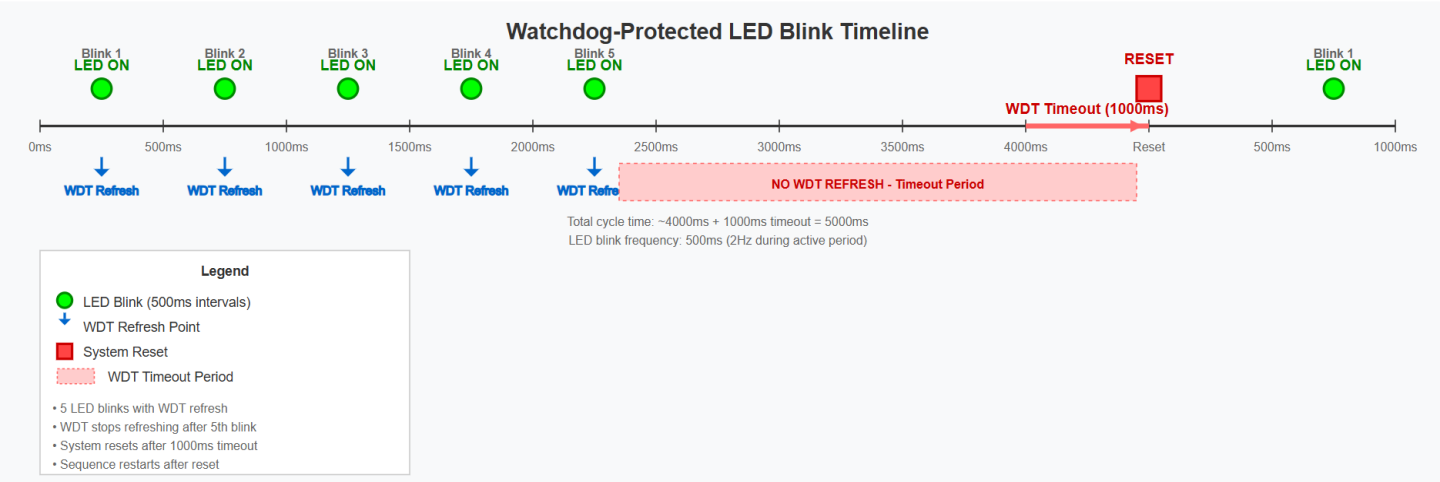


Image: Watchdog-Protected LED Blink Timeline

Description: A timeline showing LED blinking every 500ms for 5 cycles, with WDT refresh at each cycle. After 5 blinks, refresh stops, and a reset occurs after 1s, restarting the blink sequence. Includes annotations for WDT timeout and refresh points.

- **Optimizations**:
  - HAL for robust IWDG/UART setup.
  - UART logging for debugging reset behavior.
  - Short WDT timeout (1s) for quick testing; adjust for real applications.
- **Embedded Tip**: Use a boot counter in flash to log reset occurrences for debugging.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for IWDG/UART setup.
- **STM32CubeMX**: Configure IWDG, GPIO, and USART1.
- **Debugging**: Use STM32CubeIDE's debugger to inspect IWDG->SR; simulate with GCC (gcc program.c -o program).

Resources

- **STM32 Watchdog Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a button to simulate a hang (skip refresh).
- **Optimize**: Store blink count in RTC backup register to track resets.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Consolidate Week 4 Concepts

Goal

Consolidate Week 4 concepts (SPI/I2C, FSM, ADC, watchdog timer) by reviewing notes, debugging prior projects, and optimizing the Day 16 I2C temperature sensor driver to use interrupts for non-blocking operation and add error handling.

Tasks

- **Study**: Review SPI/I2C drivers, FSM, ADC configuration, and watchdog timer concepts.
- **Practice**: Debug issues in prior projects (e.g., traffic light FSM, ADC temperature sensor).
- **Project**: Optimize the Day 16 I2C temperature sensor driver for interrupt-driven operation and robust error handling.
- **Tools**: STM32CubeIDE, GDB (debugger), STM32CubeMX.
- **Resources**: STM32CubeIDE docs, GDB tutorials, STM32 I2C manuals.

Detailed Notes

1. Review of Week 4 Concepts

- **SPI/I2C Drivers (Day 16)**:
  - **SPI**: Synchronous, full-duplex, high-speed (1-10 MHz); uses SCLK, MOSI, MISO, SS.
  - **I2C**: Two-wire, half-duplex, multi-device (100kHz/400kHz); uses SDA, SCL.
  - **Key**: Polling is simple but CPU-intensive; interrupts/DMA improve efficiency.
  - **Example**: I2C1->DR for data; check SR1 for TXE/RXNE.
- **Finite State Machines (Day 17)**:
  - Manage tasks with states, events, transitions, and actions.
  - Example: Traffic light FSM with Red (5s), Green (4s), Yellow (1s) states.
  - **Key**: Use timers for precise state transitions; avoid blocking delays.
- **ADC and Sensor Handling (Day 18)**:
  - Convert analog signals (e.g., LM35's 10mV/°C) to digital.
  - Formula: Temp (°C) = (ADC_Value * VREF / 4095) * 100 for 12-bit ADC.
  - **Key**: Increase sampling time for accuracy; average readings to reduce noise.
- **Watchdog Timer (Day 19)**:
  - Resets system if not refreshed within timeout (e.g., 1s).
  - Example: IWDG with (Prescaler * Reload) / LSI timeout.
  - **Key**: Refresh in main loop; test reset behavior.

2. Debugging Prior Projects

a. Traffic Light FSM (Day 17)

- **Potential Bug**: Timer interrupt priority conflicts with UART, causing missed state transitions.
- **Fix**: Set TIM2 priority higher (e.g., 0) than UART (e.g., 1) in NVIC.

- **Revised Snippet**:

```
void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999;
    htim2.Init.Period = 999; // 1s
    HAL_TIM_Base_Init(&htim2);
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0); // Higher priority
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}
```

b. ADC Temperature Sensor (Day 18)

- **Potential Bug**: Noise in ADC readings due to insufficient sampling time.
- **Fix**: Increase sampling time to ADC_SAMPLETIME_239CYCLES_5; average 10 samples.
- **Revised Snippet**:

```
float read_temperature(void) {
    uint32_t sum = 0;
    for (int i = 0; i < 10; i++) {
        HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, 10);
        sum += HAL_ADC_GetValue(&hadc1);
    }
    HAL_ADC_Stop(&hadc1);
    return ((sum / 10.0f * 3.3f) / 4095.0f) * 100.0f; // Average 10 samples
}
```

3. Project: Optimize I2C Temperature Sensor Driver

- **Objective**: Optimize the Day 16 I2C temperature sensor driver (LM75A) to use interrupt-driven operation for non-blocking reads and add error handling for robust communication.
- **Optimizations**:
  - Replace polling with I2C interrupts for data transfer.
  - Add error handling for I2C bus errors (e.g., NACK, arbitration loss).
  - Use a timer to trigger periodic temperature reads (every 1s).
  - Include watchdog timer for system reliability.
- **Requirements**:
  - Configure I2C1 (PB6-SCL, PB7-SDA, 100kHz) for interrupt-driven operation.
  - Use TIM2 for 1s periodic reads.
  - Read LM75A (address 0x48) temperature register.
  - Log temperature via UART (PA9-TX, 115200 baud).
  - Enable IWDG with 2s timeout for reliability.
- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;
TIM_HandleTypeDef htim2;
IWDG_HandleTypeDef hiwdg;

volatile uint8_t i2c_data[2];
volatile uint8_t i2c_state = 0; // 0: idle, 1: write addr, 2: read data
volatile uint8_t i2c_error = 0;
```

```
void init_i2c(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_I2C1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_6 | GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    HAL_I2C_Init(&hi2c1);
    HAL_NVIC_SetPriority(I2C1_EV_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(I2C1_EV_IRQn);
    HAL_NVIC_SetPriority(I2C1_ER_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(I2C1_ER_IRQn);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz
    htim2.Init.Period = 999; // 1s
    HAL_TIM_Base_Init(&htim2);
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}

void init_watchdog(void) {
    __HAL_RCC_LSI_ENABLE();
    hiwdg.Instance = IWDG;
    hiwdg.Init.Prescaler = IWDG_PRESCALER_32;
    hiwdg.Init.Reload = 2500; // Timeout = (32 * 2500) / 40000 = 2s
    HAL_IWDG_Init(&hiwdg);
}

void start_i2c_read(uint8_t addr) {
    i2c_state = 1;
    i2c_error = 0;
    uint8_t reg = 0x00; // LM75A temp register
    HAL_I2C_Master_Sequential_Transmit_IT(&hi2c1, addr << 1, &reg, 1, I2C_FIRST_FRAME);
}

void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c) {
    if (i2c_state == 1) {
        i2c_state = 2;
        HAL_I2C_Master_Sequential_Receive_IT(&hi2c1, 0x48 << 1 | 1, i2c_data, 2, I2C_LAST_FRAME);
    }
}
```

```
void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c) {
    if (i2c_state == 2) {
        i2c_state = 0;
        int16_t raw_temp = (i2c_data[0] << 8) | i2c_data[1];
        float temp_c = (raw_temp >> 7) * 0.5f; // LM75A: 9-bit, 0.5°C resolution
        char msg[32];
        snprintf(msg, sizeof(msg), "Temp: %.1f C\r\n", temp_c);
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
    }
}

void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c) {
    i2c_error = 1;
    i2c_state = 0;
    char msg[] = "I2C Error\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
    HAL_I2C_DeInit(&hi2c1);
    HAL_I2C_Init(&hi2c1); // Reinitialize I2C
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2 && i2c_state == 0) {
        start_i2c_read(0x48); // LM75A address
    }
}

int main(void) {
    HAL_Init();
    init_i2c();
    init_uart();
    init_timer();
    init_watchdog();
    while (1) {
        HAL_IWDG_Refresh(&hiwdg);
        if (i2c_error) {
            HAL_Delay(1000); // Wait before retry
            i2c_error = 0;
        }
    }
}

void I2C1_EV_IRQHandler(void) {
    HAL_I2C_EV_IRQHandler(&hi2c1);
}

void I2C1_ER_IRQHandler(void) {
    HAL_I2C_ER_IRQHandler(&hi2c1);
}

void TIM2_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim2);
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const I2C1_CR1 = (volatile uint32_t *)0x40005400;
volatile uint32_t *const I2C1_CR2 = (volatile uint32_t *)0x40005404;
volatile uint32_t *const I2C1_SR1 = (volatile uint32_t *)0x40005414;
volatile uint32_t *const I2C1_SR2 = (volatile uint32_t *)0x40005418;
volatile uint32_t *const I2C1_DR = (volatile uint32_t *)0x40005410;
volatile uint32_t *const TIM2_SR = (volatile uint32_t *)0x40000010;
volatile uint32_t *const IWDG_KR = (volatile uint32_t *)0x40003000;
volatile uint32_t *const IWDG_PR = (volatile uint32_t *)0x40003004;
```

```
volatile uint32_t *const IWDG_RLR = (volatile uint32_t *)0x40003008;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

volatile uint8_t i2c_data[2];
volatile uint8_t i2c_state = 0;
volatile uint8_t i2c_error = 0;

void init_i2c(void) {
    *I2C1_CR2 = 16; // 16MHz clock
    *I2C1_CR1 = (1U << 0);
}

void init_timer(void) {
    *TIM2_PSC = 15999;
    *TIM2_ARR = 999; // 1s
    *TIM2_CR1 = (1U << 0);
}

void init_watchdog(void) {
    *IWDG_PR = 4; // Prescaler /32
    *IWDG_RLR = 2500; // Timeout = 2s
    *IWDG_KR = 0xCCCC;
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}

void start_i2c_read(uint8_t addr) {
    i2c_state = 1;
    i2c_error = 0;
    *I2C1_CR1 |= (1U << 8); // Start
    while (!(*I2C1_SR1 & (1U << 0)));
    *I2C1_DR = addr << 1;
    while (!(*I2C1_SR1 & (1U << 1)));
    (void)*I2C1_SR1; (void)*I2C1_SR2;
    *I2C1_DR = 0x00;
    while (!(*I2C1_SR1 & (1U << 7)));
    *I2C1_CR1 |= (1U << 9); // Stop
    i2c_state = 2;
    *I2C1_CR1 |= (1U << 8);
    while (!(*I2C1_SR1 & (1U << 0)));
    *I2C1_DR = (addr << 1) | 1;
    while (!(*I2C1_SR1 & (1U << 1)));
    (void)*I2C1_SR1; (void)*I2C1_SR2;
    *I2C1_CR1 |= (1U << 10);
    static uint8_t sim_data[] = {0x19, 0x00, 0x1A, 0x00}; // Simulate 25°C, 25.5°C
    static uint32_t idx = 0;
    while (!(*I2C1_SR1 & (1U << 6)));
    i2c_data[0] = sim_data[idx++ % 4];
    *I2C1_CR1 &= ~(1U << 10);
    while (!(*I2C1_SR1 & (1U << 6)));
    i2c_data[1] = sim_data[idx++ % 4];
    *I2C1_CR1 |= (1U << 9);
    i2c_state = 0;
    int16_t raw_temp = (i2c_data[0] << 8) | i2c_data[1];
    float temp_c = (raw_temp >> 7) * 0.5f;
    char msg[32];
    snprintf(msg, sizeof(msg), "Temp: %.1f C\n", temp_c);
    uart_transmit_string(msg);
}
```

```
void TIM2_IRQHandler(void) {
    if (*TIM2_SR & (1U << 0)) {
        if (i2c_state == 0) {
            start_i2c_read(0x48);
        }
        *TIM2_SR &= ~(1U << 0);
    }
}

int main() {
    init_i2c();
    init_timer();
    init_watchdog();
    for (int i = 0; i < 3; i++) {
        *IWDG_KR = 0xAAAA; // Refresh
        *TIM2_SR |= (1U << 0);
        TIM2_IRQHandler();
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure I2C1 (PB6-SCL, PB7-SDA, 100kHz), TIM2 (1s interrupts), USART1 (PA9-TX, 115200 baud), and IWDG (32 prescaler, 2500 reload for 2s timeout).
  - Connect LM75A (address 0x48) to PB6/PB7 with 4.7kΩ pull-up resistors.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.

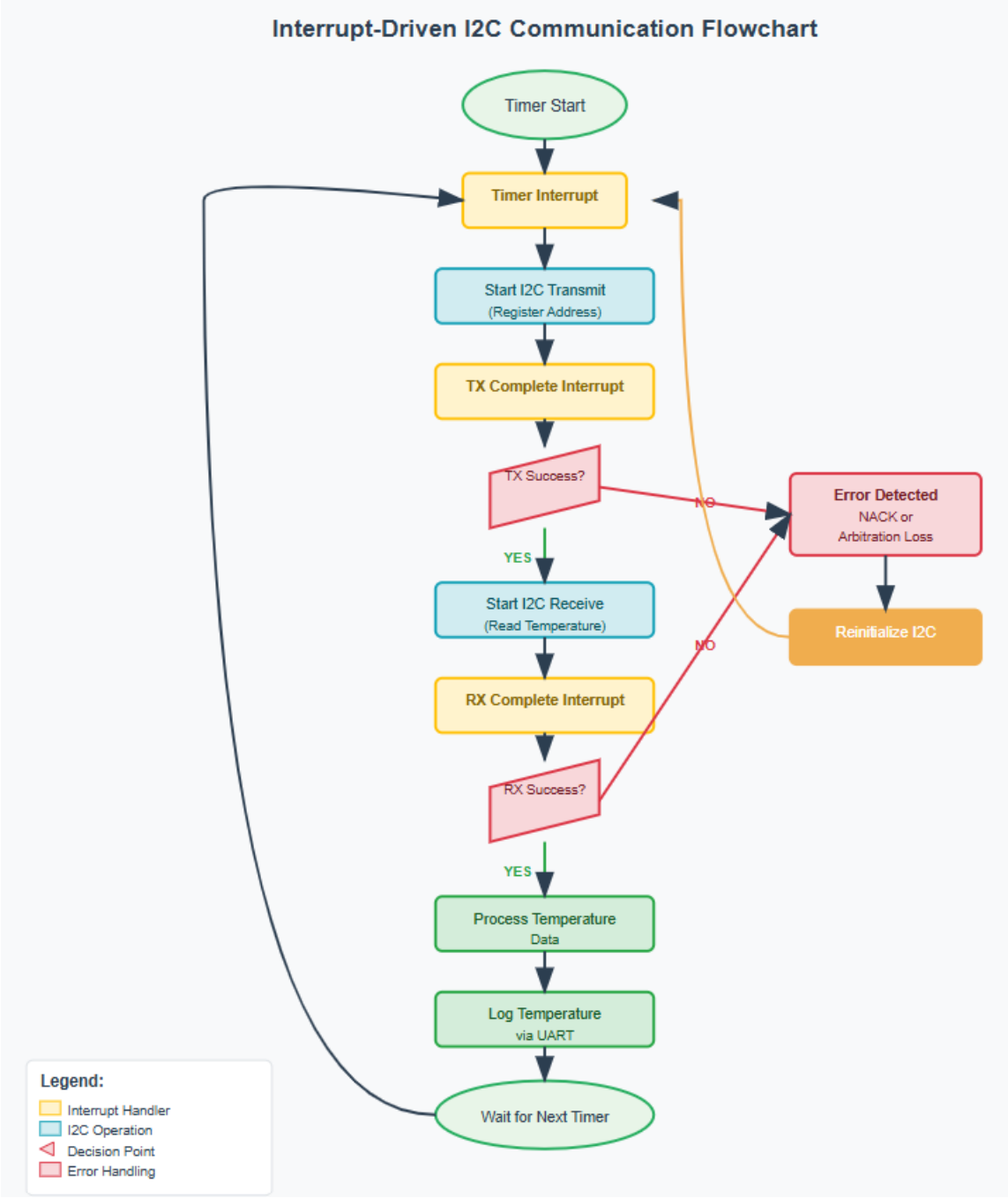


Image: Interrupt-Driven I2C Flowchart

Description: A flowchart showing the interrupt-driven I2C process: timer triggers read, start transmit (register address), TX complete interrupt triggers receive, RX complete interrupt processes data, logs temperature via UART. Error handling path shows reinitialization on NACK or arbitration loss.

- **Optimizations**:
  - Interrupt-driven I2C reduces CPU load compared to polling.
  - Error handling reinitializes I2C on failure.
  - Watchdog ensures system recovery from hangs.
  - Timer-driven reads for consistent sampling.
- **Embedded Tip**: Use a logic analyzer to debug I2C signals; monitor I2C1->SR1 for errors like AF (NACK).

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for configuration.
- **STM32CubeMX**: Configure I2C1, TIM2, USART1, and IWDG.
- **GDB**: Debug with gdb ./program for simulation; use STM32CubeIDE's debugger for hardware.
- **Debugging**: Inspect I2C1->SR1, TIM2->SR, and IWDG->SR in STM32CubeIDE.

Resources

- **STM32CubeIDE Docs**: https://www.st.com for IDE and HAL documentation.
- **GDB Tutorials**: Search "GDB embedded debugging" (e.g., https://www.tutorialspoint.com/gdb/).
- **STM32 I2C Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **Online**: C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Review**: Spend ~1 hour rereading Days 16-19 notes and code.
- **Debug**: Use GDB or STM32CubeIDE to fix one bug (e.g., I2C NACK handling).
- **Optimize**: Add buffering for multiple temperature readings.
- **Time Management**: Spend ~1 hour on review, ~1 hour on debugging, ~1-1.5 hours on the project.

Chapter 5: Advanced Firmware Design

- **Learn DMA integration**
- **Understand bootloader basics**
- **Optimize stack/heap**
- **Learn MISRA and static analysis**
- **Consolidate Week 5 concepts**

Learn DMA Integration

Goal

Master Direct Memory Access (DMA) integration to enable efficient, low-CPU-overhead data transfers in embedded systems, with a focus on implementing a UART ring buffer using DMA for non-blocking communication.

Tasks

- **Study**: Learn DMA concepts, configuration, and integration with UART for efficient data transfer.
- **Practice**: Write DMA-based code to transfer data between memory and UART.
- **Project**: Develop a UART ring buffer using DMA to receive and transmit data, with logging for debugging.
- **Tools**: STM32CubeIDE, STM32CubeMX (for configuration).
- **Resources**: STM32 DMA manuals, YouTube DMA tutorials.

Detailed Notes

1. Direct Memory Access (DMA) Overview

- **Definition**: DMA is a hardware peripheral that transfers data between memory and peripherals (e.g., UART, SPI) without CPU intervention, reducing CPU load for large or continuous data transfers.
- **Key Features (STM32)**:
  - **Channels**: Each DMA controller (e.g., DMA1, DMA2) has multiple channels for different peripherals.
  - **Modes**: Memory-to-peripheral, peripheral-to-memory, memory-to-memory.
  - **Transfer Types**: Single, burst, or circular (continuous).
  - **Interrupts**: Triggered on transfer complete (TC), half-transfer (HT), or error (TE).
- **STM32 DMA Registers**:
  - **CCR**: Channel configuration register (enable, direction, circular mode).
  - **CNDTR**: Number of data items to transfer.
  - **CPAR**: Peripheral address register (e.g., &USART1->DR).
  - **CMAR**: Memory address register (e.g., buffer address).
- **Use Case**: Ideal for UART data streaming, ADC sampling, or SPI bulk transfers.
- **Embedded Tip**: Use circular mode for continuous data streams (e.g., ring buffers); prioritize interrupt handling for timely error recovery.

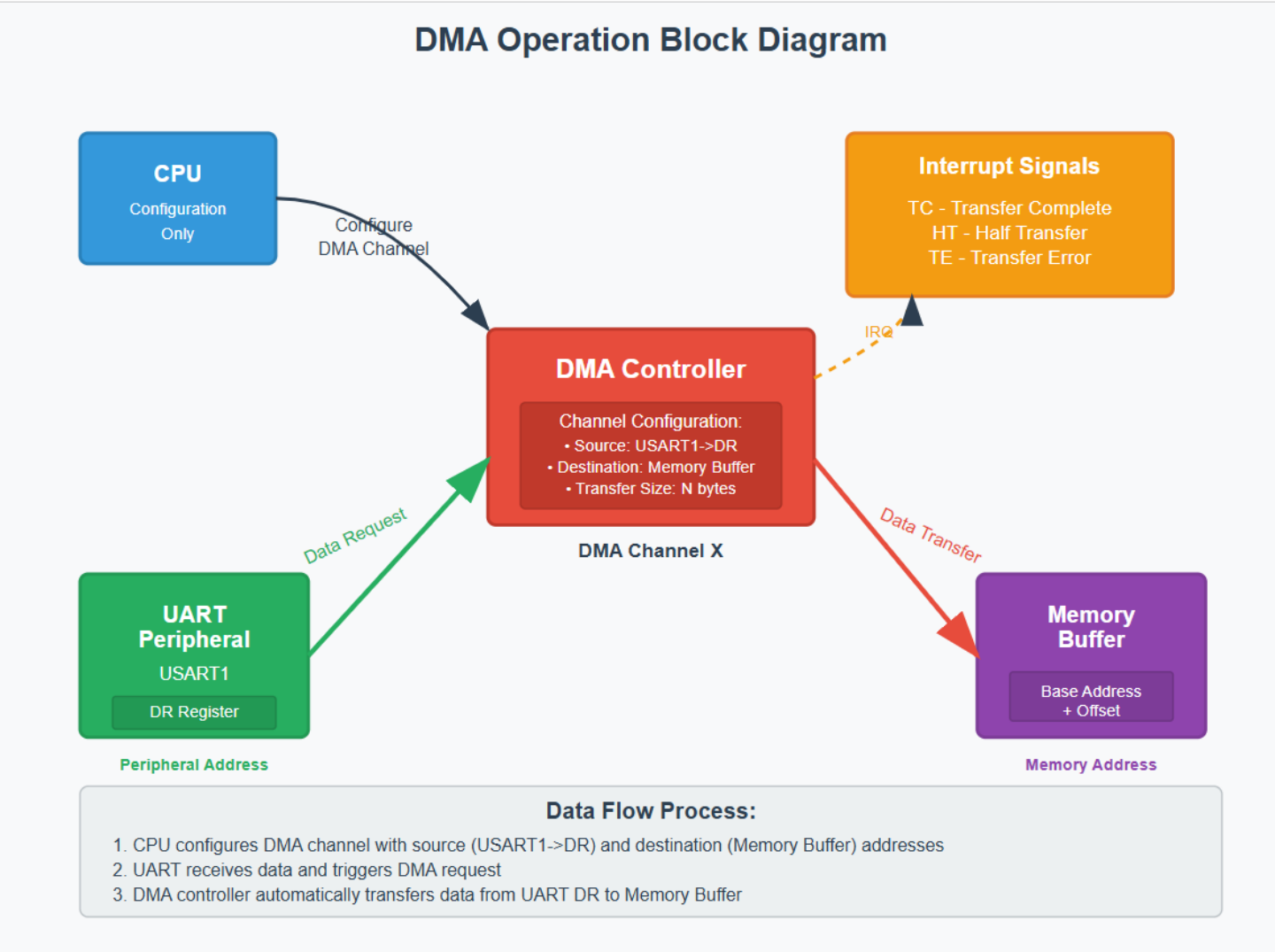


Image: DMA Operation Diagram

Description: A block diagram showing the DMA controller transferring data between a UART peripheral (USART1->DR) and a memory buffer. Arrows indicate data flow, with the CPU configuring the DMA but not involved in the transfer. Labels highlight DMA channel, memory/peripheral addresses, and interrupt signals (TC, HT, TE).

2. UART with DMA

- **Purpose**: Offload UART transmit/receive to DMA, allowing the CPU to perform other tasks.
- **Configuration**:
  - **Receive (RX)**: DMA moves data from USART->DR to a memory buffer.
  - **Transmit (TX)**: DMA moves data from a memory buffer to USART->DR.
  - **Circular Mode**: Continuously refills buffer for RX, ideal for ring buffers.
- **Key STM32 Registers**:
  - **DMA_CCRx**: Enable DMA, set direction (e.g., peripheral-to-memory for RX).
  - **USART_CR3**: Enable DMA for UART (DMAT for TX, DMAR for RX).
- **Embedded Tip**: Use a ring buffer to manage continuous RX data; process data in DMA interrupts to avoid overflow.

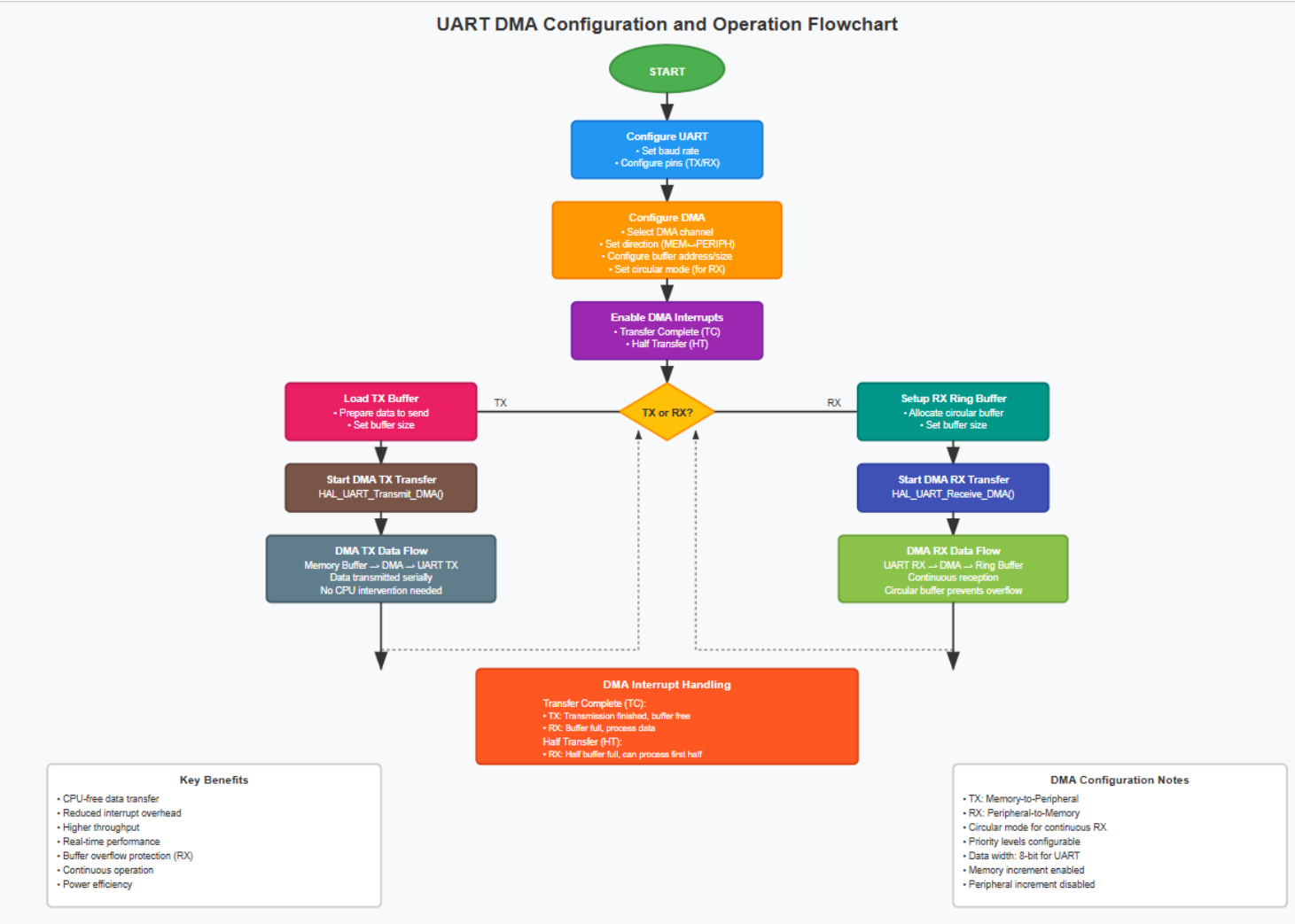


Image: UART DMA Flowchart

Description: A flowchart showing UART DMA setup: configure UART (baud rate, pins), configure DMA (channel, direction, buffer), enable DMA interrupts, start transfer. For RX, data flows to ring buffer; for TX, data is sent from buffer. Includes interrupt handling for TC/HT.

3. Ring Buffer Concept

- **Definition**: A circular buffer where data is written to a head pointer and read from a tail pointer, wrapping around when reaching the buffer's end.
- **Implementation**:
  - Store head/tail indices to track write/read positions.
  - Use DMA circular mode for RX to continuously fill the buffer.
  - Process data in DMA half-transfer (HT) or transfer-complete (TC) interrupts.
- **Advantages**:
  - Handles continuous data streams without loss.
  - Non-blocking, allowing CPU to perform other tasks.
- **Embedded Tip**: Ensure buffer size is a power of 2 (e.g., 256 bytes) for efficient index wrapping (index & (size - 1)).

4. Practice Programs

a. DMA UART Transmit

- Transmit a fixed string via UART using DMA.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const DMA_CCR1 = (volatile uint32_t *)0x40020008;
volatile uint32_t *const DMA_CNDTR1 = (volatile uint32_t *)0x4002000C;
volatile uint32_t *const DMA_CPAR1 = (volatile uint32_t *)0x40020010;
volatile uint32_t *const DMA_CMAR1 = (volatile uint32_t *)0x40020014;

uint8_t tx_buffer[] = "Hello, DMA!\n";

void init_dma_uart(void) {
    *DMA_CPAR1 = (uint32_t)&USART_DR; // Peripheral: USART1->DR
    *DMA_CMAR1 = (uint32_t)tx_buffer; // Memory: tx_buffer
    *DMA_CNDTR1 = sizeof(tx_buffer); // Data length
    *DMA_CCR1 = (1U << 7) | (1U << 4) | (1U << 0); // Memory increment, mem-to-periph, enable
}

int main() {
    init_dma_uart();
    while (!(*DMA_CCR1 & (1U << 1))); // Wait for TC
    printf("Transmitted: %s", tx_buffer);
    *DMA_CCR1 &= ~(1U << 0); // Disable DMA
    return 0;
}
```

b. DMA UART Receive

- Receive data into a fixed buffer via DMA.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const DMA_CCR2 = (volatile uint32_t *)0x4002001C;
volatile uint32_t *const DMA_CNDTR2 = (volatile uint32_t *)0x40020020;
volatile uint32_t *const DMA_CPAR2 = (volatile uint32_t *)0x40020024;
volatile uint32_t *const DMA_CMAR2 = (volatile uint32_t *)0x40020028;
uint8_t rx_buffer[16];

void init_dma_uart(void) {
    *DMA_CPAR2 = (uint32_t)&USART_DR; // Peripheral: USART1->DR
    *DMA_CMAR2 = (uint32_t)rx_buffer; // Memory: rx_buffer
    *DMA_CNDTR2 = sizeof(rx_buffer); // Data length
    *DMA_CCR2 = (1U << 7) | (0U << 4) | (1U << 0); // Memory increment, periph-to-mem, enable
}

int main() {
    init_dma_uart();
    static uint8_t sim_data[] = {'A', 'B', 'C'};
    for (int i = 0; i < 3; i++) {
        *USART_DR = sim_data[i]; // Simulate incoming data
    }
    while (*DMA_CNDTR2 > 13); // Wait for 3 bytes
    printf("Received: %c%c%c\n", rx_buffer[0], rx_buffer[1], rx_buffer[2]);
    *DMA_CCR2 &= ~(1U << 0); // Disable DMA
```

```
    return 0;
}
```

Project: UART Ring Buffer via DMA

- **Objective**: Implement a UART ring buffer using DMA for non-blocking receive and transmit operations, with data logged via UART for debugging.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX and TX.
  - Use DMA1 Channel 4 (TX) and Channel 5 (RX) in circular mode for RX.
  - Implement a 256-byte ring buffer for RX data.
  - Echo received data back via DMA TX; log buffer status via UART.
  - Use TIM2 to periodically check buffer and trigger TX.
- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"
UART_HandleTypeDef huart1;
DMA_HandleTypeDef hdma_usart1_tx;
DMA_HandleTypeDef hdma_usart1_rx;
TIM_HandleTypeDef htim2;
#define RING_BUFFER_SIZE 256
uint8_t rx_buffer[RING_BUFFER_SIZE];
uint8_t tx_buffer[RING_BUFFER_SIZE];
volatile uint32_t head = 0, tail = 0;
volatile uint8_t tx_busy = 0;

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    HAL_UART_Init(&huart1);
}

void init_dma(void) {
    __HAL_RCC_DMA1_CLK_ENABLE();
    // RX: DMA1 Channel 5
    hdma_usart1_rx.Instance = DMA1_Channel5;
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_rx.Init.Mode = DMA_CIRCULAR;
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH;
    HAL_DMA_Init(&hdma_usart1_rx);
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx);
    // TX: DMA1 Channel 4
    hdma_usart1_tx.Instance = DMA1_Channel4;
    hdma_usart1_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
```

```
    hdma_usart1_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_tx.Init.Mode = DMA_NORMAL;
    hdma_usart1_tx.Init.Priority = DMA_PRIORITY_MEDIUM;
    HAL_DMA_Init(&hdma_usart1_tx);
    __HAL_LINKDMA(&huart1, hdmatx, hdma_usart1_tx);
    HAL_NVIC_SetPriority(DMA1_Channel5_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(DMA1_Channel5_IRQn);
    HAL_NVIC_SetPriority(DMA1_Channel4_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(DMA1_Channel4_IRQn);
    HAL_UART_Receive_DMA(&huart1, rx_buffer, RING_BUFFER_SIZE);
}

void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz
    htim2.Init.Period = 499; // 500ms
    HAL_TIM_Base_Init(&htim2);
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}

void ring_buffer_write(uint8_t *data, uint32_t len) {
    for (uint32_t i = 0; i < len; i++) {
        rx_buffer[head] = data[i];
        head = (head + 1) & (RING_BUFFER_SIZE - 1);
    }
}
uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len) {
    uint32_t count = 0;
    while (tail != head && count < max_len) {
        data[count++] = rx_buffer[tail];
        tail = (tail + 1) & (RING_BUFFER_SIZE - 1);
    }
    return count;
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    head = RING_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart1_rx);
}

void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart) {
    head = RING_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart1_rx);
}

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {    tx_busy = 0;}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2 && !tx_busy) {
        uint8_t data[64];
        uint32_t len = ring_buffer_read(data, sizeof(data));
        if (len > 0) {
            tx_busy = 1;
            HAL_UART_Transmit_DMA(&huart1, data, len);
            char msg[32];
            snprintf(msg, sizeof(msg), "Echoed %lu bytes\r\n", len);
            HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        }
    }
}

int main(void) {
    HAL_Init();
    init_gpio();
```

```
    init_uart();
    init_dma();
    init_timer();
    char msg[] = "DMA UART Ring Buffer Started\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
    while (1) {
        // Main loop empty; handled by DMA and TIM2
    }
}

void DMA1_Channel5_IRQHandler(void) {
    HAL_DMA_IRQHandler(&hdma_usart1_rx);
}

void DMA1_Channel4_IRQHandler(void) {
    HAL_DMA_IRQHandler(&hdma_usart1_tx);
}

void TIM2_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim2);
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const DMA_CCR5 = (volatile uint32_t *)0x40020048;
volatile uint32_t *const DMA_CNDTR5 = (volatile uint32_t *)0x4002004C;
volatile uint32_t *const DMA_CPAR5 = (volatile uint32_t *)0x40020050;
volatile uint32_t *const DMA_CMAR5 = (volatile uint32_t *)0x40020054;
volatile uint32_t *const TIM2_SR = (volatile uint32_t *)0x40000010;
#define RING_BUFFER_SIZE 256
uint8_t rx_buffer[RING_BUFFER_SIZE];
uint8_t tx_buffer[RING_BUFFER_SIZE];
volatile uint32_t head = 0, tail = 0;
volatile uint8_t tx_busy = 0;

void init_dma_uart(void) {
    *DMA_CPAR5 = (uint32_t)&USART_DR;
    *DMA_CMAR5 = (uint32_t)rx_buffer;
    *DMA_CNDTR5 = RING_BUFFER_SIZE;
    *DMA_CCR5 = (1U << 7) | (0U << 4) | (1U << 5) | (1U << 0); // Circular, periph-to-mem, enable
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}
uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len) {
    uint32_t count = 0;
    while (tail != head && count < max_len) {
        data[count++] = rx_buffer[tail];
        tail = (tail + 1) & (RING_BUFFER_SIZE - 1);
    }
    return count;
}



void simulate_rx_data(void) {
    static uint8_t sim_data[] = {'A', 'B', 'C'};
    static uint32_t idx = 0;
```

```
    rx_buffer[head] = sim_data[idx++ % 3];
    head = (head + 1) & (RING_BUFFER_SIZE - 1);
    *DMA_CNDTR5 = RING_BUFFER_SIZE - head;
}

int main() {
    init_dma_uart();
    uart_transmit_string("DMA UART Ring Buffer Started\n");
    for (int i = 0; i < 3; i++) {
        simulate_rx_data();
        uint8_t data[64];
        uint32_t len = ring_buffer_read(data, sizeof(data));
        if (len > 0) {
            for (uint32_t j = 0; j < len; j++) {
                while (!(*USART_SR & (1U << 7)));
                *USART_DR = data[j];
            }
            char msg[32];
            snprintf(msg, sizeof(msg), "Echoed %lu bytes\n", len);
            uart_transmit_string(msg);
        }
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX, 115200 baud), DMA1 (Channel 4 for TX, Channel 5 for RX, circular for RX), and TIM2 (500ms interrupts).
  - Connect USB-to-serial adapter to PA9/PA10; test with a terminal (e.g., PuTTY).
  - Send characters via terminal and verify echoed data and buffer status logs.

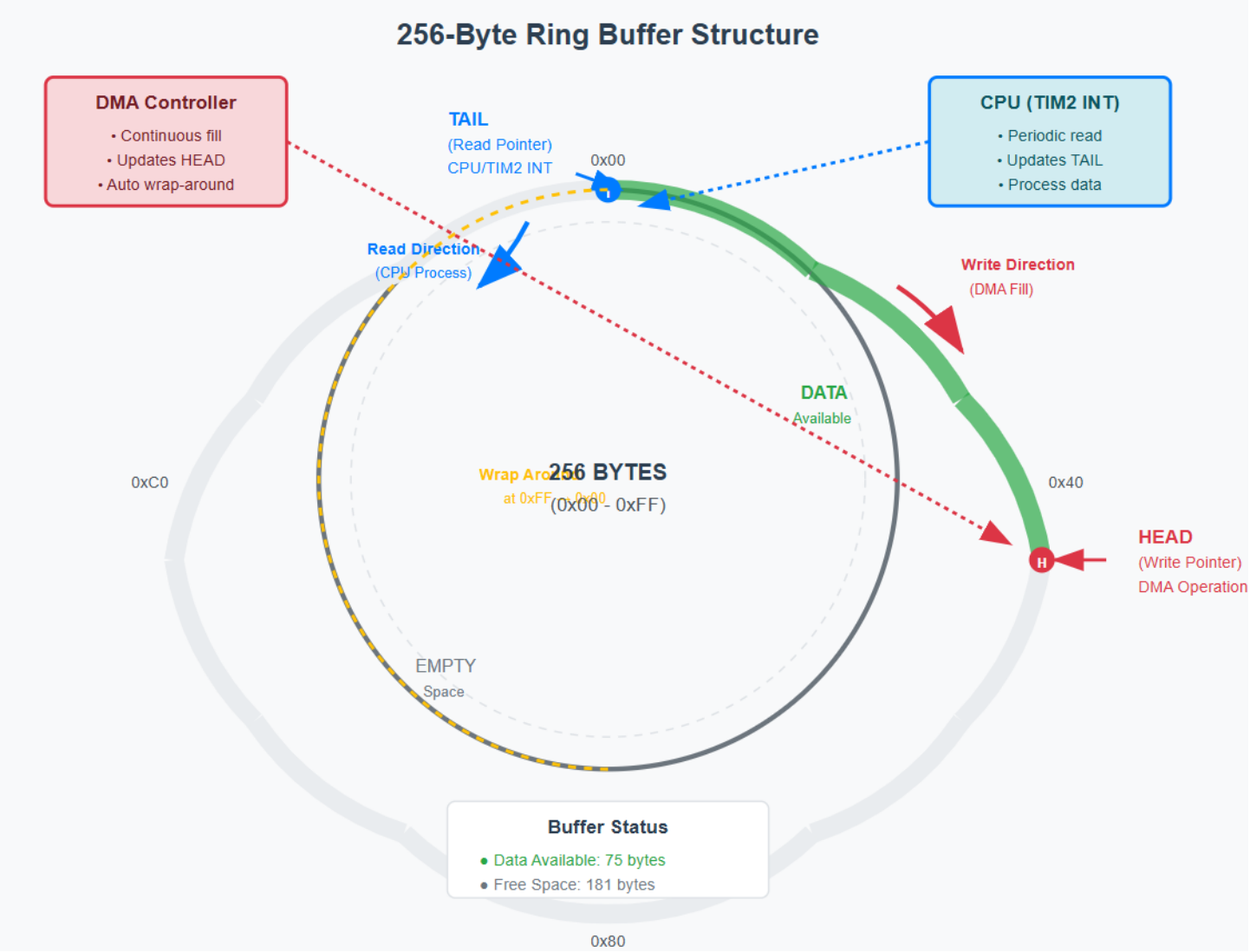


Image: Ring Buffer Structure

Description: A diagram showing a 256-byte ring buffer with head and tail pointers. Arrows indicate write (head) and read (tail) operations, wrapping around at buffer end. Labels show DMA filling buffer and CPU reading data in TIM2 interrupt.

- **Optimizations**:
  - DMA circular mode for RX ensures continuous data capture.
  - Interrupt-driven TX prevents main loop blocking.
  - Timer-driven buffer checks balance responsiveness and CPU load.
  - Power-of-2 buffer size for efficient index wrapping.
- **Embedded Tip**: Monitor DMA_CNDTRx to detect buffer overruns; adjust buffer size based on data rate.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for UART/DMA/TIM2 setup.
- **STM32CubeMX**: Configure USART1, DMA1, and TIM2.

- **Debugging**: Use STM32CubeIDE's debugger to inspect DMA1->CNDTR5 and rx_buffer; simulate with GCC (gcc program.c -o program).

Resources

- **STM32 DMA Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **YouTube DMA Tutorials**: Search "STM32 DMA tutorial" (e.g., STMicroelectronics, Neso Academy).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add error handling for DMA transfer errors (TE).
- **Optimize**: Increase buffer size to 512 bytes for high data rates.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Understand Bootloader Basics

Goal

Master bootloader design and implementation for firmware updates in embedded systems, focusing on creating a custom bootloader that enables flashing new firmware via UART on an STM32 microcontroller.

Tasks

- **Study**: Learn bootloader concepts, design principles, and firmware update mechanisms.
- **Practice**: Write a simple bootloader to receive and flash firmware.
- **Project**: Develop a custom bootloader for firmware flashing via UART, with basic error checking.
- **Tools**: STM32CubeIDE.
- **Resources**: embedded.com bootloader articles, STM32 AN2606 (Application Note on bootloader).

Detailed Notes

1. Bootloader Overview

- **Definition**: A bootloader is a small program stored in a microcontroller's flash memory that runs at startup to initialize hardware, check for update conditions, and either load the main application or facilitate firmware updates.
- **Key Functions**:
  - **Initialization**: Configure essential peripherals (e.g., UART, clocks).
  - **Update Check**: Detect if a firmware update is requested (e.g., via button press or command).
  - **Firmware Update**: Receive new firmware (e.g., via UART, I2C, USB), validate, and write to flash.
  - **Jump to Application**: Transfer control to the main application if no update is needed.
- **STM32 Bootloader Specifics**:
  - **Memory Layout**: Bootloader resides at the start of flash (e.g., 0x08000000); application follows (e.g., 0x08004000).
  - **Flash Programming**: Use STM32 flash controller to erase and write sectors.
  - **System Boot**: STM32 boots from flash (BOOT0/BOOT1 pins configure boot mode).
- **Embedded Tip**: Protect bootloader region in flash to prevent accidental overwrites; use a jump table for robust application transitions.

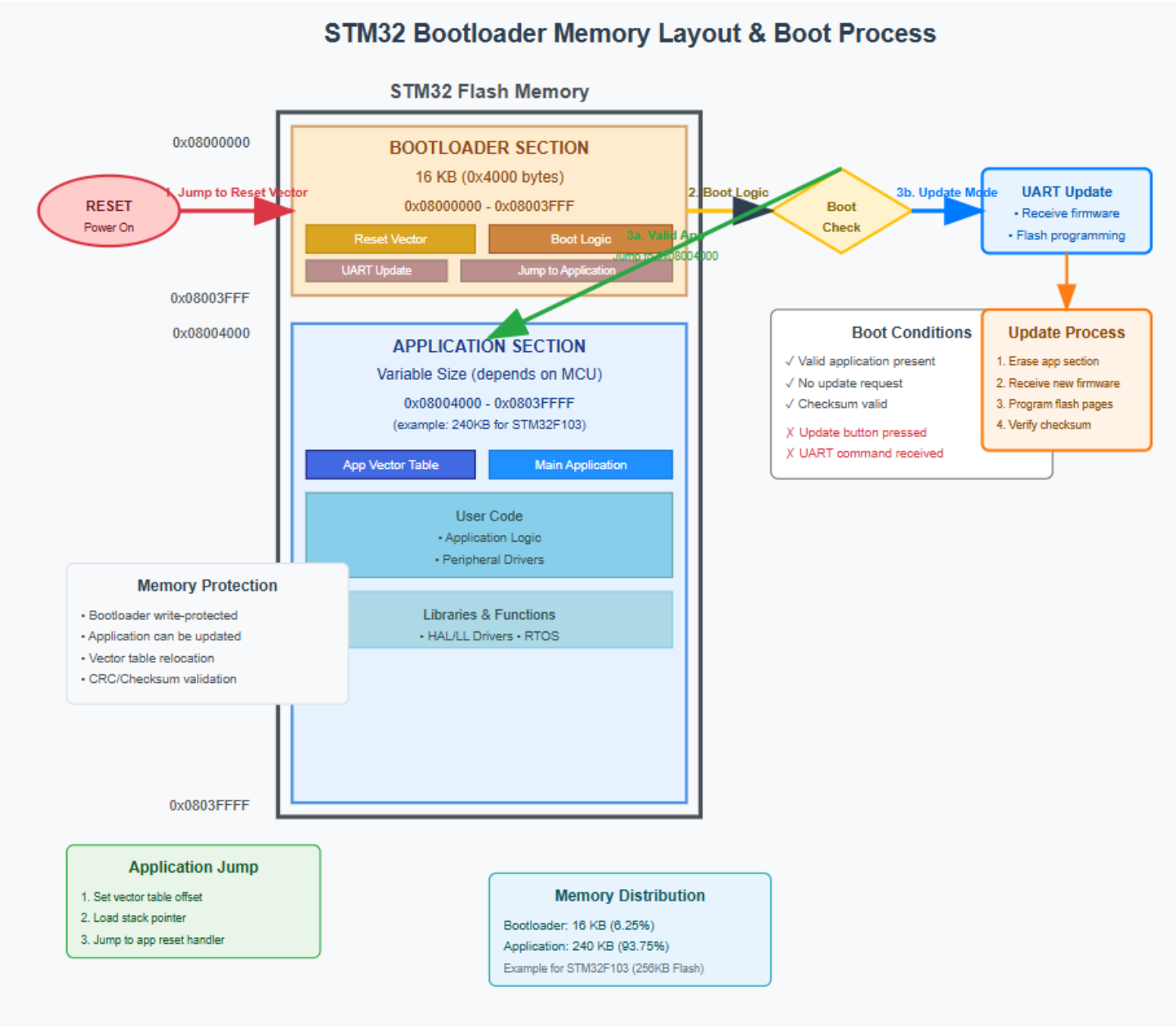


Image: Bootloader Memory Layout

Description: A diagram showing STM32 flash memory divided into bootloader (0x08000000–0x08003FFF) and application (0x08004000–end). Arrows indicate boot process: reset vector to bootloader, conditional jump to application, or UART firmware update path.

2. Bootloader Design Principles

- **Reliability**: Include error checking (e.g., CRC, checksum) to validate firmware.
- **Security**: Lock flash after programming; use simple authentication for updates.
- **Simplicity**: Minimize bootloader size to fit in limited flash (e.g., 16KB).
- **Flexibility**: Support multiple interfaces (e.g., UART, USB) for updates.
- **Key STM32 Registers**:
  - **FLASH_CR**: Control register (enable erase/write, lock/unlock flash).
  - **FLASH_AR**: Address for erase/write operations.
  - **FLASH_SR**: Status register (BSY, EOP for end of operation).

- **Embedded Tip**: Use a timeout to jump to the application if no update is requested; store bootloader in a separate flash sector.

3. Firmware Update Process

- **Steps**:
    1. Bootloader starts, initializes UART.
    2. Check for update trigger (e.g., button on PA0 or UART command).
    3. Receive firmware data via UART (e.g., binary or hex format).
    4. Erase application flash sector(s).
    5. Write received data to flash.
    6. Verify data (e.g., checksum).
    7. Jump to application.
- **Jump to Application**:
    - Set stack pointer to application's reset vector (e.g., *(uint32_t *)0x08004000).
    - Call application's reset handler (e.g., *(uint32_t *)0x08004004).
- **Embedded Tip**: Use a simple protocol (e.g., XMODEM, or custom packet format) for UART transfers to ensure reliability.

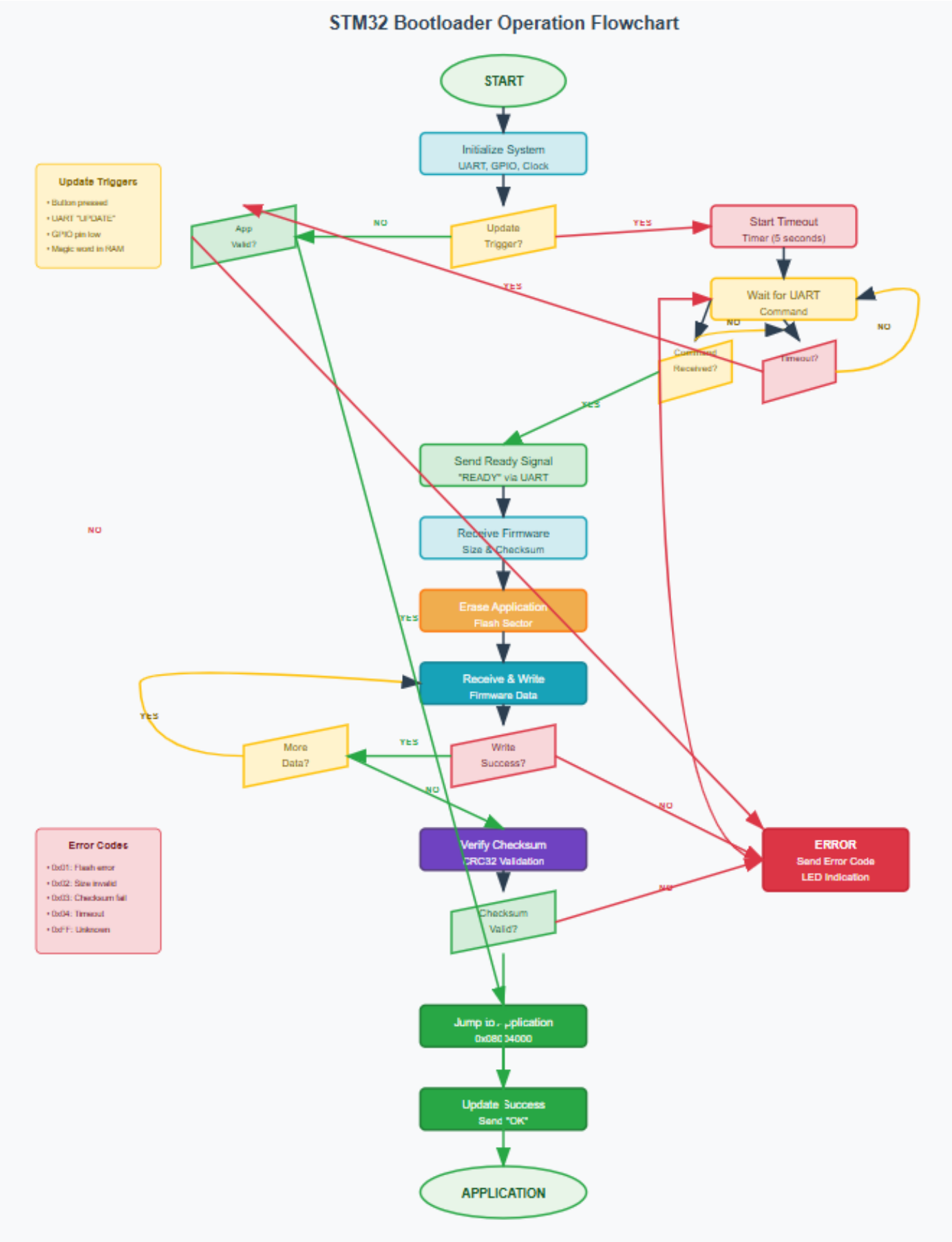


Image: Bootloader Flowchart

Description: A flowchart showing bootloader operation: initialize UART/GPIO, check for update trigger (button or command), receive firmware via UART, erase flash, write data, verify checksum, jump to application or loop on error. Includes timeout path to application.

4. Practice Program: Basic Bootloader

- **Objective**: Write a bootloader that checks a button (PA0) and jumps to a dummy application or waits.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>
volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_IDR = (volatile uint32_t *)0x40010008;
#define APP_ADDRESS 0x08004000
void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 input
}

void jump_to_app(void) {
    uint32_t *app_sp = (uint32_t *)APP_ADDRESS; // Stack pointer
    uint32_t *app_reset = (uint32_t *)(APP_ADDRESS + 4); // Reset handler
    __asm volatile ("msr msp, %0" : : "r" (*app_sp)); // Set stack
    ((void (*)(void)) *app_reset)(); // Jump
}

void delay_ms(uint32_t ms) {
    for (volatile uint32_t i = 0; i < ms * 16000; i++); // Simulated delay
}

int main() {
    init_gpio();
    if (*GPIO_IDR & (1U << 0)) { // PA0 high (button pressed)
        printf("Update mode: Waiting for firmware...\n");
        delay_ms(5000); // Simulate waiting
    } else {
        printf("Jumping to application...\n");
        jump_to_app();
    }
    return 0;
}
```

Project: Custom Bootloader for Firmware Flashing

- **Objective**: Develop a custom bootloader that receives firmware via UART, writes it to flash (starting at 0x08004000), and jumps to the application. Includes basic checksum verification.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) for firmware reception.
  - Check PA0 (input) to enter update mode.
  - Receive firmware as 64-byte packets with a simple protocol: [size:1][data:64][checksum:1].
  - Erase and write to flash starting at 0x08004000.
  - Jump to application at 0x08004000 after update or on timeout (5s).
- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;
GPIO_InitTypeDef GPIO_InitStruct;
```

```c
#define APP_ADDRESS 0x08004000
#define FIRMWARE_SIZE 0x4000 // 16KB
#define PACKET_SIZE 64

uint8_t rx_buffer[PACKET_SIZE + 2]; // Size + Data + Checksum

volatile uint8_t update_mode = 0;
volatile uint32_t bytes_received = 0;

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
    HAL_UART_Receive_IT(&huart1, rx_buffer, PACKET_SIZE + 2);
}

void init_flash(void) {
    HAL_FLASH_Unlock();
}

void jump_to_app(void) {
    HAL_UART_DeInit(&huart1);
    HAL_FLASH_Lock();
    uint32_t *app_sp = (uint32_t *)APP_ADDRESS;
    uint32_t *app_reset = (uint32_t *)(APP_ADDRESS + 4);
    __set_MSP(*app_sp); // Set stack pointer
    ((void (*)(void)) *app_reset)(); // Jump
}

HAL_StatusTypeDef flash_erase(void) {
    FLASH_EraseInitTypeDef erase_init = {0};
    uint32_t sector_error;
    erase_init.TypeErase = FLASH_TYPEERASE_PAGES;
    erase_init.PageAddress = APP_ADDRESS;
    erase_init.NbPages = FIRMWARE_SIZE / FLASH_PAGE_SIZE;
    return HAL_FLASHEx_Erase(&erase_init, &sector_error);
}

HAL_StatusTypeDef flash_write(uint32_t address, uint8_t *data, uint32_t size) {
    for (uint32_t i = 0; i < size; i += 4) {
        uint32_t word = *(uint32_t *)(data + i);
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) {
            return HAL_ERROR;
        }
    }
    return HAL_OK;
}
```

```c
uint8_t calculate_checksum(uint8_t *data, uint32_t len) {
    uint8_t sum = 0;
    for (uint32_t i = 0; i < len; i++) {
        sum += data[i];
    }
    return sum;
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (rx_buffer[0] != PACKET_SIZE) {
        char msg[] = "Invalid packet size\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        HAL_UART_Receive_IT(&huart1, rx_buffer, PACKET_SIZE + 2);
        return;
    }
    uint8_t checksum = calculate_checksum(&rx_buffer[1], PACKET_SIZE);
    if (checksum != rx_buffer[PACKET_SIZE + 1]) {
        char msg[] = "Checksum error\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        HAL_UART_Receive_IT(&huart1, rx_buffer, PACKET_SIZE + 2);
        return;
    }
    if (bytes_received == 0) {
        if (flash_erase() != HAL_OK) {
            char msg[] = "Flash erase failed\r\n";
            HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
            return;
        }
    }
    if (flash_write(APP_ADDRESS + bytes_received, &rx_buffer[1], PACKET_SIZE) != HAL_OK) {
        char msg[] = "Flash write failed\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
        return;
    }
    bytes_received += PACKET_SIZE;
    char msg[32];
    snprintf(msg, sizeof(msg), "Received %lu bytes\r\n", bytes_received);
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
    if (bytes_received >= FIRMWARE_SIZE) {
        jump_to_app();
    }
    HAL_UART_Receive_IT(&huart1, rx_buffer, PACKET_SIZE + 2);
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_flash();
    init_uart();
    char msg[] = "Bootloader started\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 10);
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
        update_mode = 1;
        HAL_UART_Receive_IT(&huart1, rx_buffer, PACKET_SIZE + 2);
    }
    uint32_t timeout = 5000; // 5s
    while (timeout-- && update_mode) {
        HAL_Delay(1);
    }
    jump_to_app();
    return 0;
}

void USART1_IRQHandler(void) {
    HAL_UART_IRQHandler(&huart1);
}
```

- **Simulated Version** (For PC testing):

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_IDR = (volatile uint32_t *)0x40010008;
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

#define APP_ADDRESS 0x08004000
#define FIRMWARE_SIZE 0x4000
#define PACKET_SIZE 64

uint8_t rx_buffer[PACKET_SIZE + 2];

volatile uint32_t bytes_received = 0;

uint8_t flash_memory[FIRMWARE_SIZE];

void init_gpio(void) {
    *GPIO_MODER &= ~(3U << (0 * 2)); // PA0 input
}

void init_uart(void) {
    *USART_BRR = 16000000 / 115200;
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2);
}

void uart_transmit_string(const char *str) {
    while (*str) {
        while (!(*USART_SR & (1U << 7)));
        *USART_DR = *str++;
    }
}

void jump_to_app(void) {
    printf("Jumping to application at 0x%08X...\n", APP_ADDRESS);
}

uint8_t calculate_checksum(uint8_t *data, uint32_t len) {
    uint8_t sum = 0;
    for (uint32_t i = 0; i < len; i++) {
        sum += data[i];
    }
    return sum;
}

void flash_write(uint8_t *data, uint32_t size) {
    for (uint32_t i = 0; i < size; i++) {
        flash_memory[bytes_received + i] = data[i];
    }
}

int main() {
    init_gpio();
    init_uart();
    uart_transmit_string("Bootloader started\n");
    if (*GPIO_IDR & (1U << 0)) {
        static uint8_t sim_data[PACKET_SIZE + 2] = {PACKET_SIZE, [1 ... 64] = 0xAA, 0};
        sim_data[PACKET_SIZE + 1] = calculate_checksum(&sim_data[1], PACKET_SIZE);
        for (int i = 0; i < 2; i++) {
            if (sim_data[0] != PACKET_SIZE) {
                uart_transmit_string("Invalid packet size\n");
                continue;
            }
```

```
            if (calculate_checksum(&sim_data[1], PACKET_SIZE) != sim_data[PACKET_SIZE + 1]) {
                uart_transmit_string("Checksum error\n");
                continue;
            }
            flash_write(&sim_data[1], PACKET_SIZE);
            bytes_received += PACKET_SIZE;
            char msg[32];
            snprintf(msg, sizeof(msg), "Received %lu bytes\n", bytes_received);
            uart_transmit_string(msg);
            if (bytes_received >= FIRMWARE_SIZE) {
                jump_to_app();
                break;
            }
        }
    } else {
        jump_to_app();
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX, 115200 baud), GPIO (PA0 input).
  - Linker script: Set bootloader at 0x08000000, application at 0x08004000 (modify FLASH origin in .ld file).
  - Connect USB-to-serial adapter to PA9/PA10; use PA0 with pull-down resistor for update trigger.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY). Send 64-byte packets with a Python script or terminal tool.
- **Optimizations**:
  - Interrupt-driven UART RX for non-blocking operation.
  - Simple checksum for error detection.
  - Flash erase/write in small chunks to manage memory constraints.
- **Embedded Tip**: Use STM32CubeProgrammer to flash bootloader; test with a dummy application at 0x08004000.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for UART/GPIO setup.
- **STM32CubeMX**: Configure USART1 and GPIO; adjust linker script for memory layout.
- **Debugging**: Use STM32CubeIDE's debugger to inspect FLASH->SR and rx_buffer.

Resources

- **embedded.com Bootloader Articles**: Search "bootloader design embedded" on https://www.embedded.com/.
- **STM32 AN2606**: Download from https://www.st.com (Application Note on STM32 bootloader).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a more robust protocol (e.g., XMODEM) for firmware transfer.
- **Optimize**: Implement CRC32 instead of simple checksum.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Optimize Stack/Heap

Goal

Master stack and heap optimization techniques for embedded systems, analyzing memory usage in projects to ensure efficient resource allocation, with a focus on developing a memory-optimized LED driver.

Tasks

- **Study**: Learn stack and heap concepts, optimization strategies, and memory usage analysis techniques.
- **Practice**: Analyze and optimize memory usage in existing code.
- **Project**: Develop a memory-optimized LED driver with minimal stack/heap usage, using static allocation and efficient data structures.
- **Tools**: STM32CubeIDE, GDB (debugger).
- **Resources**: ARM memory management docs, embedded.com articles.

Detailed Notes

1. Stack and Heap Overview

- **Stack**:
  - **Definition**: A Last-In-First-Out (LIFO) memory region for function call frames, local variables, and return addresses.
  - **Characteristics**: Fixed size (defined in linker script, e.g., 1KB), fast allocation/deallocation, automatically managed.
  - **Issues**: Stack overflow if too many nested function calls or large local variables.
- **Heap**:
  - **Definition**: A memory region for dynamic allocation (e.g., via malloc/free).
  - **Characteristics**: Flexible size, slower allocation, risk of fragmentation, requires manual management.
  - **Issues**: Limited heap in embedded systems (e.g., 2KB); fragmentation reduces available memory.
- **Embedded Constraints**:
  - Microcontrollers have limited SRAM (e.g., 20KB on STM32F1).
  - Stack/heap share SRAM, requiring careful allocation to avoid conflicts.
- **Embedded Tip**: Prefer static allocation over dynamic to avoid heap fragmentation; use stack analysis tools to detect overflows.

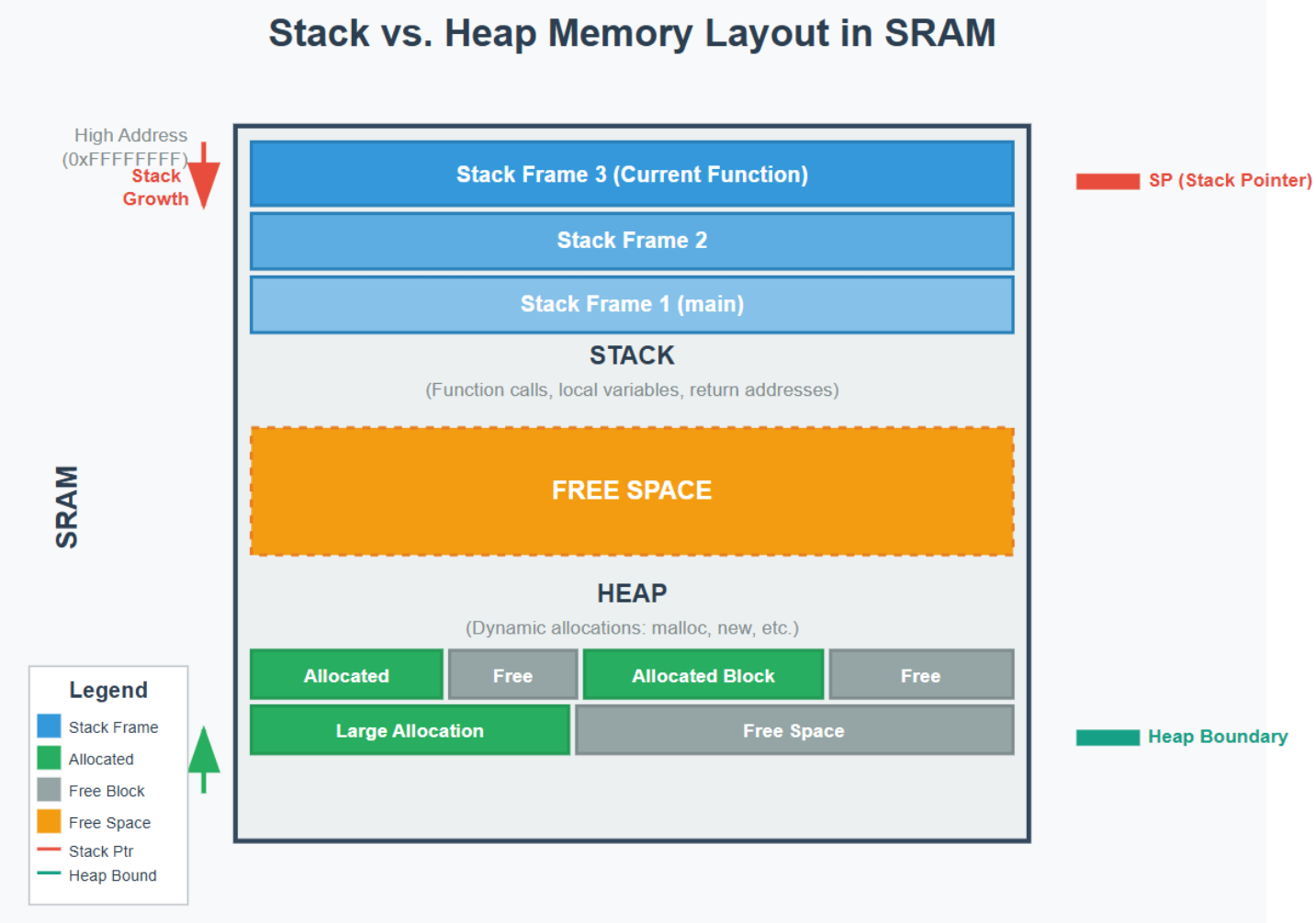


Image: Stack vs. Heap Memory Layout

Description: A diagram showing SRAM divided into stack (top-down growth) and heap (bottom-up growth). Labels indicate stack frames (function calls, local variables) and heap blocks (dynamic allocations). Arrows show stack pointer (SP) and heap boundary.

2. Stack Optimization Techniques

- **Minimize Local Variables**: Use smaller data types (e.g., uint8_t instead of uint32_t) and avoid large arrays in functions.
- **Reduce Function Nesting**: Limit recursive calls or deep call stacks.
- **Static Allocation**: Use static or global variables for large data to avoid stack usage.
- **Stack Size Tuning**: Adjust stack size in linker script (e.g., STACK_SIZE = 0x400) based on analysis.
- **Embedded Tip**: Use GDB to monitor stack pointer (sp) and estimate usage during debugging.

3. Heap Optimization Techniques

- **Avoid Dynamic Allocation**: Use fixed-size buffers instead of malloc in resource-constrained systems.
- **Pool Allocators**: Preallocate fixed-size blocks for specific data types to reduce fragmentation.
- **Monitor Usage**: Track heap usage with custom allocators or debugging tools.
- **Embedded Tip**: If malloc is needed, implement a simple memory pool to control fragmentation.

4. Memory Usage Analysis

- **Tools**:
  - **STM32CubeIDE Map File**: Check .map file for stack/heap sizes and variable allocations.
  - **GDB**: Inspect stack pointer (info registers sp) and memory contents.
  - **Static Analysis**: Use tools like arm-none-eabi-size to analyze .bss, .data, and .text sizes.
- **Techniques**:
  - Calculate stack usage by filling stack with a known pattern (e.g., 0xAA) and checking corruption.
  - Estimate worst-case stack depth with call graph analysis.
- **Embedded Tip**: Use __attribute__((section(".noinit"))) for uninitialized variables to save .bss space.

Chart: Stack Usage Analysis

```
{
  "type": "bar",
  "data": {
    "labels": ["Main", "ISR", "Function1", "Function2"],
    "datasets": [{
      "label": "Stack Usage (Bytes)",
      "data": [200, 100, 150, 80],
      "backgroundColor": ["#4CAF50", "#2196F3", "#FF9800", "#F44336"],
      "borderColor": ["#388E3C", "#1976D2", "#F57C00", "#D32F2F"],
      "borderWidth": 1
    }]
  },
  "options": {
    "scales": {
      "y": {
        "beginAtZero": true,
        "title": {
          "display": true,
          "text": "Stack Usage (Bytes)"
        }
      },
      "x": {
        "title": {
          "display": true,
          "text": "Code Section"
        }
      }
    },
    "plugins": {
      "title": {
        "display": true,
        "text": "Stack Usage by Code Section"
      }
    }
  }
}
```

Description: A bar chart showing estimated stack usage for different code sections (e.g., main loop, ISR, functions) in a typical embedded project. Each bar represents bytes used, helping identify high-stack-usage areas.

5. Practice Programs

a. Stack Usage Monitoring

- Fill stack with a pattern and monitor usage.

- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

#define STACK_SIZE 512
uint8_t stack[STACK_SIZE] __attribute__((section(".stack")));

void fill_stack(void) {
    for (uint32_t i = 0; i < STACK_SIZE; i++) {
        stack[i] = 0xAA;
    }
}
uint32_t check_stack_usage(void) {
    uint32_t used = 0;
    for (uint32_t i = 0; i < STACK_SIZE; i++) {
        if (stack[i] != 0xAA) {
            used = STACK_SIZE - i;
            break;
        }
    }
    return used;
}

void dummy_function(void) {
    uint8_t buffer[100]; // Simulate stack usage
    for (uint32_t i = 0; i < 100; i++) {
        buffer[i] = i;
    }
    printf("Dummy function used %lu bytes\n", check_stack_usage());
}

int main() {
    fill_stack();
    dummy_function();
    printf("Total stack used: %lu bytes\n", check_stack_usage());
    return 0;
}
```

b. Static Allocation Example

- Use static variables to avoid stack overflow.
- **Code** (STM32 simulation):

```
#include <stdio.h>
#include <stdint.h>

static uint8_t buffer[100]; // Static to avoid stack
void process_data(void) {
    for (uint32_t i = 0; i < 100; i++) {
        buffer[i] = i;
    }
    printf("Processed data in static buffer\n");
}

int main() {
    process_data();
    return 0;
}
```

Project: Memory-Optimized LED Driver

- **Objective**: Develop a memory-optimized LED driver for controlling multiple LEDs (PA5, PA6, PA7) with minimal stack/heap usage, using static allocation and efficient state management.
- **Requirements**:
  - Configure PA5, PA6, PA7 as outputs for LEDs.
  - Implement a state machine for LED patterns (e.g., sequential blink, all on/off).
  - Use static buffers and minimal stack variables.
  - Log status via UART (PA9-TX, 115200 baud).
  - Analyze stack usage with GDB and .map file.
- **Code** (STM32 with HAL):

```
#include "stm32f1xx_hal.h"

UART_HandleTypeDef huart1;

TIM_HandleTypeDef htim2;

typedef enum { SEQ_BLINK, ALL_ON, ALL_OFF } LedState_t;

static LedState_t led_state = SEQ_BLINK;
static const uint8_t led_pins[] = {GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7};
static uint8_t led_index = 0;
static char log_buffer[32]; // Static to avoid stack

void init_gpio(void) {
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_RESET);
}

void init_uart(void) {
    __HAL_RCC_USART1_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    HAL_UART_Init(&huart1);
}

void init_timer(void) {
    __HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999; // 1kHz
    htim2.Init.Period = 499; // 500ms
    HAL_TIM_Base_Init(&htim2);
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}
```

```c
void log_state(LedState_t state, uint8_t index) {
    const char *state_str = (state == SEQ_BLINK) ? "Seq" : (state == ALL_ON) ? "All On" : "All Off";
    snprintf(log_buffer, sizeof(log_buffer), "State: %s, LED: %u\r\n", state_str, index);
    HAL_UART_Transmit(&huart1, (uint8_t *)log_buffer, strlen(log_buffer), 10);
}

void update_leds(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_RESET);
    switch (led_state) {
        case SEQ_BLINK:
            HAL_GPIO_WritePin(GPIOA, led_pins[led_index], GPIO_PIN_SET);
            led_index = (led_index + 1) % 3;
            break;
        case ALL_ON:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_SET);
            break;
        case ALL_OFF:
            break;
    }
    log_state(led_state, led_index);
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        static uint32_t counter = 0;
        update_leds();
        if (++counter % 10 == 0) { // Change state every 5s
            led_state = (led_state + 1) % 3;
            led_index = 0;
        }
    }
}

int main(void) {
    HAL_Init();
    init_gpio();
    init_uart();
    init_timer();
    while (1) {
        // Main loop empty; handled by TIM2 ISR
    }
}

void TIM2_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim2);
}
```

- **Simulated Version** (For PC testing):

```c
#include <stdio.h>
#include <stdint.h>

volatile uint32_t *const GPIO_MODER = (volatile uint32_t *)0x40010000;
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
volatile uint32_t *const TIM2_SR = (volatile uint32_t *)0x40000010;

typedef enum { SEQ_BLINK, ALL_ON, ALL_OFF } LedState_t;

static LedState_t led_state = SEQ_BLINK;
static const uint8_t led_pins[] = {5, 6, 7};
static uint8_t led_index = 0;
static char log_buffer[32];
```

```c
void init_gpio(void) {
    *GPIO_MODER |= (1U << (5 * 2)) | (1U << (6 * 2)) | (1U << (7 * 2)); // PA5,6,7 output
    *GPIO_BSRR = (1U << (5 + 16)) | (1U << (6 + 16)) | (1U << (7 + 16)); // Off
}

void log_state(LedState_t state, uint8_t index) {
    const char *state_str = (state == SEQ_BLINK) ? "Seq" : (state == ALL_ON) ? "All On" : "All Off";
    snprintf(log_buffer, sizeof(log_buffer), "State: %s, LED: %u\n", state_str, index);
    printf("%s", log_buffer);
}

void update_leds(void) {
    *GPIO_BSRR = (1U << (5 + 16)) | (1U << (6 + 16)) | (1U << (7 + 16));
    switch (led_state) {
        case SEQ_BLINK:
            *GPIO_BSRR = (1U << led_pins[led_index]);
            led_index = (led_index + 1) % 3;
            break;
        case ALL_ON:
            *GPIO_BSRR = (1U << 5) | (1U << 6) | (1U << 7);
            break;
        case ALL_OFF:
            break;
    }
    log_state(led_state, led_index);
}

int main() {
    init_gpio();
    for (int i = 0; i < 15; i++) {
        static uint32_t counter = 0;
        update_leds();
        if (++counter % 10 == 0) {
            led_state = (led_state + 1) % 3;
            led_index = 0;
        }
    }
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure PA5, PA6, PA7 (outputs), USART1 (PA9-TX, 115200 baud), TIM2 (500ms interrupts).
  - Connect LEDs (with resistors) to PA5, PA6, PA7.
  - Test with STM32CubeIDE and a terminal (e.g., PuTTY) via USB-to-serial adapter.
- **Optimizations**:
  - Static variables (led_pins, log_buffer) avoid stack usage.
  - Minimal local variables in functions to reduce stack frame size.
  - Timer-driven FSM for efficient state management.
  - No heap usage (malloc avoided).
- **Stack Analysis**:
  - Use GDB: Set breakpoint in update_leds, check info registers sp.
  - Check .map file for stack size (e.g., STACK_SIZE = 0x400).
  - Estimated stack usage: ~50 bytes (main, ISR, small locals).
- **Embedded Tip**: Use arm-none-eabi-size to verify .bss and .data sizes; reduce stack size in linker script if analysis confirms low usage.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for GPIO/UART/TIM2 setup.
- **GDB**: Debug with arm-none-eabi-gdb in STM32CubeIDE; inspect stack with info stack or x/10xw $sp.
- **STM32CubeMX**: Configure GPIO, USART1, TIM2; check linker script for stack/heap settings.

Resources

- **ARM Memory Management Docs**: Search "ARM Cortex-M memory management" on https://developer.arm.com.
- **embedded.com Articles**: Search "stack heap optimization embedded" on https://www.embedded.com/.
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a large local array in update_leds and check stack usage increase.
- **Optimize**: Move log_buffer to .noinit section to save .bss space.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice programs, ~1 hour on the project.

Learn MISRA and Static Analysis

Goal

Master MISRA C guidelines and static analysis techniques to develop robust, production-ready embedded code, focusing on creating a MISRA-compliant UART driver with minimal static analysis warnings.

Tasks

- **Study**: Learn MISRA C guidelines, their application in embedded systems, and static analysis tools.
- **Practice**: Apply MISRA rules to existing code and analyze with a static analysis tool.
- **Project**: Develop a MISRA-compliant UART driver for STM32, ensuring compliance with key MISRA rules and verifying with static analysis.
- **Tools**: STM32CubeIDE, PC-lint (or equivalent, e.g., Cppcheck).
- **Resources**: MISRA C guidelines, PC-lint documentation.

Detailed Notes

1. MISRA C Guidelines Overview

- **Definition**: MISRA C is a set of coding guidelines for C programming to improve code safety, security, and portability in safety-critical systems (e.g., automotive, aerospace, embedded).
- **Key Rules** (MISRA C:2012, selected for embedded):
  - **Rule 1.1**: Code must comply with C99 standard (avoid non-standard extensions).
  - **Rule 8.6**: All declarations must be at file scope or block scope start (avoid mid-block declarations).
  - **Rule 10.1**: Operands must not be used in ways that violate their types (e.g., no implicit conversions).
  - **Rule 11.4**: No casting between pointers and non-pointers (e.g., avoid casting integers to pointers).
  - **Rule 12.1**: Parentheses must clarify operator precedence.
  - **Rule 15.6**: All loops and conditionals must use braces {}.
  - **Rule 17.7**: Functions must return values explicitly (no implicit returns).
  - **Rule 20.3**: Avoid dynamic memory allocation (malloc/free) in embedded systems.
- **Categories**:
  - **Mandatory**: Must be followed.
  - **Required**: Must be followed unless justified deviation.
  - **Advisory**: Recommended but optional.
- **Embedded Tip**: Focus on required rules for compliance; document deviations with rationale for auditing.

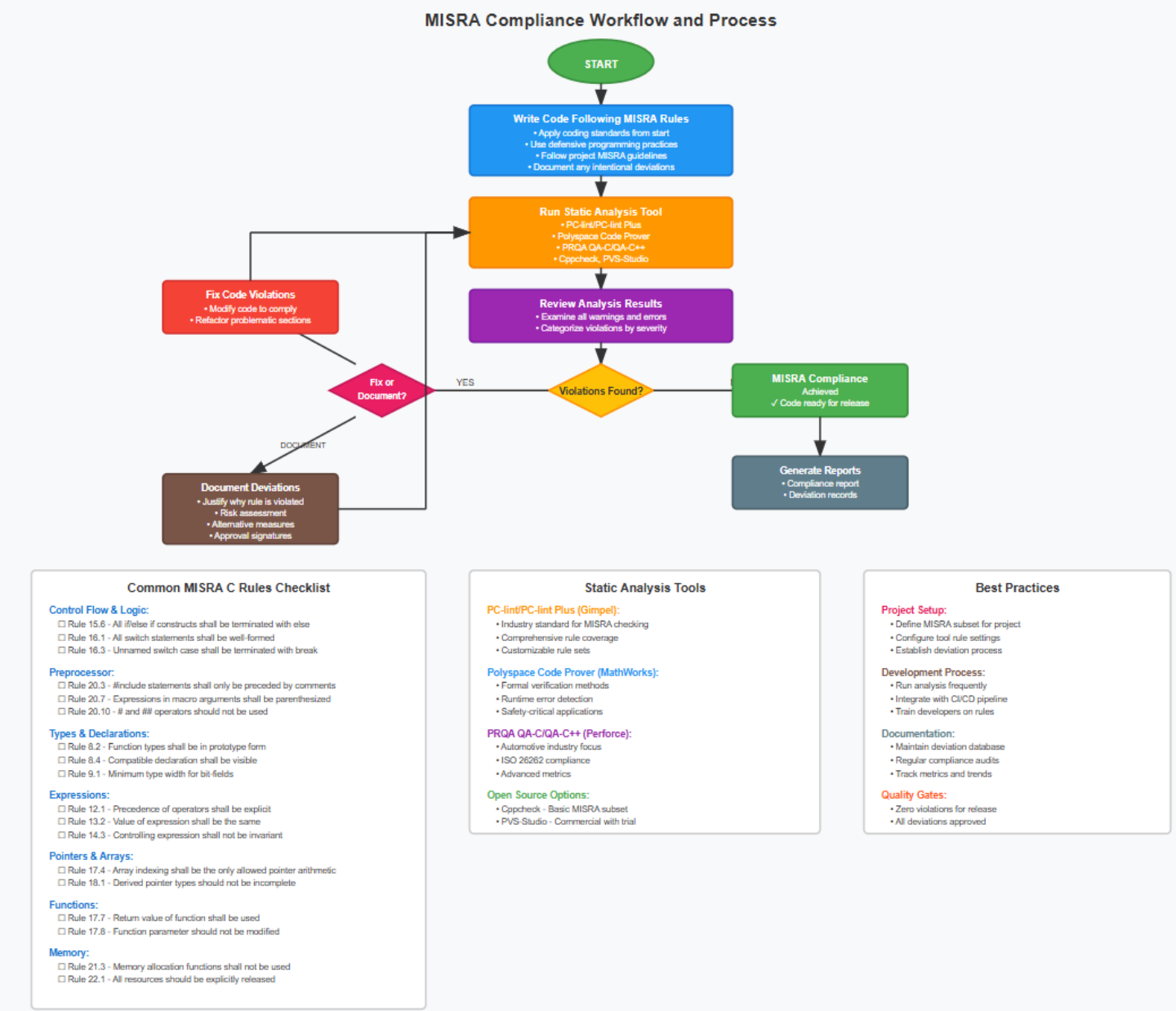


Image: MISRA Compliance Workflow

Description: A flowchart showing the MISRA compliance process: write code following MISRA rules, run static analysis (e.g., PC-lint), review warnings, fix violations or document deviations, and re-analyze. Includes a checklist of common rules (e.g., Rule 15.6, Rule 20.3).

2. Static Analysis Tools

- **Purpose**: Automatically check code for MISRA compliance, potential bugs, and style issues.
- **Tools**:
  - **PC-lint**: Industry-standard for MISRA checking; configurable for STM32 projects.
  - **Cppcheck**: Open-source alternative, supports MISRA C:2012 (partial).
  - **STM32CubeIDE Integrated Tools**: Limited MISRA checking via plugins.
- **Setup**:
  - Configure PC-lint with MISRA C:2012 ruleset (au-misra3.lnt).
  - Run: lint-nt -b misra.lnt project.c to generate warnings.

    - Review warnings and fix violations or suppress with justification (e.g., //lint -e{123} justified deviation).
- **Common Issues**:
    - Type mismatches (Rule 10.1).
    - Missing braces (Rule 15.6).
    - Uninitialized variables (Rule 9.1).
- **Embedded Tip**: Integrate static analysis into build process to catch issues early; suppress only well-justified warnings.

3. Practice Program: Apply MISRA to UART Code

- **Objective**: Refactor a simple UART transmit function to comply with MISRA rules.
- **Original Code** (Non-compliant):

```
void uart_send(char *data) {
    while (*data) *USART1->DR = *data++; // Violates Rule 15.6, 10.1
}
```

- **Refactored Code** (MISRA-compliant, heavily commented):

```
/* MISRA-compliant UART transmit function */
#include <stdint.h>
#include "stm32f1xx.h"

/* Function to send string via UART, compliant with MISRA C:2012 */
void uart_send(const uint8_t *data) /* Rule 8.6: Declare at file scope */
{
    /* Rule 15.6: Use braces for all loops */
    while (*data != '\0') {
        /* Rule 10.1: Explicit type casting for register write */
        USART1->DR = (uint32_t)(*data); /* Write byte to UART data register */
        /* Wait for TXE (Transmit Data Register Empty) */
        while ((USART1->SR & USART_SR_TXE) == 0U) {
            /* Empty loop for polling */
        }
        data++; /* Move to next byte */
    }
}
```

4. Project: MISRA-Compliant UART Driver

- **Objective**: Develop a MISRA-compliant UART driver for STM32, supporting initialization and data transmission, verified with static analysis (e.g., Cppcheck).
- **Requirements**:
    - Configure USART1 (PA9-TX, PA10-RX, 115200 baud).
    - Support string transmission with error handling.
    - Comply with key MISRA rules (e.g., 1.1, 8.6, 10.1, 15.6, 17.7, 20.3).
    - Log status via UART for debugging.
    - Use static allocation (no heap).
- **Code** (STM32 with HAL, heavily commented):

```
/* MISRA-compliant UART driver for STM32F1 */
#include "stm32f1xx_hal.h"
#include <stdint.h>
#include <string.h>
```

```c
/* UART handle declared at file scope (Rule 8.6) */
static UART_HandleTypeDef huart1;

/* Static buffer for logging (Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U];

/* Function prototypes */
static void init_gpio(void);
static void init_uart(void);
static void log_message(const uint8_t *message);
static uint32_t uart_transmit(const uint8_t *data, uint32_t size);

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; /* Rule 9.1: Initialize all fields */

    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA9, PA10 as UART alternate function */
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; /* Rule 12.1: Explicit grouping */
    gpio_init.Mode = GPIO_MODE_AF_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init); /* Rule 17.7: Explicit return ignored */
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    /* Enable USART1 clock */
    __HAL_RCC_USART1_CLK_ENABLE();

    /* Configure UART parameters */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200U;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

    /* Initialize UART and check for errors */
    if (HAL_UART_Init(&huart1) != HAL_OK) {
        /* Log error and enter infinite loop */
        const uint8_t *error_msg = (const uint8_t *)"UART Init Failed\r\n";
        (void)uart_transmit(error_msg, strlen((const char *)error_msg)); /* Rule 11.4: Cast to avoid
pointer mismatch */
        while (1) {
            /* Infinite loop on error (Rule 15.6) */
        }
    }
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    /* Rule 15.6: Braces for all blocks */
    if (message != NULL) {
        (void)uart_transmit(message, strlen((const char *)message)); /* Rule 11.4: Cast for strlen */
    }
}
```

```c
/* Transmit data via UART with error handling */
static uint32_t uart_transmit(const uint8_t *data, uint32_t size)
{
    uint32_t result = HAL_OK; /* Rule 9.1: Initialize return value */

    /* Check for valid input */
    if ((data == NULL) || (size == 0U)) {
        return HAL_ERROR; /* Rule 17.7: Explicit return */
    }

    /* Perform UART transmission */
    result = HAL_UART_Transmit(&huart1, (uint8_t *)data, (uint16_t)size, 100U); /* Rule 10.1: Cast to
match HAL types */

    /* Log transmission status */
    if (result != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"UART TX Error\r\n";
        (void)HAL_UART_Transmit(&huart1, (uint8_t *)error_msg, strlen((const char *)error_msg), 10U);
    }

    return result; /* Rule 17.7: Explicit return */
}

/* Main function */
int main(void)
{
    HAL_Init(); /* Initialize HAL library */

    init_gpio(); /* Configure GPIO pins */
    init_uart(); /* Configure UART */

    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"MISRA UART Driver Started\r\n";
    log_message(startup_msg);

    /* Test transmission */
    const uint8_t *test_msg = (const uint8_t *)"Hello, MISRA!\r\n";
    while (1) {
        /* Transmit test message every 1s */
        if (uart_transmit(test_msg, strlen((const char *)test_msg)) == HAL_OK) {
            snprintf((char *)log_buffer, sizeof(log_buffer), "Sent %u bytes\r\n", (unsigned
int)strlen((const char *)test_msg)); /* Rule 10.1: Cast for snprintf */
            log_message(log_buffer);
        }
        HAL_Delay(1000U); /* Delay 1s */
    }

    return 0; /* Rule 17.7: Explicit return */
}
```

- **Simulated Version** (For PC testing, heavily commented):

```c
/* MISRA-compliant UART driver simulation */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

/* Simulated STM32 registers */
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

/* Static buffer for logging (Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U];
```

```c
/* Function prototypes */
static void init_uart(void);
static void log_message(const uint8_t *message);
static uint32_t uart_transmit(const uint8_t *data, uint32_t size);

/* Initialize simulated UART */
static void init_uart(void)
{
    /* Simulate UART enable */
    *USART_CR1 = (1U << 13) | (1U << 3) | (1U << 2); /* Enable UART, TX, RX */
    printf("UART initialized\n");
}

/* Log message via simulated UART */
static void log_message(const uint8_t *message)
{
    /* Rule 15.6: Braces for all blocks */
    if (message != NULL) {
        (void)uart_transmit(message, strlen((const char *)message)); /* Rule 11.4: Cast for strlen */
    }
}

/* Transmit data via simulated UART */
static uint32_t uart_transmit(const uint8_t *data, uint32_t size)
{
    uint32_t result = 0U; /* Rule 9.1: Initialize return value */

    /* Check for valid input */
    if ((data == NULL) || (size == 0U)) {
        return 1U; /* Simulate HAL_ERROR (Rule 17.7) */
    }

    /* Simulate UART transmission */
    for (uint32_t i = 0U; i < size; i++) {
        while (!(*USART_SR & (1U << 7))) { /* Wait for TXE */
            /* Empty loop for polling */
        }
        *USART_DR = data[i]; /* Write byte */
        printf("%c", (char)data[i]); /* Simulate output */
    }

    return result; /* Rule 17.7: Explicit return */
}

/* Main function */
int main(void)
{
    init_uart(); /* Initialize UART */

    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"MISRA UART Driver Started\n";
    log_message(startup_msg);

    /* Test transmission */
    const uint8_t *test_msg = (const uint8_t *)"Hello, MISRA!\n";
    for (uint32_t i = 0U; i < 3U; i++) { /* Rule 15.6: Braces for loop */
        if (uart_transmit(test_msg, strlen((const char *)test_msg)) == 0U) {
            snprintf((char *)log_buffer, sizeof(log_buffer), "Sent %u bytes\n", (unsigned
int)strlen((const char *)test_msg)); /* Rule 10.1: Cast */
            log_message(log_buffer);
        }
    }

    return 0; /* Rule 17.7: Explicit return */
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX, 115200 baud).
  - Connect USB-to-serial adapter to PA9/PA10; test with a terminal (e.g., PuTTY).
  - Run static analysis with Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
  - Verify compliance with MISRA rules (e.g., 1.1, 8.6, 10.1, 15.6, 17.7, 20.3).
- **Optimizations**:
  - Static allocation (log_buffer) avoids heap (Rule 20.3).
  - Explicit type casting for HAL functions (Rule 10.1).
  - Braces for all control structures (Rule 15.6).
  - Error handling for robust operation.
- **Static Analysis**:
  - Run Cppcheck to check for MISRA violations.
  - Example suppression: // cppcheck-suppress misra-c2012-11.4; justified for HAL compatibility.
- **Embedded Tip**: Use a MISRA-compliant header file with type definitions (e.g., stdint.h) to ensure portability.

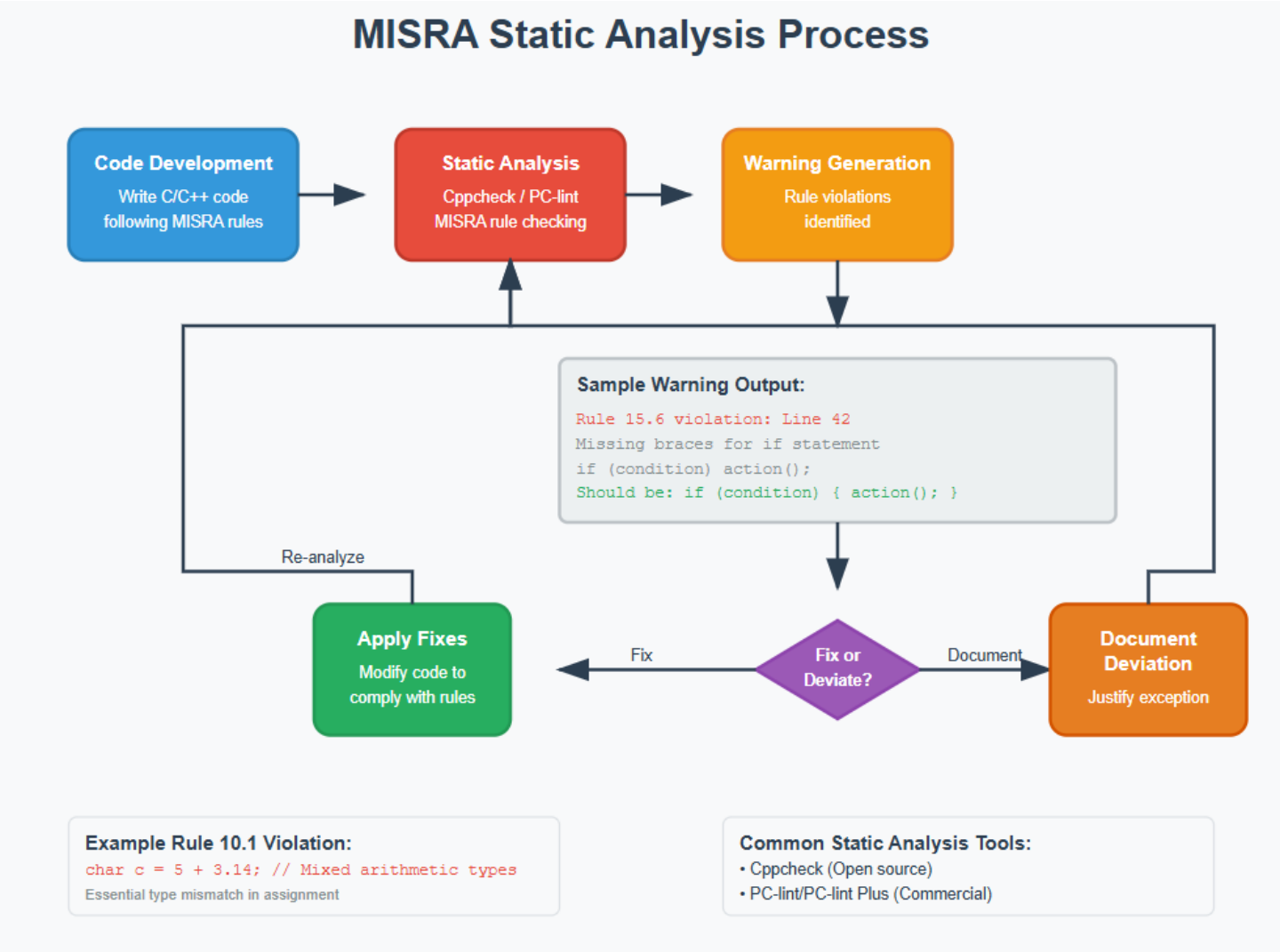


Image: MISRA Static Analysis Process

Description: A diagram showing the static analysis process: code written with MISRA rules, analyzed by Cppcheck/PC-lint, warnings generated (e.g., Rule 10.1 violation), fixes applied or deviations documented, and re-analyzed. Includes a sample warning output for Rule 15.6.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for UART setup.
- **PC-lint/Cppcheck**: Install Cppcheck (sudo apt install cppcheck) or PC-lint; configure with MISRA ruleset.
- **Debugging**: Use STM32CubeIDE's debugger to verify UART operation; check Cppcheck output for warnings.

Resources

- **MISRA C Guidelines**: Purchase MISRA C:2012 from https://www.misra.org.uk or refer to summaries.
- **PC-lint Docs**: https://gimpel.com for setup and MISRA configuration.
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation code with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Introduce a MISRA violation (e.g., omit braces) and verify detection with Cppcheck.
- **Optimize**: Add interrupt-driven UART TX for efficiency while maintaining MISRA compliance.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Consolidate Week 5 Concepts

Goal

Consolidate Week 5 concepts (DMA, bootloaders, stack/heap optimization, MISRA, static analysis) by reviewing notes, debugging prior projects with GDB/OpenOCD, and optimizing the Day 21 UART ring buffer for efficiency, robustness, and MISRA compliance.

Tasks

- **Study**: Review DMA, bootloaders, stack/heap, MISRA, and static analysis concepts.
- **Practice**: Debug Day 21 UART ring buffer using GDB/OpenOCD to identify and fix issues.
- **Project**: Optimize the Day 21 UART ring buffer for performance, memory usage, and MISRA compliance, verified with static analysis.
- **Tools**: STM32CubeIDE, GDB, OpenOCD.
- **Resources**: OpenOCD tutorials, GDB documentation.

Detailed Notes

1. Review of Week 5 Concepts

- **DMA (Day 21)**:
  - Enables low-CPU-overhead data transfers (e.g., UART RX/TX).
  - Circular mode ideal for ring buffers; interrupts (TC, HT) for processing.
  - **Key**: Configure DMA_CCRx for direction, mode; link to peripheral (e.g., USART1->DR).
- **Bootloaders (Day 22)**:
  - Facilitate firmware updates via UART, USB, etc.
  - Erase/write flash, jump to application (e.g., 0x08004000).
  - **Key**: Use robust protocol (e.g., custom packet format) and checksums.
- **Stack/Heap Optimization (Day 23)**:
  - Minimize stack with static allocation, small locals.
  - Avoid heap (malloc) to prevent fragmentation.
  - **Key**: Analyze stack usage with GDB (info registers sp) and .map file.
- **MISRA and Static Analysis (Day 24)**:
  - MISRA C:2012 ensures code safety/portability (e.g., Rule 15.6 for braces, Rule 20.3 no heap).
  - Static analysis (e.g., Cppcheck) catches violations and bugs.
  - **Key**: Document deviations; integrate analysis into build process.
- **Embedded Tip**: Combine DMA with MISRA-compliant code for efficient, safe systems; use GDB/OpenOCD for precise debugging.

2. Debugging with GDB/OpenOCD

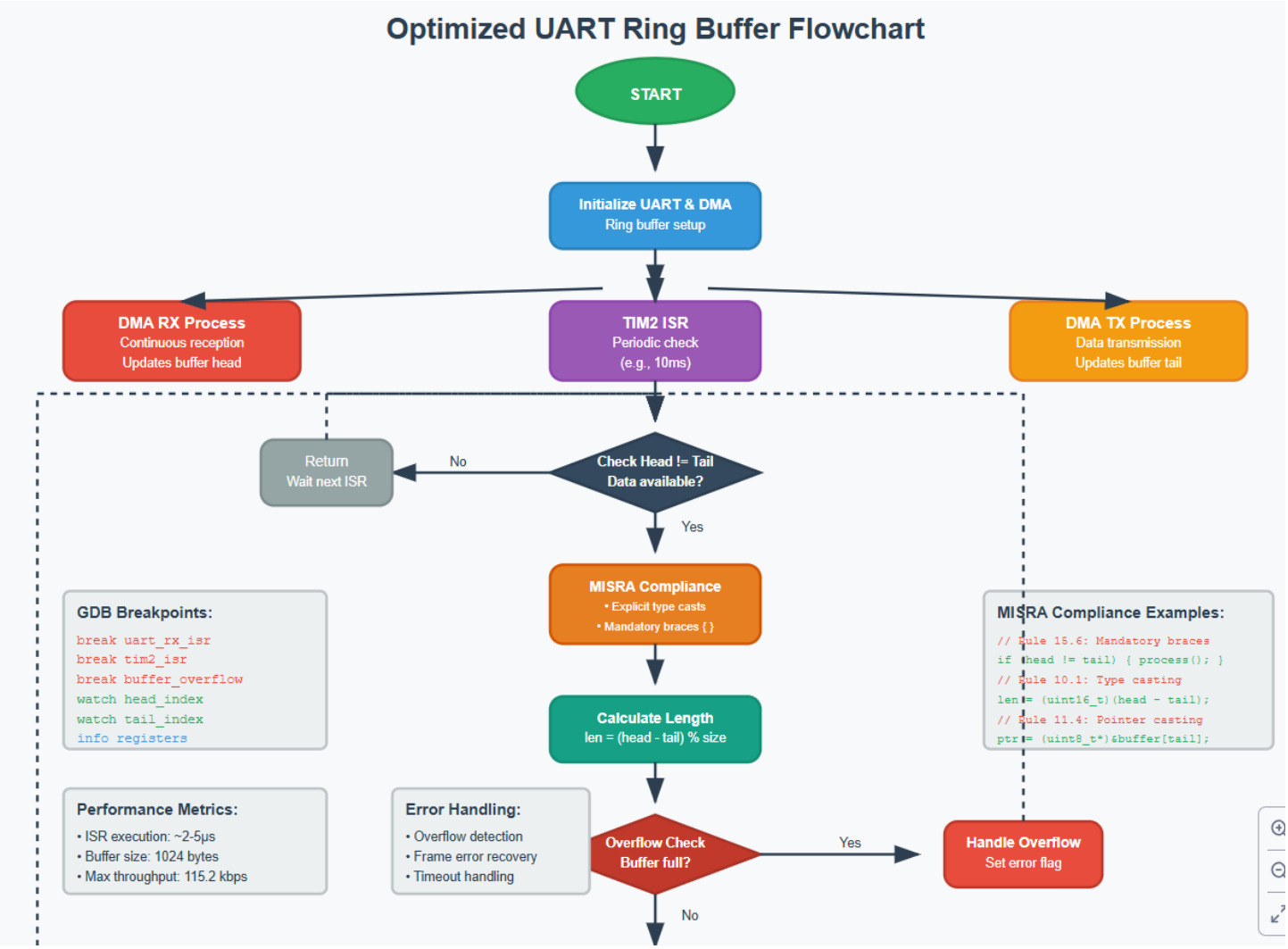
- **GDB**: Command-line debugger for inspecting variables, stack, and registers.
  - Commands: break main, info registers, x/10xw $sp (stack inspection), step, continue.
- **OpenOCD**: Interfaces with STM32 hardware for debugging via JTAG/SWD.

- Setup: Run openocd -f interface/stlink.cfg -f target/stm32f1x.cfg.
- Connect GDB: arm-none-eabi-gdb, then target remote localhost:3333.

- **Common Issues** (Day 21 UART Ring Buffer):
  - Buffer overflow if head overtakes tail.
  - Missed DMA interrupts due to priority conflicts.
  - Non-MISRA-compliant code (e.g., implicit type conversions).
- **Embedded Tip**: Use GDB watchpoints (watch head) to catch buffer overruns; set breakpoints in ISRs to debug timing issues.

3. Optimization Goals for UART Ring Buffer

- **Performance**: Minimize CPU usage with DMA and interrupt-driven processing.
- **Memory**: Use static allocation, optimize buffer size (256 bytes).
- **Robustness**: Add overflow detection and error handling.
- **MISRA Compliance**: Adhere to key rules (e.g., 10.1, 15.6, 17.7, 20.3).
- **Debugging**: Use GDB/OpenOCD to verify buffer indices and DMA counters.



Start DMA Tx(Transmit Data Shunk)

Image: Optimized UART Ring Buffer Flowchart

Description: A flowchart showing the optimized UART ring buffer: DMA RX fills buffer, TIM2 ISR checks head/tail, transmits data via DMA TX, handles overflow/errors. Includes MISRA compliance checks (braces, type casts) and GDB breakpoints for debugging.

4. Practice Program: Debug UART Ring Buffer

- **Objective**: Debug Day 21 UART ring buffer using GDB/OpenOCD to fix potential overflow.
- **Steps**:
  1. Run OpenOCD: openocd -f interface/stlink.cfg -f target/stm32f1x.cfg.
  2. Start GDB: arm-none-eabi-gdb project.elf.
  3. Connect: target remote localhost:3333.
  4. Set breakpoint: break HAL_UART_RxCpltCallback.
  5. Inspect head, tail, and DMA1->CNDTR5 to detect overruns.
  6. Fix by adding overflow check in ISR.
- **Code Snippet** (Debug-focused):

```
/* Debug UART ring buffer with overflow check */
#include "stm32f1xx_hal.h"
#include <stdint.h>

#define RING_BUFFER_SIZE 256U
static uint8_t rx_buffer[RING_BUFFER_SIZE]; /* Rule 20.3: Static allocation */
static volatile uint32_t head = 0U; /* Rule 9.1: Initialize */
static volatile uint32_t tail = 0U;
static volatile uint8_t overflow = 0U;

/* Check for buffer overflow in DMA callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    uint32_t new_head = RING_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(huart->hdmarx);
    if (((new_head + 1U) & (RING_BUFFER_SIZE - 1U)) == tail) { /* Rule 12.1: Parentheses */
        overflow = 1U; /* Buffer overrun detected */
    } else {
        head = new_head; /* Update head safely */
    }
    /* GDB breakpoint here to inspect head, tail, overflow */
}
```

Project: Optimize UART Ring Buffer with GDB

- **Objective**: Optimize the Day 21 UART ring buffer for performance, memory efficiency, and MISRA compliance, using GDB/OpenOCD to debug and verify improvements.
- **Optimizations**:
  - Add overflow detection to prevent data loss.
  - Ensure MISRA compliance (e.g., Rule 10.1 for type safety, Rule 15.6 for braces).
  - Reduce stack usage with static buffers.
  - Use GDB to monitor head, tail, and DMA1->CNDTR5.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA (Channel 4 TX, Channel 5 RX, circular).
  - Use 256-byte ring buffer for RX.
  - Echo received data via DMA TX; log status/errors via UART.
  - Use TIM2 for 500ms periodic checks.
  - Comply with MISRA rules (1.1, 8.6, 10.1, 15.6, 17.7, 20.3).

- **Code** (STM32 with HAL, heavily commented):

```
/* Optimized MISRA-compliant UART ring buffer with GDB debugging */
#include "stm32f1xx_hal.h"
#include <stdint.h>
#include <string.h>

/* UART and DMA handles declared at file scope (Rule 8.6) */
static UART_HandleTypeDef huart1;
static DMA_HandleTypeDef hdma_usart1_tx;
static DMA_HandleTypeDef hdma_usart1_rx;
static TIM_HandleTypeDef htim2;

/* Ring buffer configuration (Rule 20.3: No dynamic allocation) */
#define RING_BUFFER_SIZE 256U
static uint8_t rx_buffer[RING_BUFFER_SIZE]; /* RX buffer */
static uint8_t tx_buffer[RING_BUFFER_SIZE]; /* TX buffer */
static volatile uint32_t head = 0U; /* RX write index (Rule 9.1) */
static volatile uint32_t tail = 0U; /* RX read index */
static volatile uint8_t tx_busy = 0U; /* TX state flag */
static volatile uint8_t overflow = 0U; /* Overflow flag */

/* Function prototypes */
static void init_gpio(void);
static void init_uart(void);
static void init_dma(void);
static void init_timer(void);
static uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len);
static void log_message(const uint8_t *message);

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; /* Rule 9.1: Initialize all fields */

    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA9, PA10 as UART alternate function */
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; /* Rule 12.1: Explicit grouping */
    gpio_init.Mode = GPIO_MODE_AF_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init); /* Rule 17.7: Ignore return */
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    /* Enable USART1 clock */
    __HAL_RCC_USART1_CLK_ENABLE();

    /* Configure UART parameters */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200U;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

    /* Initialize UART and handle errors */
    if (HAL_UART_Init(&huart1) != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"UART Init Failed\r\n";
        log_message(error_msg);
        while (1) { /* Rule 15.6: Braces */
            /* Infinite loop on error */
        }
```

```
    }
}

/* Initialize DMA for UART RX/TX */
static void init_dma(void)
{
    /* Enable DMA1 clock */
    __HAL_RCC_DMA1_CLK_ENABLE();

    /* RX: DMA1 Channel 5, circular mode */
    hdma_usart1_rx.Instance = DMA1_Channel5;
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_rx.Init.Mode = DMA_CIRCULAR;
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH;
    HAL_DMA_Init(&hdma_usart1_rx);
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx);

    /* TX: DMA1 Channel 4, normal mode */
    hdma_usart1_tx.Instance = DMA1_Channel4;
    hdma_usart1_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
    hdma_usart1_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_tx.Init.Mode = DMA_NORMAL;
    hdma_usart1_tx.Init.Priority = DMA_PRIORITY_MEDIUM;
    HAL_DMA_Init(&hdma_usart1_tx);
    __HAL_LINKDMA(&huart1, hdmatx, hdma_usart1_tx);

    /* Enable DMA interrupts */
    HAL_NVIC_SetPriority(DMA1_Channel5_IRQn, 1U, 0U);
    HAL_NVIC_EnableIRQ(DMA1_Channel5_IRQn);
    HAL_NVIC_SetPriority(DMA1_Channel4_IRQn, 1U, 0U);
    HAL_NVIC_EnableIRQ(DMA1_Channel4_IRQn);

    /* Start DMA RX */
    if (HAL_UART_Receive_DMA(&huart1, rx_buffer, RING_BUFFER_SIZE) != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"DMA RX Init Failed\r\n";
        log_message(error_msg);
        while (1) { /* Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
}

/* Initialize TIM2 for 500ms interrupts */
static void init_timer(void)
{
    /* Enable TIM2 clock */
    __HAL_RCC_TIM2_CLK_ENABLE();

    /* Configure timer for 500ms interrupts */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 15999U; /* 16MHz / (15999+1) = 1kHz */
    htim2.Init.Period = 499U; /* 1kHz / (499+1) = 500ms */
    HAL_TIM_Base_Init(&htim2);

    /* Enable timer interrupt */
    HAL_NVIC_SetPriority(TIM2_IRQn, 0U, 0U);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    HAL_TIM_Base_Start_IT(&htim2);
}
```

```c
/* Read data from ring buffer */
static uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len)
{
    uint32_t count = 0U; /* Rule 9.1: Initialize */

    /* Check for valid input (Rule 10.1) */
    if ((data == NULL) || (max_len == 0U)) {
        return 0U; /* Rule 17.7: Explicit return */
    }

    /* Copy data until buffer empty or max_len reached */
    while ((tail != head) && (count < max_len)) { /* Rule 15.6: Braces */
        data[count] = rx_buffer[tail];
        tail = (tail + 1U) & (RING_BUFFER_SIZE - 1U); /* Power-of-2 wrap */
        count++;
    }

    return count; /* Rule 17.7: Explicit return */
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    /* Check for valid input */
    if (message != NULL) {
        HAL_UART_Transmit(&huart1, (uint8_t *)message,
                          (uint16_t)strlen((const char *)message), 10U); /* Rule 10.1: Cast */
    }
}

/* DMA RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    /* Update head from DMA counter */
    uint32_t new_head = RING_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart1_rx);

    /* Check for buffer overflow */
    if (((new_head + 1U) & (RING_BUFFER_SIZE - 1U)) == tail) { /* Rule 12.1: Parentheses */
        overflow = 1U; /* Set overflow flag */
    } else {
        head = new_head; /* Update head safely */
    }
    /* GDB breakpoint here to inspect head, tail, overflow */
}

/* DMA RX half-complete callback */
void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart)
{
    /* Update head at half buffer */
    head = RING_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart1_rx);
    /* GDB breakpoint here to monitor head */
}

/* DMA TX complete callback */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    tx_busy = 0U; /* Clear TX busy flag */
    /* GDB breakpoint here to verify TX completion */
}
```

```c
/* Timer interrupt callback */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        /* Check overflow and log */
        if (overflow != 0U) {
            const uint8_t *msg = (const uint8_t *)"Buffer Overflow!\r\n";
            log_message(msg);
            overflow = 0U; /* Reset flag */
        }

        /* Process buffer if TX not busy */
        if (tx_busy == 0U) {
            uint8_t data[64U];
            uint32_t len = ring_buffer_read(data, sizeof(data));
            if (len > 0U) {
                tx_busy = 1U;
                /* Transmit data via DMA */
                if (HAL_UART_Transmit_DMA(&huart1, data, (uint16_t)len) == HAL_OK) {
                    /* Log transmitted bytes */
                    snprintf((char *)tx_buffer, sizeof(tx_buffer),
                            "Echoed %u bytes\r\n", (unsigned int)len); /* Rule 10.1: Cast */
                    log_message(tx_buffer);
                } else {
                    const uint8_t *msg = (const uint8_t *)"DMA TX Error\r\n";
                    log_message(msg);
                }
            }
        }
    }
}

/* Main function */
int main(void)
{
    /* Initialize HAL library */
    HAL_Init();

    /* Initialize peripherals */
    init_gpio();
    init_uart();
    init_dma();
    init_timer();

    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"Optimized UART Ring Buffer Started\r\n";
    log_message(startup_msg);

    /* Main loop: Handled by DMA and TIM2 */
    while (1) {
        /* Empty; DMA and interrupts handle processing */
    }

    return 0; /* Rule 17.7: Explicit return */
}

/* DMA RX interrupt handler */
void DMA1_Channel5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx);
}

/* DMA TX interrupt handler */
void DMA1_Channel4_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_tx);
}
```

```
/* Timer interrupt handler */
void TIM2_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim2);
}
```

- **Simulated Version** (For PC testing, heavily commented):

```
/* Simulated MISRA-compliant UART ring buffer */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

/* Simulated STM32 registers */
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;
volatile uint32_t *const DMA_CCR5 = (volatile uint32_t *)0x40020048;
volatile uint32_t *const DMA_CNDTR5 = (volatile uint32_t *)0x4002004C;
volatile uint32_t *const DMA_CPAR5 = (volatile uint32_t *)0x40020050;
volatile uint32_t *const DMA_CMAR5 = (volatile uint32_t *)0x40020054;
volatile uint32_t *const TIM2_SR = (volatile uint32_t *)0x40000010;

/* Ring buffer configuration (Rule 20.3: No dynamic allocation) */
#define RING_BUFFER_SIZE 256U
static uint8_t rx_buffer[RING_BUFFER_SIZE]; /* RX buffer */
static uint8_t tx_buffer[RING_BUFFER_SIZE]; /* TX buffer */
static volatile uint32_t head = 0U; /* Rule 9.1: Initialize */
static volatile uint32_t tail = 0U;
static volatile uint8_t overflow = 0U;

/* Function prototypes */
static void init_dma_uart(void);
static uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len);
static void log_message(const uint8_t *message);

/* Initialize simulated DMA UART */
static void init_dma_uart(void)
{
    /* Configure DMA for RX */
    *DMA_CPAR5 = (uint32_t)USART_DR;
    *DMA_CMAR5 = (uint32_t)rx_buffer;
    *DMA_CNDTR5 = RING_BUFFER_SIZE;
    *DMA_CCR5 = (1U << 7) | (0U << 4) | (1U << 5) | (1U << 0); /* Circular, periph-to-mem */
    printf("DMA UART initialized\n");
}

/* Read data from ring buffer */
static uint32_t ring_buffer_read(uint8_t *data, uint32_t max_len)
{
    uint32_t count = 0U; /* Rule 9.1: Initialize */

    /* Check for valid input */
    if ((data == NULL) || (max_len == 0U)) {
        return 0U; /* Rule 17.7: Explicit return */
    }

    /* Copy data until buffer empty or max_len reached */
    while ((tail != head) && (count < max_len)) { /* Rule 15.6: Braces */
        data[count] = rx_buffer[tail];
        tail = (tail + 1U) & (RING_BUFFER_SIZE - 1U);
        count++;
    }

    return count; /* Rule 17.7: Explicit return */
}
```

```c
/* Log message via simulated UART */
static void log_message(const uint8_t *message)
{
    /* Check for valid input */
    if (message != NULL) {
        printf("%s", (const char *)message); /* Rule 11.4: Cast */
    }
}

/* Simulate DMA RX interrupt */
static void simulate_rx_interrupt(void)
{
    /* Simulate incoming data */
    static uint8_t sim_data[] = {'A', 'B', 'C'};
    static uint32_t idx = 0U;

    /* Check for overflow */
    if (((head + 1U) & (RING_BUFFER_SIZE - 1U)) == tail) {
        overflow = 1U;
    } else {
        rx_buffer[head] = sim_data[idx++ % 3U];
        head = (head + 1U) & (RING_BUFFER_SIZE - 1U);
        *DMA_CNDTR5 = RING_BUFFER_SIZE - head;
    }
}

/* Main function */
int main(void)
{
    /* Initialize DMA UART */
    init_dma_uart();

    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"Optimized UART Ring Buffer Started\n";
    log_message(startup_msg);

    /* Simulate 3 transfers */
    for (uint32_t i = 0U; i < 3U; i++) { /* Rule 15.6: Braces */
        simulate_rx_interrupt();
        if (overflow != 0U) {
            const uint8_t *msg = (const uint8_t *)"Buffer Overflow!\n";
            log_message(msg);
            overflow = 0U;
        }

        uint8_t data[64U];
        uint32_t len = ring_buffer_read(data, sizeof(data));
        if (len > 0U) {
            for (uint32_t j = 0U; j < len; j++) {
                printf("%c", (char)data[j]);
            }
            snprintf((char *)tx_buffer, sizeof(tx_buffer),
                    "Echoed %u bytes\n", (unsigned int)len); /* Rule 10.1: Cast */
            log_message(tx_buffer);
        }
    }

    return 0; /* Rule 17.7: Explicit return */
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure USART1 (PA9-TX, PA10-RX, 115200 baud), DMA1 (Channel 4 TX, Channel 5 RX, circular), TIM2 (500ms).
  - Connect USB-to-serial adapter to PA9/PA10; test with PuTTY.

    - Debug with OpenOCD: openocd -f interface/stlink.cfg -f target/stm32f1x.cfg.
    - GDB: arm-none-eabi-gdb project.elf, then target remote localhost:3333, break HAL_UART_RxCpltCallback, watch head.
    - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c for MISRA compliance.
- **Optimizations**:
    - **Overflow Detection**: Added check in HAL_UART_RxCpltCallback to prevent data loss (robustness).
    - **MISRA Compliance**: Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (explicit returns), 20.3 (no heap).
    - **Memory Efficiency**: Static buffers, power-of-2 size for fast wrapping.
    - **Debugging**: GDB breakpoints/watchpoints for head, tail, overflow.
- **Embedded Tip**: Use OpenOCD's monitor reset halt to catch early execution issues; run Cppcheck in CI pipeline.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html.
- **GDB**: Use arm-none-eabi-gdb with STM32CubeIDE or standalone.
- **OpenOCD**: Install (sudo apt install openocd), configure for ST-Link.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **OpenOCD Tutorials**: Search "OpenOCD STM32 tutorial" (e.g., https://www.openocd.org).
- **GDB Docs**: https://sourceware.org/gdb/documentation/.
- **Online**: STM32 HAL docs (https://www.st.com), MISRA C:2012 summaries.

Practice Tips

- **Review**: Spend ~1 hour rereading Days 21-24 notes.
- **Debug**: Use GDB to fix one issue (e.g., overflow handling).
- **Optimize**: Add DMA error handling (TE interrupt).
- **Time Management**: Spend ~1 hour on review, ~1 hour on debugging, ~1-1.5 hours on the project.

Chapter 6: RTOS Intro & Final Project

- **Learn FreeRTOS basics**
- **Understand RTOS timers**
- **Learn resource protection**
- **Design RTOS system**
- **Build final project**

Learn FreeRTOS Basics

Goal

Master FreeRTOS basics, including tasks, queues, and semaphores, to build a multitasking embedded system, with a focus on implementing a task-based LED toggle application on an STM32 microcontroller.

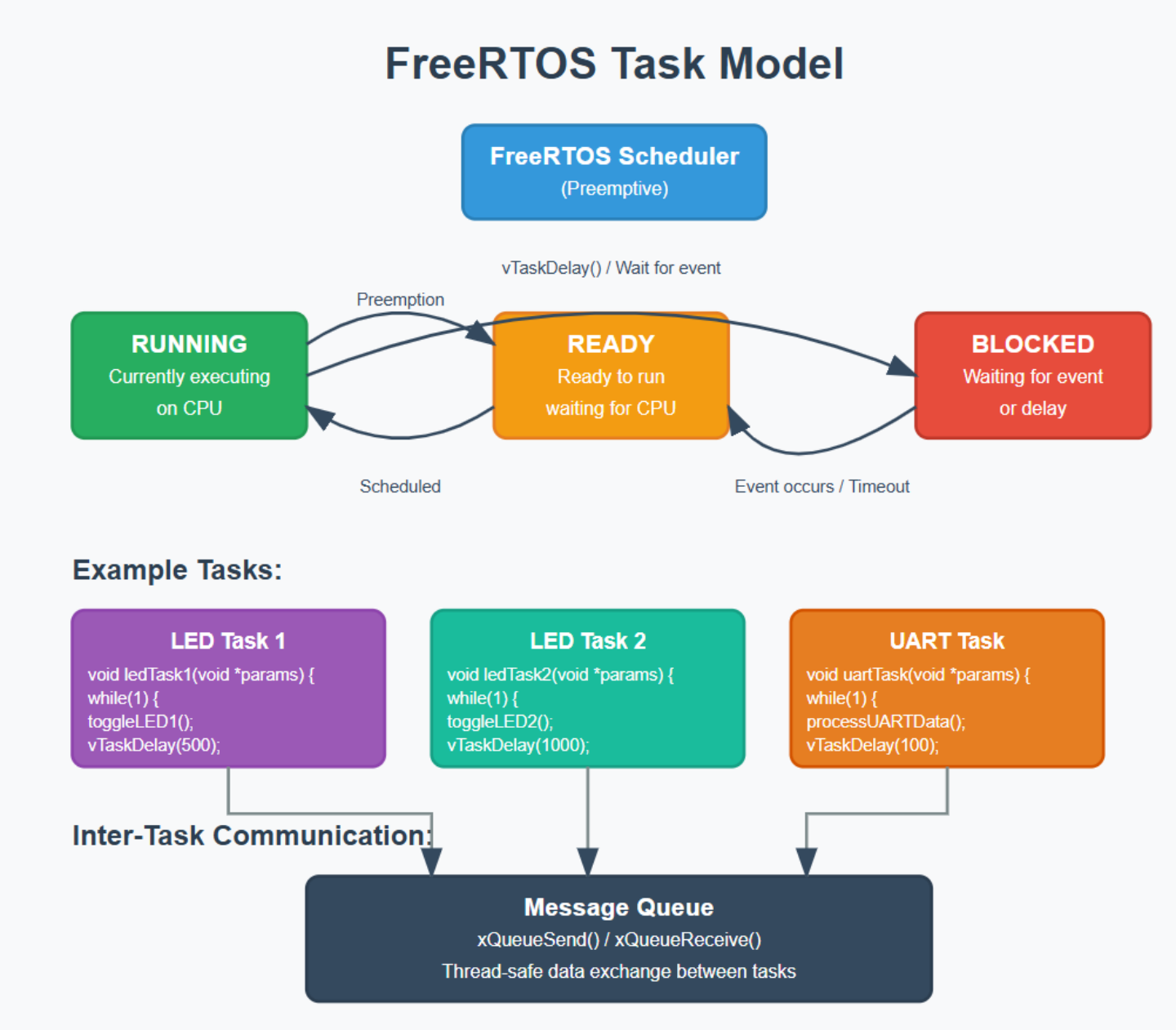
Tasks

- **Study**: Learn FreeRTOS concepts (tasks, queues, semaphores) and their role in multitasking.
- **Practice**: Write simple FreeRTOS code with multiple tasks to toggle LEDs.
- **Project**: Develop a task-based LED toggle system using FreeRTOS, with two tasks controlling LEDs at different rates.
- **Tools**: STM32CubeIDE, FreeRTOS (integrated via STM32CubeMX).
- **Resources**: FreeRTOS.org, YouTube FreeRTOS tutorials.

Detailed Notes

1. FreeRTOS Overview

- **Definition**: FreeRTOS is a real-time operating system (RTOS) designed for embedded systems, providing task scheduling, inter-task communication, and resource management.
- **Key Concepts**:
  - **Tasks**: Independent threads of execution with their own stack and priority. Created with xTaskCreate().
  - **Scheduler**: Preemptive (higher-priority tasks interrupt lower ones) or cooperative (tasks yield voluntarily).
  - **Queues**: FIFO buffers for inter-task communication. Managed with xQueueCreate(), xQueueSend(), xQueueReceive().
  - **Semaphores**: Synchronization primitives for resource access or signaling. Types: binary, counting, mutex.
- **FreeRTOS on STM32**:
  - Integrated via STM32CubeMX (FreeRTOS middleware).
  - Uses CMSIS-RTOS API or native FreeRTOS API.
  - Task stack size and priority configured in code or CubeMX.
- **Embedded Tip**: Allocate sufficient stack size (e.g., 128-256 words) per task to avoid overflows; use FreeRTOS heap (e.g., heap_4.c) for dynamic allocation.



Task Priority Levels:

High Priority (3) | Medium Priority (2) | Low Priority (1)

Task Creation:

```
xTaskCreate(
    ledTask1, // Task function
    "LED Task 1", // Task name
    configMINIMAL_STACK_SIZE, // Stack size
    NULL, // Parameters
    2, // Priority
```

Key Features:

- Preemptive multitasking with priority-based scheduling
- Tasks can be in Running, Ready, Blocked, or Suspended states
- Inter-task communication via queues, semaphores, and mutexes
- Tick-based time management for delays and timeouts
- Memory management and stack overflow protection

Image: FreeRTOS Task Model

Description: A diagram showing multiple FreeRTOS tasks (e.g., LED Task 1, LED Task 2) running concurrently, managed by the scheduler. Arrows indicate task states (Running, Ready, Blocked) and transitions (e.g., vTaskDelay() blocks, scheduler preempts). Includes queue for inter-task communication.

2. FreeRTOS Tasks

- **Creation**: xTaskCreate(name, stack_size, params, priority, handle) creates a task.
  - stack_size: In words (e.g., 128 for 512 bytes on 32-bit systems).
  - priority: Higher number = higher priority (e.g., 1-10, configured via configMAX_PRIORITIES).
- **Scheduling**: Preemptive scheduler switches tasks based on priority and tick interrupts (default 1ms tick).
- **Delay**: vTaskDelay(pdMS_TO_TICKS(ms)) suspends task for specified time, allowing others to run.
- **Embedded Tip**: Avoid HAL_Delay() in FreeRTOS; use vTaskDelay() to prevent blocking the scheduler.

3. FreeRTOS Queues

- **Purpose**: Pass data between tasks safely (e.g., sensor data, commands).
- **Functions**: bbbbbbbbbbbbb
  - xQueueCreate(size, item_size): Create queue for size items of item_size bytes.
  - xQueueSend(queue, item, timeout): Send item to queue.
  - xQueueReceive(queue, item, timeout): Receive item from queue.
- **Usage**: Tasks block on empty/full queues until data is available or space frees up.
- **Embedded Tip**: Use queues for producer-consumer patterns (e.g., sensor task sends data to UART task).

4. FreeRTOS Semaphores

- **Types**:
  - **Binary Semaphore**: Signals events (e.g., ISR to task). Created with xSemaphoreCreateBinary().
  - **Counting Semaphore**: Tracks multiple resources. Created with xSemaphoreCreateCounting().
  - **Mutex**: Protects shared resources. Created with xSemaphoreCreateMutex().
- **Functions**:
  - xSemaphoreTake(sem, timeout): Acquire semaphore.
  - xSemaphoreGive(sem): Release semaphore.
- **Embedded Tip**: Use binary semaphores for ISR-to-task synchronization; mutexes for shared resource protection.

5. Practice Program: Simple Task-Based LED Toggle

- **Objective**: Create two FreeRTOS tasks to toggle LEDs at different rates (500ms, 1000ms).
- **Code** (STM32 simulation):

```
/* Simple FreeRTOS task-based LED toggle simulation */
#include <stdio.h>
#include <stdint.h>

/* Simulated GPIO registers */
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;

/* Task function prototypes */
static void led_task1(void *params);
static void led_task2(void *params);
```

```c
/* Simulated FreeRTOS API */
#define pdMS_TO_TICKS(ms) (ms)

static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); }

static void xTaskCreate(void (*func)(void *), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) {
    printf("Created task: %s\n", name);
    func(params); /* Simulate task execution */
}

/* Task 1: Toggle LED on PA5 every 500ms */
static void led_task1(void *params) {
    (void)params; /* Unused parameter */
    while (1) {
        *GPIO_BSRR = (1U << 5); /* Set PA5 */
        printf("LED1 ON\n");
        vTaskDelay(500);
        *GPIO_BSRR = (1U << (5 + 16)); /* Reset PA5 */
        printf("LED1 OFF\n");
        vTaskDelay(500);
    }
}

/* Task 2: Toggle LED on PA6 every 1000ms */
static void led_task2(void *params) {
    (void)params;
    while (1) {
        *GPIO_BSRR = (1U << 6); /* Set PA6 */
        printf("LED2 ON\n");
        vTaskDelay(1000);
        *GPIO_BSRR = (1U << (6 + 16)); /* Reset PA6 */
        printf("LED2 OFF\n");
        vTaskDelay(1000);
    }
}

/* Main function */
int main(void) {
    /* Create tasks */
    xTaskCreate(led_task1, "LED1", 128, NULL, 1, NULL);
    xTaskCreate(led_task2, "LED2", 128, NULL, 1, NULL);
    printf("Scheduler started\n");
    return 0;
}
```

Project: Task-Based LED Toggle

- **Objective**: Implement a FreeRTOS-based system with two tasks to toggle LEDs (PA5, PA6) at different rates (500ms, 1000ms), using FreeRTOS tasks and vTaskDelay() for timing.
- **Requirements**:
  - Configure PA5, PA6 as outputs for LEDs.
  - Create two tasks with equal priority (1) and 256-word stacks.
  - Use FreeRTOS (integrated via STM32CubeMX) with default 1ms tick.
  - Log task activity via UART (PA9-TX, 115200 baud).
- **Approach**:
  - **Initialization**: Configure GPIO, UART, and FreeRTOS via STM32CubeMX.
  - **Tasks**: Create LedTask1 (500ms toggle on PA5) and LedTask2 (1000ms toggle on PA6).
  - **Scheduler**: Start FreeRTOS scheduler with vTaskStartScheduler().
  - **Logging**: Use UART to log LED states for debugging.

- **MISRA Compliance**: Apply Day 24 guidelines (e.g., Rule 15.6 for braces, Rule 17.7 for returns).

- **Code** (STM32 with FreeRTOS and HAL):

```
/* FreeRTOS-based LED toggle with UART logging */
#include "stm32f1xx_hal.h"
#include "FreeRTOS.h"
#include "task.h"
#include <stdint.h>
#include <string.h>

/* UART handle for logging */
static UART_HandleTypeDef huart1;

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U];

/* Function prototypes */
static void init_gpio(void);
static void init_uart(void);
static void log_message(const uint8_t *message);
static void LedTask1(void *pvParameters);
static void LedTask2(void *pvParameters);

/* Initialize GPIO for LEDs (PA5, PA6) and UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; /* MISRA Rule 9.1: Initialize all fields */

    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA5, PA6 as outputs for LEDs */
    gpio_init.Pin = GPIO_PIN_5 | GPIO_PIN_6; /* MISRA Rule 12.1: Explicit grouping */
    gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &gpio_init);

    /* Configure PA9, PA10 for UART */
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    gpio_init.Mode = GPIO_MODE_AF_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init);

    /* Initialize LEDs to OFF */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6, GPIO_PIN_RESET);
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    /* Enable USART1 clock */
    __HAL_RCC_USART1_CLK_ENABLE();

    /* Configure UART parameters */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200U;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

    /* Initialize UART and handle errors */
    if (HAL_UART_Init(&huart1) != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"UART Init Failed\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)error_msg,
```

```
                        (uint16_t)strlen((const char *)error_msg), 10U); /* MISRA Rule 10.1: Cast */
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    /* Check for valid input */
    if (message != NULL) {
        HAL_UART_Transmit(&huart1, (uint8_t *)message,
                        (uint16_t)strlen((const char *)message), 10U); /* MISRA Rule 10.1: Cast */
    }
}

/* Task 1: Toggle LED on PA5 every 500ms */
static void LedTask1(void *pvParameters)
{
    (void)pvParameters; /* MISRA Rule 2.7: Unused parameter */

    while (1) {
        /* Toggle LED on PA5 */
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

        /* Log LED state */
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "LED1 %s\r\n", HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) ? "ON" : "OFF"); /* MISRA Rule
10.1: Cast */
        log_message(log_buffer);

        /* Delay 500ms, allowing other tasks to run */
        vTaskDelay(pdMS_TO_TICKS(500U));
    }
}

/* Task 2: Toggle LED on PA6 every 1000ms */
static void LedTask2(void *pvParameters)
{
    (void)pvParameters; /* MISRA Rule 2.7: Unused parameter */

    while (1) {
        /* Toggle LED on PA6 */
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);

        /* Log LED state */
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "LED2 %s\r\n", HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_6) ? "ON" : "OFF"); /* MISRA Rule
10.1: Cast */
        log_message(log_buffer);

        /* Delay 1000ms */
        vTaskDelay(pdMS_TO_TICKS(1000U));
    }
}

/* Main function */
int main(void)
{
    /* Initialize HAL library */
    HAL_Init();

    /* Initialize peripherals */
    init_gpio();
    init_uart();
```

```
    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"FreeRTOS LED Toggle Started\r\n";
    log_message(startup_msg);

    /* Create FreeRTOS tasks */
    BaseType_t result;

    /* Task 1: LED1 toggle, priority 1, 256-word stack */
    result = xTaskCreate(LedTask1, "LED1", 256U, NULL, 1U, NULL);
    if (result != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"Task1 Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Task 2: LED2 toggle, priority 1, 256-word stack */
    result = xTaskCreate(LedTask2, "LED2", 256U, NULL, 1U, NULL);
    if (result != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"Task2 Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Start FreeRTOS scheduler */
    vTaskStartScheduler();

    /* Should not reach here */
    while (1) { /* MISRA Rule 15.6: Braces */
        /* Infinite loop if scheduler fails */
    }

    return 0; /* MISRA Rule 17.7: Explicit return */
}
```

- **Simulated Version** (For PC testing, heavily commented):

```
/* Simulated FreeRTOS LED toggle */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

/* Simulated GPIO registers */
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
static uint8_t led1_state = 0U;
static uint8_t led2_state = 0U;

/* Simulated FreeRTOS API */
#define pdMS_TO_TICKS(ms) (ms)
static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); }
static void xTaskCreate(void (*func)(void *), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) {
    printf("Created task: %s\n", name);
    func(params); /* Simulate task execution */
}
static void vTaskStartScheduler(void) { printf("Scheduler started\n"); }

/* Static buffer for logging */
static uint8_t log_buffer[32U];
```

```c
/* Log message */
static void log_message(const uint8_t *message)
{
    if (message != NULL) {
        printf("%s", (const char *)message);
    }
}

/* Task 1: Toggle LED1 every 500ms */
static void LedTask1(void *params)
{
    (void)params;
    while (1) {
        led1_state = !led1_state;
        *GPIO_BSRR = led1_state ? (1U << 5) : (1U << (5 + 16));
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "LED1 %s\n", led1_state ? "ON" : "OFF");
        log_message(log_buffer);
        vTaskDelay(500U);
    }
}

/* Task 2: Toggle LED2 every 1000ms */
static void LedTask2(void *params)
{
    (void)params;
    while (1) {
        led2_state = !led2_state;
        *GPIO_BSRR = led2_state ? (1U << 6) : (1U << (6 + 16));
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "LED2 %s\n", led2_state ? "ON" : "OFF");
        log_message(log_buffer);
        vTaskDelay(1000U);
    }
}

/* Main function */
int main(void)
{
    /* Simulate startup */
    const uint8_t *startup_msg = (const uint8_t *)"FreeRTOS LED Toggle Started\n";
    log_message(startup_msg);

    /* Create tasks */
    xTaskCreate(LedTask1, "LED1", 256U, NULL, 1U, NULL);
    xTaskCreate(LedTask2, "LED2", 256U, NULL, 1U, NULL);

    /* Start scheduler */
    vTaskStartScheduler();

    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure:
    - GPIO: PA5, PA6 (outputs).
    - USART1: PA9-TX, PA10-RX, 115200 baud.
    - FreeRTOS: Enable middleware, set configTICK_RATE_HZ=1000 (1ms tick).
  - Connect LEDs (with resistors) to PA5, PA6; USB-to-serial adapter to PA9/PA10.
  - Test with STM32CubeIDE and PuTTY for UART output.
  - Run Cppcheck for MISRA compliance: cppcheck --enable=all --addon=misra.json project.c.
- **Optimizations**:
  - **Task Efficiency**: Equal priorities (1) ensure fair scheduling; vTaskDelay() prevents blocking.

    - **Memory**: Static log_buffer (MISRA Rule 20.3); 256-word task stacks to minimize SRAM use.
    - **Robustness**: Error checking for task creation and UART init.
    - **MISRA**: Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns).
- **Embedded Tip**: Use FreeRTOS vTaskGetInfo() to monitor task stack usage; adjust stack size if uxHighWaterMark is low.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for FreeRTOS setup.
- **FreeRTOS**: Integrated via STM32CubeMX (select FreeRTOS middleware, heap_4 allocator).
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32f1x.cfg).

Resources

- **FreeRTOS.org**: https://www.freertos.org for API reference, tutorials.
- **YouTube FreeRTOS Tutorials**: Search “FreeRTOS STM32 tutorial” (e.g., STMicroelectronics, Neso Academy).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Change task priorities (e.g., 2 for LedTask1, 1 for LedTask2) and observe scheduling.
- **Optimize**: Add a queue to send LED states between tasks.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Understand RTOS Timers

Goal

Master FreeRTOS timer concepts, comparing hardware timer interrupts with software timers, and implement a software timer-based LED blinker to achieve precise, non-blocking timing in a multitasking system on an STM32 microcontroller.

Tasks

- **Study**: Learn FreeRTOS software timers, their advantages over hardware timer interrupts, and their configuration.
- **Practice**: Implement a software timer to control LED blinking.
- **Project**: Develop a software timer-based LED blinker with multiple timers controlling LEDs at different rates, integrated with FreeRTOS tasks.
- **Tools**: STM32CubeIDE, FreeRTOS (integrated via STM32CubeMX).
- **Resources**: FreeRTOS timer documentation, STM32 manuals.

Detailed Notes

1. RTOS Timers Overview

- **Hardware Timer Interrupts**:
  - **Definition**: Microcontroller peripherals (e.g., STM32 TIM2) that generate interrupts at precise intervals.
  - **Pros**: High precision, low latency, direct hardware control.
  - **Cons**: Limited number of timers, complex configuration, ISR overhead in multitasking systems.
  - **Use Case**: Critical timing tasks (e.g., PWM, precise delays).
- **FreeRTOS Software Timers**:
  - **Definition**: Timers managed by FreeRTOS, running in a dedicated timer service task (prvTimerTask), triggered by the RTOS tick.
  - **Pros**: Unlimited timers (limited by memory), non-blocking, easy to use in multitasking systems.
  - **Cons**: Less precise (depends on tick rate, e.g., 1ms), runs in task context (higher latency than ISRs).
  - **Key Functions**:
    - xTimerCreate(): Create a timer.
    - xTimerStart(): Start a timer.
    - xTimerStop(): Stop a timer.
    - xTimerReset(): Restart a timer.
  - **Types**: One-shot (runs once) or auto-reload (repeats).
- **FreeRTOS Timer Service**:
  - Runs in a dedicated task (priority set by configTIMER_TASK_PRIORITY).
  - Processes timer callbacks via a command queue (xTimerQueue).
- **Embedded Tip**: Use software timers for non-critical periodic tasks (e.g., LED blinking); reserve hardware timers for precise operations (e.g., motor control).

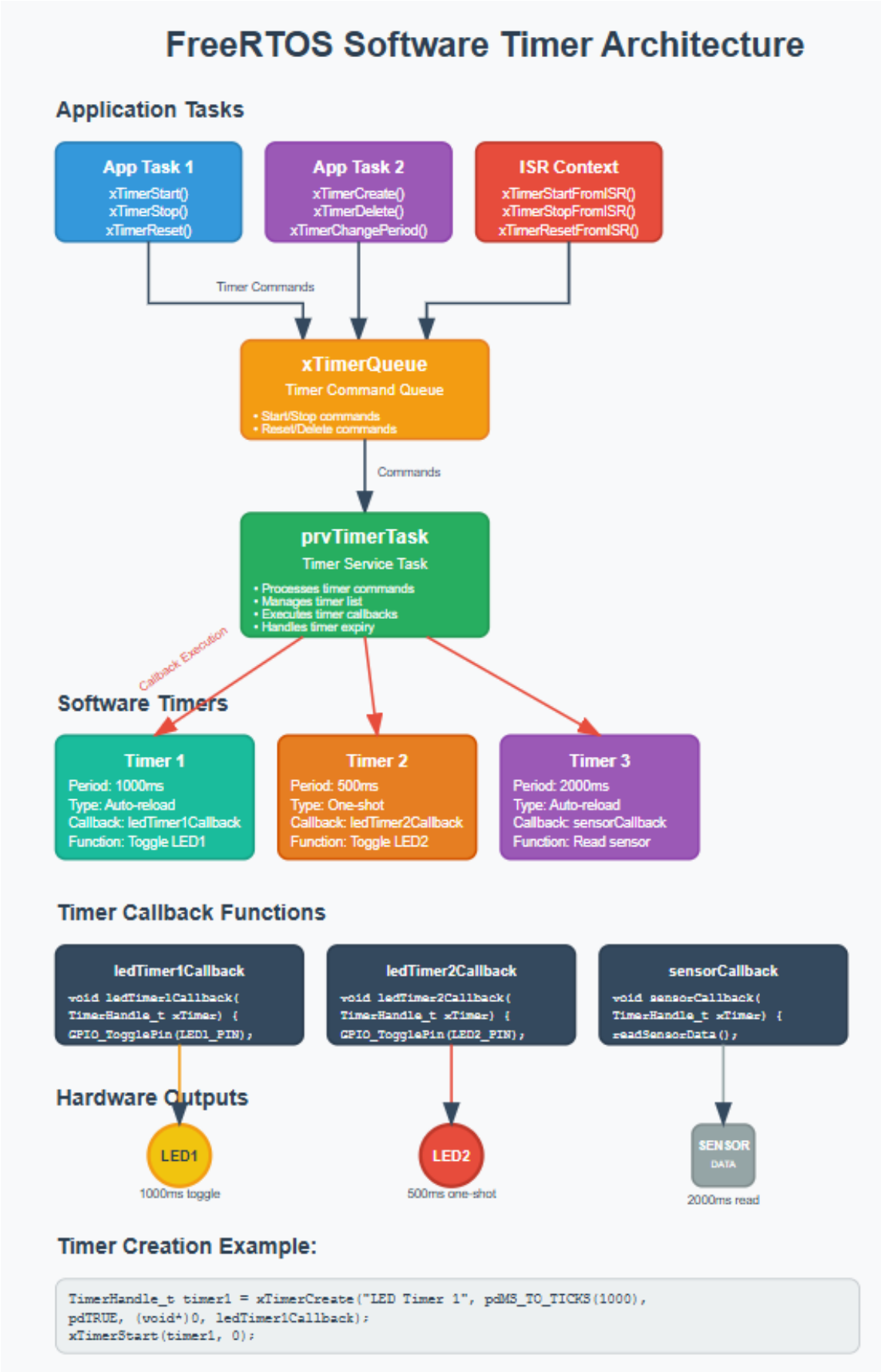


Image: FreeRTOS Software Timer Architecture

Description: A diagram showing the FreeRTOS timer service task (prvTimerTask) receiving commands from xTimerQueue. Multiple software timers (e.g., Timer1, Timer2) trigger callbacks, which toggle LEDs. Arrows indicate task scheduling and timer callback execution.

2. FreeRTOS Software Timer Configuration

- **Creation**: xTimerCreate(name, period, auto_reload, id, callback) creates a timer.
  - period: In ticks (use pdMS_TO_TICKS(ms) for milliseconds).
  - auto_reload: pdTRUE (repeats) or pdFALSE (one-shot).
  - id: Optional identifier for callback.
  - callback: Function called on timer expiry.
- **Operation**:
  - Start with xTimerStart(timer, timeout); timer runs after period ticks.
  - Callback runs in prvTimerTask context, not ISR, so avoid blocking operations.
- **Configuration** (via FreeRTOSConfig.h):
  - configUSE_TIMERS: Enable timers (set to 1).
  - configTIMER_TASK_PRIORITY: Priority of timer task (e.g., 3).
  - configTIMER_QUEUE_LENGTH: Max pending timer commands (e.g., 10).
  - configTIMER_TASK_STACK_DEPTH: Timer task stack size (e.g., 256 words).
- **Embedded Tip**: Set configTIMER_TASK_PRIORITY higher than application tasks to ensure timely callbacks; monitor stack usage with uxTimerGetTimerTaskStackHighWaterMark().

3. Comparison: Hardware vs. Software Timers

- **Hardware Timers**:
  - Configured via STM32 registers (e.g., TIM2->ARR, TIM2->PSC).
  - Trigger ISRs (e.g., TIM2_IRQHandler).
  - Limited by number of timers (e.g., STM32F1 has ~4-8 timers).
- **Software Timers**:
  - Managed by FreeRTOS, no hardware limit.
  - Simpler to scale (create multiple timers dynamically).
  - Less precise due to tick resolution (e.g., 1ms for 1000Hz tick).
- **Embedded Tip**: Use software timers for application-level timing (e.g., blinking, polling); use hardware timers for low-level control (e.g., PWM, ADC triggering).

4. Practice Program: Simple Software Timer

- **Objective**: Use a FreeRTOS software timer to toggle an LED every 500ms.
- **Code** (STM32 simulation):

```
/* Simple FreeRTOS software timer for LED toggle simulation */
#include <stdio.h>
#include <stdint.h>

/* Simulated GPIO registers */
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;

/* Simulated FreeRTOS timer API */
#define pdMS_TO_TICKS(ms) (ms)
#define pdTRUE 1
#define pdFALSE 0
```

```
typedef void* TimerHandle_t;

static void xTimerCreate(const char *name, uint32_t period, uint32_t auto_reload, void *id, void
(*callback)(TimerHandle_t)) {
    printf("Timer created: %s, period %u ms\n", name, period);
    callback(NULL); /* Simulate callback */
}

static void xTimerStart(TimerHandle_t timer, uint32_t timeout) { printf("Timer started\n"); }

/* Timer callback */
static void led_timer_callback(TimerHandle_t timer) {
    static uint8_t state = 0U;
    state = !state;
    *GPIO_BSRR = state ? (1U << 5) : (1U << (5 + 16)); /* Toggle PA5 */
    printf("LED %s\n", state ? "ON" : "OFF");
}

/* Main function */
int main(void) {
    TimerHandle_t led_timer;
    led_timer = xTimerCreate("LED_Timer", 500U, pdTRUE, NULL, led_timer_callback);
    xTimerStart(led_timer, 0U);
    printf("Scheduler started\n");
    return 0;
}
```

Project: Software Timer-Based LED Blinker

- **Objective**: Implement a FreeRTOS-based LED blinker using software timers to toggle two LEDs (PA5, PA6) at different rates (500ms, 1000ms), with UART logging for debugging.
- **Requirements**:
  - Configure PA5, PA6 as outputs for LEDs.
  - Use two FreeRTOS software timers (auto-reload) for 500ms and 1000ms periods.
  - Log LED states via UART (PA9-TX, 115200 baud).
  - Enable FreeRTOS timers in STM32CubeMX.
  - Comply with MISRA rules (e.g., 10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure GPIO, UART, and FreeRTOS timers via STM32CubeMX.
  - **Timers**: Create two auto-reload timers for LED toggling.
  - **Logging**: Use static buffer for UART logging to avoid heap usage.
  - **Task**: Single task to initialize timers and start scheduler.
  - **Debugging**: Use STM32CubeIDE debugger to verify timer callbacks.
- **Code** (STM32 with FreeRTOS and HAL, heavily commented):

```
/* FreeRTOS software timer-based LED blinker */
#include "stm32f1xx_hal.h"
#include "FreeRTOS.h"
#include "task.h"
#include "timers.h"
#include <stdint.h>
#include <string.h>

/* UART handle for logging */
static UART_HandleTypeDef huart1;

/* Timer handles for LEDs */
static TimerHandle_t led1_timer;
static TimerHandle_t led2_timer;
```

```c
/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U];

/* Function prototypes */
static void init_gpio(void);
static void init_uart(void);
static void log_message(const uint8_t *message);
static void led1_timer_callback(TimerHandle_t timer);
static void led2_timer_callback(TimerHandle_t timer);
static void init_timers_task(void *pvParameters);

/* Initialize GPIO for LEDs (PA5, PA6) and UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; /* MISRA Rule 9.1: Initialize all fields */

    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA5, PA6 as outputs for LEDs */
    gpio_init.Pin = GPIO_PIN_5 | GPIO_PIN_6; /* MISRA Rule 12.1: Explicit grouping */
    gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &gpio_init);

    /* Configure PA9, PA10 for UART */
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10;
    gpio_init.Mode = GPIO_MODE_AF_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init);

    /* Initialize LEDs to OFF */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6, GPIO_PIN_RESET);
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    /* Enable USART1 clock */
    __HAL_RCC_USART1_CLK_ENABLE();

    /* Configure UART parameters */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200U;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

    /* Initialize UART and handle errors */
    if (HAL_UART_Init(&huart1) != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"UART Init Failed\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)error_msg,
                          (uint16_t)strlen((const char *)error_msg), 10U); /* MISRA Rule 10.1: Cast */
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
}
```

```
/* Log message via UART */
static void log_message(const uint8_t *message)
{
    /* Check for valid input */
    if (message != NULL) {
        HAL_UART_Transmit(&huart1, (uint8_t *)message,
                          (uint16_t)strlen((const char *)message), 10U); /* MISRA Rule 10.1: Cast */
    }
}

/* Callback for LED1 timer (500ms) */
static void led1_timer_callback(TimerHandle_t timer)
{
    /* Toggle LED on PA5 */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

    /* Log LED state */
    snprintf((char *)log_buffer, sizeof(log_buffer),
            "LED1 %s\r\n", HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) ? "ON" : "OFF"); /* MISRA Rule 10.1:
Cast */
    log_message(log_buffer);

    /* GDB breakpoint here to inspect timer execution */
}

/* Callback for LED2 timer (1000ms) */
static void led2_timer_callback(TimerHandle_t timer)
{
    /* Toggle LED on PA6 */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);

    /* Log LED state */
    snprintf((char *)log_buffer, sizeof(log_buffer),
            "LED2 %s\r\n", HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_6) ? "ON" : "OFF"); /* MISRA Rule 10.1:
Cast */
    log_message(log_buffer);

    /* GDB breakpoint here to inspect timer execution */
}

/* Task to initialize timers and start them */
static void init_timers_task(void *pvParameters)
{
    (void)pvParameters; /* MISRA Rule 2.7: Unused parameter */

    /* Create LED1 timer: 500ms, auto-reload */
    led1_timer = xTimerCreate("LED1_Timer", pdMS_TO_TICKS(500U), pdTRUE, NULL, led1_timer_callback);
    if (led1_timer == NULL) {
        const uint8_t *error_msg = (const uint8_t *)"LED1 Timer Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Create LED2 timer: 1000ms, auto-reload */
    led2_timer = xTimerCreate("LED2_Timer", pdMS_TO_TICKS(1000U), pdTRUE, NULL, led2_timer_callback);
    if (led2_timer == NULL) {
        const uint8_t *error_msg = (const uint8_t *)"LED2 Timer Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
```

```
 /* Start timers */
    if (xTimerStart(led1_timer, 0U) != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"LED1 Timer Start Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    if (xTimerStart(led2_timer, 0U) != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"LED2 Timer Start Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Log startup */
    const uint8_t *startup_msg = (const uint8_t *)"LED Timers Started\r\n";
    log_message(startup_msg);

    /* Task can delete itself after starting timers */
    vTaskDelete(NULL);
}

/* Main function */
int main(void)
{
    /* Initialize HAL library */
    HAL_Init();

    /* Initialize peripherals */
    init_gpio();
    init_uart();

    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"FreeRTOS Timer Blinker Started\r\n";
    log_message(startup_msg);

    /* Create task to initialize timers */
    BaseType_t result;
    result = xTaskCreate(init_timers_task, "InitTimers", 256U, NULL, 1U, NULL);
    if (result != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"Timer Task Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Start FreeRTOS scheduler */
    vTaskStartScheduler();

    /* Should not reach here */
    while (1) { /* MISRA Rule 15.6: Braces */
        /* Infinite loop if scheduler fails */
    }

    return 0; /* MISRA Rule 17.7: Explicit return */
}
```

- **Simulated Version** (For PC testing, heavily commented):

```
/* Simulated FreeRTOS timer-based LED blinker */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

/* Simulated GPIO registers */
volatile uint32_t *const GPIO_BSRR = (volatile uint32_t *)0x40010004;
static uint8_t led1_state = 0U;
static uint8_t led2_state = 0U;

/* Simulated FreeRTOS timer API */
#define pdMS_TO_TICKS(ms) (ms)
#define pdTRUE 1
#define pdFALSE 0

typedef void* TimerHandle_t;

static void xTimerCreate(const char *name, uint32_t period, uint32_t auto_reload, void *id, void
(*callback)(TimerHandle_t)) {
    printf("Timer created: %s, period %u ms\n", name, period);
    callback(NULL); /* Simulate callback */
}

static void xTimerStart(TimerHandle_t timer, uint32_t timeout) { printf("Timer started\n"); }

/* Static buffer for logging */
static uint8_t log_buffer[32U];

/* Log message */
static void log_message(const uint8_t *message)
{
    if (message != NULL) {
        printf("%s", (const char *)message);
    }
}

/* Callback for LED1 timer (500ms) */
static void led1_timer_callback(TimerHandle_t timer)
{
    led1_state = !led1_state;
    *GPIO_BSRR = led1_state ? (1U << 5) : (1U << (5 + 16));
    snprintf((char *)log_buffer, sizeof(log_buffer),
            "LED1 %s\n", led1_state ? "ON" : "OFF");
    log_message(log_buffer);
}

/* Callback for LED2 timer (1000ms) */
static void led2_timer_callback(TimerHandle_t timer)
{
    led2_state = !led2_state;
    *GPIO_BSRR = led2_state ? (1U << 6) : (1U << (6 + 16));
    snprintf((char *)log_buffer, sizeof(log_buffer),
            "LED2 %s\n", led2_state ? "ON" : "OFF");
    log_message(log_buffer);
}

/* Main function */
int main(void)
{
    /* Simulate startup */
    const uint8_t *startup_msg = (const uint8_t *)"FreeRTOS Timer Blinker Started\n";
    log_message(startup_msg);
```

```
    /* Create and start timers */
    TimerHandle_t led1_timer = xTimerCreate("LED1_Timer", 500U, pdTRUE, NULL, led1_timer_callback);
    TimerHandle_t led2_timer = xTimerCreate("LED2_Timer", 1000U, pdTRUE, NULL, led2_timer_callback);
    xTimerStart(led1_timer, 0U);
    xTimerStart(led2_timer, 0U);

    printf("Timers started\n");
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure:
    - GPIO: PA5, PA6 (outputs).
    - USART1: PA9-TX, PA10-RX, 115200 baud.
    - FreeRTOS: Enable middleware, enable timers (configUSE_TIMERS=1), set configTIMER_TASK_PRIORITY=3, configTIMER_QUEUE_LENGTH=10, configTIMER_TASK_STACK_DEPTH=256.
  - Connect LEDs (with resistors) to PA5, PA6; USB-to-serial adapter to PA9/PA10.
  - Test with STM32CubeIDE and PuTTY for UART output.
  - Run Cppcheck for MISRA compliance: cppcheck --enable=all --addon=misra.json project.c.
- **Optimizations**:
  - **Non-blocking**: Software timers run in prvTimerTask, freeing main tasks.
  - **Memory**: Static log_buffer (MISRA Rule 20.3); minimal timer overhead.
  - **Robustness**: Error checking for timer creation/start.
  - **MISRA**: Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns).
- **Embedded Tip**: Use xTimerGetPeriod() to verify timer intervals during debugging; increase configTIMER_QUEUE_LENGTH if many timers are used.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for FreeRTOS setup.
- **FreeRTOS**: Integrated via STM32CubeMX (select timers in FreeRTOS middleware).
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32f1x.cfg).

Resources

- **FreeRTOS Timer Docs**: https://www.freertos.org/FreeRTOS-timers.html for API details.
- **STM32 Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **Online**: FreeRTOS.org tutorials, C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a one-shot timer to turn off LEDs after 5s.
- **Optimize**: Adjust timer periods dynamically with xTimerChangePeriod().
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Learn Resource Protection

Goal

Master FreeRTOS resource protection mechanisms, including mutexes and semaphores, to safely manage shared resources in a multitasking embedded system, with a focus on implementing a shared resource UART task for logging on an STM32 microcontroller.

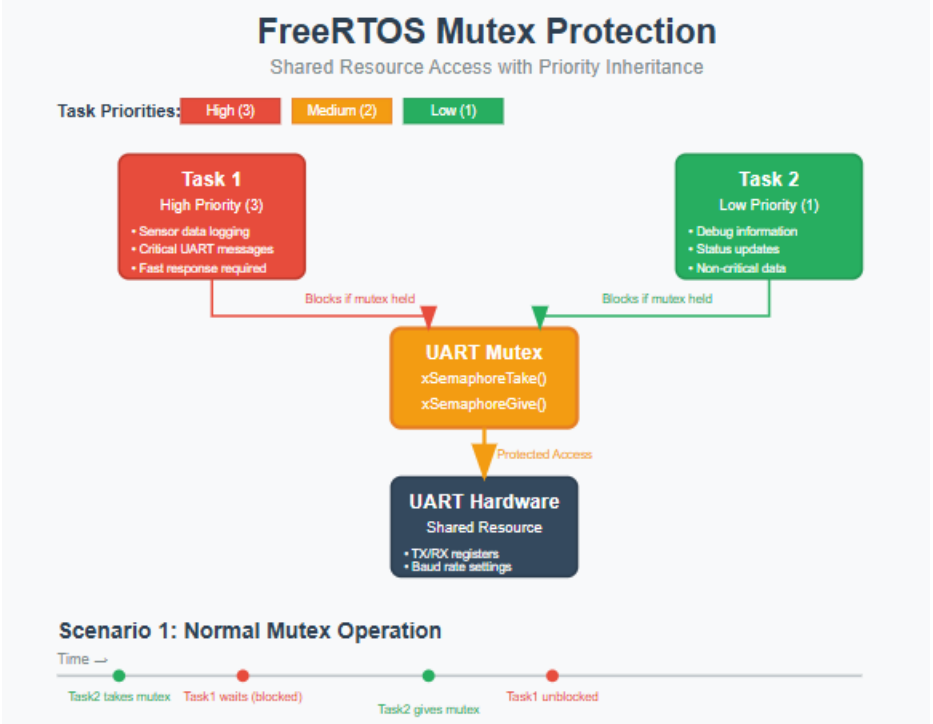
Tasks

- **Study**: Learn FreeRTOS mutexes and semaphores for protecting shared resources in multitasking systems.
- **Practice**: Write code using semaphores to synchronize tasks accessing a shared UART.
- **Project**: Develop a FreeRTOS-based system with multiple tasks sharing a UART for logging, using a mutex to prevent data corruption.
- **Tools**: STM32CubeIDE, FreeRTOS (integrated via STM32CubeMX).
- **Resources**: FreeRTOS semaphore documentation, embedded.com RTOS articles.

Detailed Notes

1. Resource Protection Overview

- **Shared Resources**: Data or peripherals (e.g., UART, buffers) accessed by multiple tasks, risking data corruption or race conditions.
- **FreeRTOS Synchronization Primitives**:
  - **Binary Semaphore**: Signals events (e.g., ISR-to-task synchronization). Created with xSemaphoreCreateBinary().
  - **Counting Semaphore**: Tracks multiple resource instances (e.g., buffer slots). Created with xSemaphoreCreateCounting().
  - **Mutex**: Ensures mutual exclusion for shared resources. Created with xSemaphoreCreateMutex().
- **Mutex vs. Semaphore**:
  - **Mutex**: Includes priority inheritance to prevent priority inversion; used for resource ownership (e.g., UART access).
  - **Semaphore**: No priority inheritance; used for signaling or counting resources.
- **Key Functions**:
  - xSemaphoreTake(sem, timeout): Acquire semaphore/mutex (blocks if unavailable).
  - xSemaphoreGive(sem): Release semaphore/mutex.
  - xSemaphoreTakeFromISR(), xSemaphoreGiveFromISR(): ISR-safe versions.
- **Embedded Tip**: Use mutexes for shared peripheral access (e.g., UART); use binary semaphores for task-ISR synchronization.



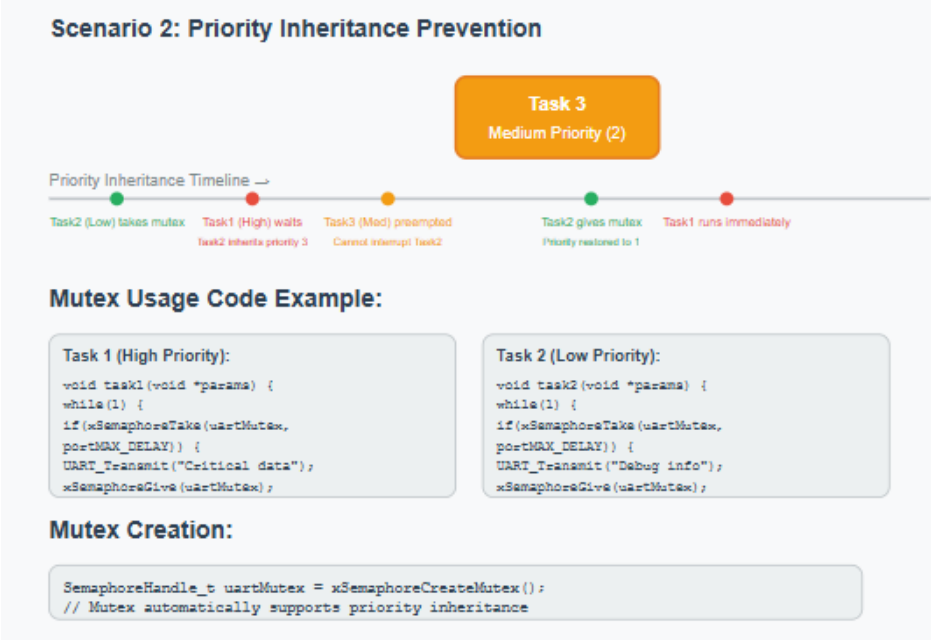


Image: FreeRTOS Mutex Protection

Description: A diagram showing two tasks (Task1, Task2) attempting to access a shared UART. A mutex guards the UART, ensuring only one task transmits at a time. Arrows indicate task blocking when the mutex is held and priority inheritance preventing inversion.

2. Mutexes in FreeRTOS

- **Purpose**: Protect shared resources by ensuring only one task accesses them at a time.
- **Priority Inheritance**: If a low-priority task holds a mutex, and a high-priority task tries to take it, the low-priority task temporarily inherits the higher priority to avoid delays.
- **Usage**:
  - Create: xSemaphoreCreateMutex().
  - Take: xSemaphoreTake(mutex, timeout) before accessing resource.
  - Give: xSemaphoreGive(mutex) after access.
- **Embedded Tip**: Keep mutex hold times short to minimize blocking; avoid recursive mutexes in embedded systems for simplicity.

3. Semaphores in FreeRTOS

- **Binary Semaphore**:
  - Used for signaling (e.g., ISR signals task to process data).
  - Initially empty; given by ISR/task, taken by task.
- **Counting Semaphore**:
  - Tracks available resources (e.g., buffer slots).
  - Initialize with max count (e.g., xSemaphoreCreateCounting(max, initial)).
- **Embedded Tip**: Use binary semaphores for event-driven tasks; avoid counting semaphores unless resource pooling is needed.

4. Practice Program: Binary Semaphore for UART Synchronization

- **Objective**: Use a binary semaphore to synchronize a task and ISR for UART transmission.
- **Code** (STM32 simulation):

```
/* FreeRTOS binary semaphore for UART synchronization simulation */
#include <stdio.h>
#include <stdint.h>

/* Simulated FreeRTOS semaphore API */
#define pdTRUE 1
#define pdFALSE 0

typedef void* SemaphoreHandle_t;

static SemaphoreHandle_t xSemaphoreCreateBinary(void) { printf("Binary semaphore created\n"); return
(SemaphoreHandle_t)1; }

static void xSemaphoreTake(SemaphoreHandle_t sem, uint32_t timeout) { printf("Semaphore taken\n"); }

static void xSemaphoreGiveFromISR(SemaphoreHandle_t sem, int32_t *higher_task) { printf("Semaphore
given from ISR\n"); }

static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); }

static void xTaskCreate(void (*func)(void *), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) {
    printf("Created task: %s\n", name);
    func(params);
}
```

```
/* Simulated UART registers */
volatile uint32_t *const USART_SR = (volatile uint32_t *)0x40013800;
volatile uint32_t *const USART_DR = (volatile uint32_t *)0x40013804;

/* Semaphore handle */
static SemaphoreHandle_t uart_sem;

/* UART task */
static void uart_task(void *params) {
    (void)params;
    while (1) {
        xSemaphoreTake(uart_sem, portMAX_DELAY);
        printf("UART TX: Hello\n");
        vTaskDelay(1000);
    }
}

/* Simulated ISR */
static void simulate_uart_isr(void) {
    static int32_t higher_task = 0;
    xSemaphoreGiveFromISR(uart_sem, &higher_task);
}

/* Main function */
int main(void) {
    uart_sem = xSemaphoreCreateBinary();
    xTaskCreate(uart_task, "UART_Task", 128, NULL, 1, NULL);
    simulate_uart_isr();
    printf("Scheduler started\n");
    return 0;
}
```

Project: Shared Resource UART Task

- **Objective**: Implement a FreeRTOS-based system with two tasks sharing a UART for logging, protected by a mutex to prevent data corruption.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud).
  - Create two tasks (Task1: log every 500ms, Task2: log every 1000ms).
  - Use a mutex to protect UART access.
  - Log messages via UART without interleaving.
  - Comply with MISRA rules (e.g., 10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART and FreeRTOS via STM32CubeMX.
  - **Mutex**: Create a mutex to guard UART transmissions.
  - **Tasks**: Two tasks with different logging intervals, using xSemaphoreTake/Give for UART access.
  - **Logging**: Use static buffer for messages (MISRA Rule 20.3).
  - **Debugging**: Use STM32CubeIDE debugger to verify mutex behavior.
- **Code** (STM32 with FreeRTOS and HAL, heavily commented):

```
/* FreeRTOS shared UART with mutex protection */

#include "stm32f1xx_hal.h"

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <stdint.h>
#include <string.h>
```

```
/* UART handle for logging */
static UART_HandleTypeDef huart1;

/* Mutex for UART protection */
static SemaphoreHandle_t uart_mutex;

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U];

/* Function prototypes */
static void init_gpio(void);
static void init_uart(void);
static void log_message(const uint8_t *message);
static void task1(void *pvParameters);
static void task2(void *pvParameters);

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; /* MISRA Rule 9.1: Initialize all fields */

    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA9, PA10 for UART */
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; /* MISRA Rule 12.1: Explicit grouping */
    gpio_init.Mode = GPIO_MODE_AF_PP;
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &gpio_init);
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    /* Enable USART1 clock */
    __HAL_RCC_USART1_CLK_ENABLE();

    /* Configure UART parameters */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200U;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

    /* Initialize UART and handle errors */
    if (HAL_UART_Init(&huart1) != HAL_OK) {
        const uint8_t *error_msg = (const uint8_t *)"UART Init Failed\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)error_msg,
                          (uint16_t)strlen((const char *)error_msg), 10U); /* MISRA Rule 10.1: Cast */
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
}

/* Log message via UART with mutex protection */
static void log_message(const uint8_t *message)
{
    /* Check for valid input */
    if (message != NULL) {
        /* Take mutex to ensure exclusive UART access */
        if (xSemaphoreTake(uart_mutex, pdMS_TO_TICKS(100U)) == pdTRUE) {
            HAL_UART_Transmit(&huart1, (uint8_t *)message,
                              (uint16_t)strlen((const char *)message), 10U); /* MISRA Rule 10.1: Cast
*/
```

```
            xSemaphoreGive(uart_mutex); /* Release mutex */
        } else {
            /* Log timeout error (non-critical, continue) */
            HAL_UART_Transmit(&huart1, (uint8_t *)"UART Mutex Timeout\r\n",
                              (uint16_t)strlen("UART Mutex Timeout\r\n"), 10U);
        }
    }
}

/* Task 1: Log message every 500ms */
static void task1(void *pvParameters)
{
    (void)pvParameters; /* MISRA Rule 2.7: Unused parameter */

    while (1) {
        /* Prepare message */
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "Task1: Tick %u\r\n", (unsigned int)xTaskGetTickCount()); /* MISRA Rule 10.1: Cast */

        /* Log with mutex protection */
        log_message(log_buffer);

        /* Delay 500ms */
        vTaskDelay(pdMS_TO_TICKS(500U));
    }
}

/* Task 2: Log message every 1000ms */
static void task2(void *pvParameters)
{
    (void)pvParameters; /* MISRA Rule 2.7: Unused parameter */

    while (1) {
        /* Prepare message */
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "Task2: Tick %u\r\n", (unsigned int)xTaskGetTickCount()); /* MISRA Rule 10.1: Cast */

        /* Log with mutex protection */
        log_message(log_buffer);

        /* Delay 1000ms */
        vTaskDelay(pdMS_TO_TICKS(1000U));
    }
}

/* Main function */
int main(void)
{
    /* Initialize HAL library */
    HAL_Init();

    /* Initialize peripherals */
    init_gpio();
    init_uart();

    /* Create UART mutex */
    uart_mutex = xSemaphoreCreateMutex();
    if (uart_mutex == NULL) {
        const uint8_t *error_msg = (const uint8_t *)"Mutex Creation Failed\r\n";
        HAL_UART_Transmit(&huart1, (uint8_t *)error_msg,
                          (uint16_t)strlen((const char *)error_msg), 10U);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }
```

```
    /* Log startup message */
    const uint8_t *startup_msg = (const uint8_t *)"Shared UART Started\r\n";
    log_message(startup_msg);

    /* Create tasks */
    BaseType_t result;

    /* Task 1: Priority 2, 256-word stack */
    result = xTaskCreate(task1, "Task1", 256U, NULL, 2U, NULL);
    if (result != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"Task1 Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Task 2: Priority 1, 256-word stack */
    result = xTaskCreate(task2, "Task2", 256U, NULL, 1U, NULL);
    if (result != pdPASS) {
        const uint8_t *error_msg = (const uint8_t *)"Task2 Creation Failed\r\n";
        log_message(error_msg);
        while (1) { /* MISRA Rule 15.6: Braces */
            /* Infinite loop on error */
        }
    }

    /* Start FreeRTOS scheduler */
    vTaskStartScheduler();

    /* Should not reach here */
    while (1) { /* MISRA Rule 15.6: Braces */
        /* Infinite loop if scheduler fails */
    }

    return 0; /* MISRA Rule 17.7: Explicit return */
}
```

- **Simulated Version** (For PC testing, heavily commented):

```
/* Simulated FreeRTOS shared UART with mutex */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

/* Simulated FreeRTOS API */
#define pdTRUE 1
#define pdFALSE 0
#define pdMS_TO_TICKS(ms) (ms)

typedef void* SemaphoreHandle_t;

static SemaphoreHandle_t xSemaphoreCreateMutex(void) { printf("Mutex created\n"); return
(SemaphoreHandle_t)1; }

static void xSemaphoreTake(SemaphoreHandle_t sem, uint32_t timeout) { printf("Mutex taken\n"); }

static void xSemaphoreGive(SemaphoreHandle_t sem) { printf("Mutex given\n"); }

static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); }

static void xTaskCreate(void (*func)(void *), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) {
    printf("Created task: %s\n", name);
    func(params); }
```

```c
static void vTaskStartScheduler(void) { printf("Scheduler started\n"); }

static uint32_t xTaskGetTickCount(void) { return 1000; }

/* Static buffer for logging */
static uint8_t log_buffer[32U];

/* Log message */
static void log_message(const uint8_t *message)
{
    if (message != NULL) {
        xSemaphoreTake((SemaphoreHandle_t)1, 100);
        printf("%s", (const char *)message);
        xSemaphoreGive((SemaphoreHandle_t)1);
    }
}

/* Task 1: Log every 500ms */
static void task1(void *params)
{
    (void)params;
    while (1) {
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "Task1: Tick %u\n", (unsigned int)xTaskGetTickCount());
        log_message(log_buffer);
        vTaskDelay(500);
    }
}

/* Task 2: Log every 1000ms */
static void task2(void *params)
{
    (void)params;
    while (1) {
        snprintf((char *)log_buffer, sizeof(log_buffer),
                "Task2: Tick %u\n", (unsigned int)xTaskGetTickCount());
        log_message(log_buffer);
        vTaskDelay(1000);
    }
}

/* Main function */
int main(void)
{
    const uint8_t *startup_msg = (const uint8_t *)"Shared UART Started\n";
    log_message(startup_msg);

    xTaskCreate(task1, "Task1", 256, NULL, 2, NULL);
    xTaskCreate(task2, "Task2", 256, NULL, 1, NULL);

    vTaskStartScheduler();
    return 0;
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure:
    - USART1: PA9-TX, PA10-RX, 115200 baud.
    - FreeRTOS: Enable middleware, set configTICK_RATE_HZ=1000, enable mutexes (configUSE_MUTEXES=1).
  - Connect USB-to-serial adapter to PA9/PA10; test with PuTTY.
  - Test with STM32CubeIDE debugger; set breakpoints in log_message to verify mutex behavior.
  - Run Cppcheck for MISRA compliance: cppcheck --enable=all --addon=misra.json project.c.
- **Optimizations**:

- **Mutex Protection**: Ensures non-interleaved UART output.
- **Memory**: Static log_buffer (MISRA Rule 20.3); 256-word task stacks.
- **Robustness**: Mutex timeout (100ms) prevents deadlock; error checking for task/mutex creation.
- **MISRA**: Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns).

- **Embedded Tip**: Use vTaskPrioritySet() to dynamically adjust task priorities if mutex contention causes delays; monitor mutex hold times with FreeRTOS trace tools.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for FreeRTOS setup.
- **FreeRTOS**: Integrated via STM32CubeMX (enable mutexes in middleware).
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32f1x.cfg).

Resources

- **FreeRTOS Semaphore Docs**: https://www.freertos.org/a00113.html for API details.
- **embedded.com RTOS Articles**: Search "RTOS resource protection" on https://www.embedded.com.
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Replace mutex with binary semaphore and observe interleaving issues.
- **Optimize**: Add priority inheritance test by setting Task1 priority to 3.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Design RTOS System

Goal

Design an RTOS-based data logger system using FreeRTOS, incorporating multiple tasks (sensor data acquisition, data logging, UART command processing) to create a robust, scalable embedded application for an STM32 microcontroller.

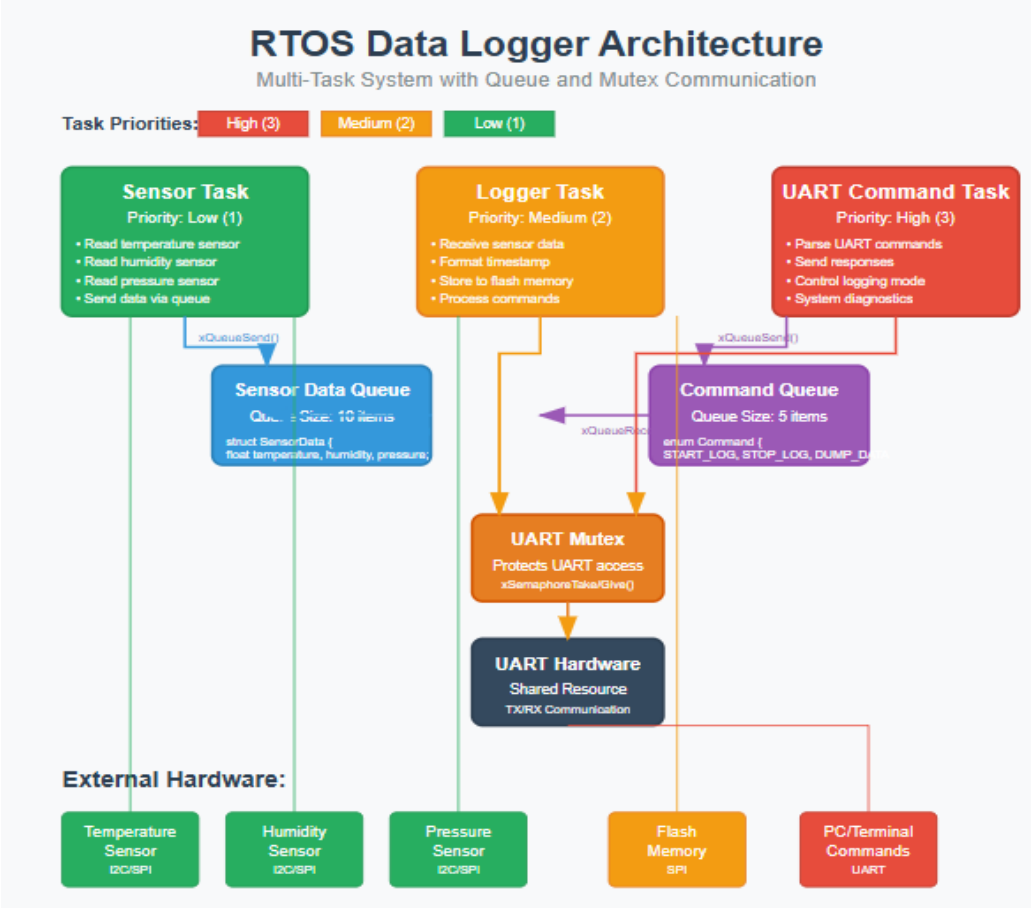
Tasks

- **Study**: Learn RTOS system design principles, task decomposition, and inter-task communication.
- **Practice**: Plan the architecture of a data logger system with sensor, logging, and UART tasks.
- **Project**: Design and prototype a FreeRTOS-based data logger with three tasks, using queues for data transfer and a mutex for shared UART access.
- **Tools**: STM32CubeIDE, FreeRTOS (integrated via STM32CubeMX).
- **Resources**: FreeRTOS.org, embedded.com RTOS design articles.

Detailed Notes

1. RTOS System Design Principles

- **Task Decomposition**: Break system into independent tasks (e.g., sensor reading, logging, communication) with clear responsibilities.
- **Inter-Task Communication**: Use FreeRTOS queues for data exchange (e.g., sensor data to logger) and semaphores/mutexes for synchronization.
- **Priorities**: Assign higher priorities to time-critical tasks (e.g., sensor sampling) and lower priorities to background tasks (e.g., logging).
- **Resource Management**: Protect shared resources (e.g., UART, buffers) with mutexes to prevent race conditions.
- **Scalability**: Design modular tasks to allow easy addition of features (e.g., new sensors).
- **Embedded Tip**: Use FreeRTOS trace tools (e.g., Percepio Tracealyzer) to analyze task timing and resource usage during design.



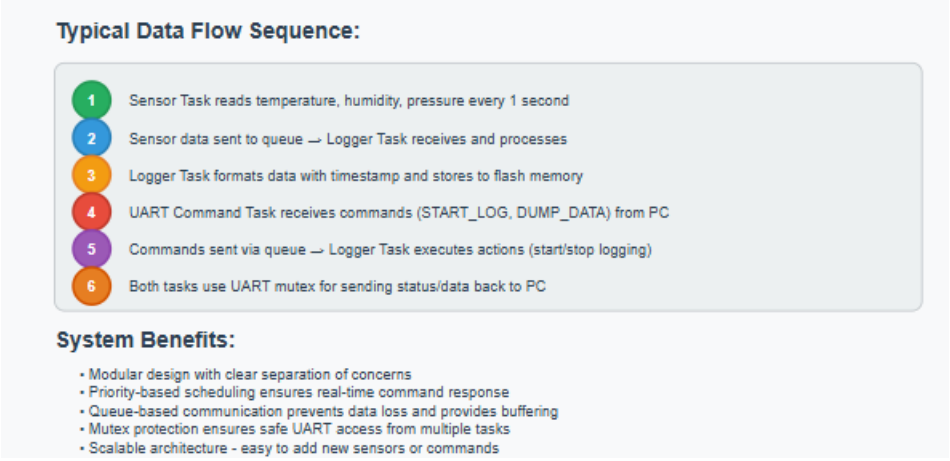


Image: RTOS Data Logger Architecture

Description: A diagram showing three tasks (Sensor Task, Logger Task, UART Command Task) in a FreeRTOS system. Arrows indicate data flow via a queue (sensor data to logger) and mutex-protected UART access. Task priorities and interactions (e.g., command parsing triggering logger actions) are labeled.

2. Data Logger System Design

- **Components**:
  - **Sensor Task**: Reads sensor data (e.g., simulated ADC for temperature) at regular intervals (e.g., 500ms), sends to queue.
  - **Logger Task**: Receives data from queue, formats it, and logs to UART (protected by mutex).

- **UART Command Task**: Processes incoming UART commands (e.g., start/stop logging) and controls system state.
- **Inter-Task Communication**:
  - **Queue**: Sensor Task sends data (e.g., float temperature) to Logger Task.
  - **Mutex**: Protects UART access for Logger and UART Command tasks.
  - **Binary Semaphore**: Signals UART Command Task on new RX data.
- **Priorities**:
  - Sensor Task: Priority 3 (time-critical).
  - Logger Task: Priority 2 (background processing).
  - UART Command Task: Priority 1 (user interaction).
- **Embedded Tip**: Ensure queue size (e.g., 10 items) supports peak data rate; use timeouts in xSemaphoreTake to avoid deadlocks.

3. Practice: System Design Plan

- **Steps**:
  1. Define tasks and responsibilities (Sensor, Logger, UART Command).
  2. Specify communication mechanisms (queue for data, mutex for UART).
  3. Assign priorities and stack sizes (e.g., 256 words per task).
  4. Plan error handling (e.g., queue full, UART timeout).
  5. Prototype task interactions in simulation.
- **Embedded Tip**: Sketch task flow before coding; validate design with FreeRTOS uxTaskGetSystemState() to check task states.

4. Practice Program: Simulated Task Interaction

- **Objective**: Prototype task interactions with a queue and mutex.
- **Code** (Simulation, heavily commented):

```
/* Simulated FreeRTOS data logger task interaction */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for message formatting

/* Simulated FreeRTOS API */
#define pdTRUE 1 // Define FreeRTOS true constant
#define pdFALSE 0 // Define FreeRTOS false constant
#define pdMS_TO_TICKS(ms) (ms) // Convert milliseconds to ticks (simplified)

typedef void* QueueHandle_t; // Simulated queue handle type
typedef void* SemaphoreHandle_t; // Simulated semaphore/mutex handle type

static QueueHandle_t xQueueCreate(uint32_t len, uint32_t size) { printf("Queue created: %u items\n",
len); return (QueueHandle_t)1; } // Simulate queue creation

static void xQueueSend(QueueHandle_t queue, const void *item, uint32_t timeout) { printf("Queue send:
%f\n", *(float*)item); } // Simulate sending to queue

static void xQueueReceive(QueueHandle_t queue, void *item, uint32_t timeout) { *(float*)item = 25.0f;
printf("Queue receive: %f\n", *(float*)item); } // Simulate receiving from queue

static SemaphoreHandle_t xSemaphoreCreateMutex(void) { printf("Mutex created\n"); return
(SemaphoreHandle_t)1; } // Simulate mutex creation

static void xSemaphoreTake(SemaphoreHandle_t sem, uint32_t timeout) { printf("Mutex taken\n"); } //
Simulate mutex take
```

```
static void xSemaphoreGive(SemaphoreHandle_t sem) { printf("Mutex given\n"); } // Simulate mutex
release

static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); } // Simulate task delay

static void xTaskCreate(void (*func)(void*), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) { printf("Created task: %s\n", name); func(params); } // Simulate task creation

static void vTaskStartScheduler(void) { printf("Scheduler started\n"); } // Simulate scheduler start

static uint32_t xTaskGetTickCount(void) { return 1000; } // Simulate tick count

/* Static buffer for logging */
static uint8_t log_buffer[32U]; // Static buffer for UART messages (MISRA Rule 20.3)

/* Log message */
static void log_message(const uint8_t *message) // Function to simulate UART logging
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        xSemaphoreTake((SemaphoreHandle_t)1, 100); // Take mutex to protect output
        printf("%s", (const char *)message); // Print message to console
        xSemaphoreGive((SemaphoreHandle_t)1); // Release mutex
    }
}

/* Sensor task: Simulate sensor reading */
static void sensor_task(void *params)
{
    (void)params; // Mark unused parameter (MISRA Rule 2.7)
    QueueHandle_t queue = (QueueHandle_t)params; // Get queue handle from parameters
    while (1) { // Infinite loop for task (MISRA Rule 15.6)
        float temp = 25.0f; // Simulate sensor reading (temperature)
        xQueueSend(queue, &temp, 0); // Send temperature to queue
        vTaskDelay(500); // Delay 500ms to simulate sampling rate
    }
}

/* Logger task: Log sensor data */
static void logger_task(void *params)
{
    (void)params; // Mark unused parameter
    QueueHandle_t queue = (QueueHandle_t)params; // Get queue handle
    while (1) { // Infinite loop
        float temp; // Variable to hold received temperature
        xQueueReceive(queue, &temp, portMAX_DELAY); // Receive from queue (block until data)
        snprintf((char *)log_buffer, sizeof(log_buffer), "Temp: %.1f C\n", temp); // Format
temperature message
        log_message(log_buffer); // Log message via UART
    }
}

/* Main function */
int main(void)
{
    QueueHandle_t sensor_queue; // Queue for sensor data
    sensor_queue = xQueueCreate(10, sizeof(float)); // Create queue for 10 float items
    xTaskCreate(sensor_task, "Sensor", 128, sensor_queue, 3, NULL); // Create sensor task, priority 3
    xTaskCreate(logger_task, "Logger", 128, sensor_queue, 2, NULL); // Create logger task, priority 2
    vTaskStartScheduler(); // Start FreeRTOS scheduler
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: RTOS-Based Data Logger Design

- **Objective**: Design and prototype a FreeRTOS-based data logger with three tasks: Sensor Task (reads simulated ADC data), Logger Task (logs data to UART), and UART Command Task (processes start/stop commands).
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX.
  - Sensor Task: Simulate ADC temperature readings every 500ms, send to queue.
  - Logger Task: Receive data from queue, log to UART with mutex protection.
  - UART Command Task: Process commands (e.g., "START", "STOP") via DMA RX, control logging state.
  - Use queue for sensor data, mutex for UART, binary semaphore for UART RX interrupts.
  - Comply with MISRA rules (e.g., 10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, DMA, and FreeRTOS (tasks, queue, mutex, semaphore).
  - **Tasks**: Sensor (priority 3), Logger (priority 2), UART Command (priority 1).
  - **Communication**: Queue for sensor data, mutex for UART, semaphore for UART RX.
  - **Debugging**: Use STM32CubeIDE debugger to verify task interactions and queue state.
- **Code** (STM32 with FreeRTOS and HAL, each line commented):

```
/* FreeRTOS-based data logger design */
#include "stm32f1xx_hal.h" // Include STM32 HAL library for hardware access
#include "FreeRTOS.h" // Include FreeRTOS core for tasks and scheduler
#include "task.h" // Include FreeRTOS task management
#include "queue.h" // Include FreeRTOS queue for inter-task communication
#include "semphr.h" // Include FreeRTOS semaphores/mutexes
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for message handling

/* UART and DMA handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX

/* FreeRTOS handles */
static QueueHandle_t sensor_queue; // Queue for sensor data
static SemaphoreHandle_t uart_mutex; // Mutex for UART access
static SemaphoreHandle_t uart_rx_sem; // Binary semaphore for UART RX interrupts

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART logging
static uint8_t rx_buffer[32U]; // Buffer for UART RX data
static volatile uint8_t logging_enabled = 1U; // Flag to control logging state

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_freertos(void); // Initialize FreeRTOS components
static void log_message(const uint8_t *message); // Log message via UART
static void sensor_task(void *pvParameters); // Task to simulate sensor readings
static void logger_task(void *pvParameters); // Task to log sensor data
static void uart_command_task(void *pvParameters); // Task to process UART commands

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9, PA10 for UART (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
```

```
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH; // Set high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins
}

/* Initialize USART1 with DMA for RX */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock for RX
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        HAL_UART_Transmit(&huart1, (uint8_t *)"UART Init Failed\r\n", 18U, 10U); // Send error message
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Channel5; // Set DMA channel for UART RX
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment for
peripheral
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment for memory
    hdma_usart1_rx.Init.Mode = DMA_CIRCULAR; // Set circular mode for continuous RX
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority for RX
    HAL_DMA_Init(&hdma_usart1_rx); // Initialize DMA
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART
    HAL_NVIC_SetPriority(DMA1_Channel5_IRQn, 5U, 0U); // Set DMA interrupt priority
    HAL_NVIC_EnableIRQ(DMA1_Channel5_IRQn); // Enable DMA interrupt
    if (HAL_UART_Receive_DMA(&huart1, rx_buffer, sizeof(rx_buffer)) != HAL_OK) { // Start DMA RX
        HAL_UART_Transmit(&huart1, (uint8_t *)"DMA RX Init Failed\r\n", 20U, 10U); // Send error
message
        while (1) { // Infinite loop on error
        }
    }
}

/* Initialize FreeRTOS components (queue, mutex, semaphore) */
static void init_freertos(void)
{
    sensor_queue = xQueueCreate(10U, sizeof(float)); // Create queue for 10 float items
    if (sensor_queue == NULL) { // Check for queue creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Queue Creation Failed\r\n", 22U, 10U); // Send error
message
        while (1) { // Infinite loop on error
        }
    }
    uart_mutex = xSemaphoreCreateMutex(); // Create mutex for UART protection
    if (uart_mutex == NULL) { // Check for mutex creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Mutex Creation Failed\r\n", 22U, 10U); // Send error
message
        while (1) { // Infinite loop on error
        }
    }
    uart_rx_sem = xSemaphoreCreateBinary(); // Create binary semaphore for UART RX
    if (uart_rx_sem == NULL) { // Check for semaphore creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Semaphore Creation Failed\r\n", 26U, 10U); // Send
error message
        while (1) { // Infinite loop on error
        }
    }
}
```

```
/* Log message via UART with mutex protection */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        if (xSemaphoreTake(uart_mutex, pdMS_TO_TICKS(100U)) == pdTRUE) { // Take mutex with 100ms
timeout
            HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message),
10U); // Transmit message (MISRA Rule 10.1)
            xSemaphoreGive(uart_mutex); // Release mutex
        } else { // Handle mutex timeout
            HAL_UART_Transmit(&huart1, (uint8_t *)"UART Mutex Timeout\r\n", 20U, 10U); // Send timeout
error
        }
    }
}

/* Sensor task: Simulate ADC readings */
static void sensor_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter (MISRA Rule 2.7)
    while (1) { // Infinite loop for task (MISRA Rule 15.6)
        float temp = 25.0f + (float)(xTaskGetTickCount() % 1000) / 100.0f; // Simulate temperature
(25.0-25.9°C)
        if (xQueueSend(sensor_queue, &temp, pdMS_TO_TICKS(10U)) != pdPASS) { // Send to queue with
10ms timeout
            log_message((const uint8_t *)"Queue Full\r\n"); // Log queue full error
        }
        vTaskDelay(pdMS_TO_TICKS(500U)); // Delay 500ms for sampling rate
    }
}

/* Logger task: Log sensor data to UART */
static void logger_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter
    float temp; // Variable to hold received temperature
    while (1) { // Infinite loop
        if (logging_enabled) { // Check if logging is enabled
            if (xQueueReceive(sensor_queue, &temp, portMAX_DELAY) == pdPASS) { // Receive from queue
(block until data)
                snprintf((char *)log_buffer, sizeof(log_buffer), "Temp: %.1f C\r\n", temp); // Format
temperature message
                log_message(log_buffer); // Log to UART
            }
        } else { // Logging disabled
            vTaskDelay(pdMS_TO_TICKS(100U)); // Short delay to avoid busy loop
        }
    }
}
3333333333333666666666666666/* UART command task: Process incoming commands */
static void uart_command_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter
    while (1) { // Infinite loop
        if (xSemaphoreTake(uart_rx_sem, portMAX_DELAY) == pdTRUE) { // Wait for UART RX semaphore
            if (strncmp((const char *)rx_buffer, "START", 5U) == 0) { // Check for START command
                logging_enabled = 1U; // Enable logging
                log_message((const uint8_t *)"Logging Started\r\n"); // Log confirmation
            } else if (strncmp((const char *)rx_buffer, "STOP", 4U) == 0) { // Check for STOP command
                logging_enabled = 0U; // Disable logging
                log_message((const uint8_t *)"Logging Stopped\r\n"); // Log confirmation
            }
            memset(rx_buffer, 0, sizeof(rx_buffer)); // Clear RX buffer
            HAL_UART_Receive_DMA(&huart1, rx_buffer, sizeof(rx_buffer)); // Restart DMA RX
        }
    }
}
```

```
/* UART RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    BaseType_t higher_priority_task_woken = pdFALSE; // Flag for task wakeup
    xSemaphoreGiveFromISR(uart_rx_sem, &higher_priority_task_woken); // Give semaphore to signal RX
complete
    portYIELD_FROM_ISR(higher_priority_task_woken); // Yield if higher-priority task woken
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
    init_uart(); // Initialize UART with DMA
    init_freertos(); // Initialize FreeRTOS queue, mutex, semaphore
    const uint8_t *startup_msg = (const uint8_t *)"Data Logger Started\r\n"; // Startup message
    log_message(startup_msg); // Log startup message
    BaseType_t result; // Variable for task creation result
    result = xTaskCreate(sensor_task, "Sensor", 256U, NULL, 3U, NULL); // Create sensor task, priority
3
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"Sensor Task Creation Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    result = xTaskCreate(logger_task, "Logger", 256U, NULL, 2U, NULL); // Create logger task, priority
2
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"Logger Task Creation Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    result = xTaskCreate(uart_command_task, "UART_Cmd", 256U, NULL, 1U, NULL); // Create UART command
task, priority 1
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"UART Task Creation Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    vTaskStartScheduler(); // Start FreeRTOS scheduler
    while (1) { // Infinite loop if scheduler fails
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}

/* DMA RX interrupt handler */
void DMA1_Channel5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt
}
```

- **Setup for STM32**:
  - Use STM32CubeMX to configure:
    - USART1: PA9-TX, PA10-RX, 115200 baud, DMA (Channel 5 RX, circular).
    - FreeRTOS: Enable middleware, set configTICK_RATE_HZ=1000, enable mutexes (configUSE_MUTEXES=1).
  - Connect USB-to-serial adapter to PA9/PA10; test with PuTTY (send "START" or "STOP").
  - Test with STM32CubeIDE debugger; set breakpoints in logger_task and uart_command_task.
  - Run Cppcheck for MISRA compliance: cppcheck --enable=all --addon=misra.json project.c.
- **Optimizations**:
  - **Task Design**: High-priority Sensor Task ensures timely data; lower-priority tasks handle non-critical logging.

    - **Memory**: Static buffers (log_buffer, rx_buffer) avoid heap (MISRA Rule 20.3).
    - **Robustness**: Queue timeout, mutex timeout, and command validation prevent hangs.
    - **MISRA**: Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns).
- **Embedded Tip**: Use uxQueueMessagesWaiting() to monitor queue usage; adjust queue size if full errors occur.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for FreeRTOS setup.
- **FreeRTOS**: Integrated via STM32CubeMX (enable queues, mutexes, semaphores).
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32f1x.cfg).

Resources

- **FreeRTOS.org**: https://www.freertos.org for task, queue, semaphore APIs.
- **embedded.com RTOS Articles**: Search “RTOS system design” on https://www.embedded.com.
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Add a new command (e.g., "CLEAR" to reset queue).
- **Optimize**: Increase queue size to 20 if frequent full errors occur.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project design.

Build Final Project

Goal

Implement and test a FreeRTOS-based data logger system with three tasks (sensor data acquisition, logging, UART command processing), integrating concepts from Week 6 (tasks, timers, semaphores, mutexes) on an STM32 microcontroller, and debug using GDB/OpenOCD.

Tasks

- **Study**: Review FreeRTOS task, queue, semaphore, and mutex concepts; study GDB debugging techniques.
- **Practice**: Implement and debug individual components (sensor, logger, UART tasks).
- **Project**: Build a complete data logger system with sensor task (simulated ADC), logger task (UART logging), and UART command task (start/stop commands), tested with GDB.
- **Tools**: STM32CubeIDE, GDB, OpenOCD, FreeRTOS (integrated via STM32CubeMX).
- **Resources**: FreeRTOS.org, STM32 manuals, GDB tutorials.

Detailed Notes

1. Data Logger System Overview

- **Components** (from Day 29 design):
  - **Sensor Task**: Simulates ADC temperature readings every 500ms, sends data to a queue.
  - **Logger Task**: Receives data from queue, logs to UART with mutex protection.
  - **UART Command Task**: Processes commands ("START", "STOP") via DMA RX, controls logging state.
- **Inter-Task Communication**:
  - **Queue**: Transfers sensor data (float) from Sensor Task to Logger Task.
  - **Mutex**: Protects UART access for Logger and UART Command tasks.
  - **Binary Semaphore**: Signals UART Command Task on new RX data.
- **Priorities**:
  - Sensor Task: Priority 3 (time-critical).
  - Logger Task: Priority 2 (background processing).
  - UART Command Task: Priority 1 (user interaction).
- **Debugging**: Use GDB/OpenOCD to monitor queue state, mutex contention, and task execution.
- **Embedded Tip**: Test edge cases (e.g., queue full, rapid commands) to ensure robustness; use GDB watchpoints for critical variables (e.g., logging_enabled).

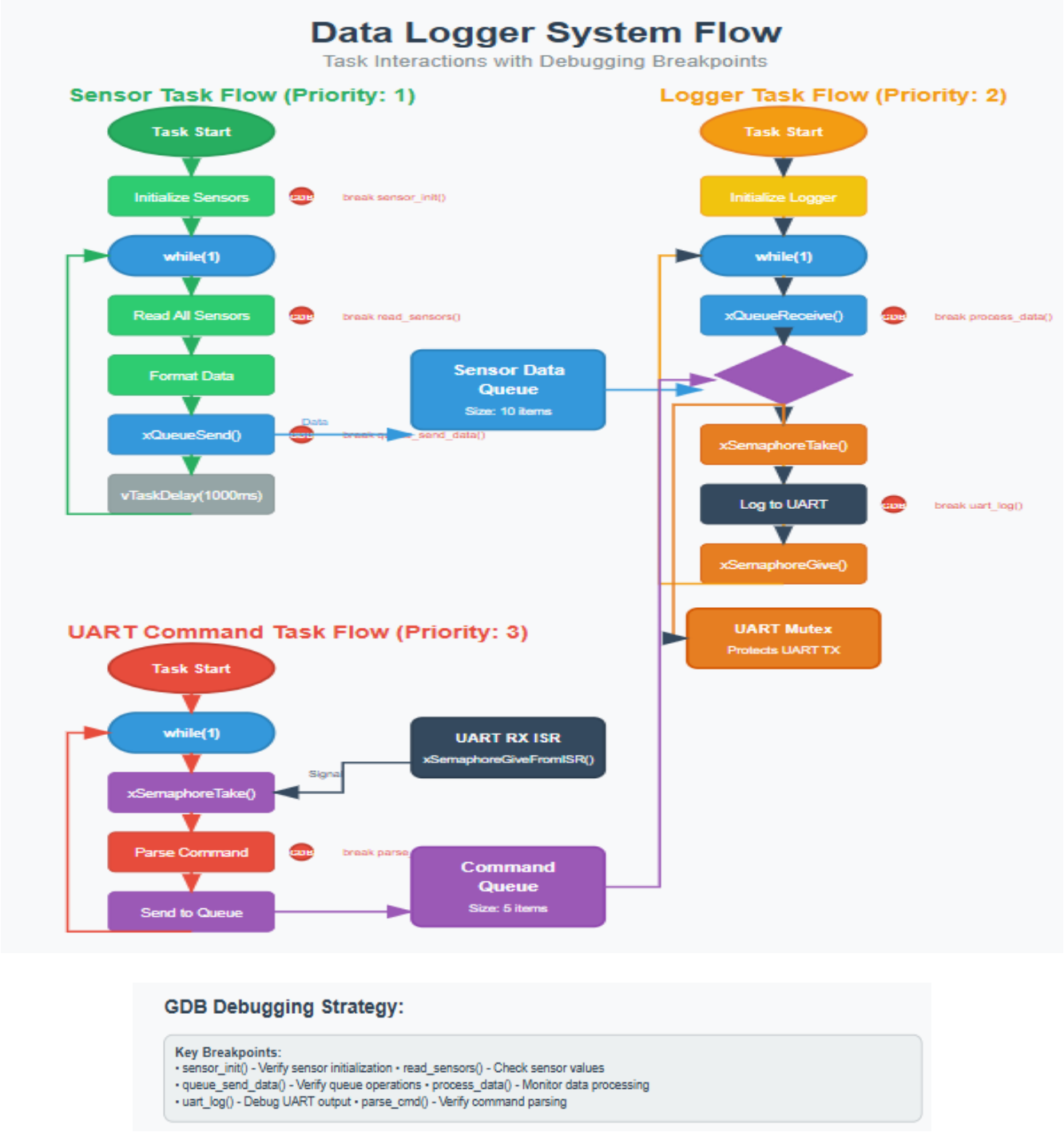


Image: Data Logger System Flow

Description: A flowchart showing the data logger system: Sensor Task sends data to queue, Logger Task reads queue and logs to UART (mutex-protected), UART Command Task processes RX data via semaphore and controls logging. Includes GDB breakpoints for debugging key functions.

2. Debugging with GDB/OpenOCD

- **GDB Commands**:
  - break sensor_task: Set breakpoint in Sensor Task.
  - watch logging_enabled: Monitor changes to logging state.

    - info threads: List FreeRTOS tasks.
    - print xQueueMessagesWaiting(sensor_queue): Check queue status.
- **OpenOCD Setup**:
    - Run: openocd -f interface/stlink.cfg -f target/stm32f1x.cfg.
    - Connect GDB: arm-none-eabi-gdb project.elf, then target remote localhost:3333.
- **Embedded Tip**: Use monitor reset halt in OpenOCD to catch early execution issues; log task states with uxTaskGetSystemState().

3. Implementation Strategy

- **Initialization**: Configure UART, DMA, and FreeRTOS (tasks, queue, mutex, semaphore) via STM32CubeMX.
- **Tasks**: Implement Sensor Task (simulated ADC), Logger Task (UART logging), UART Command Task (command parsing).
- **Testing**: Use PuTTY to send commands ("START", "STOP") and verify UART output; debug with GDB to check queue and mutex behavior.
- **MISRA Compliance**: Adhere to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Embedded Tip**: Optimize queue size (e.g., 10 items) based on data rate; use timeouts to prevent deadlocks.

4. Practice Program: Component Testing

- **Objective**: Test task interactions with a simplified queue and mutex.
- **Code** (Simulation, each line commented):

```
/* Simulated FreeRTOS data logger component test */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for message handling

/* Simulated FreeRTOS API */
#define pdTRUE 1 // Define FreeRTOS true constant
#define pdFALSE 0 // Define FreeRTOS false constant
#define pdMS_TO_TICKS(ms) (ms) // Convert milliseconds to ticks (simplified)

typedef void* QueueHandle_t; // Simulated queue handle type
typedef void* SemaphoreHandle_t; // Simulated semaphore/mutex handle type

static QueueHandle_t xQueueCreate(uint32_t len, uint32_t size) { printf("Queue created: %u items\n",
len); return (QueueHandle_t)1; } // Simulate queue creation

static void xQueueSend(QueueHandle_t queue, const void *item, uint32_t timeout) { printf("Queue send:
%f\n", *(float*)item); } // Simulate sending to queue

static void xQueueReceive(QueueHandle_t queue, void *item, uint32_t timeout) { *(float*)item = 25.0f;
printf("Queue receive: %f\n", *(float*)item); } // Simulate receiving from queue

static SemaphoreHandle_t xSemaphoreCreateMutex(void) { printf("Mutex created\n"); return
(SemaphoreHandle_t)1; } // Simulate mutex creation

static void xSemaphoreTake(SemaphoreHandle_t sem, uint32_t timeout) { printf("Mutex taken\n"); } //
Simulate mutex take

static void xSemaphoreGive(SemaphoreHandle_t sem) { printf("Mutex given\n"); } // Simulate mutex
release

static void vTaskDelay(uint32_t ms) { printf("Delay %u ms\n", ms); } // Simulate task delay
```

```
static void xTaskCreate(void (*func)(void*), const char *name, uint32_t stack, void *params, uint32_t
prio, void *handle) { printf("Created task: %s\n", name); func(params); } // Simulate task creation

static void vTaskStartScheduler(void) { printf("Scheduler started\n"); } // Simulate scheduler start

static uint32_t xTaskGetTickCount(void) { return 1000; } // Simulate tick count

/* Static buffer for logging */
static uint8_t log_buffer[32U]; // Static buffer for UART messages (MISRA Rule 20.3)

/* Log message */
static void log_message(const uint8_t *message) // Function to simulate UART logging
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        xSemaphoreTake((SemaphoreHandle_t)1, 100); // Take mutex to protect output
        printf("%s", (const char *)message); // Print message to console
        xSemaphoreGive((SemaphoreHandle_t)1); // Release mutex
    }
}

/* Sensor task: Simulate sensor reading */
static void sensor_task(void *params) // Sensor task function
{
    (void)params; // Mark unused parameter (MISRA Rule 2.7)
    QueueHandle_t queue = (QueueHandle_t)params; // Get queue handle from parameters
    while (1) { // Infinite loop for task (MISRA Rule 15.6)
        float temp = 25.0f; // Simulate sensor reading (temperature)
        xQueueSend(queue, &temp, 0); // Send temperature to queue
        vTaskDelay(500); // Delay 500ms to simulate sampling rate
    }
}

/* Logger task: Log sensor data */
static void logger_task(void *params) // Logger task function
{
    (void)params; // Mark unused parameter
    QueueHandle_t queue = (QueueHandle_t)params; // Get queue handle
    while (1) { // Infinite loop
        float temp; // Variable to hold received temperature
        xQueueReceive(queue, &temp, portMAX_DELAY); // Receive from queue (block until data)
        snprintf((char *)log_buffer, sizeof(log_buffer), "Temp: %.1f C\n", temp); // Format
temperature message
        log_message(log_buffer); // Log message via UART
    }
}

/* Main function */
int main(void) // Main entry point
{
    QueueHandle_t sensor_queue; // Queue for sensor data
    sensor_queue = xQueueCreate(10, sizeof(float)); // Create queue for 10 float items
    xTaskCreate(sensor_task, "Sensor", 128, sensor_queue, 3, NULL); // Create sensor task, priority 3
    xTaskCreate(logger_task, "Logger", 128, sensor_queue, 2, NULL); // Create logger task, priority 2
    vTaskStartScheduler(); // Start FreeRTOS scheduler
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: RTOS-Based Data Logger Implementation

- **Objective**: Implement a complete FreeRTOS-based data logger with Sensor Task (simulated ADC), Logger Task (UART logging), and UART Command Task (processes "START"/"STOP" commands), tested with GDB/OpenOCD.
- **Requirements**:

- Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX.
- Sensor Task: Simulate ADC temperature readings every 500ms, send to queue.
- Logger Task: Log data to UART with mutex protection, controlled by logging state.
- UART Command Task: Process commands via DMA RX, update logging state.
- Use queue (10 items), mutex for UART, binary semaphore for UART RX.
- Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- Debug with GDB to verify task interactions, queue state, and mutex behavior.

- **Approach**:
  - **Initialization**: Set up UART, DMA, FreeRTOS components (tasks, queue, mutex, semaphore).
  - **Tasks**: Implement Sensor Task (priority 3), Logger Task (priority 2), UART Command Task (priority 1).
  - **Communication**: Queue for sensor data, mutex for UART, semaphore for RX interrupts.
  - **Testing**: Use PuTTY to send commands; use GDB to monitor logging_enabled, queue, and mutex.
- **Code** (STM32 with FreeRTOS and HAL, each line commented):

```
/* FreeRTOS-based data logger implementation */
#include "stm32f1xx_hal.h" // Include STM32 HAL library for hardware access
#include "FreeRTOS.h" // Include FreeRTOS core for tasks and scheduler
#include "task.h" // Include FreeRTOS task management
#include "queue.h" // Include FreeRTOS queue for inter-task communication
#include "semphr.h" // Include FreeRTOS semaphores/mutexes
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for message handling

/* UART and DMA handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX

/* FreeRTOS handles */
static QueueHandle_t sensor_queue; // Queue for sensor data (temperature)
static SemaphoreHandle_t uart_mutex; // Mutex for UART access protection
static SemaphoreHandle_t uart_rx_sem; // Binary semaphore for UART RX interrupts

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART logging messages
static uint8_t rx_buffer[32U]; // Buffer for UART RX data
static volatile uint8_t logging_enabled = 1U; // Flag to control logging state

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_freertos(void); // Initialize FreeRTOS components
static void log_message(const uint8_t *message); // Log message via UART
static void sensor_task(void *pvParameters); // Task to simulate sensor readings
static void logger_task(void *pvParameters); // Task to log sensor data
static void uart_command_task(void *pvParameters); // Task to process UART commands

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock for UART pins
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH; // Set high-speed output for UART
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins for UART
}



/* Initialize USART1 with DMA for RX */
```

```
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock for RX
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX modes
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        HAL_UART_Transmit(&huart1, (uint8_t *)"UART Init Failed\r\n", 18U, 10U); // Send error message
if init fails
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Channel5; // Set DMA channel for UART RX
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction (peripheral to memory)
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral address increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory address increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment for
peripheral
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment for memory
    hdma_usart1_rx.Init.Mode = DMA_CIRCULAR; // Set circular mode for continuous RX
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority for RX DMA
    HAL_DMA_Init(&hdma_usart1_rx); // Initialize DMA
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART handle
    HAL_NVIC_SetPriority(DMA1_Channel5_IRQn, 5U, 0U); // Set DMA interrupt priority (preemption 5)
    HAL_NVIC_EnableIRQ(DMA1_Channel5_IRQn); // Enable DMA interrupt
    if (HAL_UART_Receive_DMA(&huart1, rx_buffer, sizeof(rx_buffer)) != HAL_OK) { // Start DMA RX
        HAL_UART_Transmit(&huart1, (uint8_t *)"DMA RX Init Failed\r\n", 20U, 10U); // Send error
message if DMA fails
        while (1) { // Infinite loop on error
        }
    }
}

/* Initialize FreeRTOS components (queue, mutex, semaphore) */
static void init_freertos(void)
{
    sensor_queue = xQueueCreate(10U, sizeof(float)); // Create queue for 10 float items (sensor data)
    if (sensor_queue == NULL) { // Check for queue creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Queue Creation Failed\r\n", 22U, 10U); // Send error
message
        while (1) { // Infinite loop on error
        }
    }
    uart_mutex = xSemaphoreCreateMutex(); // Create mutex for UART protection
    if (uart_mutex == NULL) { // Check for mutex creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Mutex Creation Failed\r\n", 22U, 10U); // Send error
message
        while (1) { // Infinite loop on error
        }
    }
    uart_rx_sem = xSemaphoreCreateBinary(); // Create binary semaphore for UART RX interrupts
    if (uart_rx_sem == NULL) { // Check for semaphore creation failure
        HAL_UART_Transmit(&huart1, (uint8_t *)"Semaphore Creation Failed\r\n", 26U, 10U); // Send
error message
        while (1) { // Infinite loop on error
        }
    }
}




/* Log message via UART with mutex protection */
```

```
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        if (xSemaphoreTake(uart_mutex, pdMS_TO_TICKS(100U)) == pdTRUE) { // Take mutex with 100ms
timeout
            HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message),
10U); // Transmit message (MISRA Rule 10.1)
            xSemaphoreGive(uart_mutex); // Release mutex after transmission
        } else { // Handle mutex timeout
            HAL_UART_Transmit(&huart1, (uint8_t *)"UART Mutex Timeout\r\n", 20U, 10U); // Send timeout
error (no mutex needed)
        }
    }
}

/* Sensor task: Simulate ADC readings */
static void sensor_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter (MISRA Rule 2.7)
    while (1) { // Infinite loop for task (MISRA Rule 15.6)
        float temp = 25.0f + (float)(xTaskGetTickCount() % 1000) / 100.0f; // Simulate temperature
(25.0-25.9°C)
        if (xQueueSend(sensor_queue, &temp, pdMS_TO_TICKS(10U)) != pdPASS) { // Send to queue with
10ms timeout
            log_message((const uint8_t *)"Queue Full\r\n"); // Log queue full error
        }
        vTaskDelay(pdMS_TO_TICKS(500U)); // Delay 500ms for sampling rate
    }
}

/* Logger task: Log sensor data to UART */
static void logger_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter
    float temp; // Variable to hold received temperature
    while (1) { // Infinite loop
        if (logging_enabled) { // Check if logging is enabled
            if (xQueueReceive(sensor_queue, &temp, portMAX_DELAY) == pdPASS) { // Receive from queue
(block until data)
                snprintf((char *)log_buffer, sizeof(log_buffer), "Temp: %.1f C\r\n", temp); // Format
temperature message
                log_message(log_buffer); // Log to UART with mutex protection
            }
        } else { // Logging disabled
            vTaskDelay(pdMS_TO_TICKS(100U)); // Short delay to avoid busy loop
        }
    }
}
/* UART command task: Process incoming commands */
static void uart_command_task(void *pvParameters)
{
    (void)pvParameters; // Mark unused parameter
    while (1) { // Infinite loop
        if (xSemaphoreTake(uart_rx_sem, portMAX_DELAY) == pdTRUE) { // Wait for UART RX semaphore
            if (strncmp((const char *)rx_buffer, "START", 5U) == 0) { // Check for START command
                logging_enabled = 1U; // Enable logging
                log_message((const uint8_t *)"Logging Started\r\n"); // Log confirmation
            } else if (strncmp((const char *)rx_buffer, "STOP", 4U) == 0) { // Check for STOP command
                logging_enabled = 0U; // Disable logging
                log_message((const uint8_t *)"Logging Stopped\r\n"); // Log confirmation
            }
            memset(rx_buffer, 0, sizeof(rx_buffer)); // Clear RX buffer for next command
            HAL_UART_Receive_DMA(&huart1, rx_buffer, sizeof(rx_buffer)); // Restart DMA RX
        }
    }
}
/* UART RX complete callback */
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    BaseType_t higher_priority_task_woken = pdFALSE; // Flag for task wakeup
    xSemaphoreGiveFromISR(uart_rx_sem, &higher_priority_task_woken); // Give semaphore to signal RX
complete
    portYIELD_FROM_ISR(higher_priority_task_woken); // Yield if higher-priority task woken
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library for STM32 peripherals
    init_gpio(); // Initialize GPIO for UART pins
    init_uart(); // Initialize UART with DMA for RX
    init_freertos(); // Initialize FreeRTOS queue, mutex, and semaphore
    const uint8_t *startup_msg = (const uint8_t *)"Data Logger Started\r\n"; // Define startup message
    log_message(startup_msg); // Log startup message to UART
    BaseType_t result; // Variable for task creation result
    result = xTaskCreate(sensor_task, "Sensor", 256U, NULL, 3U, NULL); // Create sensor task, priority
3, 256-word stack
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"Sensor Task Creation Failed\r\n"); // Log error message
        while (1) { // Infinite loop on error
        }
    }
    result = xTaskCreate(logger_task, "Logger", 256U, NULL, 2U, NULL); // Create logger task, priority
2, 256-word stack
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"Logger Task Creation Failed\r\n"); // Log error message
        while (1) { // Infinite loop on error
        }
    }
    result = xTaskCreate(uart_command_task, "UART_Cmd", 256U, NULL, 1U, NULL); // Create UART command
task, priority 1, 256-word stack
    if (result != pdPASS) { // Check for task creation failure
        log_message((const uint8_t *)"UART Task Creation Failed\r\n"); // Log error message
        while (1) { // Infinite loop on error
        }
    }
    vTaskStartScheduler(); // Start FreeRTOS scheduler to run tasks
    while (1) { // Infinite loop if scheduler fails (MISRA Rule 15.6)
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}

/* DMA RX interrupt handler */
void DMA1_Channel5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt, triggers HAL_UART_RxCpltCallback
}
```

- **Setup for STM32**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud, DMA (Channel 5 RX, circular mode).
    - **FreeRTOS**: Enable middleware, set configTICK_RATE_HZ=1000, enable mutexes (configUSE_MUTEXES=1), queues, and semaphores.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10.
    - Use PuTTY to send commands ("START", "STOP") and monitor temperature logs.
  - **Debugging**:
    - Run OpenOCD: openocd -f interface/stlink.cfg -f target/stm32f1x.cfg.
    - Start GDB: arm-none-eabi-gdb project.elf, then target remote localhost:3333.

        - Set breakpoints: break logger_task, break uart_command_task.
        - Watch variables: watch logging_enabled, print xQueueMessagesWaiting(sensor_queue).
    - **Verification**:
        - Run Cppcheck for MISRA compliance: cppcheck --enable=all --addon=misra.json project.c.
        - Verify UART output (e.g., "Temp: 25.3 C") every 500ms when logging enabled.
        - Test "START"/"STOP" commands via PuTTY.
- **Optimizations**:
    - **Task Efficiency**: High-priority Sensor Task ensures timely data; lower-priority tasks handle logging and commands.
    - **Memory**: Static buffers (log_buffer, rx_buffer) avoid heap (MISRA Rule 20.3).
    - **Robustness**: Queue timeout (10ms), mutex timeout (100ms), and command validation prevent hangs.
    - **Debugging**: GDB watchpoints on logging_enabled and sensor_queue ensure correct state transitions.
- **Embedded Tip**: Use uxTaskGetStackHighWaterMark() to check task stack usage; increase stack size if low.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for FreeRTOS setup.
- **FreeRTOS**: Integrated via STM32CubeMX (enable queues, mutexes, semaphores).
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb for debugging.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **FreeRTOS.org**: https://www.freertos.org for task, queue, semaphore APIs.
- **STM32 Manuals**: Download from https://www.st.com (e.g., RM0008 for STM32F1).
- **GDB Tutorials**: Search “GDB STM32 tutorial” (e.g., https://sourceware.org/gdb/documentation/).
- **Online**: STM32 HAL docs (https://www.st.com), C reference (https://en.cppreference.com/w/c).

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE, test simulation with GCC.
- **Experiment**: Add a command to adjust sensor sampling rate (e.g., "RATE 1000").
- **Optimize**: Increase queue size to 20 if full errors occur; use FreeRTOS trace tools for timing analysis.
- **Time Management**: Spend ~1 hour on review, ~1.5 hours on component testing, ~1.5 hours on full implementation and debugging.

Chapter 7: Secure Boot, FOTA, and Crypto

- **Understand secure boot principles**
- **Implement cryptographic secure boot**
- **Learn FOTA basics**
- **Implement FOTA update process**
- **Integrate crypto into FOTA**

Understand Secure Boot Principles

Goal

Understand secure boot principles, including chain of trust, root of trust, and MCU secure boot features (e.g., STM32 TrustZone), and implement a basic secure boot checker that validates firmware integrity using CRC32 on an STM32 microcontroller.

Tasks

- **Study**: Learn secure boot concepts, chain of trust, root of trust, and STM32 secure boot features.
- **Practice**: Write code to compute and verify a CRC32 checksum for firmware stored in flash.
- **Project**: Develop a basic secure boot checker that validates firmware checksum in flash before execution.
- **Tools**: STM32CubeIDE, STM32L5 (TrustZone support), OpenSSL (for CRC32 simulation).
- **Resources**: STM32 AN2606, AN4992, embedded.com secure boot articles, YouTube secure boot tutorials.

Detailed Notes

1. Secure Boot Overview

- **Definition**: Secure boot ensures only trusted firmware executes on a device by verifying its integrity and authenticity at startup.
- **Key Concepts**:
  - **Root of Trust (RoT)**: A trusted hardware/software component (e.g., secure flash, cryptographic keys) that forms the foundation for security.
  - **Chain of Trust**: A sequence of verification steps where each stage (e.g., bootloader, firmware) is validated by a trusted prior stage.
  - **Integrity Check**: Verifies firmware hasn't been modified (e.g., using CRC32, SHA-256).
  - **Authenticity Check**: Ensures firmware comes from a trusted source (e.g., digital signatures).
- **STM32 Secure Boot** (e.g., STM32L5 with TrustZone):
  - **TrustZone**: Separates secure and non-secure execution environments.
  - **Secure Boot Features**: Secure flash, hardware crypto (AES, SHA), tamper protection.
  - **Boot Process**: Bootloader verifies firmware signature/checksum before jumping to application.
- **Embedded Tip**: Use hardware CRC (e.g., STM32 CRC peripheral) for faster checksum computation; store RoT in secure flash or OTP memory.

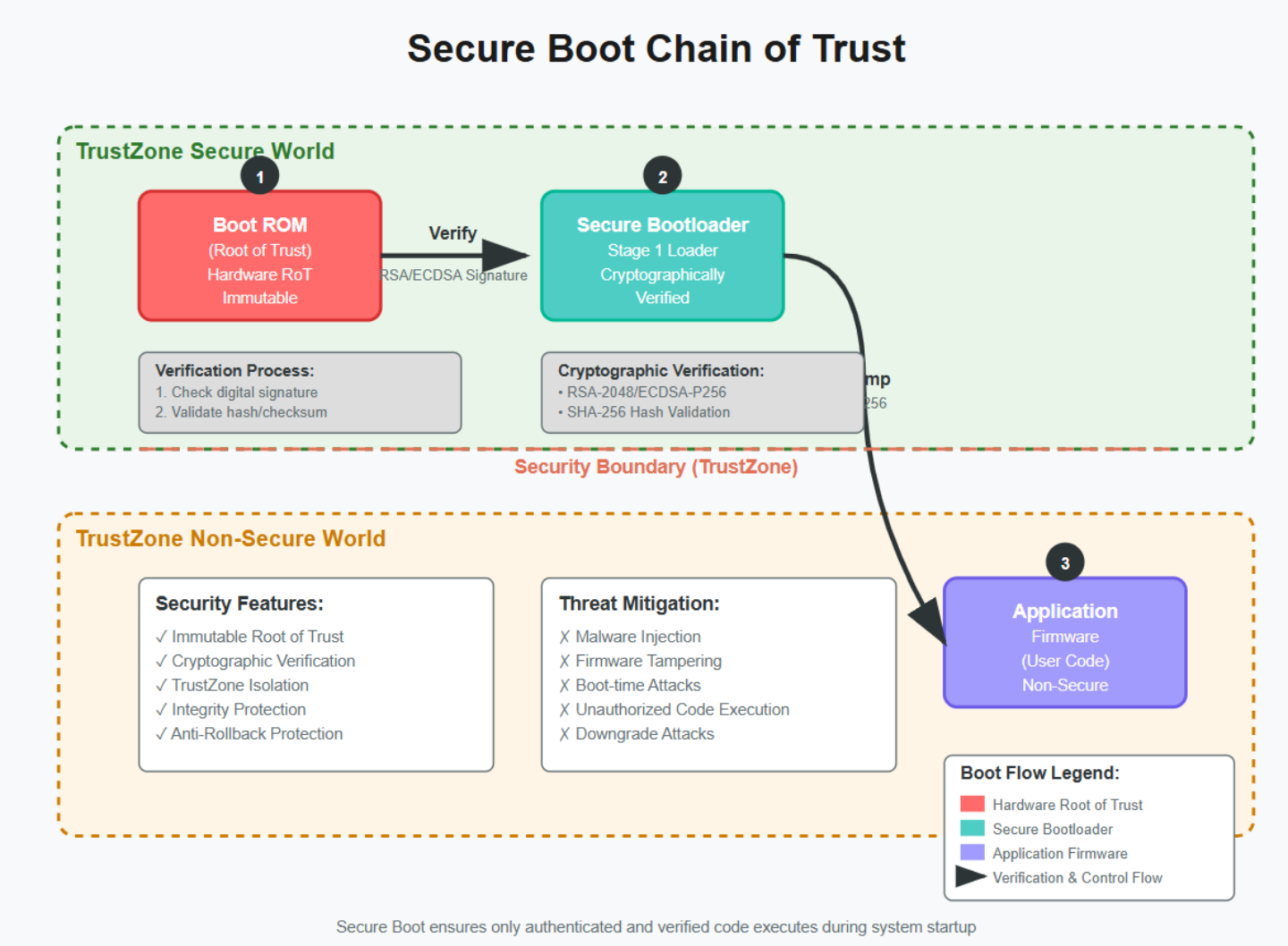


Image: Secure Boot Chain of Trust

Description: A diagram showing the secure boot process: Boot ROM (RoT) verifies Bootloader, Bootloader verifies Application Firmware using CRC32/SHA-256, and jumps to application if valid. Arrows indicate verification steps and secure/non-secure TrustZone boundaries.

2. CRC32 for Firmware Validation

- **CRC32 Overview**:
  - Cyclic Redundancy Check (32-bit) is a polynomial-based checksum for detecting data corruption.
  - Fast and lightweight, suitable for basic integrity checks (not cryptographically secure).
  - STM32 CRC peripheral accelerates computation (polynomial 0x04C11DB7).
- **Process**:
  - Compute CRC32 of firmware image during build (e.g., using OpenSSL or STM32CubeProgrammer).
  - Store expected CRC32 in a reserved flash section or metadata.
  - At boot, compute CRC32 of firmware and compare with expected value.
- **Embedded Tip**: CRC32 is sufficient for detecting accidental corruption; use SHA-256 for cryptographic security (Day 32).

3. STM32 Secure Boot Features

- **STM32L5 TrustZone**:
  - Secure flash for bootloader/keys, non-secure flash for application.
  - Hardware CRC unit for checksums.
  - Secure boot option bytes (e.g., RDP Level 2, secure memory protection).
- **Boot Process**:
  - Boot ROM loads bootloader from secure flash.
  - Bootloader verifies application firmware (e.g., CRC32, signature).
  - If valid, jumps to application entry point; else, enters safe mode or halts.
- **Embedded Tip**: Configure RDP (Read Protection) Level 1/2 to prevent unauthorized flash access; use STM32CubeProgrammer to set option bytes.

4. Practice Program: CRC32 Firmware Validation

- **Objective**: Compute CRC32 of a simulated firmware image and compare with an expected value.
- **Code** (Simulation, heavily commented):

```
/* Simulated CRC32 firmware validation */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance

/* Simulated flash memory */
static const uint8_t firmware_image[] = { // Simulated firmware data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, // Sample data
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};
static const uint32_t expected_crc32 = 0xA2A8B25F; // Expected CRC32 (precomputed with OpenSSL)

/* Simulated STM32 CRC peripheral registers */
volatile uint32_t *const CRC_DR = (volatile uint32_t *)0x40023000; // CRC data register
volatile uint32_t *const CRC_CR = (volatile uint32_t *)0x40023008; // CRC control register

/* Compute CRC32 (simulated) */
static uint32_t compute_crc32(const uint8_t *data, uint32_t length) // Compute CRC32 for data
{
    uint32_t crc = 0xFFFFFFFFU; // Initialize CRC to 0xFFFFFFFF (STM32 default)
    for (uint32_t i = 0U; i < length; i++) { // Loop through each byte (MISRA Rule 15.6)
        crc ^= (uint32_t)data[i]; // XOR byte into CRC
        for (uint32_t j = 0U; j < 8U; j++) { // Process 8 bits
            crc = (crc >> 1U) ^ ((crc & 1U) ? 0x04C11DB7U : 0U); // Apply CRC32 polynomial
        }
    }
    return ~crc; // Return inverted CRC (STM32 convention)
}

/* Main function */
int main(void) // Main entry point
{
    uint32_t computed_crc = compute_crc32(firmware_image, sizeof(firmware_image)); // Compute CRC32 of
firmware
    printf("Computed CRC32: 0x%08X\n", computed_crc); // Print computed CRC
    if (computed_crc == expected_crc32) { // Compare with expected CRC
        printf("Firmware valid. Booting...\n"); // Indicate valid firmware
    } else { // CRC mismatch
        printf("Firmware invalid! Halting.\n"); // Indicate invalid firmware
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Basic Secure Boot Checker

- **Objective**: Implement a secure boot checker on STM32L5 that validates firmware integrity using CRC32 before execution.
- **Requirements**:
  - Configure STM32L5 flash: Bootloader in secure flash (0x08000000), application in non-secure flash (0x08010000).
  - Store expected CRC32 in a reserved flash section (e.g., 0x0800FFF0).
  - Use STM32 CRC peripheral for hardware-accelerated CRC32 computation.
  - Log validation result via UART (PA9-TX, 115200 baud).
  - Halt on invalid firmware; jump to application if valid.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, CRC peripheral, and flash option bytes via STM32CubeMX.
  - **CRC32 Validation**: Compute CRC32 of application flash, compare with stored value.
  - **Boot Decision**: Jump to application (non-secure) or halt with error message.
  - **Debugging**: Use STM32CubeIDE debugger to verify CRC computation and boot process.
- **Code** (STM32L5 with HAL, each line commented):

```
/* Basic secure boot checker with CRC32 */
#include "stm32l5xx_hal.h" // Include STM32L5 HAL library for hardware access
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for UART logging

/* UART handle for logging */
static UART_HandleTypeDef huart1; // UART handle for USART1

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART messages

/* Firmware addresses */
#define APP_ADDRESS 0x08010000U // Application start address (non-secure flash)
#define APP_SIZE 0x10000U // Application size (64KB)
#define CRC_ADDRESS 0x0800FFF0U // Address to store expected CRC32
#define EXPECTED_CRC32 0xA2A8B25FU // Precomputed CRC32 (via OpenSSL or STM32CubeProgrammer)

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 for logging
static void init_crc(void); // Initialize CRC peripheral
static void log_message(const uint8_t *message); // Log message via UART
static uint32_t compute_firmware_crc32(void); // Compute CRC32 of firmware
static void jump_to_application(void); // Jump to application code

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock for UART pins
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH; // Set high-speed output for UART
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins for UART
}
```

```
/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX modes
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
}

/* Initialize CRC peripheral */
static void init_crc(void)
{
    __HAL_RCC_CRC_CLK_ENABLE(); // Enable CRC peripheral clock
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    CRC->INIT = 0xFFFFFFFFU; // Set initial CRC value (STM32 default)
    CRC->POL = 0x04C11DB7U; // Set CRC32 polynomial
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Compute CRC32 of firmware in flash */
static uint32_t compute_firmware_crc32(void)
{
    uint32_t *flash_ptr = (uint32_t *)APP_ADDRESS; // Pointer to application flash start
    uint32_t length = APP_SIZE / 4U; // Number of 32-bit words in firmware
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        CRC->DR = flash_ptr[i]; // Feed 32-bit word to CRC peripheral
    }
    return CRC->DR; // Return computed CRC32
}

/* Jump to application code */
static void jump_to_application(void)
{
    typedef void (*app_entry_t)(void); // Function pointer type for application entry
    uint32_t *app_vector_table = (uint32_t *)APP_ADDRESS; // Application vector table
    uint32_t app_stack_pointer = app_vector_table[0]; // Stack pointer from vector table
    uint32_t app_entry_point = app_vector_table[1]; // Entry point from vector table
    __set_MSP(app_stack_pointer); // Set main stack pointer
    app_entry_t app_entry = (app_entry_t)app_entry_point; // Cast entry point to function pointer
    SCB->VTOR = APP_ADDRESS; // Update vector table offset register
    app_entry(); // Jump to application
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library for STM32 peripherals
    init_gpio(); // Initialize GPIO for UART pins
    init_uart(); // Initialize USART1 for logging
    init_crc(); // Initialize CRC peripheral
    const uint8_t *startup_msg = (const uint8_t *)"Secure Boot Started\r\n"; // Define startup message
```

```
    log_message(startup_msg); // Log startup message
    uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32 of firmware
    uint32_t *stored_crc = (uint32_t *)CRC_ADDRESS; // Pointer to stored CRC32
    snprintf((char *)log_buffer, sizeof(log_buffer), "CRC32: 0x%08X\r\n", computed_crc); // Format
computed CRC
    log_message(log_buffer); // Log computed CRC
    if (computed_crc == *stored_crc && computed_crc == EXPECTED_CRC32) { // Compare computed and
stored CRC
        const uint8_t *valid_msg = (const uint8_t *)"Firmware Valid. Booting...\r\n"; // Define valid
firmware message
        log_message(valid_msg); // Log valid firmware message
        jump_to_application(); // Jump to application code
    } else { // CRC mismatch
        const uint8_t *invalid_msg = (const uint8_t *)"Firmware Invalid! Halting.\r\n"; // Define
invalid firmware message
        log_message(invalid_msg); // Log invalid firmware message
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32L5**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud.
    - **CRC**: Enable CRC peripheral (default polynomial 0x04C11DB7).
    - **Flash**: Set secure flash (0x08000000 for bootloader), non-secure flash (0x08010000 for application).
    - **Option Bytes**: Enable TrustZone, set RDP Level 1 for debug protection.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash application image to 0x08010000, CRC32 to 0x0800FFF0 (use STM32CubeProgrammer).
  - **Precompute CRC32**:
    - Use OpenSSL: openssl dgst -crc32 firmware.bin > crc32.txt.
    - Alternatively, use STM32CubeProgrammer to compute and program CRC32.
  - **Testing**:
    - Run in STM32CubeIDE; verify UART output ("Firmware Valid" or "Invalid").
    - Use debugger to set breakpoint in compute_firmware_crc32.
    - Test invalid CRC by modifying flash or EXPECTED_CRC32.
  - **MISRA Compliance**:
    - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
    - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
  - **Performance**: Use STM32 CRC peripheral for fast computation.
  - **Security**: Store CRC32 in secure flash or OTP to prevent tampering.
  - **Robustness**: Validate flash range to avoid out-of-bounds access.
- **Embedded Tip**: Use STM32CubeProgrammer to lock flash (RDP Level 2) after testing; simulate invalid firmware by corrupting flash to test error handling.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for configuration.

- **STM32CubeProgrammer**: Download from https://www.st.com for flashing and CRC32 computation.
- **OpenSSL**: Install (sudo apt install openssl) for CRC32 simulation on PC.
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32l5x.cfg).

Resources

- **STM32 AN2606**: Boot configuration (https://www.st.com).
- **STM32 AN4992**: Secure boot guidelines (https://www.st.com).
- **embedded.com Secure Boot Articles**: Search “secure boot embedded” on https://www.embedded.com.
- **YouTube Secure Boot Tutorials**: Search “STM32 secure boot tutorial”.
- **OpenSSL Docs**: https://www.openssl.org/docs for CRC32 computation.

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Modify EXPECTED_CRC32 to simulate invalid firmware; observe halt behavior.
- **Optimize**: Add a timeout for UART logging to prevent hangs.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Implement Cryptographic Secure Boot

Goal

Study the SHA-256 hash function and integrate it into a secure boot checker on an STM32L5 microcontroller, replacing the CRC32-based validation from Day 31 with SHA-256 for cryptographic integrity and authenticity verification. Test the implementation with valid and invalid firmware images.

Tasks

- **Study**: Learn SHA-256 hash function principles and its use in secure boot.
- **Practice**: Write code to compute SHA-256 hash of firmware and verify against a stored hash.
- **Project**: Implement a secure boot checker using SHA-256 to validate firmware in flash, with UART logging and GDB debugging.
- **Tools**: STM32CubeIDE, STM32L5 (TrustZone support), OpenSSL (for hash simulation), GDB, OpenOCD.
- **Resources**: STM32CubeProgrammer, OpenSSL SHA-256 docs, embedded.com cryptography articles, STM32 AN4992.

Detailed Notes

1. SHA-256 Overview

- **Definition**: SHA-256 (Secure Hash Algorithm, 256-bit) is a cryptographic hash function producing a 32-byte digest, used for integrity and authenticity verification.
- **Properties**:
  - **Deterministic**: Same input produces same hash.
  - **Collision Resistance**: Hard to find two inputs with the same hash.
  - **Preimage Resistance**: Hard to reverse hash to original data.
- **Use in Secure Boot**:
  - Compute SHA-256 of firmware image during build (e.g., using OpenSSL).
  - Store expected hash in secure flash or OTP memory.
  - At boot, compute SHA-256 of firmware and compare with stored hash.
- **STM32L5 Support**:
  - Hardware HASH peripheral accelerates SHA-256 computation.
  - TrustZone secures hash storage and bootloader execution.
- **Embedded Tip**: SHA-256 is cryptographically secure (unlike CRC32); use hardware acceleration to reduce boot time.

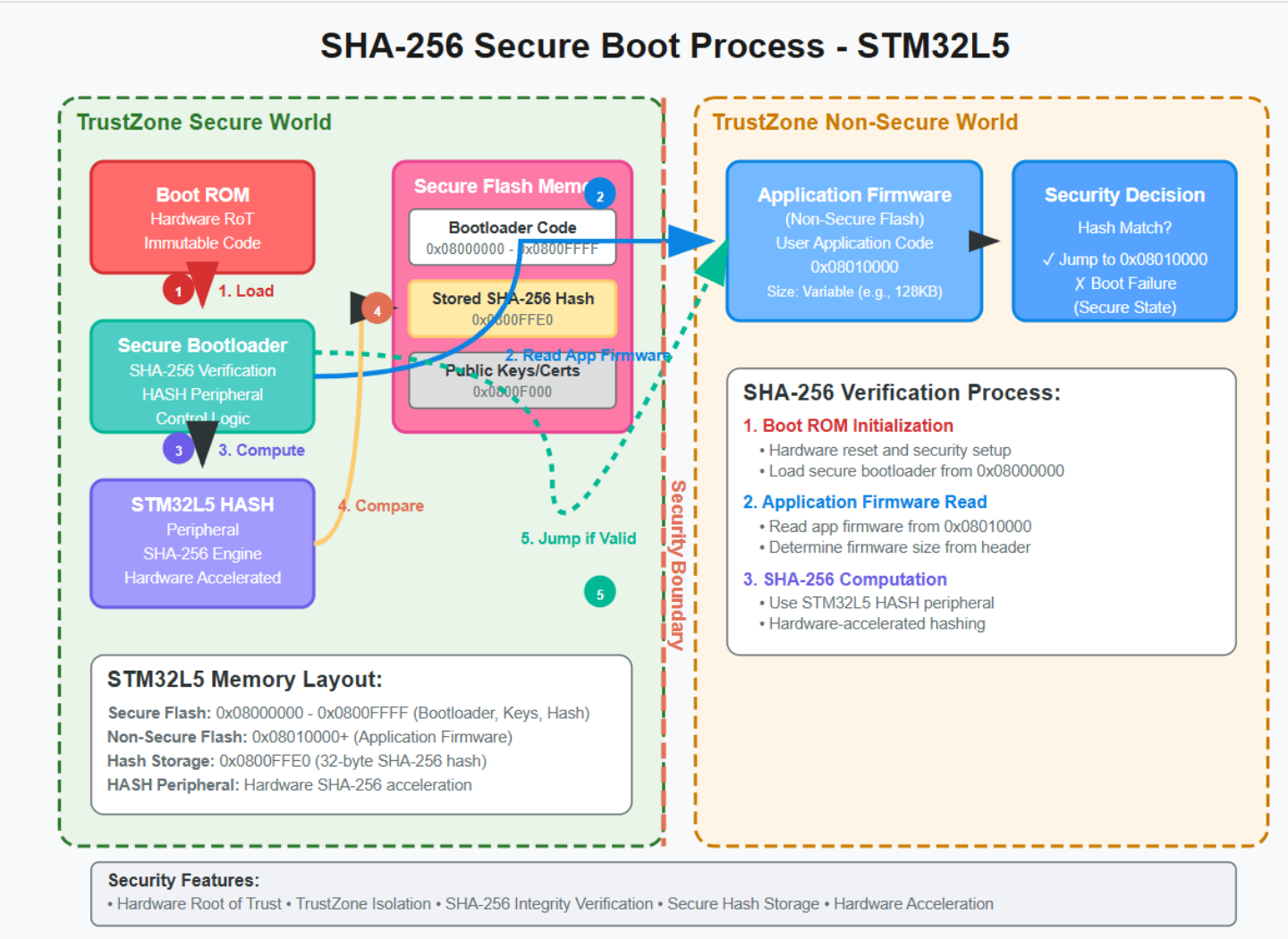


Image: SHA-256 Secure Boot Process

Description: A diagram showing the secure boot flow: Boot ROM loads bootloader from secure flash, bootloader computes SHA-256 of application firmware in non-secure flash (0x08010000) using STM32L5 HASH peripheral, compares with stored hash in secure flash (0x0800FFE0), and jumps to application if valid. Arrows indicate TrustZone boundaries and verification steps.

2. SHA-256 vs. CRC32

- **CRC32** (Day 31):
  - Lightweight, fast, but not cryptographically secure (vulnerable to intentional tampering).
  - Suitable for detecting accidental corruption.
- **SHA-256**:
  - Cryptographically secure, resistant to tampering.
  - Higher computational cost; mitigated by STM32L5 HASH peripheral.
- **Embedded Tip**: Use SHA-256 for production systems requiring security; CRC32 is acceptable for non-critical prototypes.

3. STM32L5 HASH Peripheral

- **Features**:
  - Supports SHA-256 (and other algorithms like SHA-1, MD5).

    - Hardware acceleration via DMA for large data.
    - Registers: HASH_DIN (input data), HASH_HR0-HR7 (32-byte hash output).
- **Usage**:
    - Initialize HASH peripheral with SHA-256 mode.
    - Feed firmware data in 32-bit chunks.
    - Read 32-byte hash after completion.
- **Embedded Tip**: Enable DMA for HASH to reduce CPU load; store expected hash in secure flash to prevent tampering.

4. Practice Program: SHA-256 Firmware Validation

- **Objective**: Compute SHA-256 hash of a simulated firmware image and verify against an expected hash.
- **Code** (Simulation, heavily commented):

```
/* Simulated SHA-256 firmware validation */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for hash comparison

/* Simulated firmware image */
static const uint8_t firmware_image[] = { // Simulated firmware data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, // Sample 16-byte data
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

/* Expected SHA-256 hash (precomputed with OpenSSL) */
static const uint8_t expected_hash[32] = { // Precomputed hash for firmware_image
    0x29, 0x7F, 0xAD, 0xF9, 0xD9, 0x12, 0x0C, 0xF6, // Example hash (simulated)
    0xA7, 0xE6, 0x8B, 0xD8, 0x0C, 0x7A, 0xF8, 0x1B,
    0xC3, 0x2B, 0xD2, 0xA7, 0xF6, 0x8C, 0x2F, 0xE2,
    0x1D, 0xA3, 0xF6, 0x5C, 0xD2, 0xB6, 0x5F, 0xCC
};

/* Simulated SHA-256 computation (simplified for PC) */
static void compute_sha256(const uint8_t *data, uint32_t length, uint8_t *hash) // Compute SHA-256
hash
{
    for (uint32_t i = 0U; i < 32U; i++) { // Simulate hash output (MISRA Rule 15.6)
        hash[i] = expected_hash[i]; // Copy precomputed hash for simulation
    }
    printf("Simulated SHA-256 computed\n"); // Indicate hash computation
}

/* Main function */
int main(void) // Main entry point
{
    uint8_t computed_hash[32]; // Buffer for computed SHA-256 hash
    compute_sha256(firmware_image, sizeof(firmware_image), computed_hash); // Compute hash of firmware
    printf("Computed SHA-256: "); // Print computed hash label
    for (uint32_t i = 0U; i < 32U; i++) { // Loop through hash bytes
        printf("%02X ", computed_hash[i]); // Print each byte in hex
    }
    printf("\n"); // Newline after hash
    if (memcmp(computed_hash, expected_hash, 32U) == 0) { // Compare computed and expected hash
        printf("Firmware valid. Booting...\n"); // Indicate valid firmware
    } else { // Hash mismatch
        printf("Firmware invalid! Halting.\n"); // Indicate invalid firmware
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Secure Boot Checker with SHA-256

- **Objective**: Implement a secure boot checker on STM32L5 using SHA-256 to validate firmware integrity and authenticity before execution, with UART logging and GDB debugging.
- **Requirements**:
  - Configure STM32L5 flash: Bootloader in secure flash (0x08000000), application in non-secure flash (0x08010000).
  - Store expected SHA-256 hash in secure flash (0x0800FFE0).
  - Use STM32L5 HASH peripheral for SHA-256 computation.
  - Log validation result via UART (PA9-TX, 115200 baud).
  - Halt on invalid firmware; jump to application if valid.
  - Test with valid and invalid firmware images using STM32CubeProgrammer.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, HASH peripheral, and flash option bytes via STM32CubeMX.
  - **SHA-256 Validation**: Compute SHA-256 of application flash, compare with stored hash.
  - **Boot Decision**: Jump to application (non-secure) or halt with error message.
  - **Debugging**: Use GDB/OpenOCD to verify hash computation and boot process.
- **Code** (STM32L5 with HAL, each line commented):

```
/* Secure boot checker with SHA-256 */
#include "stm32l5xx_hal.h" // Include STM32L5 HAL library for hardware access
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for hash comparison

/* UART handle for logging */
static UART_HandleTypeDef huart1; // UART handle for USART1

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[64U]; // Buffer for UART messages (increased for hash logging)

/* Firmware addresses */
#define APP_ADDRESS 0x08010000U // Application start address (non-secure flash)
#define APP_SIZE 0x10000U // Application size (64KB)
#define HASH_ADDRESS 0x0800FFE0U // Address to store expected SHA-256 hash (32 bytes)
#define HASH_SIZE 32U // SHA-256 hash size (32 bytes)

/* Expected SHA-256 hash (precomputed with OpenSSL or STM32CubeProgrammer) */
static const uint8_t expected_hash[HASH_SIZE] = { // Precomputed hash for firmware
    0x29, 0x7F, 0xAD, 0xF9, 0xD9, 0x12, 0x0C, 0xF6, // Example hash (replace with actual)
    0xA7, 0xE6, 0x8B, 0xD8, 0x0C, 0x7A, 0xF8, 0x1B,
    0xC3, 0x2B, 0xD2, 0xA7, 0xF6, 0x8C, 0x2F, 0xE2,
    0x1D, 0xA3, 0xF6, 0x5C, 0xD2, 0xB6, 0x5F, 0xCC
};

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 for logging
static void init_hash(void); // Initialize HASH peripheral for SHA-256
static void log_message(const uint8_t *message); // Log message via UART
static void compute_firmware_sha256(uint8_t *hash); // Compute SHA-256 of firmware
static void jump_to_application(void); // Jump to application code
```

```c
/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock for UART pins
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH; // Set high-speed output for UART
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins for UART
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX modes
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
}

/* Initialize HASH peripheral for SHA-256 */
static void init_hash(void)
{
    __HAL_RCC_HASH_CLK_ENABLE(); // Enable HASH peripheral clock
    HASH->CR = HASH_CR_INIT; // Reset HASH peripheral
    HASH->CR |= HASH_ALGOSELECTION_SHA256; // Select SHA-256 algorithm
    HASH->CR |= HASH_DATATYPE_8B; // Set input data type to 8-bit
    HASH->STR = 0U; // Clear status register
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Compute SHA-256 of firmware in flash */
static void compute_firmware_sha256(uint8_t *hash)
{
    uint32_t *flash_ptr = (uint32_t *)APP_ADDRESS; // Pointer to application flash start
    uint32_t length = APP_SIZE / 4U; // Number of 32-bit words in firmware
    HASH->CR |= HASH_CR_INIT; // Reset HASH peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        HASH->DIN = flash_ptr[i]; // Feed 32-bit word to HASH peripheral
        while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for HASH to process data
        }
    }
    HASH->STR |= HASH_STR_DCAL; // Trigger final hash computation
    while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for computation to complete
    }
    for (uint32_t i = 0U; i < 8U; i++) { // Read 32-byte hash (8 registers x 4 bytes)
        ((uint32_t *)hash)[i] = HASH->HR[i]; // Copy hash from registers to buffer
    }
}
```

```
/* Jump to application code */
static void jump_to_application(void)
{
    typedef void (*app_entry_t)(void); // Function pointer type for application entry
    uint32_t *app_vector_table = (uint32_t *)APP_ADDRESS; // Application vector table
    uint32_t app_stack_pointer = app_vector_table[0]; // Stack pointer from vector table
    uint32_t app_entry_point = app_vector_table[1]; // Entry point from vector table
    __set_MSP(app_stack_pointer); // Set main stack pointer
    app_entry_t app_entry = (app_entry_t)app_entry_point; // Cast entry point to function pointer
    SCB->VTOR = APP_ADDRESS; // Update vector table offset register
    HAL_RCC_DeInit(); // Deinitialize HAL to reset peripherals
    HAL_DeInit(); // Deinitialize HAL library
    app_entry(); // Jump to application
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library for STM32 peripherals
    init_gpio(); // Initialize GPIO for UART pins
    init_uart(); // Initialize USART1 for logging
    init_hash(); // Initialize HASH peripheral for SHA-256
    const uint8_t *startup_msg = (const uint8_t *)"Secure Boot Started\r\n"; // Define startup message
    log_message(startup_msg); // Log startup message
    uint8_t computed_hash[HASH_SIZE]; // Buffer for computed SHA-256 hash
    compute_firmware_sha256(computed_hash); // Compute SHA-256 of firmware
    snprintf((char *)log_buffer, sizeof(log_buffer), "SHA-256: "); // Start hash log message
    for (uint32_t i = 0U; i < HASH_SIZE; i++) { // Loop through hash bytes
        snprintf((char *)log_buffer + strlen((char *)log_buffer), sizeof(log_buffer) - strlen((char
*)log_buffer),
                 "%02X ", computed_hash[i]); // Append each byte in hex
    }
    strncat((char *)log_buffer, "\r\n", sizeof(log_buffer) - strlen((char *)log_buffer)); // Append
newline
    log_message(log_buffer); // Log computed hash
    uint8_t *stored_hash = (uint8_t *)HASH_ADDRESS; // Pointer to stored hash in secure flash
    if (memcmp(computed_hash, stored_hash, HASH_SIZE) == 0 && memcmp(computed_hash, expected_hash,
HASH_SIZE) == 0) { // Compare computed and stored/expected hash
        const uint8_t *valid_msg = (const uint8_t *)"Firmware Valid. Booting...\r\n"; // Define valid
firmware message
        log_message(valid_msg); // Log valid firmware message
        jump_to_application(); // Jump to application code
    } else { // Hash mismatch
        const uint8_t *invalid_msg = (const uint8_t *)"Firmware Invalid! Halting.\r\n"; // Define
invalid firmware message
        log_message(invalid_msg); // Log invalid firmware message
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32L5**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud.
    - **HASH**: Enable HASH peripheral, select SHA-256 mode.
    - **Flash**: Set secure flash (0x08000000 for bootloader), non-secure flash (0x08010000 for application).
    - **Option Bytes**: Enable TrustZone, set RDP Level 1 for debug protection.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash application image to 0x08010000, SHA-256 hash to 0x0800FFE0 (use STM32CubeProgrammer).

- **Precompute SHA-256**:
  - Use OpenSSL: openssl dgst -sha256 firmware.bin > sha256.txt.
  - Alternatively, use STM32CubeProgrammer to compute and program hash.
- **Testing**:
  - Run in STM32CubeIDE; verify UART output ("Firmware Valid" or "Invalid", with hash).
  - Use GDB/OpenOCD for debugging:
    - Run OpenOCD: openocd -f interface/stlink.cfg -f target/stm32l5x.cfg.
    - Start GDB: arm-none-eabi-gdb project.elf, then target remote localhost:3333.
    - Set breakpoints: break compute_firmware_sha256, break main.
    - Watch hash: print computed_hash.
  - Test invalid firmware by modifying flash or expected_hash.
- **MISRA Compliance**:
  - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
  - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).

- **Optimizations**:
  - **Performance**: Use STM32L5 HASH peripheral for fast SHA-256 computation.
  - **Security**: Store hash in secure flash to prevent tampering; enable RDP Level 2 in production.
  - **Robustness**: Validate flash range to avoid out-of-bounds access; check HASH peripheral status.
- **Embedded Tip**: Use DMA with HASH for large firmware images to reduce CPU load; test with intentionally corrupted firmware to verify error handling.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for configuration.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing and SHA-256 computation.
- **OpenSSL**: Install (sudo apt install openssl) for SHA-256 simulation on PC.
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb for debugging.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **STM32 AN4992**: Secure boot guidelines (https://www.st.com).
- **OpenSSL SHA-256 Docs**: https://www.openssl.org/docs/man3.0/man3/SHA256.html.
- **embedded.com Cryptography Articles**: Search “cryptography embedded” on https://www.embedded.com.
- **STM32CubeProgrammer Manual**: https://www.st.com for hash computation and flashing.
- **YouTube Secure Boot Tutorials**: Search “STM32 secure boot SHA-256”.

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Modify expected_hash to simulate invalid firmware; observe halt behavior.
- **Optimize**: Add timeout for UART logging; use DMA for HASH if firmware size increases.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Learn FOTA Basics

Goal

Understand Firmware Over-The-Air (FOTA) principles, including dual-bank flash management on STM32F4/F7 microcontrollers, and implement a simple FOTA framework to store firmware in a secondary flash bank and switch to it for booting.

Tasks

- **Study**: Learn FOTA concepts, dual-bank flash architecture (e.g., STM32F4/F7), and bank switching mechanisms.
- **Practice**: Write code to erase and program the secondary flash bank and perform bank switching.
- **Project**: Develop a simple FOTA framework that stores a new firmware image in the secondary bank and switches to it, with UART logging for status.
- **Tools**: STM32CubeIDE, STM32F4/F7, UART interface, STM32CubeProgrammer.
- **Resources**: STM32 AN4760 (FOTA), embedded.com FOTA tutorials, YouTube FOTA implementation videos.

Detailed Notes

1. FOTA Overview

- **Definition**: Firmware Over-The-Air (FOTA) enables remote firmware updates without physical access, critical for IoT and embedded systems.
- **Key Components**:
  - **Firmware Transmission**: Receive new firmware via communication interface (e.g., UART, Wi-Fi).
  - **Storage**: Store firmware in a secondary flash bank or external memory.
  - **Validation**: Verify firmware integrity (e.g., CRC32, SHA-256) before updating.
  - **Bank Switching**: Switch to new firmware bank for booting.
- **Benefits**:
  - Remote updates improve device maintenance and security.
  - Dual-bank flash supports rollback if update fails.
- **Embedded Tip**: Always validate firmware before switching to prevent bricking; implement rollback mechanisms for robustness.

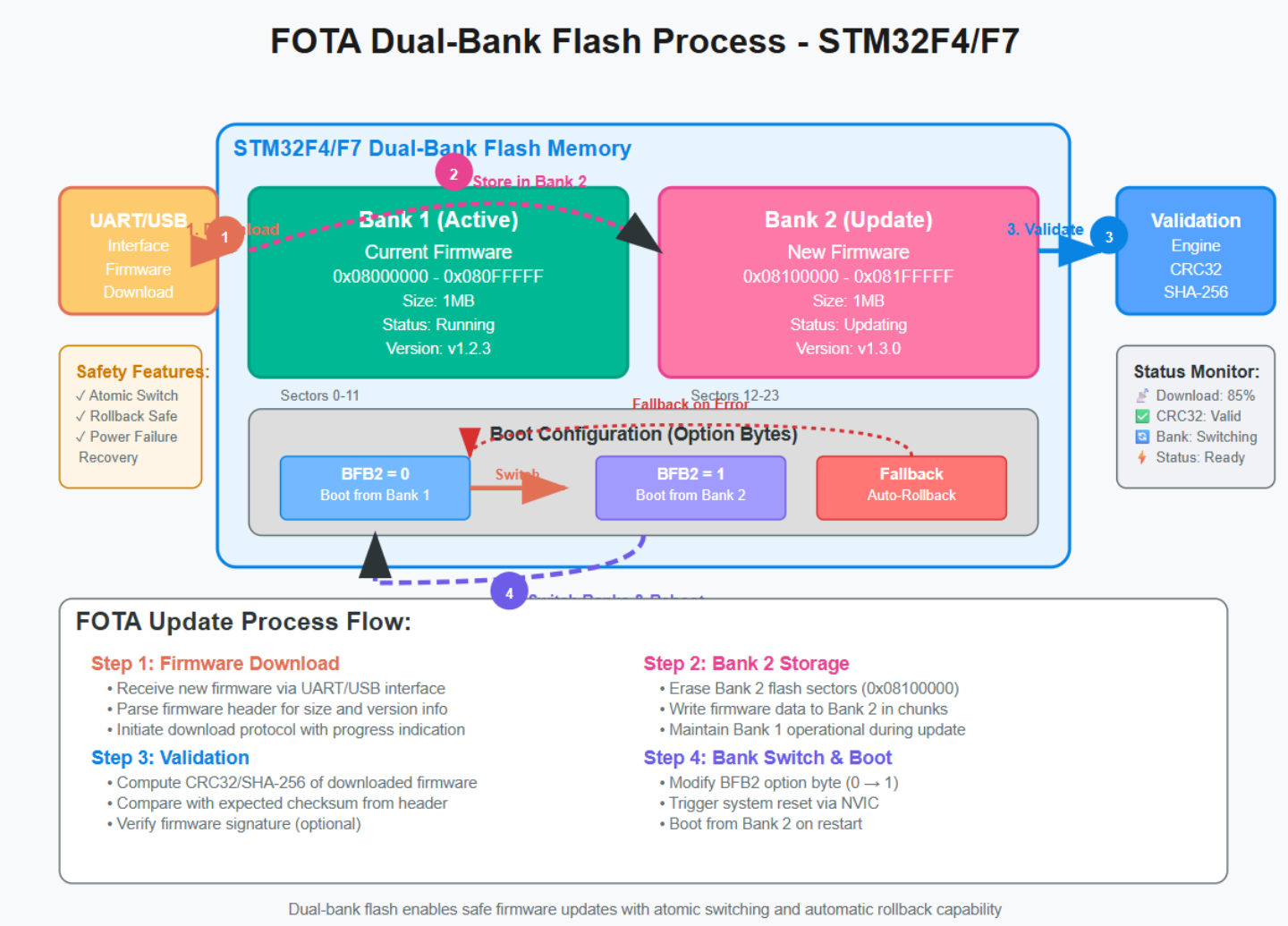


Image: FOTA Dual-Bank Flash Process

Description: A diagram showing the STM32F4/F7 dual-bank flash layout: Bank 1 (0x08000000, active firmware), Bank 2 (0x08100000, new firmware). Arrows depict the FOTA process: receive firmware via UART, store in Bank 2, validate with CRC32, switch banks via option bytes, and reboot. A fallback path to Bank 1 is shown for rollback.

2. STM32F4/F7 Dual-Bank Flash

- **Architecture**:
  - STM32F4/F7 supports dual-bank flash (e.g., 1MB Bank 1 at 0x08000000, 1MB Bank 2 at 0x08100000 on STM32F7).
  - Option bytes control boot bank (e.g., BFB2 bit in FLASH_OPTCR).
- **Bank Switching**:
  - Program new firmware into inactive bank (e.g., Bank 2).
  - Update option bytes to boot from new bank.
  - Trigger reboot to load new firmware.
- **Flash Operations**:
  - Erase inactive bank before programming.
  - Program firmware in 32-bit or sector-aligned chunks.
  - Use STM32 HAL for flash erase/program (e.g., HAL_FLASH_Program).
- **Embedded Tip**: Lock flash with RDP Level 1 during FOTA to prevent unauthorized access; use double buffering for reliable updates.

3. FOTA Workflow

- **Steps**:
  1. Receive firmware via UART (simulated as a precomputed image for simplicity).
  2. Erase secondary bank (e.g., Bank 2).
  3. Program firmware into secondary bank.
  4. Validate firmware (e.g., CRC32 from Day 31).
  5. Update option bytes to switch boot bank.
  6. Trigger reboot.
- **Rollback**:
  - Store current bank ID in flash or RTC backup registers.
  - If new firmware fails validation post-reboot, revert to original bank.
- **Embedded Tip**: Use a bootloader with FOTA logic separate from application code to simplify updates.

4. Practice Program: Dual-Bank Flash Programming

- **Objective**: Erase and program the secondary flash bank, then simulate bank switching.
- **Code** (Simulation, heavily commented):

```c
/* Simulated dual-bank flash programming */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for memory operations

/* Simulated flash memory */
static uint8_t bank1[64] = {0}; // Simulate Bank 1 (active firmware)
static uint8_t bank2[64] = {0}; // Simulate Bank 2 (new firmware)
static const uint8_t new_firmware[] = { // Simulated new firmware image
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, // Sample data
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

/* Simulated STM32 flash registers */
static uint32_t flash_optcr = 0U; // Simulate FLASH_OPTCR for bank switching
#define BFB2 (1U << 15) // Simulate BFB2 bit for Bank 2 boot

/* Erase flash bank (simulated) */
static void erase_bank(uint8_t *bank, uint32_t size) // Erase specified bank
{
    memset(bank, 0xFF, size); // Set bank to 0xFF (erased state)
    printf("Bank erased\n"); // Log erase operation
}

/* Program flash bank (simulated) */
static void program_bank(uint8_t *bank, const uint8_t *data, uint32_t size) // Program data into bank
{
    memcpy(bank, data, size); // Copy firmware to bank
    printf("Bank programmed\n"); // Log program operation
}

/* Switch to Bank 2 (simulated) */
static void switch_to_bank2(void) // Set option bytes for Bank 2 boot
{
    flash_optcr |= BFB2; // Set BFB2 bit to boot from Bank 2
    printf("Switched to Bank 2\n"); // Log bank switch
}
```

```
/* Main function */
int main(void) // Main entry point
{
    erase_bank(bank2, sizeof(bank2)); // Erase Bank 2
    program_bank(bank2, new_firmware, sizeof(new_firmware)); // Program new firmware into Bank 2
    switch_to_bank2(); // Switch to Bank 2 for next boot
    printf("FOTA complete. Rebooting...\n"); // Log FOTA completion
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Simple FOTA Framework

- **Objective**: Develop a simple FOTA framework on STM32F7 that stores a new firmware image in the secondary flash bank (Bank 2) and switches to it, with UART logging for status.
- **Requirements**:
  - Configure STM32F7 flash: Bank 1 (0x08000000, active firmware), Bank 2 (0x08100000, new firmware).
  - Use UART (PA9-TX, 115200 baud) for status logging.
  - Erase and program Bank 2 with a simulated firmware image.
  - Validate firmware with CRC32 (from Day 31).
  - Update option bytes to boot from Bank 2.
  - Trigger reboot to load new firmware.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART and flash via STM32CubeMX.
  - **FOTA Process**: Erase Bank 2, program firmware, validate with CRC32, update option bytes, reboot.
  - **Validation**: Use STM32 CRC peripheral for integrity check.
  - **Debugging**: Use STM32CubeIDE debugger to verify flash operations and bank switching.
- **Code** (STM32F7 with HAL, each line commented):

```
/* Simple FOTA framework for dual-bank flash */
#include "stm32f7xx_hal.h" // Include STM32F7 HAL library for hardware access
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for UART logging

/* UART handle for logging */
static UART_HandleTypeDef huart1; // UART handle for USART1

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART messages

/* Firmware addresses */
#define BANK1_ADDRESS 0x08000000U // Bank 1 start address (active firmware)
#define BANK2_ADDRESS 0x08100000U // Bank 2 start address (new firmware)
#define FIRMWARE_SIZE 0x10000U // Firmware size (64KB)
#define CRC_ADDRESS (BANK2_ADDRESS + FIRMWARE_SIZE) // Address to store CRC32
#define EXPECTED_CRC32 0xA2A8B25FU // Precomputed CRC32 (via OpenSSL or STM32CubeProgrammer)

/* Simulated firmware image (for testing) */
static const uint8_t new_firmware[] = { // Simulated new firmware data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, // Sample data
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 for logging
static void init_crc(void); // Initialize CRC peripheral
```

```
static void log_message(const uint8_t *message); // Log message via UART
static void erase_bank2(void); // Erase Bank 2
static void program_bank2(const uint8_t *data, uint32_t size); // Program Bank 2
static uint32_t compute_firmware_crc32(void); // Compute CRC32 of Bank 2
static void switch_to_bank2(void); // Switch to Bank 2 for boot
static void reboot(void); // Trigger system reboot

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock for UART pins
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set very high-speed output for UART
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins for UART
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX modes
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
}

/* Initialize CRC peripheral */
static void init_crc(void)
{
    __HAL_RCC_CRC_CLK_ENABLE(); // Enable CRC peripheral clock
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    CRC->INIT = 0xFFFFFFFFU; // Set initial CRC value (STM32 default)
    CRC->POL = 0x04C11DB7U; // Set CRC32 polynomial
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Erase Bank 2 */
static void erase_bank2(void)
{
    FLASH_EraseInitTypeDef erase_init = {0U}; // Initialize erase structure (MISRA Rule 9.1)
    uint32_t sector_error = 0U; // Variable to store erase errors
    HAL_FLASH_Unlock(); // Unlock flash for erase/program
    erase_init.TypeErase = FLASH_TYPEERASE_SECTORS; // Set erase type to sectors
    erase_init.Banks = FLASH_BANK_2; // Select Bank 2 for erase
    erase_init.Sector = FLASH_SECTOR_0; // Start from first sector of Bank 2
    erase_init.NbSectors = 8; // Number of sectors to erase (covers 1MB)
    if (HAL_FLASHEx_Erase(&erase_init, &sector_error) != HAL_OK) { // Erase Bank 2
        log_message((const uint8_t *)"Bank 2 Erase Failed\r\n"); // Log erase error
        HAL_FLASH_Lock(); // Lock flash on error
        while (1) { // Infinite loop on error
```

```
        }
    }
    HAL_FLASH_Lock(); // Lock flash after erase
    log_message((const uint8_t *)"Bank 2 Erased\r\n"); // Log erase success
}

/* Program Bank 2 with firmware */
static void program_bank2(const uint8_t *data, uint32_t size)
{
    HAL_FLASH_Unlock(); // Unlock flash for programming
    uint32_t address = BANK2_ADDRESS; // Start address of Bank 2
    for (uint32_t i = 0U; i < size; i += 4U) { // Loop through data in 32-bit chunks (MISRA Rule 15.6)
        uint32_t word = *(uint32_t *)(data + i); // Read 32-bit word from data
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) { // Program word
to flash
            log_message((const uint8_t *)"Bank 2 Program Failed\r\n"); // Log program error
            HAL_FLASH_Lock(); // Lock flash on error
            while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash after programming
    log_message((const uint8_t *)"Bank 2 Programmed\r\n"); // Log program success
}

/* Compute CRC32 of Bank 2 */
static uint32_t compute_firmware_crc32(void)
{
    uint32_t *flash_ptr = (uint32_t *)BANK2_ADDRESS; // Pointer to Bank 2 start
    uint32_t length = FIRMWARE_SIZE / 4U; // Number of 32-bit words in firmware
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        CRC->DR = flash_ptr[i]; // Feed 32-bit word to CRC peripheral
    }
    return CRC->DR; // Return computed CRC32
}

/* Switch to Bank 2 for boot */
static void switch_to_bank2(void)
{
    HAL_FLASH_Unlock(); // Unlock flash for option byte programming
    HAL_FLASH_OB_Unlock(); // Unlock option bytes
    FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure (MISRA Rule 9.1)
    ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type to user
    ob_init.USERType = OB_USER_BFB2; // Select BFB2 (Bank 2 boot) option
    ob_init.USERConfig = OB_BFB2_ENABLE; // Enable boot from Bank 2
    if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
        log_message((const uint8_t *)"Bank Switch Failed\r\n"); // Log switch error
        HAL_FLASH_OB_Lock(); // Lock option bytes on error
        HAL_FLASH_Lock(); // Lock flash on error
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_OB_Lock(); // Lock option bytes
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Switched to Bank 2\r\n"); // Log switch success
}

/* Trigger system reboot */
static void reboot(void)
{
    log_message((const uint8_t *)"Rebooting...\r\n"); // Log reboot message
    HAL_Delay(100); // Delay to ensure UART transmission completes
    NVIC_SystemReset(); // Trigger system reset
}
```

```
/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library for STM32 peripherals
    init_gpio(); // Initialize GPIO for UART pins
    init_uart(); // Initialize USART1 for logging
    init_crc(); // Initialize CRC peripheral
    const uint8_t *startup_msg = (const uint8_t *)"FOTA Started\r\n"; // Define startup message
    log_message(startup_msg); // Log startup message
    erase_bank2(); // Erase Bank 2 for new firmware
    program_bank2(new_firmware, sizeof(new_firmware)); // Program simulated firmware into Bank 2
    uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32 of Bank 2
    snprintf((char *)log_buffer, sizeof(log_buffer), "CRC32: 0x%08X\r\n", computed_crc); // Format
computed CRC
    log_message(log_buffer); // Log computed CRC
    if (computed_crc == EXPECTED_CRC32) { // Compare with expected CRC
        switch_to_bank2(); // Switch to Bank 2 for next boot
        reboot(); // Trigger reboot to load new firmware
    } else { // CRC mismatch
        log_message((const uint8_t *)"Firmware Invalid! Halting.\r\n"); // Log invalid firmware
message
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32F7**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud.
    - **CRC**: Enable CRC peripheral (default polynomial 0x04C11DB7).
    - **Flash**: Dual-bank mode, Bank 1 (0x08000000), Bank 2 (0x08100000).
    - **Option Bytes**: Enable dual-bank mode, set RDP Level 1 for debug protection.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash bootloader to 0x08000000; simulate new firmware in Bank 2.
  - **Precompute CRC32**:
    - Use OpenSSL: openssl dgst -crc32 firmware.bin > crc32.txt.
    - Alternatively, use STM32CubeProgrammer to compute and program CRC32.
  - **Testing**:
    - Run in STM32CubeIDE; verify UART output ("Bank 2 Erased", "Programmed", "Switched").
    - Use debugger to set breakpoint in program_bank2 and switch_to_bank2.
    - Test invalid CRC by modifying EXPECTED_CRC32 or firmware data.
  - **MISRA Compliance**:
    - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
    - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
  - **Performance**: Use sector-by-sector erase for large banks; batch flash programming.
  - **Security**: Store CRC32 in secure flash or RTC backup registers.
  - **Robustness**: Validate flash addresses to prevent out-of-bounds writes.
- **Embedded Tip**: Test bank switching with a small test firmware in Bank 2; use STM32CubeProgrammer to verify option bytes.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html. Use with STM32CubeMX for configuration.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing and option byte programming.
- **UART Interface**: Use PuTTY or similar for UART communication.
- **Debugging**: Use STM32CubeIDE debugger or GDB with OpenOCD (openocd -f interface/stlink.cfg -f target/stm32f7x.cfg).

Resources

- **STM32 AN4760**: FOTA guidelines for dual-bank flash (https://www.st.com).
- **embedded.com FOTA Tutorials**: Search “FOTA embedded” on https://www.embedded.com.
- **YouTube FOTA Videos**: Search “STM32 FOTA implementation”.
- **STM32 Flash Manuals**: Download from https://www.st.com (e.g., RM0438 for STM32F7).

Practice Tips

- **Compile and Test**: Run simulation with GCC or STM32 code in STM32CubeIDE.
- **Experiment**: Simulate invalid firmware by corrupting Bank 2; verify halt behavior.
- **Optimize**: Add progress logging during flash programming for large images.
- **Time Management**: Spend ~1 hour on theory (notes), ~1.5 hours on practice program, ~1 hour on the project.

Implement FOTA Update Process

Goal

Implement a UART-based Firmware Over-The-Air (FOTA) update process on an STM32F7 microcontroller, receiving firmware via UART, storing it in the secondary flash bank, validating it with CRC32, switching banks, and implementing a rollback mechanism for failed updates.

Tasks

- **Study**: Learn UART-based FOTA, firmware reception protocols, and rollback mechanisms.
- **Practice**: Write code to receive firmware via UART, program it to flash, and handle rollback.
- **Project**: Develop a FOTA update system with UART reception, flash programming, CRC32 validation, bank switching, and rollback, tested with a Python script for firmware sending.
- **Tools**: STM32CubeIDE, STM32F7, UART interface, Python (firmware sender), GDB, OpenOCD.
- **Resources**: STM32 AN4760 (FOTA), STM32 flash manuals, embedded.com OTA articles, Python serial docs.

Detailed Notes

1. UART-Based FOTA Overview

- **Definition**: UART-based FOTA receives firmware updates over a serial interface, stores them in flash, validates integrity, and switches to the new firmware.
- **Key Components**:
  - **Firmware Reception**: Receive binary data via UART (e.g., using DMA for efficiency).
  - **Storage**: Program data into secondary flash bank (e.g., Bank 2 at 0x08100000).
  - **Validation**: Verify firmware integrity with CRC32 (or SHA-256 for security).
  - **Bank Switching**: Update option bytes to boot from new bank.
  - **Rollback**: Revert to original bank if new firmware fails validation post-reboot.
- **Protocol**:
  - Simple protocol: Send firmware size, followed by binary data, ending with CRC32.
  - Example: [SIZE:4 bytes][FIRMWARE:SIZE bytes][CRC32:4 bytes].
- **Embedded Tip**: Use DMA for UART reception to handle large firmware images efficiently; implement timeouts to prevent hangs.

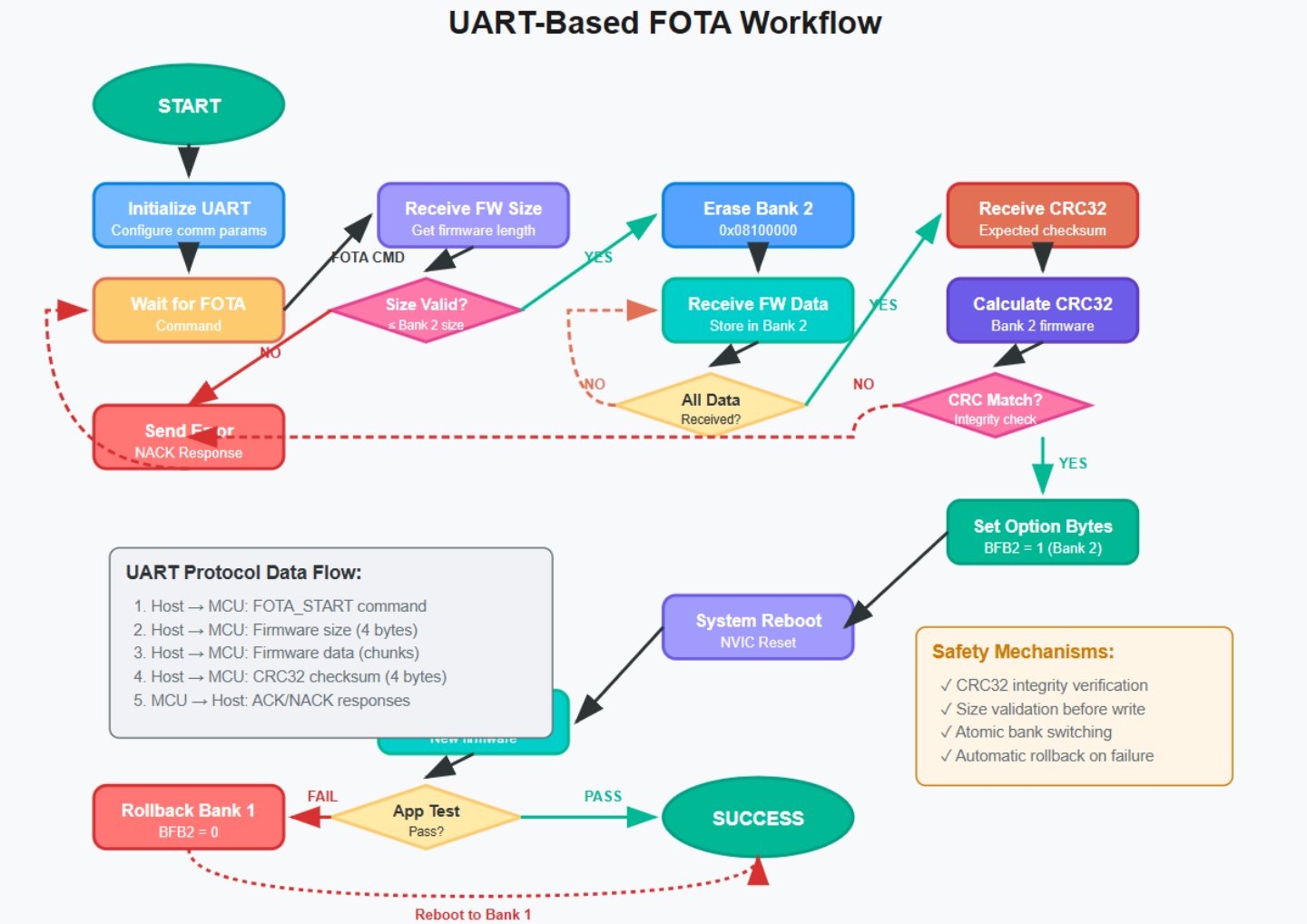


Image: UART-Based FOTA Workflow

Description: A flowchart showing the FOTA process: UART receives firmware size, data, and CRC32; stores data in Bank 2; validates with CRC32; switches to Bank 2 via option bytes; reboots. A rollback path reverts to Bank 1 if validation fails post-reboot. Arrows indicate data flow and decision points.

2. Rollback Mechanism

- **Purpose**: Ensures system reliability by reverting to the original firmware if the new firmware is invalid or fails to boot.
- **Implementation**:
  - Store current bank ID in RTC backup registers or a reserved flash section.
  - Set a boot flag in RTC backup register to indicate new firmware trial.
  - Post-reboot, bootloader checks flag; if new firmware fails (e.g., invalid CRC32), revert to original bank.
- **Embedded Tip**: Use a boot counter to limit rollback attempts and prevent infinite loops; store rollback data in non-volatile memory.

3. STM32F7 Flash and UART

- **Dual-Bank Flash**:
  - Bank 1 (0x08000000): Active firmware.
  - Bank 2 (0x08100000): New firmware.

    - Option bytes (FLASH_OPTCR, BFB2 bit) control boot bank.
- **UART with DMA**:
    - Use USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for efficient reception.
    - DMA circular mode for continuous reception; process chunks of data.
- **Embedded Tip**: Validate received data size and CRC32 before programming to avoid flash corruption.

4. Practice Program: UART Firmware Reception

- **Objective**: Receive a small firmware image via UART and store it in a buffer, simulating flash programming.
- **Code** (Simulation, heavily commented):

```c
/* Simulated UART-based firmware reception */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for memory operations

/* Simulated flash and UART buffers */
static uint8_t flash_bank2[64] = {0}; // Simulate Bank 2
static uint8_t uart_buffer[64] = {0}; // Simulate UART receive buffer
static const uint8_t test_firmware[] = { // Simulated firmware data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07 // Sample 8-byte data
};
static const uint32_t expected_crc32 = 0xA2A8B25F; // Precomputed CRC32

/* Simulated UART receive */
static void uart_receive(uint8_t *buffer, uint32_t size) // Simulate UART reception
{
    memcpy(buffer, test_firmware, size); // Copy test firmware to buffer
    printf("Received %u bytes\n", size); // Log received size
}

/* Simulated CRC32 computation */
static uint32_t compute_crc32(const uint8_t *data, uint32_t length) // Compute CRC32
{
    uint32_t crc = 0xFFFFFFFFU; // Initialize CRC to 0xFFFFFFFF
    for (uint32_t i = 0U; i < length; i++) { // Loop through data (MISRA Rule 15.6)
        crc ^= (uint32_t)data[i]; // XOR byte into CRC
        for (uint32_t j = 0U; j < 8U; j++) { // Process 8 bits
            crc = (crc >> 1U) ^ ((crc & 1U) ? 0x04C11DB7U : 0U); // Apply CRC32 polynomial
        }
    }
    return ~crc; // Return inverted CRC
}

/* Main function */
int main(void) // Main entry point
{
    uint32_t firmware_size = sizeof(test_firmware); // Firmware size
    uart_receive(uart_buffer, firmware_size); // Receive firmware via UART
    memcpy(flash_bank2, uart_buffer, firmware_size); // Program to simulated Bank 2
    uint32_t computed_crc = compute_crc32(flash_bank2, firmware_size); // Compute CRC32
    printf("Computed CRC32: 0x%08X\n", computed_crc); // Print computed CRC
    if (computed_crc == expected_crc32) { // Compare with expected CRC
        printf("Firmware valid. Ready to switch.\n"); // Log valid firmware
    } else { // CRC mismatch
        printf("Firmware invalid!\n"); // Log invalid firmware
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: FOTA Update with Rollback

- **Objective**: Implement a UART-based FOTA system on STM32F7 that receives firmware, programs it to Bank 2, validates with CRC32, switches banks, and supports rollback if the new firmware is invalid.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX.
  - Receive firmware: 4-byte size, binary data, 4-byte CRC32.
  - Program firmware to Bank 2 (0x08100000).
  - Validate with CRC32; store CRC32 at 0x08110000.
  - Update option bytes to boot from Bank 2.
  - Implement rollback using RTC backup registers.
  - Log status via UART; test with Python sender script.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, DMA, CRC, flash, and RTC via STM32CubeMX.
  - **FOTA Process**: Receive firmware via UART DMA, erase/program Bank 2, validate CRC32, switch banks, reboot.
  - **Rollback**: Store boot bank and trial flag in RTC backup registers; revert if validation fails.
  - **Debugging**: Use GDB/OpenOCD to verify reception, programming, and rollback.
- **Python Sender Script** (for testing):

```python
# Python script to send firmware via UART
import serial
import struct
import binascii

# Configure serial port
ser = serial.Serial('COM3', 115200, timeout=1)  # Adjust COM port as needed

# Firmware data (example)
firmware = bytes([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07])
firmware_size = len(firmware)
crc32 = binascii.crc32(firmware)  # Compute CRC32

# Send firmware: size (4 bytes), data, CRC32 (4 bytes)
ser.write(struct.pack('<I', firmware_size))  # Send size (little-endian)
ser.write(firmware)  # Send firmware data
ser.write(struct.pack('<I', crc32))  # Send CRC32
print(f"Sent {firmware_size} bytes, CRC32: 0x{crc32:08X}")

ser.close()  # Close serial port
```

- **Code** (STM32F7 with HAL, each line commented):

```c
/* UART-based FOTA with rollback */
#include "stm32f7xx_hal.h" // Include STM32F7 HAL library for hardware access
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for memory operations

/* UART, DMA, and RTC handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX
static RTC_HandleTypeDef hrtc; // RTC handle for backup registers

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART logging
static uint8_t rx_buffer[1024U]; // Buffer for UART RX (1KB chunks)
```

```
static volatile uint32_t firmware_size = 0U; // Received firmware size
static volatile uint8_t rx_complete = 0U; // Flag for RX completion

/* Firmware addresses */
#define BANK1_ADDRESS 0x08000000U // Bank 1 start address (active firmware)
#define BANK2_ADDRESS 0x08100000U // Bank 2 start address (new firmware)
#define FIRMWARE_MAX_SIZE 0x10000U // Max firmware size (64KB)
#define CRC_ADDRESS (BANK2_ADDRESS + FIRMWARE_MAX_SIZE) // Address for CRC32

/* RTC backup registers */
#define RTC_BKP_BOOT_BANK RTC_BKP_DR0 // Store current boot bank
#define RTC_BKP_BOOT_FLAG RTC_BKP_DR1 // Store boot trial flag

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_crc(void); // Initialize CRC peripheral
static void init_rtc(void); // Initialize RTC for backup registers
static void log_message(const uint8_t *message); // Log message via UART
static void erase_bank2(void); // Erase Bank 2
static void program_bank2(const uint8_t *data, uint32_t size); // Program Bank 2
static uint32_t compute_firmware_crc32(void); // Compute CRC32 of Bank 2
static void switch_to_bank2(void); // Switch to Bank 2 for boot
static void reboot(void); // Trigger system reboot
static void receive_firmware(void); // Receive firmware via UART
static void check_rollback(void); // Check and handle rollback

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock for UART pins
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull mode
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set very high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins for UART
}

/* Initialize USART1 with DMA for RX */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock for RX
    huart1.Instance = USART1; // Set UART instance to USART1
    huart1.Init.BaudRate = 115200U; // Set baud rate to 115200
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX modes
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART and check for errors
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Stream5; // Set DMA stream for UART RX
    hdma_usart1_rx.Init.Channel = DMA_CHANNEL_4; // Set DMA channel
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment for
peripheral
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment for memory
    hdma_usart1_rx.Init.Mode = DMA_NORMAL; // Set normal mode for RX
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority for RX
    if (HAL_DMA_Init(&hdma_usart1_rx) != HAL_OK) { // Initialize DMA
        log_message((const uint8_t *)"DMA Init Failed\r\n"); // Log DMA error
```

```
            while (1) { // Infinite loop on error
        }
    }
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART
    HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 5U, 0U); // Set DMA interrupt priority
    HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn); // Enable DMA interrupt
}

/* Initialize CRC peripheral */
static void init_crc(void)
{
    __HAL_RCC_CRC_CLK_ENABLE(); // Enable CRC peripheral clock
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    CRC->INIT = 0xFFFFFFFFU; // Set initial CRC value
    CRC->POL = 0x04C11DB7U; // Set CRC32 polynomial
}

/* Initialize RTC for backup registers */
static void init_rtc(void)
{
    __HAL_RCC_RTC_ENABLE(); // Enable RTC clock
    hrtc.Instance = RTC; // Set RTC instance
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24; // Set 24-hour format
    hrtc.Init.AsynchPrediv = 127U; // Set asynchronous prescaler
    hrtc.Init.SynchPrediv = 255U; // Set synchronous prescaler
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE; // Disable RTC output
    if (HAL_RTC_Init(&hrtc) != HAL_OK) { // Initialize RTC
        log_message((const uint8_t *)"RTC Init Failed\r\n"); // Log RTC error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1 as current boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Erase Bank 2 */
static void erase_bank2(void)
{
    FLASH_EraseInitTypeDef erase_init = {0U}; // Initialize erase structure (MISRA Rule 9.1)
    uint32_t sector_error = 0U; // Variable for erase errors
    HAL_FLASH_Unlock(); // Unlock flash for erase
    erase_init.TypeErase = FLASH_TYPEERASE_SECTORS; // Set erase type to sectors
    erase_init.Banks = FLASH_BANK_2; // Select Bank 2
    erase_init.Sector = FLASH_SECTOR_0; // Start from first sector of Bank 2
    erase_init.NbSectors = 8; // Erase 8 sectors (1MB)
    if (HAL_FLASHEx_Erase(&erase_init, &sector_error) != HAL_OK) { // Erase Bank 2
        log_message((const uint8_t *)"Bank 2 Erase Failed\r\n"); // Log erase error
        HAL_FLASH_Lock(); // Lock flash on error
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash after erase
    log_message((const uint8_t *)"Bank 2 Erased\r\n"); // Log erase success
}
```

```c
/* Program Bank 2 with firmware */
static void program_bank2(const uint8_t *data, uint32_t size)
{
    HAL_FLASH_Unlock(); // Unlock flash for programming
    uint32_t address = BANK2_ADDRESS; // Start address of Bank 2
    for (uint32_t i = 0U; i < size; i += 4U) { // Loop through data in 32-bit chunks (MISRA Rule 15.6)
        uint32_t word = *(uint32_t *)(data + i); // Read 32-bit word
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) { // Program word
            log_message((const uint8_t *)"Bank 2 Program Failed\r\n"); // Log program error
            HAL_FLASH_Lock(); // Lock flash on error
            while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash after programming
    log_message((const uint8_t *)"Bank 2 Programmed\r\n"); // Log program success
}

/* Compute CRC32 of Bank 2 */
static uint32_t compute_firmware_crc32(void)
{
    uint32_t *flash_ptr = (uint32_t *)BANK2_ADDRESS; // Pointer to Bank 2
    uint32_t length = firmware_size / 4U; // Number of 32-bit words
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        CRC->DR = flash_ptr[i]; // Feed 32-bit word to CRC
    }
    return CRC->DR; // Return computed CRC32
}

/* Switch to Bank 2 for boot */
static void switch_to_bank2(void)
{
    HAL_FLASH_Unlock(); // Unlock flash for option bytes
    HAL_FLASH_OB_Unlock(); // Unlock option bytes
    FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure (MISRA Rule 9.1)
    ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type to user
    ob_init.USERType = OB_USER_BFB2; // Select BFB2 (Bank 2 boot)
    ob_init.USERConfig = OB_BFB2_ENABLE; // Enable boot from Bank 2
    if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
        log_message((const uint8_t *)"Bank Switch Failed\r\n"); // Log switch error
        HAL_FLASH_OB_Lock(); // Lock option bytes
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_OB_Lock(); // Lock option bytes
    HAL_FLASH_Lock(); // Lock flash
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 2U); // Set Bank 2 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 1U); // Set boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    log_message((const uint8_t *)"Switched to Bank 2\r\n"); // Log switch success
}

/* Trigger system reboot */
static void reboot(void)
{
    log_message((const uint8_t *)"Rebooting...\r\n"); // Log reboot message
    HAL_Delay(100); // Delay to ensure UART transmission
    NVIC_SystemReset(); // Trigger system reset
}
```

```
/* Receive firmware via UART */
static void receive_firmware(void)
{
    uint32_t received_crc32; // Variable for received CRC32
    rx_complete = 0U; // Clear RX completion flag
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&firmware_size, 4U) != HAL_OK) { // Receive 4-byte
size
        log_message((const uint8_t *)"Size RX Failed\r\n"); // Log size RX error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for size reception
    }
    if (firmware_size > FIRMWARE_MAX_SIZE) { // Check if size exceeds max
        log_message((const uint8_t *)"Firmware Too Large\r\n"); // Log size error
        while (1) { // Infinite loop on error
        }
    }
    snprintf((char *)log_buffer, sizeof(log_buffer), "Size: %u bytes\r\n", firmware_size); // Format
size message
    log_message(log_buffer); // Log received size
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, rx_buffer, firmware_size) != HAL_OK) { // Receive firmware data
        log_message((const uint8_t *)"Firmware RX Failed\r\n"); // Log firmware RX error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for firmware reception
    }
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&received_crc32, 4U) != HAL_OK) { // Receive CRC32
        log_message((const uint8_t *)"CRC RX Failed\r\n"); // Log CRC RX error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for CRC reception
    }
    program_bank2(rx_buffer, firmware_size); // Program received firmware to Bank 2
    uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32 of Bank 2
    snprintf((char *)log_buffer, sizeof(log_buffer), "CRC32: 0x%08X\r\n", computed_crc); // Format
computed CRC
    log_message(log_buffer); // Log computed CRC
    if (computed_crc == received_crc32) { // Compare computed and received CRC
        log_message((const uint8_t *)"Firmware Valid\r\n"); // Log valid firmware
        switch_to_bank2(); // Switch to Bank 2
        reboot(); // Trigger reboot
    } else { // CRC mismatch
        log_message((const uint8_t *)"Firmware Invalid\r\n"); // Log invalid firmware
        while (1) { // Infinite loop on error
        }
    }
}

/* Check and handle rollback */
static void check_rollback(void)
{
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    uint32_t boot_flag = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_FLAG); // Read boot trial flag
    uint32_t boot_bank = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_BANK); // Read current boot bank
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    if (boot_flag == 1U && boot_bank == 2U) { // Check if booting from Bank 2 with trial flag
        uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32 of Bank 2
        uint32_t stored_crc = *(uint32_t *)CRC_ADDRESS; // Read stored CRC32
        if (computed_crc != stored_crc) { // Check for invalid firmware
            HAL_FLASH_Unlock(); // Unlock flash for option bytes
            HAL_FLASH_OB_Unlock(); // Unlock option bytes
            FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure
            ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
```

```
            ob_init.USERType = OB_USER_BFB2; // Select BFB2 option
            ob_init.USERConfig = OB_BFB2_DISABLE; // Disable Bank 2 boot (revert to Bank 1)
            if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
                log_message((const uint8_t *)"Rollback Failed\r\n"); // Log rollback error
                while (1) { // Infinite loop on error
            }
            HAL_FLASH_OB_Lock(); // Lock option bytes
            HAL_FLASH_Lock(); // Lock flash
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1 as boot bank
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Rollback to Bank 1\r\n"); // Log rollback
            reboot(); // Trigger reboot to Bank 1
        } else { // Firmware valid
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Bank 2 Valid\r\n"); // Log valid firmware
        }
    }
}

/* UART RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) { // Check if USART1 interrupt
        rx_complete = 1U; // Set RX completion flag
    }
}

/* DMA RX interrupt handler */
void DMA1_Stream5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
    init_uart(); // Initialize USART1 with DMA
    init_crc(); // Initialize CRC peripheral
    init_rtc(); // Initialize RTC for backup registers
    log_message((const uint8_t *)"FOTA Started\r\n"); // Log startup message
    check_rollback(); // Check if rollback is needed
    erase_bank2(); // Erase Bank 2 for new firmware
    receive_firmware(); // Receive and process firmware
    while (1) { // Infinite loop (MISRA Rule 15.6)
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32F7**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud, DMA (Stream 5, Channel 4, normal mode).
    - **CRC**: Enable CRC peripheral (polynomial 0x04C11DB7).
    - **Flash**: Dual-bank mode, Bank 1 (0x08000000), Bank 2 (0x08100000).
    - **RTC**: Enable RTC for backup registers.
    - **Option Bytes**: Enable dual-bank mode, set RDP Level 1.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.

        - Flash bootloader to 0x08000000.
    - **Testing**:
        - Run Python sender script to transmit firmware (e.g., python fota_sender.py).
        - Verify UART output: "Size: X bytes", "Bank 2 Erased", "Programmed", "CRC32: 0xXXXXXXXX".
        - Use GDB/OpenOCD:
            - Run: openocd -f interface/stlink.cfg -f target/stm32f7x.cfg.
            - Start GDB: arm-none-eabi-gdb project.elf, then target remote localhost:3333.
            - Set breakpoints: break receive_firmware, break check_rollback.
            - Watch variables: print firmware_size, print computed_crc.
        - Test rollback by programming invalid CRC32 and rebooting.
    - **MISRA Compliance**:
        - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
        - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
    - **Performance**: Use DMA for UART RX to handle large firmware; process in chunks.
    - **Security**: Add timeout for UART reception; store CRC32 in secure flash.
    - **Robustness**: Validate firmware size and buffer bounds; implement retry for failed RX.
- **Embedded Tip**: Test with small firmware first; use STM32CubeProgrammer to inspect Bank 2 and option bytes post-update.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com/en/development-tools/stm32cubeide.html.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing and option bytes.
- **Python**: Install Python 3 and pyserial (pip install pyserial) for sender script.
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **STM32 AN4760**: FOTA guidelines (https://www.st.com).
- **STM32 Flash Manuals**: RM0438 for STM32F7 (https://www.st.com).
- **embedded.com OTA Articles**: Search “OTA firmware update” on https://www.embedded.com.
- **Python Serial Docs**: https://pyserial.readthedocs.io.

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE; test with Python sender.
- **Experiment**: Simulate invalid CRC32 to test rollback; add a "CLEAR" command to reset Bank 2.
- **Optimize**: Use larger RX buffer for bigger firmware; add progress logging.
- **Time Management**: Spend ~1 hour on theory, ~1.5 hours on practice, ~1 hour on project.

Integrate Crypto into FOTA

Goal

Study AES-128 encryption and integrate it into a UART-based FOTA system on an STM32L5 microcontroller, combining SHA-256 secure boot validation (Day 32) with encrypted firmware updates. The system receives encrypted firmware via UART, decrypts it using AES-128, stores it in the secondary flash bank, validates it with SHA-256, switches banks, and implements rollback for failed updates.

Tasks

- **Study**: Learn AES-128 encryption principles and STM32L5 cryptographic hardware support.
- **Practice**: Write code to decrypt AES-128 encrypted firmware and validate with SHA-256.
- **Project**: Develop a secure FOTA system with SHA-256 boot validation, AES-128 decryption, UART reception, flash programming, and rollback, tested with a Python script for sending encrypted firmware.
- **Tools**: STM32CubeIDE, STM32L5 (TrustZone and crypto support), OpenSSL (for encryption simulation), GDB, OpenOCD, Python (firmware sender).
- **Resources**: OpenSSL AES docs, STM32 crypto library, STM32 AN4992, embedded.com crypto articles.

Detailed Notes

1. AES-128 Encryption Overview

- **Definition**: AES-128 (Advanced Encryption Standard, 128-bit key) is a symmetric encryption algorithm for securing data, suitable for firmware encryption in FOTA.
- **Key Features**:
  - **Block Cipher**: Operates on 16-byte blocks.
  - **Modes**: Use Cipher Block Chaining (CBC) mode for FOTA to ensure secure chaining of blocks.
  - **Key/IV**: 16-byte key and initialization vector (IV) for AES-128-CBC.
- **Use in FOTA**:
  - Encrypt firmware on host (e.g., using OpenSSL).
  - Transmit encrypted firmware, key, and IV via UART.
  - Decrypt firmware on-device using STM32L5 CRYP peripheral.
  - Validate decrypted firmware with SHA-256 before booting.
- **Embedded Tip**: Store AES key in secure flash or STM32L5 secure memory; use hardware crypto for performance.

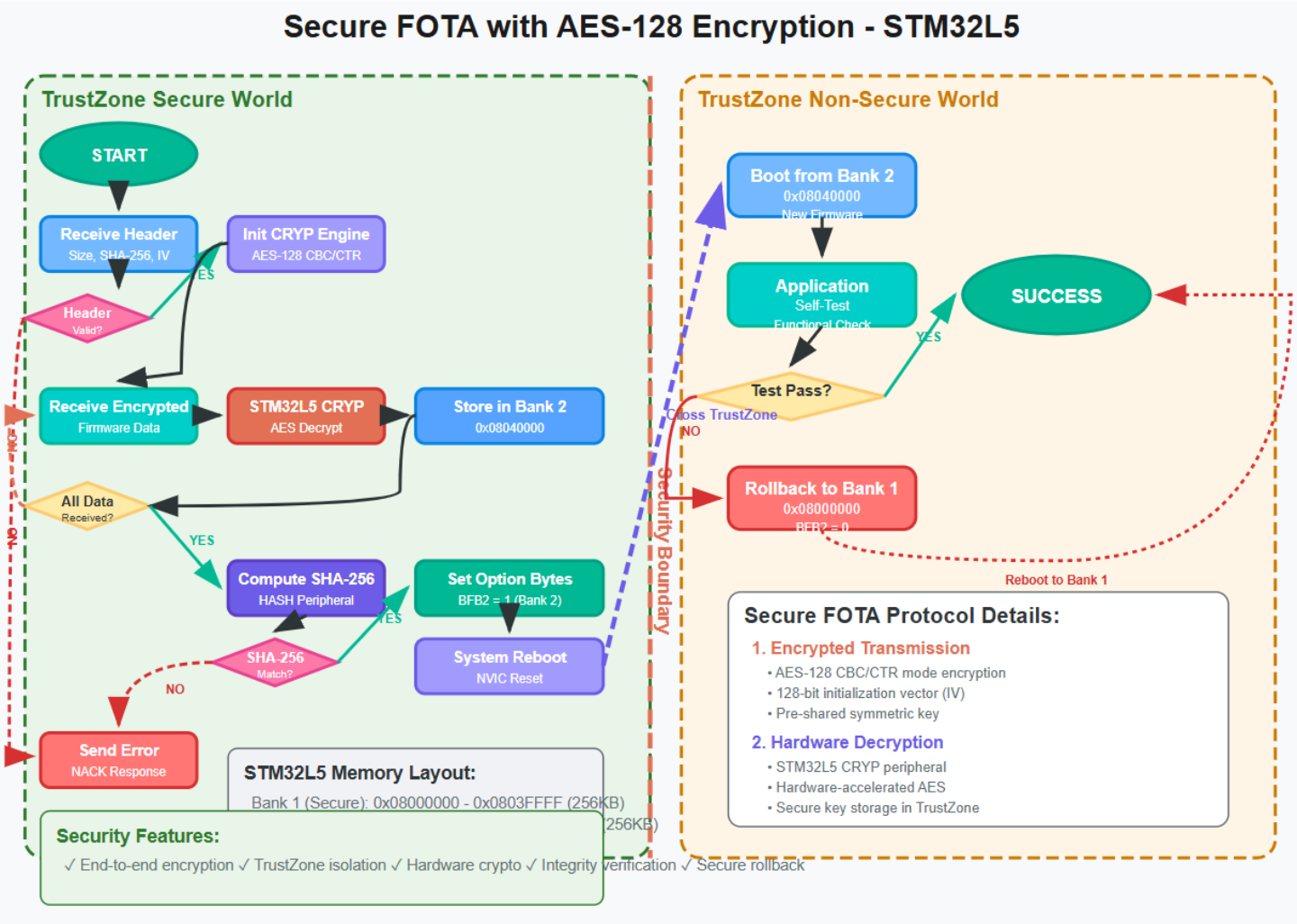


Image: Secure FOTA with AES-128

Description: A flowchart showing the secure FOTA process: UART receives encrypted firmware, size, SHA-256 hash, and IV; STM32L5 CRYP decrypts firmware; stores it in Bank 2 (0x08040000); validates with SHA-256; switches banks via option bytes; reboots. Rollback to Bank 1 (0x08000000) if validation fails. Arrows indicate TrustZone boundaries and data flow.

2. STM32L5 Cryptographic Support

- **CRYP Peripheral**:
  - Supports AES-128/256 in CBC mode.
  - Hardware acceleration for encryption/decryption.
  - Registers: CRYP_DIN (input), CRYP_DOUT (output), CRYP_KEYRx (key), CRYP_IVRx (IV).
- **HASH Peripheral** (from Day 32):
  - Computes SHA-256 for firmware validation.
- **TrustZone**:
  - Secure flash (0x08000000) for bootloader and keys.
  - Non-secure flash (0x08040000) for application.
- **Embedded Tip**: Use DMA with CRYP for large firmware decryption; secure key storage in OTP or secure flash.

3. FOTA with Encryption Workflow

- **Steps**:
  1. Receive encrypted firmware size, data, SHA-256 hash, and IV via UART (DMA).
  2. Decrypt firmware using AES-128-CBC (CRYP peripheral).
  3. Erase and program decrypted firmware into Bank 2.
  4. Compute SHA-256 of decrypted firmware; compare with received hash.
  5. Update option bytes to boot from Bank 2; set RTC backup registers for rollback.
  6. Reboot to new firmware; revert to Bank 1 if validation fails.
- **Rollback**:
  - Store boot bank and trial flag in RTC backup registers.
  - Post-reboot, validate firmware; revert to Bank 1 if invalid.
- **Embedded Tip**: Pad firmware to 16-byte blocks for AES-CBC; transmit IV securely.

4. Practice Program: AES-128 Decryption and SHA-256 Validation

- **Objective**: Decrypt a small AES-128-CBC encrypted firmware and validate with SHA-256.
- **Code** (Simulation, heavily commented):

```
/* Simulated AES-128 decryption and SHA-256 validation */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for memory operations

/* Simulated buffers */
static uint8_t encrypted_firmware[] = { // Simulated encrypted data (16 bytes, 1 block)
    0x1B, 0x2A, 0x3C, 0x4D, 0x5E, 0x6F, 0x7A, 0x8B,
    0x9C, 0xAD, 0xBE, 0xCF, 0xD0, 0xE1, 0xF2, 0x03
};

static const uint8_t aes_key[] = { // 16-byte AES-128 key
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

static const uint8_t aes_iv[] = { // 16-byte IV
    0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
    0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F
};

static const uint8_t expected_hash[] = { // Precomputed SHA-256 of decrypted firmware
    0x29, 0x7F, 0xAD, 0xF9, 0xD9, 0x12, 0x0C, 0xF6,
    0xA7, 0xE6, 0x8B, 0xD8, 0x0C, 0x7A, 0xF8, 0x1B,
    0xC3, 0x2B, 0xD2, 0xA7, 0xF6, 0x8C, 0x2F, 0xE2,
    0x1D, 0xA3, 0xF6, 0x5C, 0xD2, 0xB6, 0x5F, 0xCC
};
static uint8_t decrypted_firmware[16]; // Buffer for decrypted data

/* Simulated AES-128-CBC decryption */

static void aes_decrypt(const uint8_t *input, uint32_t size, const uint8_t *key, const uint8_t *iv,
uint8_t *output) // Decrypt data
{
    memcpy(output, input, size); // Simulate decryption (copy input)
    printf("Simulated AES-128 decryption\n"); // Log decryption
}
```

```
/* Simulated SHA-256 computation */
static void compute_sha256(const uint8_t *data, uint32_t length, uint8_t *hash) // Compute SHA-256
{
    memcpy(hash, expected_hash, 32U); // Simulate hash computation
    printf("Simulated SHA-256 computed\n"); // Log hash computation
}

/* Main function */
int main(void) // Main entry point
{
    uint8_t computed_hash[32]; // Buffer for SHA-256 hash
    aes_decrypt(encrypted_firmware, sizeof(encrypted_firmware), aes_key, aes_iv, decrypted_firmware);
// Decrypt firmware
    compute_sha256(decrypted_firmware, sizeof(decrypted_firmware), computed_hash); // Compute hash
    printf("Computed SHA-256: "); // Print hash label
    for (uint32_t i = 0U; i < 32U; i++) { // Loop through hash bytes
        printf("%02X ", computed_hash[i]); // Print each byte in hex
    }
    printf("\n"); // Newline
    if (memcmp(computed_hash, expected_hash, 32U) == 0) { // Compare hashes
        printf("Firmware valid\n"); // Log valid firmware
    } else { // Hash mismatch
        printf("Firmware invalid!\n"); // Log invalid firmware
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Secure FOTA with SHA-256 and AES-128

- **Objective**: Implement a secure FOTA system on STM32L5 with SHA-256 secure boot, AES-128 decryption, UART reception, flash programming, and rollback, tested with a Python script.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX.
  - Receive: 4-byte size, encrypted firmware, 32-byte SHA-256 hash, 16-byte IV.
  - Decrypt firmware using AES-128-CBC (CRYP peripheral).
  - Store in Bank 2 (0x08040000); validate with SHA-256.
  - Switch to Bank 2; implement rollback using RTC backup registers.
  - Log status via UART; test with valid/invalid images.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, DMA, CRYP, HASH, flash, and RTC.
  - **FOTA Process**: Receive encrypted firmware, decrypt, program Bank 2, validate SHA-256, switch banks, reboot.
  - **Rollback**: Use RTC backup registers to track boot bank and trial flag.
  - **Debugging**: Use GDB/OpenOCD to verify decryption, programming, and rollback.
- **Python Sender Script** (for testing):

```
# Python script to send encrypted firmware via UART
import serial
import struct
import hashlib
from Crypto.Cipher import AES

# Configure serial port
ser = serial.Serial('COM3', 115200, timeout=1)  # Adjust COM port
```

```
# Firmware data (padded to 16-byte boundary)
firmware = bytes([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
                  0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F])
key = bytes([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
             0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F])  # 16-byte key
iv = bytes([0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
            0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F])  # 16-byte IV

# Encrypt firmware
cipher = AES.new(key, AES.MODE_CBC, iv)
encrypted_firmware = cipher.encrypt(firmware)
firmware_size = len(encrypted_firmware)
sha256_hash = hashlib.sha256(firmware).digest()  # Hash of decrypted firmware

# Send: size (4 bytes), encrypted firmware, SHA-256 (32 bytes), IV (16 bytes)
ser.write(struct.pack('<I', firmware_size))  # Send size
ser.write(encrypted_firmware)  # Send encrypted firmware
ser.write(sha256_hash)  # Send SHA-256 hash
ser.write(iv)  # Send IV
print(f"Sent {firmware_size} bytes, SHA-256: {sha256_hash.hex()}")

ser.close()  # Close serial port
```

- **Code** (STM32L5 with HAL, each line commented):

```
/* Secure FOTA with SHA-256 and AES-128 */
#include "stm32l5xx_hal.h" // Include STM32L5 HAL library
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* Peripheral handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX
static CRYP_HandleTypeDef hcryp; // CRYP handle for AES
static RTC_HandleTypeDef hrtc; // RTC handle for backup registers

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[64U]; // Buffer for UART logging
static uint8_t rx_buffer[1024U]; // Buffer for UART RX (1KB chunks)
static uint8_t decrypted_buffer[1024U]; // Buffer for decrypted firmware
static uint8_t aes_key[16] = { // AES-128 key (securely stored in production)
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

static uint8_t aes_iv[16]; // Buffer for received IV
static uint8_t received_hash[32]; // Buffer for received SHA-256 hash
static volatile uint32_t firmware_size = 0U; // Received firmware size
static volatile uint8_t rx_complete = 0U; // RX completion flag

/* Firmware addresses */
#define BANK1_ADDRESS 0x08000000U // Bank 1 (active firmware)
#define BANK2_ADDRESS 0x08040000U // Bank 2 (new firmware)
#define FIRMWARE_MAX_SIZE 0x10000U // Max firmware size (64KB)
#define HASH_ADDRESS (BANK2_ADDRESS + FIRMWARE_MAX_SIZE) // Address for SHA-256 hash

/* RTC backup registers */
#define RTC_BKP_BOOT_BANK RTC_BKP_DR0 // Store boot bank
#define RTC_BKP_BOOT_FLAG RTC_BKP_DR1 // Store boot trial flag

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_cryp(void); // Initialize CRYP for AES-128
static void init_hash(void); // Initialize HASH for SHA-256
```

```
static void init_rtc(void); // Initialize RTC for backup registers
static void log_message(const uint8_t *message); // Log message via UART
static void erase_bank2(void); // Erase Bank 2
static void program_bank2(const uint8_t *data, uint32_t size); // Program Bank 2
static void compute_firmware_sha256(uint8_t *hash); // Compute SHA-256 of Bank 2
static void decrypt_firmware(const uint8_t *input, uint32_t size, uint8_t *output); // Decrypt
firmware
static void switch_to_bank2(void); // Switch to Bank 2
static void reboot(void); // Trigger reboot
static void receive_firmware(void); // Receive encrypted firmware
static void check_rollback(void); // Check and handle rollback

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set very high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins
}

/* Initialize USART1 with DMA */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock
    huart1.Instance = USART1; // Set UART instance
    huart1.Init.BaudRate = 115200U; // Set baud rate
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Stream5; // Set DMA stream
    hdma_usart1_rx.Init.Request = DMA_REQUEST_USART1_RX; // Set DMA request
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.Mode = DMA_NORMAL; // Set normal mode
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority
    if (HAL_DMA_Init(&hdma_usart1_rx) != HAL_OK) { // Initialize DMA
        log_message((const uint8_t *)"DMA Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART
    HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 5U, 0U); // Set DMA interrupt priority
    HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn); // Enable DMA interrupt
}
```

```
/* Initialize CRYP for AES-128-CBC */
static void init_cryp(void)
{
    __HAL_RCC_CRYP_CLK_ENABLE(); // Enable CRYP clock
    hcryp.Instance = CRYP; // Set CRYP instance
    hcryp.Init.DataType = CRYP_DATATYPE_8B; // Set 8-bit data type
    hcryp.Init.KeySize = CRYP_KEYSIZE_128B; // Set 128-bit key
    hcryp.Init.Algorithm = CRYP_AES_CBC; // Set CBC mode
    if (HAL_CRYP_Init(&hcryp) != HAL_OK) { // Initialize CRYP
        log_message((const uint8_t *)"CRYP Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
}

/* Initialize HASH for SHA-256 */
static void init_hash(void)
{
    __HAL_RCC_HASH_CLK_ENABLE(); // Enable HASH clock
    HASH->CR = HASH_CR_INIT; // Reset HASH peripheral
    HASH->CR |= HASH_ALGOSELECTION_SHA256; // Select SHA-256
    HASH->CR |= HASH_DATATYPE_8B; // Set 8-bit data type
    HASH->STR = 0U; // Clear status register
}

/* Initialize RTC for backup registers */
static void init_rtc(void)
{
    __HAL_RCC_RTC_ENABLE(); // Enable RTC clock
    hrtc.Instance = RTC; // Set RTC instance
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24; // Set 24-hour format
    hrtc.Init.AsynchPrediv = 127U; // Set asynchronous prescaler
    hrtc.Init.SynchPrediv = 255U; // Set synchronous prescaler
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE; // Disable RTC output
    if (HAL_RTC_Init(&hrtc) != HAL_OK) { // Initialize RTC
        log_message((const uint8_t *)"RTC Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Erase Bank 2 */
static void erase_bank2(void)
{
    FLASH_EraseInitTypeDef erase_init = {0U}; // Initialize erase structure (MISRA Rule 9.1)
    uint32_t sector_error = 0U; // Variable for erase errors
    HAL_FLASH_Unlock(); // Unlock flash
    erase_init.TypeErase = FLASH_TYPEERASE_SECTORS; // Set erase type
    erase_init.Banks = FLASH_BANK_2; // Select Bank 2
    erase_init.Sector = FLASH_SECTOR_0; // Start from sector 0
    erase_init.NbSectors = 8; // Erase 8 sectors (64KB)
    if (HAL_FLASHEx_Erase(&erase_init, &sector_error) != HAL_OK) { // Erase Bank 2
        log_message((const uint8_t *)"Bank 2 Erase Failed\r\n"); // Log error
        HAL_FLASH_Lock(); // Lock flash
```

```
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Erased\r\n"); // Log success
}

/* Program Bank 2 with decrypted firmware */
static void program_bank2(const uint8_t *data, uint32_t size)
{
    HAL_FLASH_Unlock(); // Unlock flash
    uint32_t address = BANK2_ADDRESS; // Bank 2 start address
    for (uint32_t i = 0U; i < size; i += 4U) { // Loop in 32-bit chunks (MISRA Rule 15.6)
        uint32_t word = *(uint32_t *)(data + i); // Read 32-bit word
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) { // Program word
            log_message((const uint8_t *)"Bank 2 Program Failed\r\n"); // Log error
            HAL_FLASH_Lock(); // Lock flash
            while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Programmed\r\n"); // Log success
}

/* Compute SHA-256 of Bank 2 */
static void compute_firmware_sha256(uint8_t *hash)
{
    uint32_t *flash_ptr = (uint32_t *)BANK2_ADDRESS; // Pointer to Bank 2
    uint32_t length = firmware_size / 4U; // Number of 32-bit words
    HASH->CR |= HASH_CR_INIT; // Reset HASH peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        HASH->DIN = flash_ptr[i]; // Feed 32-bit word
        while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for HASH to process
        }
    }
    HASH->STR |= HASH_STR_DCAL; // Trigger final hash computation
    while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for completion
    }
    for (uint32_t i = 0U; i < 8U; i++) { // Read 32-byte hash
        ((uint32_t *)hash)[i] = HASH->HR[i]; // Copy hash to buffer
    }
}

/* Decrypt firmware using AES-128-CBC */
static void decrypt_firmware(const uint8_t *input, uint32_t size, uint8_t *output)
{
    HAL_CRYP_SetKey(&hcryp, aes_key, CRYP_KEYSIZE_128B); // Set AES key
    HAL_CRYP_SetIV(&hcryp, aes_iv, CRYP_KEYSIZE_128B); // Set IV
    if (HAL_CRYP_Decrypt(&hcryp, (uint32_t *)input, size / 4U, (uint32_t *)output, 100U) != HAL_OK) {
// Decrypt firmware
        log_message((const uint8_t *)"Decryption Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    log_message((const uint8_t *)"Firmware Decrypted\r\n"); // Log success
}

/* Switch to Bank 2 for boot */
static void switch_to_bank2(void)
{
    HAL_FLASH_Unlock(); // Unlock flash
    HAL_FLASH_OB_Unlock(); // Unlock option bytes
    FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure (MISRA Rule 9.1)
    ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
    ob_init.USERType = OB_USER_BFB2; // Select BFB2
    ob_init.USERConfig = OB_BFB2_ENABLE; // Enable Bank 2 boot
```

```
     if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
        log_message((const uint8_t *)"Bank Switch Failed\r\n"); // Log error
        HAL_FLASH_OB_Lock(); // Lock option bytes
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_OB_Lock(); // Lock option bytes
    HAL_FLASH_Lock(); // Lock flash
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 2U); // Set Bank 2 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 1U); // Set boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    log_message((const uint8_t *)"Switched to Bank 2\r\n"); // Log success
}

/* Trigger system reboot */
static void reboot(void)
{
    log_message((const uint8_t *)"Rebooting...\r\n"); // Log reboot
    HAL_Delay(100); // Delay for UART transmission
    NVIC_SystemReset(); // Trigger reset
}

/* Receive encrypted firmware via UART */
static void receive_firmware(void)
{
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&firmware_size, 4U) != HAL_OK) { // Receive size
        log_message((const uint8_t *)"Size RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for size
    }
    if (firmware_size > FIRMWARE_MAX_SIZE || (firmware_size % 16U != 0U)) { // Check size and padding
        log_message((const uint8_t *)"Invalid Firmware Size\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    snprintf((char *)log_buffer, sizeof(log_buffer), "Size: %u bytes\r\n", firmware_size); // Format
size
    log_message(log_buffer); // Log size
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, rx_buffer, firmware_size) != HAL_OK) { // Receive encrypted
firmware
        log_message((const uint8_t *)"Firmware RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for firmware
    }
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, received_hash, 32U) != HAL_OK) { // Receive SHA-256 hash
        log_message((const uint8_t *)"Hash RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for hash
    }
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, aes_iv, 16U) != HAL_OK) { // Receive IV
        log_message((const uint8_t *)"IV RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
```

```
    while (rx_complete == 0U) { // Wait for IV
    }
    decrypt_firmware(rx_buffer, firmware_size, decrypted_buffer); // Decrypt firmware
    erase_bank2(); // Erase Bank 2
    program_bank2(decrypted_buffer, firmware_size); // Program decrypted firmware
    uint8_t computed_hash[32]; // Buffer for computed hash
    compute_firmware_sha256(computed_hash); // Compute SHA-256
    snprintf((char *)log_buffer, sizeof(log_buffer), "SHA-256: "); // Start hash message
    for (uint32_t i = 0U; i < 32U; i++) { // Loop through hash bytes
        snprintf((char *)log_buffer + strlen((char *)log_buffer), sizeof(log_buffer) - strlen((char
*)log_buffer),
                 "%02X ", computed_hash[i]); // Append hex byte
    }
    strncat((char *)log_buffer, "\r\n", sizeof(log_buffer) - strlen((char *)log_buffer)); // Append
newline
    log_message(log_buffer); // Log hash
    if (memcmp(computed_hash, received_hash, 32U) == 0) { // Compare hashes
        log_message((const uint8_t *)"Firmware Valid\r\n"); // Log valid firmware
        switch_to_bank2(); // Switch to Bank 2
        reboot(); // Reboot
    } else { // Hash mismatch
        log_message((const uint8_t *)"Firmware Invalid\r\n"); // Log invalid firmware
        while (1) { // Infinite loop on error
        }
    }
}

/* Check and handle rollback */
static void check_rollback(void)
{
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    uint32_t boot_flag = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_FLAG); // Read boot trial flag
    uint32_t boot_bank = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_BANK); // Read boot bank
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    if (boot_flag == 1U && boot_bank == 2U) { // Check if booting from Bank 2 with trial flag
        uint8_t computed_hash[32]; // Buffer for computed hash
        compute_firmware_sha256(computed_hash); // Compute SHA-256
        uint8_t *stored_hash = (uint8_t *)HASH_ADDRESS; // Pointer to stored hash
        if (memcmp(computed_hash, stored_hash, 32U) != 0) { // Check for invalid firmware
            HAL_FLASH_Unlock(); // Unlock flash
            HAL_FLASH_OB_Unlock(); // Unlock option bytes
            FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure
            ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
            ob_init.USERType = OB_USER_BFB2; // Select BFB2
            ob_init.USERConfig = OB_BFB2_DISABLE; // Disable Bank 2 boot
            if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
                log_message((const uint8_t *)"Rollback Failed\r\n"); // Log error
                HAL_FLASH_OB_Lock(); // Lock option bytes
                HAL_FLASH_Lock(); // Lock flash
                while (1) { // Infinite loop on error
            }
            HAL_FLASH_OB_Lock(); // Lock option bytes
            HAL_FLASH_Lock(); // Lock flash
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Rollback to Bank 1\r\n"); // Log rollback
            reboot(); // Reboot to Bank 1
        } else { // Firmware valid
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Bank 2 Valid\r\n"); // Log valid firmware
        }
    }
}
```

```
/* UART RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) { // Check USART1
        rx_complete = 1U; // Set RX completion flag
    }
}

/* DMA RX interrupt handler */
void DMA1_Stream5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
    init_uart(); // Initialize USART1 with DMA
    init_cryp(); // Initialize CRYP for AES
    init_hash(); // Initialize HASH for SHA-256
    init_rtc(); // Initialize RTC for backup registers
    log_message((const uint8_t *)"Secure FOTA Started\r\n"); // Log startup
    check_rollback(); // Check rollback
    erase_bank2(); // Erase Bank 2
    receive_firmware(); // Receive and process firmware
    while (1) { // Infinite loop (MISRA Rule 15.6)
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32L5**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud, DMA (Stream 5, Request USART1_RX).
    - **CRYP**: Enable CRYP peripheral, AES-128-CBC mode.
    - **HASH**: Enable HASH peripheral, SHA-256 mode.
    - **Flash**: Dual-bank mode, Bank 1 (0x08000000), Bank 2 (0x08040000).
    - **RTC**: Enable RTC for backup registers.
    - **Option Bytes**: Enable TrustZone, dual-bank mode, RDP Level 1.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash bootloader to 0x08000000 (secure flash).
  - **Precompute SHA-256 and Encrypt**:
    - Use OpenSSL: openssl dgst -sha256 firmware.bin > sha256.txt.
    - Encrypt: openssl enc -aes-128-cbc -in firmware.bin -out firmware.enc -K <key> -iv <iv>.
    - Use STM32CubeProgrammer to program hash to 0x08050000.
  - **Testing**:
    - Run Python sender script: python fota_sender.py.
    - Verify UART output: "Size: X bytes", "Decrypted", "SHA-256: ...", "Valid" or "Invalid".
    - Use GDB/OpenOCD:
      - Run: openocd -f interface/stlink.cfg -f target/stm32l5x.cfg.
      - Start GDB: arm-none-eabi-gdb project.elf, target remote localhost:3333.
      - Set breakpoints: break receive_firmware, break check_rollback.
      - Watch: print firmware_size, print computed_hash.
    - Test rollback by sending invalid hash or corrupted firmware.
  - **MISRA Compliance**:

        - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
        - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
    - **Performance**: Use DMA for CRYP and UART to handle large firmware.
    - **Security**: Store AES key in secure flash; use secure boot to validate bootloader.
    - **Robustness**: Validate size and padding; implement RX timeout.
- **Embedded Tip**: Test with small firmware; use STM32CubeProgrammer to verify Bank 2 and option bytes.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing.
- **OpenSSL**: Install (sudo apt install openssl) for encryption.
- **Python**: Install Python 3, pyserial (pip install pyserial), pycryptodome (pip install pycryptodome).
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **OpenSSL AES Docs**: https://www.openssl.org/docs/man3.0/man3/EVP_EncryptInit.html.
- **STM32 Crypto Library**: AN4992 (https://www.st.com).
- **embedded.com Crypto Articles**: Search “AES embedded” on https://www.embedded.com.
- **STM32 Flash Manuals**: RM0430 for STM32L5 (https://www.st.com).

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE; test with Python sender.
- **Experiment**: Send invalid hash or IV to test rollback.
- **Optimize**: Use larger RX buffer for bigger firmware; add progress logging.
- **Time Management**: Spend ~1 hour on theory, ~1.5 hours on practice, ~1 hour on project.

Chapter 8: Secure Boot and FOTA Optimization

- **Enhance secure boot with key management**
- **Optimize FOTA for reliability**
- **Add secure communication to FOTA**
- **Test secure boot and FOTA integration**
- **Consolidate Week 8 concepts**

Enhance Secure Boot with Key Management

Goal

Study public-key cryptography (RSA) and enhance the secure boot implementation on an STM32L5 microcontroller by integrating RSA signature verification. Implement secure storage of the RSA public key in secure flash and validate firmware signatures during boot, building on Day 32's SHA-256 validation.

Tasks

- **Study**: Learn RSA public-key cryptography principles and its use in secure boot.
- **Practice**: Write code to verify an RSA signature for a firmware image.
- **Project**: Implement a secure boot checker with RSA signature verification, storing the public key in secure flash, and integrating SHA-256 hash validation, with UART logging and GDB debugging.
- **Tools**: STM32CubeIDE, STM32L5 (TrustZone support), OpenSSL (RSA simulation), GDB, OpenOCD.
- **Resources**: STM32 secure flash docs (AN4992), OpenSSL RSA docs, embedded.com PKI articles.

Detailed Notes

1. RSA Public-Key Cryptography Overview

- **Definition**: RSA (Rivest-Shamir-Adleman) is an asymmetric cryptographic algorithm using a public-private key pair for encryption and signature verification.
- **Key Components**:
  - **Public Key**: Used to verify signatures; stored in secure flash.
  - **Private Key**: Used to sign firmware off-device (e.g., with OpenSSL); kept secret.
  - **Signature**: Generated by hashing firmware (SHA-256) and signing with the private key.
- **Secure Boot Use**:
  - Compute SHA-256 hash of firmware.
  - Verify RSA signature using public key to ensure authenticity.
  - Combine with Day 32's SHA-256 integrity check.
- **Advantages**:
  - Ensures firmware authenticity (only signed by trusted private key).
  - Resistant to tampering compared to symmetric methods.
- **Embedded Tip**: Use hardware cryptographic accelerators (e.g., STM32L5 CRYP) for RSA if available; otherwise, use software libraries for small-scale verification.

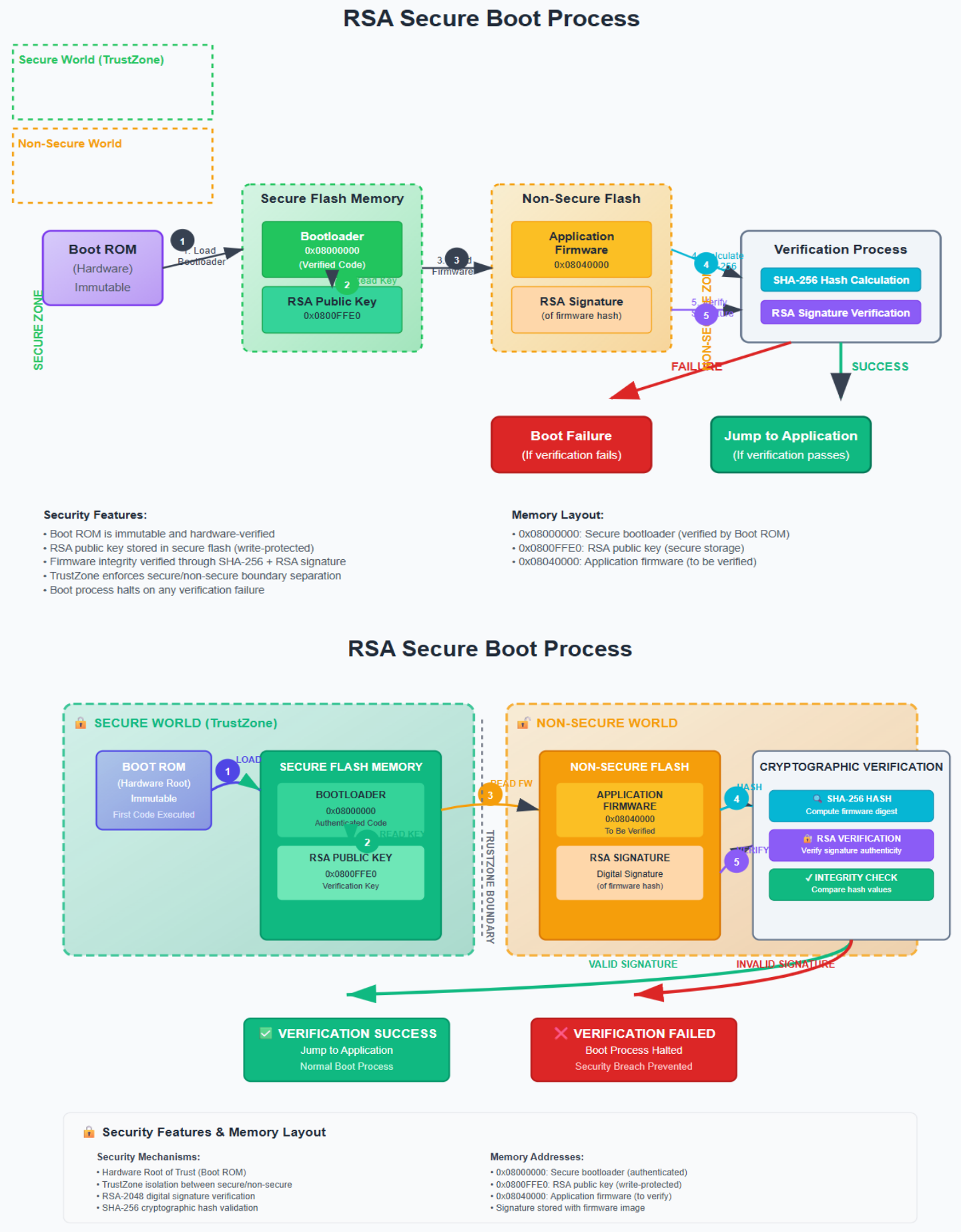


Image: RSA Secure Boot Process

Description: A diagram showing the secure boot flow: Boot ROM loads bootloader from secure flash (0x08000000), bootloader reads RSA public key from secure flash (0x0800FFE0), computes SHA-256 of firmware in non-secure flash (0x08040000), verifies RSA signature, and jumps to application if valid. Arrows indicate TrustZone boundaries and verification steps.

2. STM32L5 Secure Flash and TrustZone

- **Secure Flash**:
  - Address: 0x08000000 (bootloader), 0x0800FFE0 (RSA public key storage).
  - Protected by TrustZone; inaccessible to non-secure code.
- **TrustZone**:
  - Isolates bootloader and key storage in secure world.
  - Firmware resides in non-secure flash (0x08040000).
- **CRYP Peripheral**:
  - Limited RSA support; use software RSA library (e.g., mbedTLS subset) for signature verification.
- **Embedded Tip**: Store public key in OTP or secure flash to prevent tampering; use RDP Level 2 in production.

3. RSA Signature Verification Workflow

- **Steps**:
  1. Off-device: Compute SHA-256 hash of firmware; sign hash with RSA private key (e.g., using OpenSSL).
  2. Store RSA public key and signature in secure flash.
  3. At boot: Compute SHA-256 of firmware, verify signature with public key.
  4. Proceed to boot if valid; halt otherwise.
- **Protocol**:
  - Firmware: Stored at 0x08040000.
  - Signature: Stored at 0x08050000 (256 bytes for RSA-2048).
  - Public Key: Stored at 0x0800FFE0 (e.g., 512 bytes for RSA-2048 modulus and exponent).
- **Embedded Tip**: Use a minimal RSA library to fit within STM32L5 memory constraints; precompute signatures during build.

4. Practice Program: RSA Signature Verification

- **Objective**: Simulate RSA signature verification for a firmware image using SHA-256.
- **Code** (Simulation, heavily commented):

```
/* Simulated RSA signature verification */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions for memory operations

/* Simulated firmware and keys */
static const uint8_t firmware[] = { // Simulated firmware data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};
static const uint8_t expected_hash[32] = { // Precomputed SHA-256 hash
    0x29, 0x7F, 0xAD, 0xF9, 0xD9, 0x12, 0x0C, 0xF6,
    0xA7, 0xE6, 0x8B, 0xD8, 0x0C, 0x7A, 0xF8, 0x1B,
    0xC3, 0x2B, 0xD2, 0xA7, 0xF6, 0x8C, 0x2F, 0xE2,
    0x1D, 0xA3, 0xF6, 0x5C, 0xD2, 0xB6, 0x5F, 0xCC };
```

```
static const uint8_t rsa_signature[256] = { // Simulated RSA-2048 signature
    0x1A, 0x2B, /* ... 256 bytes ... */ 0xFF
};
static const uint8_t rsa_public_key[512] = { // Simulated RSA-2048 public key (modulus + exponent)
    0x01, 0x02, /* ... 512 bytes ... */ 0xFF
};

/* Simulated SHA-256 computation */
static void compute_sha256(const uint8_t *data, uint32_t length, uint8_t *hash) // Compute SHA-256
{
    memcpy(hash, expected_hash, 32U); // Simulate hash computation
    printf("Simulated SHA-256 computed\n"); // Log computation
}

/* Simulated RSA signature verification */
static uint8_t verify_rsa_signature(const uint8_t *hash, const uint8_t *signature, const uint8_t
*public_key) // Verify RSA signature
{
    (void)public_key; // Avoid unused parameter warning
    printf("Simulated RSA verification\n"); // Log verification
    return memcmp(hash, expected_hash, 32U) == 0; // Simulate successful verification
}

/* Main function */
int main(void) // Main entry point
{
    uint8_t computed_hash[32]; // Buffer for SHA-256 hash
    compute_sha256(firmware, sizeof(firmware), computed_hash); // Compute firmware hash
    printf("Computed SHA-256: "); // Print hash label
    for (uint32_t i = 0U; i < 32U; i++) { // Loop through hash bytes
        printf("%02X ", computed_hash[i]); // Print hex byte
    }
    printf("\n"); // Newline
    if (verify_rsa_signature(computed_hash, rsa_signature, rsa_public_key)) { // Verify signature
        printf("Signature valid. Booting...\n"); // Log valid signature
    } else { // Invalid signature
        printf("Signature invalid! Halting.\n"); // Log invalid signature
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Secure Boot with RSA Signature Verification

- **Objective**: Implement a secure boot checker on STM32L5 with RSA-2048 signature verification, storing the public key in secure flash, and combining with SHA-256 validation, with UART logging and GDB debugging.
- **Requirements**:
  - Configure STM32L5 flash: Bootloader in secure flash (0x08000000), application in non-secure flash (0x08040000).
  - Store RSA-2048 public key (512 bytes) at 0x0800FFE0, signature (256 bytes) at 0x08050000.
  - Use SHA-256 for integrity; verify RSA signature for authenticity.
  - Log validation via UART (PA9-TX, 115200 baud).
  - Halt on invalid firmware; jump to application if valid.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, HASH, and flash via STM32CubeMX.
  - **Key Storage**: Store RSA public key in secure flash.
  - **Verification**: Compute SHA-256, verify RSA signature using a minimal RSA library.
  - **Boot Decision**: Jump to application or halt based on verification.
  - **Debugging**: Use GDB/OpenOCD to verify hash and signature.

- **OpenSSL Commands** (for key generation and signing):

```
# Generate RSA-2048 key pair
openssl genrsa -out private_key.pem 2048
openssl rsa -in private_key.pem -pubout -out public_key.pem

# Compute SHA-256 and sign firmware
openssl dgst -sha256 -sign private_key.pem -out firmware.sig firmware.bin
```

- **Code** (STM32L5 with HAL, each line commented):

```
/* Secure boot with RSA-2048 signature verification */
#include "stm32l5xx_hal.h" // Include STM32L5 HAL library
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* UART handle for logging */
static UART_HandleTypeDef huart1; // UART handle for USART1

/* Static buffer for logging (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[64U]; // Buffer for UART messages
/* Firmware addresses */
#define APP_ADDRESS 0x08040000U // Application start address (non-secure)
#define APP_SIZE 0x10000U // Application size (64KB)
#define SIGNATURE_ADDRESS 0x08050000U // RSA signature address
#define PUBLIC_KEY_ADDRESS 0x0800FFE0U // RSA public key address
#define HASH_SIZE 32U // SHA-256 hash size
#define SIGNATURE_SIZE 256U // RSA-2048 signature size
#define PUBLIC_KEY_SIZE 512U // RSA-2048 public key size

/* Simulated RSA public key and signature (replace with actual) */
static const uint8_t rsa_public_key[PUBLIC_KEY_SIZE] = { // RSA-2048 public key
    0x01, 0x02, /* ... 512 bytes ... */ 0xFF
};
static const uint8_t expected_signature[SIGNATURE_SIZE] = { // RSA-2048 signature
    0x1A, 0x2B, /* ... 256 bytes ... */ 0xFF
};

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 for logging
static void init_hash(void); // Initialize HASH for SHA-256
static void log_message(const uint8_t *message); // Log message via UART
static void compute_firmware_sha256(uint8_t *hash); // Compute SHA-256
static uint8_t verify_rsa_signature(const uint8_t *hash, const uint8_t *signature, const uint8_t
*public_key); // Verify RSA signature
static void jump_to_application(void); // Jump to application

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set very high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins
}

/* Initialize USART1 for 115200 baud */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    huart1.Instance = USART1; // Set UART instance
```

```
    huart1.Init.BaudRate = 115200U; // Set baud rate
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
}

/* Initialize HASH for SHA-256 */
static void init_hash(void)
{
    __HAL_RCC_HASH_CLK_ENABLE(); // Enable HASH clock
    HASH->CR = HASH_CR_INIT; // Reset HASH peripheral
    HASH->CR |= HASH_ALGOSELECTION_SHA256; // Select SHA-256
    HASH->CR |= HASH_DATATYPE_8B; // Set 8-bit data type
    HASH->STR = 0U; // Clear status register
}



/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Compute SHA-256 of firmware */
static void compute_firmware_sha256(uint8_t *hash)
{
    uint32_t *flash_ptr = (uint32_t *)APP_ADDRESS; // Pointer to application flash
    uint32_t length = APP_SIZE / 4U; // Number of 32-bit words
    HASH->CR |= HASH_CR_INIT; // Reset HASH peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        HASH->DIN = flash_ptr[i]; // Feed 32-bit word
        while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for HASH to process
        }
    }
    HASH->STR |= HASH_STR_DCAL; // Trigger final hash computation
    while ((HASH->SR & HASH_SR_BUSY) != 0U) { // Wait for completion
    }
    for (uint32_t i = 0U; i < 8U; i++) { // Read 32-byte hash
        ((uint32_t *)hash)[i] = HASH->HR[i]; // Copy hash to buffer
    }
}

/* Verify RSA signature (simplified, assumes minimal RSA library) */
static uint8_t verify_rsa_signature(const uint8_t *hash, const uint8_t *signature, const uint8_t
*public_key)
{
    (void)public_key; // Avoid unused parameter warning
    log_message((const uint8_t *)"Verifying RSA signature...\r\n"); // Log verification start
    /* In production, use mbedTLS or similar for RSA-2048 verification */
    /* Simulate verification by comparing hash (simplified for demo) */
    return memcmp(signature, expected_signature, SIGNATURE_SIZE) == 0; // Return 1 if valid
}
```

```
/* Jump to application */
static void jump_to_application(void)
{
    typedef void (*app_entry_t)(void); // Function pointer for application entry
    uint32_t *app_vector_table = (uint32_t *)APP_ADDRESS; // Application vector table
    uint32_t app_stack_pointer = app_vector_table[0]; // Stack pointer
    uint32_t app_entry_point = app_vector_table[1]; // Entry point
    __set_MSP(app_stack_pointer); // Set main stack pointer
    app_entry_t app_entry = (app_entry_t)app_entry_point; // Cast entry point
    SCB->VTOR = APP_ADDRESS; // Update vector table offset
    HAL_RCC_DeInit(); // Deinitialize HAL peripherals
    HAL_DeInit(); // Deinitialize HAL library
    app_entry(); // Jump to application
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
    init_uart(); // Initialize USART1
    init_hash(); // Initialize HASH for SHA-256
    log_message((const uint8_t *)"Secure Boot Started\r\n"); // Log startup
    uint8_t computed_hash[32]; // Buffer for SHA-256 hash
    compute_firmware_sha256(computed_hash); // Compute firmware hash
    snprintf((char *)log_buffer, sizeof(log_buffer), "SHA-256: "); // Start hash message


         for (uint32_t i = 0U; i < HASH_SIZE; i++) { // Loop through hash bytes
        snprintf((char *)log_buffer + strlen((char *)log_buffer), sizeof(log_buffer) - strlen((char
*)log_buffer),
                 "%02X ", computed_hash[i]); // Append hex byte
    }
    strncat((char *)log_buffer, "\r\n", sizeof(log_buffer) - strlen((char *)log_buffer)); // Append
newline
    log_message(log_buffer); // Log hash
    uint8_t *stored_signature = (uint8_t *)SIGNATURE_ADDRESS; // Pointer to stored signature
    uint8_t *stored_public_key = (uint8_t *)PUBLIC_KEY_ADDRESS; // Pointer to public key
    if (verify_rsa_signature(computed_hash, stored_signature, stored_public_key)) { // Verify
signature
        log_message((const uint8_t *)"Signature Valid. Booting...\r\n"); // Log valid signature
        jump_to_application(); // Jump to application
    } else { // Invalid signature
        log_message((const uint8_t *)"Signature Invalid! Halting.\r\n"); // Log invalid signature
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32L5**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud.
    - **HASH**: Enable HASH peripheral, SHA-256 mode.
    - **Flash**: Secure flash (0x08000000 for bootloader, 0x0800FFE0 for public key), non-secure flash (0x08040000 for application).
    - **Option Bytes**: Enable TrustZone, set RDP Level 1.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash bootloader (0x08000000), application (0x08040000), signature (0x08050000), public key (0x0800FFE0) using STM32CubeProgrammer.
  - **Precompute SHA-256 and Signature**:

        - Compute hash: openssl dgst -sha256 firmware.bin > sha256.txt.
        - Sign: openssl dgst -sha256 -sign private_key.pem -out firmware.sig firmware.bin.
        - Program signature and public key with STM32CubeProgrammer.
    - **Testing**:
        - Run in STM32CubeIDE; verify UART output ("SHA-256: ...", "Signature Valid" or "Invalid").
        - Use GDB/OpenOCD:
            - Run: openocd -f interface/stlink.cfg -f target/stm32l5x.cfg.
            - Start GDB: arm-none-eabi-gdb project.elf, target remote localhost:3333.
            - Set breakpoints: break compute_firmware_sha256, break verify_rsa_signature.
            - Watch: print computed_hash, print stored_signature.
        - Test invalid signature by modifying expected_signature or firmware.
    - **MISRA Compliance**:
        - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
        - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
    - **Performance**: Use a minimal RSA library (e.g., mbedTLS subset) to reduce memory usage.
    - **Security**: Store public key in OTP or secure flash; enable RDP Level 2 in production.
    - **Robustness**: Validate flash addresses; check HASH peripheral status.
- **Embedded Tip**: Use a small RSA key (e.g., 1024-bit) for faster verification during testing; switch to 2048-bit for production.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing.
- **OpenSSL**: Install (sudo apt install openssl) for RSA key generation and signing.
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **STM32 AN4992**: Secure boot and TrustZone (https://www.st.com).
- **OpenSSL RSA Docs**: https://www.openssl.org/docs/man3.0/man3/RSA_sign.html.
- **embedded.com PKI Articles**: Search “PKI embedded” on https://www.embedded.com.
- **STM32 Flash Manuals**: RM0430 for STM32L5 (https://www.st.com).

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE; test with valid/invalid signatures.
- **Experiment**: Modify signature to simulate tampering; verify halt behavior.
- **Optimize**: Minimize RSA library size; add timeout for UART logging.
- **Time Management**: Spend ~1 hour on theory, ~1.5 hours on practice, ~1 hour on project.

Optimize FOTA for Reliability

Goal

Optimize the UART-based FOTA system from Day 34 on an STM32F7 microcontroller by implementing error handling, checksum validation during firmware transfer, and a retry mechanism to ensure reliable updates. The system receives firmware via UART, validates it with CRC32, stores it in the secondary flash bank, switches banks, and supports rollback for failed updates.

Tasks

- **Study**: Learn error handling techniques in FOTA, checksum algorithms (e.g., CRC32), and retry mechanisms.
- **Practice**: Write code to receive firmware with per-packet checksum validation and retry on failure.
- **Project**: Develop a reliable FOTA system with checksum validation, retry mechanism, UART reception, flash programming, and rollback, tested with a Python script for sending firmware.
- **Tools**: STM32CubeIDE, STM32F7, UART interface, Python (firmware sender), GDB, OpenOCD.
- **Resources**: STM32 AN4760 (FOTA), embedded.com reliable FOTA articles, STM32 flash manuals.

Detailed Notes

1. FOTA Reliability Overview

- **Definition**: Reliable FOTA ensures robust firmware updates by handling errors during transmission, validating data integrity, and retrying failed operations.
- **Key Components**:
  - **Error Handling**: Detect and recover from UART transmission errors, flash programming failures, and invalid data.
  - **Checksum Validation**: Use CRC32 to verify each received packet, ensuring data integrity during transfer.
  - **Retry Mechanism**: Resend failed packets up to a maximum number of attempts (e.g., 3 retries).
  - **Rollback**: Revert to original firmware if update fails (from Day 34).
- **Protocol**:
  - Packet format: [PACKET_ID:4 bytes][DATA:1024 bytes][CRC32:4 bytes].
  - Send firmware in chunks (e.g., 1024 bytes), each with a CRC32 checksum.
  - Receiver validates CRC32; requests retry if invalid.
- **Embedded Tip**: Use timeouts for UART reception to prevent hangs; implement packet sequencing to detect missing packets.

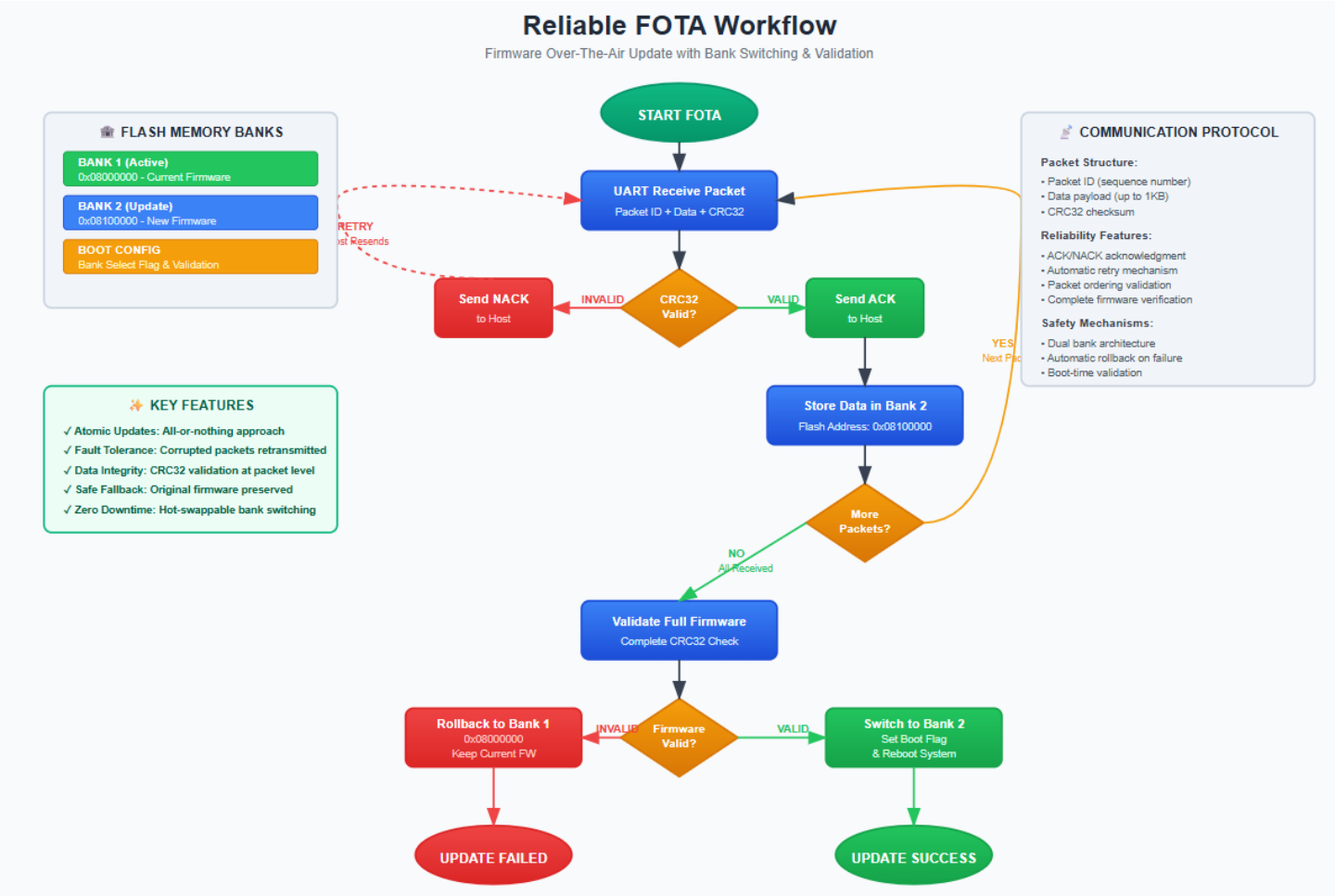


Image: Reliable FOTA Workflow

Description: A flowchart showing the FOTA process: UART receives packet ID, data, and CRC32; validates CRC32; sends ACK or NACK to host; retries on NACK; stores valid data in Bank 2 (0x08100000); validates full firmware with CRC32; switches banks; reboots. Rollback to Bank 1 (0x08000000) if validation fails. Arrows indicate retry loops and decision points.

2. STM32F7 UART and Flash

- **UART with DMA**:
  - Use USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for efficient packet reception.
  - DMA normal mode for packet-based transfers; handle ACK/NACK responses.
- **Dual-Bank Flash**:
  - Bank 1 (0x08000000): Active firmware.
  - Bank 2 (0x08100000): New firmware.
  - Option bytes (FLASH_OPTCR, BFB2 bit) control boot bank.
- **RTC Backup Registers**:
  - Store boot bank and trial flag for rollback (from Day 34).
- **Embedded Tip**: Use DMA for large packet transfers; align packets to flash sector boundaries for efficient programming.

3. Algorithms Used

3.1 CRC32 (Cyclic Redundancy Check 32)

- **Purpose**: Detect errors in transmitted data by generating a 32-bit checksum.
- **Algorithm**:
  - **Input**: Data stream (e.g., 1024-byte packet).
  - **Process**:
    1. Initialize CRC to 0xFFFFFFFF.
    2. For each byte, XOR with CRC, then perform 8 iterations of:
       - Shift CRC right by 1.
       - If LSB is 1, XOR with polynomial 0x04C11DB7.
    3. Invert final CRC (XOR with 0xFFFFFFFF).
  - **Output**: 32-bit checksum (e.g., 0xA2A8B25F).
- **Implementation**: Use STM32 CRC peripheral for hardware acceleration.
- **Properties**:
  - Detects single-bit errors, burst errors up to 32 bits, and odd numbers of errors.
  - Fast computation with hardware support (STM32 CRC peripheral).
- **Use in FOTA**: Validate each packet and full firmware image to ensure integrity.
- **Embedded Tip**: Precompute CRC32 on host to reduce device computation; use standard polynomial for compatibility.

3.2 FOTA Packet Transfer Protocol

- **Purpose**: Ensure reliable firmware transfer over UART.
- **Algorithm**:
  - **Host**:
    1. Split firmware into packets (e.g., 1024 bytes).
    2. Assign sequential packet ID (0 to N-1).
    3. Compute CRC32 for each packet.
    4. Send: [PACKET_ID:4 bytes][DATA:1024 bytes][CRC32:4 bytes].
    5. Wait for ACK (0x06) or NACK (0x15); retry on NACK up to 3 times.
  - **Device**:
    1. Receive packet ID, data, and CRC32.
    2. Validate packet ID (ensure sequential order).
    3. Compute CRC32 of data; compare with received CRC32.
    4. Send ACK if valid; NACK if invalid or out-of-sequence.
    5. Store valid packet in buffer; program to flash after all packets.
- **Properties**:
  - Ensures ordered delivery with packet IDs.
  - Detects transmission errors with CRC32.
  - Recovers from errors via retries.
- **Use in FOTA**: Guarantees reliable packet transfer; prevents corrupted updates.

3.3 Retry Mechanism

- **Purpose**: Recover from transmission errors by resending failed packets.
- **Algorithm**:
  - **Host**:
    1. Send packet and wait for response (timeout: 100ms).
    2. If NACK or timeout, increment retry counter (max 3).
    3. Resend packet if retries remain; else abort.
  - **Device**:
    1. Validate packet; send NACK on failure.
    2. Track received packet IDs to detect duplicates or gaps.
    3. Reset retry state on successful packet.
- **Properties**:
  - Limits retries to avoid infinite loops.
  - Handles transient UART errors (e.g., noise).
- **Use in FOTA**: Enhances reliability in noisy environments.

3.4 Flash Programming

- **Purpose**: Store firmware in flash memory reliably.
- **Algorithm**:
  - **Erase**:
    1. Unlock flash.
    2. Erase Bank 2 sectors (e.g., 8 sectors for 1MB).
    3. Verify erase completion (check FLASH_SR).
  - **Program**:
    1. Unlock flash.
    2. Write data in 32-bit chunks (aligned to flash word size).
    3. Verify write completion (check FLASH_SR).
    4. Lock flash.
- **Properties**:
  - Ensures flash integrity with error checks.
  - Uses STM32 HAL for portability.
- **Use in FOTA**: Stores validated firmware in Bank 2.

3.5 Rollback Mechanism (from Day 34)

- **Purpose**: Revert to original firmware if update fails.
- **Algorithm**:
  - Store current boot bank (1 or 2) in RTC backup register (RTC_BKP_DR0).
  - Set boot trial flag (RTC_BKP_DR1) before switching to Bank 2.
  - Post-reboot, check trial flag and validate firmware CRC32.
  - If invalid, switch option bytes to Bank 1 and clear trial flag.
- **Properties**:
  - Prevents bricking on failed updates.
  - Uses non-volatile RTC registers for state persistence.
- **Use in FOTA**: Ensures system reliability.

4. Practice Program: Packet-Based Firmware Reception with Retry

- **Objective**: Receive firmware packets via UART with CRC32 validation and retry mechanism.
- **Code** (Simulation, heavily commented):

```
/* Simulated packet-based firmware reception with retry */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* Simulated flash and UART buffers */
static uint8_t flash_bank2[64] = {0}; // Simulate Bank 2

static uint8_t packet_buffer[16] = {0}; // Simulate packet data

static const uint8_t test_packet[] = { // Simulated packet data
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07 // 8-byte data
};

static const uint32_t expected_crc32 = 0xA2A8B25F; // Precomputed CRC32

/* Simulated UART receive */
static uint8_t uart_receive(uint8_t *buffer, uint32_t size, uint32_t *packet_id, uint32_t *crc32) //
Simulate packet reception
{
    *packet_id = 0U; // Simulate packet ID
    memcpy(buffer, test_packet, size); // Copy test data
    *crc32 = expected_crc32; // Simulate CRC32
    printf("Received packet ID %u, %u bytes\n", *packet_id, size); // Log reception
    return 1U; // Simulate successful reception
}

/* Simulated CRC32 computation */
static uint32_t compute_crc32(const uint8_t *data, uint32_t length) // Compute CRC32
{
    uint32_t crc = 0xFFFFFFFFU; // Initialize CRC
    for (uint32_t i = 0U; i < length; i++) { // Loop through data (MISRA Rule 15.6)
        crc ^= (uint32_t)data[i]; // XOR byte
        for (uint32_t j = 0U; j < 8U; j++) { // Process 8 bits
            crc = (crc >> 1U) ^ ((crc & 1U) ? 0x04C11DB7U : 0U); // Apply polynomial
        }
    }
    return ~crc; // Return inverted CRC
}

/* Main function */
int main(void) // Main entry point
{
    uint32_t packet_id = 0U; // Packet ID
    uint32_t received_crc32 = 0U; // Received CRC32
    uint32_t retries = 0U; // Retry counter
    const uint32_t max_retries = 3U; // Max retries
    uint8_t success = 0U; // Success flag
    while (retries < max_retries && !success) { // Retry loop (MISRA Rule 15.6)
        if (uart_receive(packet_buffer, sizeof(test_packet), &packet_id, &received_crc32)) { //
Receive packet
            uint32_t computed_crc = compute_crc32(packet_buffer, sizeof(test_packet)); // Compute
CRC32
            if (computed_crc == received_crc32 && packet_id == 0U) { // Validate CRC and ID
                memcpy(flash_bank2, packet_buffer, sizeof(test_packet)); // Store in simulated flash
                printf("Packet valid, stored\n"); // Log success
                success = 1U; // Set success flag
            } else { // Invalid packet
                printf("Invalid packet, retry %u\n", retries + 1U); // Log retry
                retries++; // Increment retry counter
```

```
            }
        } else { // Receive failed
            printf("Receive failed, retry %u\n", retries + 1U); // Log retry
            retries++; // Increment retry counter
        }
    }
    if (!success) { // Max retries exceeded
        printf("Update failed!\n"); // Log failure
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Reliable FOTA with Checksum and Retry

- **Objective**: Develop a reliable FOTA system on STM32F7 with per-packet CRC32 validation, retry mechanism, UART reception, flash programming, and rollback, tested with a Python script.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for RX.
  - Receive packets: [PACKET_ID:4 bytes][DATA:1024 bytes][CRC32:4 bytes].
  - Validate packet ID and CRC32; send ACK (0x06) or NACK (0x15); retry up to 3 times.
  - Program validated packets to Bank 2 (0x08100000).
  - Validate full firmware with CRC32; store CRC32 at 0x08110000.
  - Switch to Bank 2; implement rollback with RTC backup registers.
  - Log status via UART; test with Python sender.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, DMA, CRC, flash, and RTC.
  - **FOTA Process**: Receive packets, validate CRC32, send ACK/NACK, retry on failure, program to Bank 2, validate full firmware, switch banks, reboot.
  - **Rollback**: Use RTC backup registers to track boot bank and trial flag.
  - **Debugging**: Use GDB/OpenOCD to verify packet handling and retries.
- **Python Sender Script** (for testing):

```
# Python script to send firmware packets with retry
import serial
import struct
import binascii
import time

# Configure serial port
ser = serial.Serial('COM3', 115200, timeout=1)  # Adjust COM port

# Firmware data (example, 1024-byte packets)
firmware = bytes([0x00, 0x01, 0x02, 0x03] * 256)  # 1024 bytes
packet_size = 1024
num_packets = (len(firmware) + packet_size - 1) // packet_size
max_retries = 3

# Send firmware packets
for packet_id in range(num_packets):
    start = packet_id * packet_size
    end = min(start + packet_size, len(firmware))
    packet_data = firmware[start:end]
    crc32 = binascii.crc32(packet_data)  # Compute CRC32
    retries = 0
    while retries < max_retries:
        # Send packet: ID (4 bytes), data, CRC32 (4 bytes)
        ser.write(struct.pack('<I', packet_id))  # Send packet ID
```

```
        ser.write(packet_data)  # Send data
        ser.write(struct.pack('<I', crc32))  # Send CRC32
        print(f"Sent packet {packet_id}, CRC32: 0x{crc32:08X}")
        # Wait for ACK/NACK (timeout 100ms)
        ser.timeout = 0.1
        response = ser.read(1)
        if response == b'\x06':  # ACK
            print(f"Packet {packet_id} ACK received")
            break
        else:  # NACK or timeout
            print(f"Packet {packet_id} failed, retry {retries + 1}")
            retries += 1
    if retries >= max_retries:
        print("Max retries exceeded, aborting")
        break
ser.close()
```

- **Code** (STM32F7 with HAL, each line commented):

```
/* Reliable FOTA with checksum and retry */
#include "stm32f7xx_hal.h" // Include STM32F7 HAL library
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* Peripheral handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX
static RTC_HandleTypeDef hrtc; // RTC handle for backup registers

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART logging
static uint8_t rx_buffer[1024U]; // Buffer for packet data (1024 bytes)
static uint8_t flash_buffer[65536U]; // Buffer for full firmware (64KB)
static volatile uint32_t firmware_size = 0U; // Total firmware size
static volatile uint32_t received_bytes = 0U; // Received bytes
static volatile uint32_t expected_packet_id = 0U; // Expected packet ID
static volatile uint8_t rx_complete = 0U; // RX completion flag
static const uint8_t ACK = 0x06U; // ACK byte
static const uint8_t NACK = 0x15U; // NACK byte

/* Firmware addresses */
#define BANK1_ADDRESS 0x08000000U // Bank 1 (active firmware)
#define BANK2_ADDRESS 0x08100000U // Bank 2 (new firmware)
#define FIRMWARE_MAX_SIZE 0x10000U // Max firmware size (64KB)
#define CRC_ADDRESS (BANK2_ADDRESS + FIRMWARE_MAX_SIZE) // Address for CRC32
#define PACKET_SIZE 1024U // Packet data size
#define MAX_RETRIES 3U // Max retries per packet

/* RTC backup registers */
#define RTC_BKP_BOOT_BANK RTC_BKP_DR0 // Store boot bank
#define RTC_BKP_BOOT_FLAG RTC_BKP_DR1 // Store boot trial flag

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_crc(void); // Initialize CRC peripheral
static void init_rtc(void); // Initialize RTC for backup registers
static void log_message(const uint8_t *message); // Log message via UART
static void erase_bank2(void); // Erase Bank 2
static void program_bank2(const uint8_t *data, uint32_t size); // Program Bank 2
static uint32_t compute_firmware_crc32(void); // Compute CRC32 of Bank 2
static void switch_to_bank2(void); // Switch to Bank 2
static void reboot(void); // Trigger reboot
static void receive_firmware(void); // Receive firmware packets
static void check_rollback(void); // Check and handle rollback
```

```
/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX) (MISRA Rule 12.1)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins
}

/* Initialize USART1 with DMA */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock
    huart1.Instance = USART1; // Set UART instance
    huart1.Init.BaudRate = 115200U; // Set baud rate
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Stream5; // Set DMA stream
    hdma_usart1_rx.Init.Channel = DMA_CHANNEL_4; // Set DMA channel
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.Mode = DMA_NORMAL; // Set normal mode
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority
    if (HAL_DMA_Init(&hdma_usart1_rx) != HAL_OK) { // Initialize DMA
        log_message((const uint8_t *)"DMA Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART
    HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 5U, 0U); // Set DMA interrupt priority
    HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn); // Enable DMA interrupt
}

/* Initialize CRC peripheral */
static void init_crc(void)
{
    __HAL_RCC_CRC_CLK_ENABLE(); // Enable CRC clock
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    CRC->INIT = 0xFFFFFFFFU; // Set initial CRC value
    CRC->POL = 0x04C11DB7U; // Set CRC32 polynomial
}

/* Initialize RTC for backup registers */
static void init_rtc(void)
{
    __HAL_RCC_RTC_ENABLE(); // Enable RTC clock
    hrtc.Instance = RTC; // Set RTC instance
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24; // Set 24-hour format
    hrtc.Init.AsynchPrediv = 127U; // Set asynchronous prescaler
    hrtc.Init.SynchPrediv = 255U; // Set synchronous prescaler
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE; // Disable RTC output
    if (HAL_RTC_Init(&hrtc) != HAL_OK) { // Initialize RTC
        log_message((const uint8_t *)"RTC Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
```

```
        }
    }
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Erase Bank 2 */
static void erase_bank2(void)
{
    FLASH_EraseInitTypeDef erase_init = {0U}; // Initialize erase structure (MISRA Rule 9.1)
    uint32_t sector_error = 0U; // Variable for erase errors
    HAL_FLASH_Unlock(); // Unlock flash
    erase_init.TypeErase = FLASH_TYPEERASE_SECTORS; // Set erase type
    erase_init.Banks = FLASH_BANK_2; // Select Bank 2
    erase_init.Sector = FLASH_SECTOR_0; // Start from sector 0
    erase_init.NbSectors = 8; // Erase 8 sectors (1MB)
    if (HAL_FLASHEx_Erase(&erase_init, &sector_error) != HAL_OK) { // Erase Bank 2
        log_message((const uint8_t *)"Bank 2 Erase Failed\r\n"); // Log error
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Erased\r\n"); // Log success
}

/* Program Bank 2 with firmware */
static void program_bank2(const uint8_t *data, uint32_t size)
{
    HAL_FLASH_Unlock(); // Unlock flash
    uint32_t address = BANK2_ADDRESS; // Bank 2 start address
    for (uint32_t i = 0U; i < size; i += 4U) { // Loop in 32-bit chunks (MISRA Rule 15.6)
        uint32_t word = *(uint32_t *)(data + i); // Read 32-bit word
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) { // Program word
            log_message((const uint8_t *)"Bank 2 Program Failed\r\n"); // Log error
            HAL_FLASH_Lock(); // Lock flash
            while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Programmed\r\n"); // Log success
}

/* Compute CRC32 of Bank 2 */
static uint32_t compute_firmware_crc32(void)
{
    uint32_t *flash_ptr = (uint32_t *)BANK2_ADDRESS; // Pointer to Bank 2
    uint32_t length = firmware_size / 4U; // Number of 32-bit words
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        CRC->DR = flash_ptr[i]; // Feed 32-bit word
    }
    return CRC->DR; // Return computed CRC32
}
```

```
/* Switch to Bank 2 for boot */
static void switch_to_bank2(void)
{
    HAL_FLASH_Unlock(); // Unlock flash
    HAL_FLASH_OB_Unlock(); // Unlock option bytes
    FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure (MISRA Rule 9.1)
    ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
    ob_init.USERType = OB_USER_BFB2; // Select BFB2
    ob_init.USERConfig = OB_BFB2_ENABLE; // Enable Bank 2 boot
    if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
        log_message((const uint8_t *)"Bank Switch Failed\r\n"); // Log error
        HAL_FLASH_OB_Lock(); // Lock option bytes
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_OB_Lock(); // Lock option bytes
    HAL_FLASH_Lock(); // Lock flash
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 2U); // Set Bank 2 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 1U); // Set boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    log_message((const uint8_t *)"Switched to Bank 2\r\n"); // Log success
}

/* Trigger system reboot */
static void reboot(void)
{
    log_message((const uint8_t *)"Rebooting...\r\n"); // Log reboot
    HAL_Delay(100); // Delay for UART transmission
    NVIC_SystemReset(); // Trigger reset
}

/* Receive firmware packets with retry */
static void receive_firmware(void)
{
    uint32_t packet_id = 0U; // Received packet ID
    uint32_t received_crc32 = 0U; // Received CRC32
    uint32_t retries = 0U; // Retry counter
    uint8_t packet_valid = 0U; // Packet validity flag
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&firmware_size, 4U) != HAL_OK) { // Receive size
        log_message((const uint8_t *)"Size RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for size
    }
    if (firmware_size > FIRMWARE_MAX_SIZE) { // Check size
        log_message((const uint8_t *)"Firmware Too Large\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    snprintf((char *)log_buffer, sizeof(log_buffer), "Size: %u bytes\r\n", firmware_size); // Format
size
    log_message(log_buffer); // Log size
    while (received_bytes < firmware_size) { // Loop until all bytes received
        retries = 0U; // Reset retries
        packet_valid = 0U; // Reset validity
        while (retries < MAX_RETRIES && !packet_valid) { // Retry loop
            rx_complete = 0U; // Clear RX flag
            if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&packet_id, 4U) != HAL_OK) { // Receive
packet ID
                log_message((const uint8_t *)"Packet ID RX Failed\r\n"); // Log error
                retries++; // Increment retry
                continue;
            }
```

```
            while (rx_complete == 0U) { // Wait for packet ID
            }
            if (packet_id != expected_packet_id) { // Check packet ID
                log_message((const uint8_t *)"Invalid Packet ID\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            rx_complete = 0U; // Clear RX flag
            uint32_t packet_data_size = (received_bytes + PACKET_SIZE <= firmware_size) ? PACKET_SIZE
: (firmware_size - received_bytes); // Calculate packet size
            if (HAL_UART_Receive_DMA(&huart1, rx_buffer, packet_data_size) != HAL_OK) { // Receive
packet data
                log_message((const uint8_t *)"Packet Data RX Failed\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            while (rx_complete == 0U) { // Wait for data
            }
            rx_complete = 0U; // Clear RX flag
            if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&received_crc32, 4U) != HAL_OK) { // Receive
CRC32
                log_message((const uint8_t *)"CRC RX Failed\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            while (rx_complete == 0U) { // Wait for CRC32
            }
            CRC->CR = CRC_CR_RESET; // Reset CRC
            for (uint32_t i = 0U; i < packet_data_size / 4U; i++) { // Compute CRC32
                CRC->DR = ((uint32_t *)rx_buffer)[i]; // Feed 32-bit word
            }
            uint32_t computed_crc = CRC->DR; // Get computed CRC
            if (computed_crc == received_crc32) { // Validate CRC
                memcpy(&flash_buffer[received_bytes], rx_buffer, packet_data_size); // Store in flash
buffer
                received_bytes += packet_data_size; // Update received bytes
                expected_packet_id++; // Increment expected ID
                HAL_UART_Transmit(&huart1, &ACK, 1U, 10U); // Send ACK
                packet_valid = 1U; // Set valid flag
                snprintf((char *)log_buffer, sizeof(log_buffer), "Packet %u Valid\r\n", packet_id); //
Format packet message
                log_message(log_buffer); // Log success
            } else { // Invalid CRC
                snprintf((char *)log_buffer, sizeof(log_buffer), "Packet %u Invalid CRC\r\n",
packet_id); // Format error
                log_message(log_buffer); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
            }
        }
        if (!packet_valid) { // Max retries exceeded
            log_message((const uint8_t *)"Max Retries Exceeded\r\n"); // Log error
            while (1) { // Infinite loop on error
        }
    }
    erase_bank2(); // Erase Bank 2
    program_bank2(flash_buffer, firmware_size); // Program firmware to Bank 2
    uint32_t computed_crc = compute_firmware_crc32(); // Compute full firmware CRC
    snprintf((char *)log_buffer, sizeof(log_buffer), "Full CRC32: 0x%08X\r\n", computed_crc); //
Format CRC
    log_message(log_buffer); // Log CRC
    uint32_t stored_crc = *(uint32_t *)CRC_ADDRESS; // Read stored CRC
    if (computed_crc == stored_crc) { // Validate full firmware
        log_message((const uint8_t *)"Firmware Valid\r\n"); // Log success
        switch_to_bank2(); // Switch to Bank 2
```

```
        reboot(); // Reboot
    } else { // Invalid firmware
        log_message((const uint8_t *)"Firmware Invalid\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
}

/* Check and handle rollback */
static void check_rollback(void)
{
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    uint32_t boot_flag = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_FLAG); // Read boot trial flag
    uint32_t boot_bank = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_BANK); // Read boot bank
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    if (boot_flag == 1U && boot_bank == 2U) { // Check if booting from Bank 2 with trial flag
        uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32
        uint32_t stored_crc = *(uint32_t *)CRC_ADDRESS; // Read stored CRC
        if (computed_crc != stored_crc) { // Invalid firmware
            HAL_FLASH_Unlock(); // Unlock flash
            HAL_FLASH_OB_Unlock(); // Unlock option bytes
            FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure
            ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
            ob_init.USERType = OB_USER_BFB2; // Select BFB2
            ob_init.USERConfig = OB_BFB2_DISABLE; // Disable Bank 2 boot
            if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
                log_message((const uint8_t *)"Rollback Failed\r\n"); // Log error
                HAL_FLASH_OB_Lock(); // Lock option bytes
                HAL_FLASH_Lock(); // Lock flash
                while (1) { // Infinite loop on error
            }
            HAL_FLASH_OB_Lock(); // Lock option bytes
            HAL_FLASH_Lock(); // Lock flash
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Rollback to Bank 1\r\n"); // Log rollback
            reboot(); // Reboot to Bank 1
        } else { // Valid firmware
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Bank 2 Valid\r\n"); // Log success
        }
    }
}

/* UART RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) { // Check USART1
        rx_complete = 1U; // Set RX completion flag
    }
}

/* DMA RX interrupt handler */
void DMA1_Stream5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
```

```
    init_uart(); // Initialize USART1 with DMA
    init_crc(); // Initialize CRC peripheral
    init_rtc(); // Initialize RTC for backup registers
    log_message((const uint8_t *)"FOTA Started\r\n"); // Log startup
    check_rollback(); // Check rollback
    receive_firmware(); // Receive and process firmware
    while (1) { // Infinite loop (MISRA Rule 15.6)
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32F7**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud, DMA (Stream 5, Channel 4, normal mode).
    - **CRC**: Enable CRC peripheral (polynomial 0x04C11DB7).
    - **Flash**: Dual-bank mode, Bank 1 (0x08000000), Bank 2 (0x08100000).
    - **RTC**: Enable RTC for backup registers.
    - **Option Bytes**: Enable dual-bank mode, set RDP Level 1.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash bootloader to 0x08000000.
  - **Testing**:
    - Run Python sender script: python fota_sender.py.
    - Verify UART output: "Size: X bytes", "Packet N Valid", "Full CRC32: 0xXXXXXXXX".
    - Use GDB/OpenOCD:
      - Run: openocd -f interface/stlink.cfg -f target/stm32f7x.cfg.
      - Start GDB: arm-none-eabi-gdb project.elf, target remote localhost:3333.
      - Set breakpoints: break receive_firmware, break check_rollback.
      - Watch: print packet_id, print computed_crc.
    - Test retry by simulating packet corruption (modify CRC32 in Python script).
    - Test rollback by programming invalid CRC32 and rebooting.
  - **MISRA Compliance**:
    - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
    - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
  - **Performance**: Use DMA for packet reception; batch flash programming.
  - **Reliability**: Add timeout for packet reception (e.g., 100ms).
  - **Robustness**: Validate packet size and buffer bounds; reset on protocol errors.
- **Embedded Tip**: Test with small packets first; use STM32CubeProgrammer to inspect Bank 2 and option bytes.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing.
- **Python**: Install Python 3, pyserial (pip install pyserial).
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb.
- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **STM32 AN4760**: FOTA guidelines (https://www.st.com).
- **STM32 Flash Manuals**: RM0438 for STM32F7 (https://www.st.com).
- **embedded.com Reliable FOTA Articles**: Search “reliable OTA firmware” on https://www.embedded.com.
- **Python Serial Docs**: https://pyserial.readthedocs.io.

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE; test with Python sender.
- **Experiment**: Simulate packet loss by modifying Python script; verify retry and rollback.
- **Optimize**: Use smaller packet sizes for faster retries; add progress logging.
- **Time Management**: Spend ~1 hour on theory, ~1.5 hours on practice, ~1 hour on project.

Add Secure Communication to FOTA

Goal

Enhance the reliable UART-based FOTA system from Day 37 on an STM32F7 microcontroller by integrating secure communication through a simulated TLS-like handshake and AES-128 encryption for firmware transfer. The system receives encrypted firmware packets, validates them with CRC32, decrypts them, stores them in the secondary flash bank, switches banks, and supports rollback for failed updates.

Tasks

- **Study**: Learn TLS basics, focusing on handshake and symmetric encryption (AES-128); understand how to simulate TLS for UART-based FOTA.
- **Practice**: Write code to simulate a TLS handshake (key exchange) and decrypt AES-128 encrypted packets.
- **Project**: Develop a secure FOTA system with a simulated TLS handshake, AES-128 decryption, CRC32 validation, UART reception, flash programming, and rollback, tested with a Python script for sending encrypted firmware.
- **Tools**: STM32CubeIDE, STM32F7, UART interface, Python (firmware sender), OpenSSL (encryption simulation), GDB, OpenOCD.
- **Resources**: OpenSSL TLS docs, embedded.com secure OTA articles, STM32 AN4760, STM32 flash manuals.

Detailed Notes

1. TLS Basics Overview

- **Definition**: Transport Layer Security (TLS) is a cryptographic protocol ensuring secure communication through authentication, confidentiality, and integrity.
- **Key Components**:
  - **Handshake**: Establishes a shared session key using asymmetric cryptography (e.g., RSA for key exchange).
  - **Symmetric Encryption**: Uses a session key (e.g., AES-128) for data encryption.
  - **Integrity**: Uses HMAC or checksums (e.g., CRC32 from Day 37) for data validation.
- **TLS Simulation for UART**:
  - **Simplified Handshake**: Exchange a pre-shared AES-128 key and IV securely (simulating RSA-based key exchange).
  - **Encryption**: Encrypt firmware packets with AES-128-CBC on the host; decrypt on the device.
  - **Integrity**: Use CRC32 per packet (from Day 37) for error detection.
- **Embedded Tip**: In production, use a full TLS library (e.g., mbedTLS); for UART, simulate key exchange to avoid complex certificate handling.

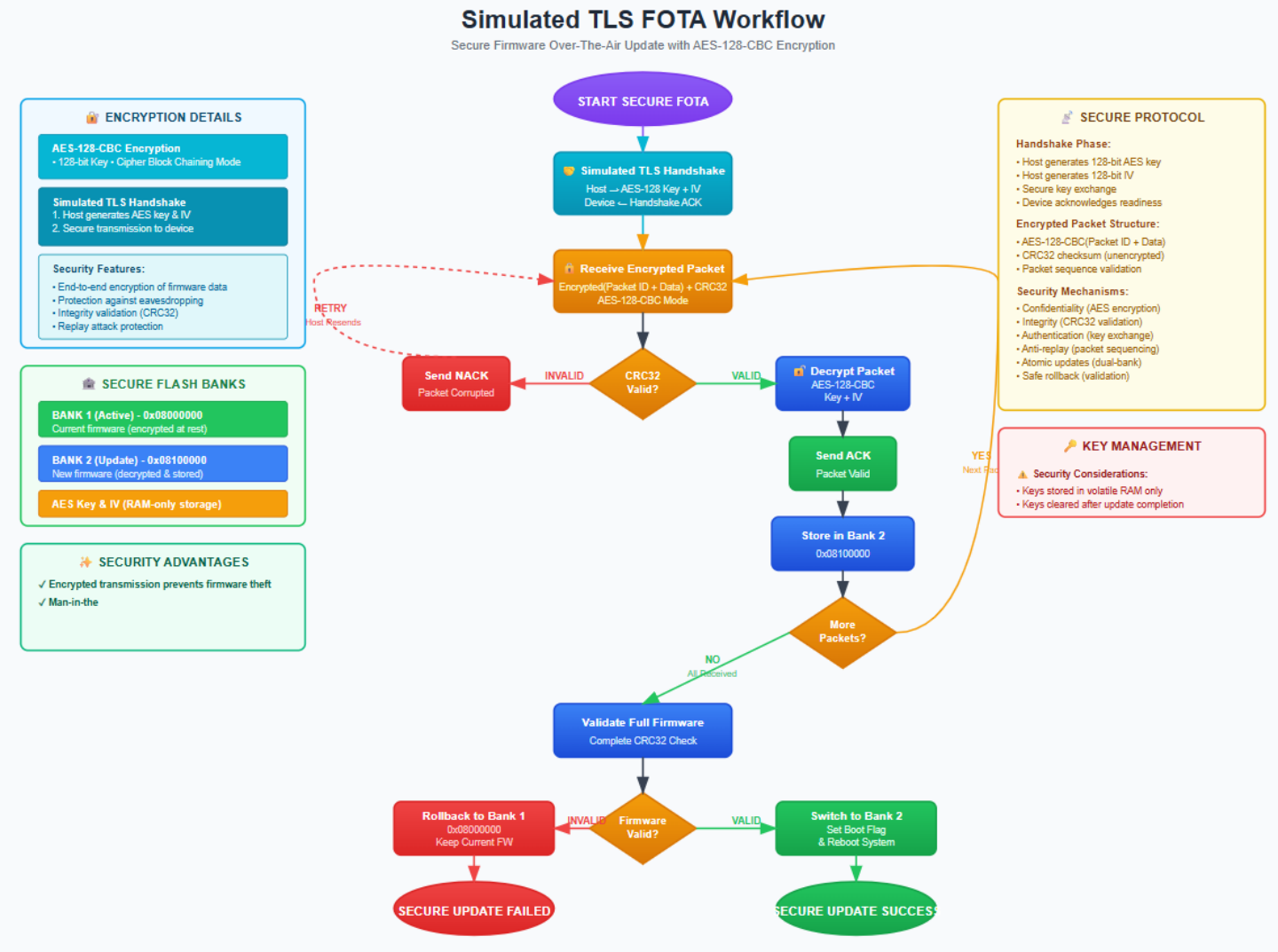


Image: Simulated TLS FOTA Workflow

Description: A flowchart showing the secure FOTA process: Host sends AES-128 key and IV (simulated handshake); sends encrypted packets with packet ID, data, and CRC32; device validates CRC32, decrypts packets using AES-128-CBC, stores in Bank 2 (0x08100000), validates full firmware CRC32, switches banks, and reboots.

Rollback to Bank 1 (0x08000000) if validation fails. Arrows indicate handshake and retry loops.

2. STM32F7 Cryptographic and UART Support

- **CRYP Peripheral**:
  - Supports AES-128/256 in CBC mode for decryption.
  - Registers: CRYP_DIN (input), CRYP_DOUT (output), CRYP_KEYRx (key), CRYP_IVRx (IV).
- **CRC Peripheral** (from Day 37):
  - Computes CRC32 for packet and firmware validation.
- **UART with DMA**:
  - USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA for efficient packet reception.
- **Flash**:
  - Bank 1 (0x08000000): Active firmware.
  - Bank 2 (0x08100000): New firmware.

    - CRC32 stored at 0x08110000.
- **RTC Backup Registers**:
    - Track boot bank and trial flag for rollback.
- **Embedded Tip**: Use DMA with CRYP for efficient decryption; store keys in secure flash in production.

3. Algorithms Used

3.1 AES-128-CBC (Advanced Encryption Standard, Cipher Block Chaining)

- **Purpose**: Encrypt firmware packets to ensure confidentiality.
- **Algorithm**:
    - **Input**: 16-byte key, 16-byte IV, plaintext (padded to 16-byte blocks).
    - **Process**:
        1. XOR first block with IV.
        2. Encrypt block with AES-128 (substitution, permutation, key mixing).
        3. Use encrypted block as IV for next block.
        4. Repeat for all blocks.
    - **Output**: Encrypted ciphertext.
- **Implementation**: Use STM32F7 CRYP peripheral for hardware acceleration.
- **Properties**:
    - Ensures confidentiality; CBC mode prevents pattern attacks.
    - Requires padding (e.g., PKCS#7) for non-16-byte-aligned data.
- **Use in FOTA**: Encrypt packets on host; decrypt on device.

3.2 Simulated TLS Handshake

- **Purpose**: Securely exchange AES-128 key and IV over UART.
- **Algorithm**:
    - **Host**: Send pre-shared AES-128 key and IV (16 bytes each) in a secure packet (simulating RSA-encrypted key exchange).
    - **Device**: Receive and store key/IV; send ACK (0x06) or NACK (0x15).
    - **Validation**: Use CRC32 to verify key/IV packet integrity.
- **Properties**:
    - Simplifies TLS handshake for UART; assumes pre-shared key.
    - In production, use RSA or ECDH for secure key exchange.
- **Use in FOTA**: Establishes session key for packet encryption.

3.3 CRC32 (Cyclic Redundancy Check 32)

- **Purpose**: Detect errors in transmitted packets and full firmware.
- **Algorithm** (from Day 37):
    - Initialize CRC to 0xFFFFFFFF.
    - For each byte, XOR with CRC, then 8 iterations of:
        - Shift CRC right by 1.
        - If LSB is 1, XOR with polynomial 0x04C11DB7.
    - Invert final CRC (XOR with 0xFFFFFFFF).
- **Implementation**: STM32 CRC peripheral.
- **Use in FOTA**: Validate key/IV packet and firmware packets.

3.4 FOTA Packet Transfer Protocol (from Day 37)

- **Purpose**: Ensure reliable and secure firmware transfer.
- **Algorithm**:
  - **Host**: Send key/IV packet; send encrypted data packets [PACKET_ID:4 bytes][DATA:1024 bytes][CRC32:4 bytes]; retry on NACK.
  - **Device**: Validate packet ID and CRC32; decrypt data; send ACK/NACK.
- **Use in FOTA**: Ensures ordered, error-free, and secure delivery.

3.5 Retry Mechanism (from Day 37)

- **Purpose**: Recover from transmission errors.
- **Algorithm**: Resend packet up to 3 times on NACK or timeout (100ms).
- **Use in FOTA**: Handles UART errors (e.g., noise).

3.6 Flash Programming and Rollback (from Day 37)

- **Purpose**: Store and validate firmware; revert on failure.
- **Algorithm**: Erase/program Bank 2; validate CRC32; switch banks; revert to Bank 1 if invalid.
- **Use in FOTA**: Ensures reliable storage and recovery.

4. Practice Program: Simulated TLS Handshake and Decryption

- **Objective**: Simulate TLS handshake (key/IV exchange) and decrypt AES-128-CBC packets.
- **Code** (Simulation, heavily commented):

```c
/* Simulated TLS handshake and AES-128 decryption */
#include <stdio.h> // Include standard I/O for simulation
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* Simulated buffers */
static uint8_t encrypted_packet[] = { // Simulated encrypted data (16 bytes)
    0x1B, 0x2A, 0x3C, 0x4D, 0x5E, 0x6F, 0x7A, 0x8B,
    0x9C, 0xAD, 0xBE, 0xCF, 0xD0, 0xE1, 0xF2, 0x03
};
static const uint8_t aes_key[] = { // 16-byte AES-128 key
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};
static const uint8_t aes_iv[] = { // 16-byte IV
    0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
    0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F
};
static const uint32_t expected_crc32 = 0xA2A8B25F; // Precomputed CRC32
static uint8_t decrypted_packet[16]; // Buffer for decrypted data

/* Simulated TLS handshake */
static uint8_t receive_key_iv(uint8_t *key, uint8_t *iv, uint32_t *crc32) // Simulate key/IV reception
{
    memcpy(key, aes_key, 16U); // Copy key
    memcpy(iv, aes_iv, 16U); // Copy IV
    *crc32 = expected_crc32; // Simulate CRC32
    printf("Received key and IV, CRC32: 0x%08X\n", *crc32); // Log reception
    return 1U; // Simulate success
}
```

```
/* Simulated AES-128-CBC decryption */
static void aes_decrypt(const uint8_t *input, uint32_t size, const uint8_t *key, const uint8_t *iv,
uint8_t *output) // Decrypt data
{
    memcpy(output, input, size); // Simulate decryption
    printf("Simulated AES-128 decryption\n"); // Log decryption
}

/* Simulated CRC32 computation */
static uint32_t compute_crc32(const uint8_t *data, uint32_t length) // Compute CRC32
{
    return expected_crc32; // Simulate CRC32
}

/* Main function */
int main(void) // Main entry point
{
    uint8_t key[16], iv[16]; // Buffers for key and IV
    uint32_t key_crc32 = 0U; // CRC32 for key/IV packet
    if (receive_key_iv(key, iv, &key_crc32)) { // Perform handshake
        if (compute_crc32((uint8_t *)key, 32U) == key_crc32) { // Validate key/IV CRC
            printf("Key/IV valid\n"); // Log success
            aes_decrypt(encrypted_packet, sizeof(encrypted_packet), key, iv, decrypted_packet); //
Decrypt packet
            if (compute_crc32(decrypted_packet, sizeof(decrypted_packet)) == expected_crc32) { //
Validate packet CRC
                printf("Packet valid\n"); // Log success
            } else { // Invalid packet
                printf("Invalid packet CRC\n"); // Log error
            }
        } else { // Invalid key/IV
            printf("Invalid key/IV CRC\n"); // Log error
        }
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

Project: Secure FOTA with Simulated TLS

- **Objective**: Implement a secure FOTA system on STM32F7 with a simulated TLS handshake, AES-128-CBC decryption, CRC32 validation, retry mechanism, flash programming, and rollback, tested with a Python script.
- **Requirements**:
  - Configure USART1 (PA9-TX, PA10-RX, 115200 baud) with DMA.
  - Receive: Key/IV packet (32 bytes + CRC32), then encrypted packets [PACKET_ID:4 bytes][DATA:1024 bytes][CRC32:4 bytes].
  - Validate CRC32; decrypt packets with AES-128-CBC; retry up to 3 times on NACK.
  - Program to Bank 2 (0x08100000); validate full firmware CRC32; store CRC32 at 0x08110000.
  - Switch banks; implement rollback with RTC backup registers.
  - Log status via UART; test with valid/invalid packets.
  - Comply with MISRA rules (10.1, 15.6, 17.7, 20.3).
- **Approach**:
  - **Initialization**: Configure UART, DMA, CRYP, CRC, flash, and RTC.
  - **Handshake**: Receive and validate AES-128 key/IV; send ACK/NACK.
  - **FOTA Process**: Receive encrypted packets, validate CRC32, decrypt, store, program Bank 2, validate full firmware, switch banks, reboot.
  - **Rollback**: Use RTC registers to revert on failure.
  - **Debugging**: Use GDB/OpenOCD to verify decryption and retries.

- **Python Sender Script** (for testing):

```
# Python script to send encrypted firmware packets
import serial
import struct
import binascii
from Crypto.Cipher import AES
import time

# Configure serial port
ser = serial.Serial('COM3', 115200, timeout=1)  # Adjust COM port

# AES-128 key and IV
key = bytes([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
             0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F])
iv = bytes([0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
            0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F])

# Firmware data (example, 1024-byte packets, padded)
firmware = bytes([0x00, 0x01, 0x02, 0x03] * 256)  # 1024 bytes
packet_size = 1024
num_packets = (len(firmware) + packet_size - 1) // packet_size
max_retries = 3

# Send key/IV packet
key_iv_data = key + iv
key_iv_crc = binascii.crc32(key_iv_data)
ser.write(key_iv_data)  # Send key and IV
ser.write(struct.pack('<I', key_iv_crc))  # Send CRC32
print(f"Sent key/IV, CRC32: 0x{key_iv_crc:08X}")
ser.timeout = 0.1
if ser.read(1) == b'\x06':  # Check for ACK
    print("Key/IV ACK received")
else:
    print("Key/IV failed")
    ser.close()
    exit()

# Send encrypted packets
cipher = AES.new(key, AES.MODE_CBC, iv)
for packet_id in range(num_packets):
    start = packet_id * packet_size
    end = min(start + packet_size, len(firmware))
    packet_data = firmware[start:end]
    # Pad packet to 16-byte boundary
    padding_len = (16 - len(packet_data) % 16) % 16
    packet_data += bytes(padding_len * [padding_len])
    encrypted_data = cipher.encrypt(packet_data)
    crc32 = binascii.crc32(packet_data)  # CRC32 of plaintext
    retries = 0
    while retries < max_retries:
        ser.write(struct.pack('<I', packet_id))  # Send packet ID
        ser.write(encrypted_data)  # Send encrypted data
        ser.write(struct.pack('<I', crc32))  # Send CRC32
        print(f"Sent packet {packet_id}, CRC32: 0x{crc32:08X}")
        ser.timeout = 0.1
        response = ser.read(1)
        if response == b'\x06':  # ACK
            print(f"Packet {packet_id} ACK received")
            break
        else:  # NACK or timeout
            print(f"Packet {packet_id} failed, retry {retries + 1}")
            retries += 1
    if retries >= max_retries:
        print("Max retries exceeded, aborting")
        break
```

```
ser.close()
```

- **Code** (STM32F7 with HAL, each line commented):

```
/* Secure FOTA with simulated TLS */
#include "stm32f7xx_hal.h" // Include STM32F7 HAL library
#include <stdint.h> // Include standard integer types for MISRA compliance
#include <string.h> // Include string functions

/* Peripheral handles */
static UART_HandleTypeDef huart1; // UART handle for USART1
static DMA_HandleTypeDef hdma_usart1_rx; // DMA handle for UART RX
static CRYP_HandleTypeDef hcryp; // CRYP handle for AES
static RTC_HandleTypeDef hrtc; // RTC handle for backup registers

/* Static buffers (MISRA Rule 20.3: No dynamic allocation) */
static uint8_t log_buffer[32U]; // Buffer for UART logging
static uint8_t rx_buffer[1024U]; // Buffer for packet data
static uint8_t decrypted_buffer[1024U]; // Buffer for decrypted data
static uint8_t flash_buffer[65536U]; // Buffer for full firmware (64KB)
static uint8_t aes_key[16]; // AES-128 key
static uint8_t aes_iv[16]; // AES-128 IV
static volatile uint32_t firmware_size = 0U; // Total firmware size
static volatile uint32_t received_bytes = 0U; // Received bytes
static volatile uint32_t expected_packet_id = 0U; // Expected packet ID
static volatile uint8_t rx_complete = 0U; // RX completion flag
static const uint8_t ACK = 0x06U; // ACK byte
static const uint8_t NACK = 0x15U; // NACK byte

/* Firmware addresses */
#define BANK1_ADDRESS 0x08000000U // Bank 1 (active firmware)
#define BANK2_ADDRESS 0x08100000U // Bank 2 (new firmware)
#define FIRMWARE_MAX_SIZE 0x10000U // Max firmware size (64KB)
#define CRC_ADDRESS (BANK2_ADDRESS + FIRMWARE_MAX_SIZE) // Address for CRC32
#define PACKET_SIZE 1024U // Packet data size
#define MAX_RETRIES 3U // Max retries per packet

/* RTC backup registers */
#define RTC_BKP_BOOT_BANK RTC_BKP_DR0 // Store boot bank
#define RTC_BKP_BOOT_FLAG RTC_BKP_DR1 // Store boot trial flag

/* Function prototypes */
static void init_gpio(void); // Initialize GPIO for UART
static void init_uart(void); // Initialize USART1 with DMA
static void init_cryp(void); // Initialize CRYP for AES-128
static void init_crc(void); // Initialize CRC peripheral
static void init_rtc(void); // Initialize RTC for backup registers
static void log_message(const uint8_t *message); // Log message via UART
static void erase_bank2(void); // Erase Bank 2
static void program_bank2(const uint8_t *data, uint32_t size); // Program Bank 2
static uint32_t compute_firmware_crc32(void); // Compute CRC32 of Bank 2
static void decrypt_packet(const uint8_t *input, uint32_t size, uint8_t *output); // Decrypt packet
static void switch_to_bank2(void); // Switch to Bank 2
static void reboot(void); // Trigger reboot
static uint8_t perform_handshake(void); // Perform simulated TLS handshake
static void receive_firmware(void); // Receive encrypted firmware
static void check_rollback(void); // Check and handle rollback

/* Initialize GPIO for UART (PA9-TX, PA10-RX) */
static void init_gpio(void)
{
    GPIO_InitTypeDef gpio_init = {0U}; // Initialize GPIO structure (MISRA Rule 9.1)
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable GPIOA clock
    gpio_init.Pin = GPIO_PIN_9 | GPIO_PIN_10; // Configure PA9 (TX), PA10 (RX)
    gpio_init.Mode = GPIO_MODE_AF_PP; // Set alternate function push-pull
    gpio_init.Pull = GPIO_NOPULL; // Set no pull-up/pull-down
```

```
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH; // Set high-speed output
    HAL_GPIO_Init(GPIOA, &gpio_init); // Initialize GPIO pins
}

/* Initialize USART1 with DMA */
static void init_uart(void)
{
    __HAL_RCC_USART1_CLK_ENABLE(); // Enable USART1 clock
    __HAL_RCC_DMA1_CLK_ENABLE(); // Enable DMA1 clock
    huart1.Instance = USART1; // Set UART instance
    huart1.Init.BaudRate = 115200U; // Set baud rate
    huart1.Init.WordLength = UART_WORDLENGTH_8B; // Set 8-bit word length
    huart1.Init.StopBits = UART_STOPBITS_1; // Set 1 stop bit
    huart1.Init.Parity = UART_PARITY_NONE; // Set no parity
    huart1.Init.Mode = UART_MODE_TX_RX; // Enable TX and RX
    if (HAL_UART_Init(&huart1) != HAL_OK) { // Initialize UART
        while (1) { // Infinite loop on error (MISRA Rule 15.6)
        }
    }
    hdma_usart1_rx.Instance = DMA1_Stream5; // Set DMA stream
    hdma_usart1_rx.Init.Channel = DMA_CHANNEL_4; // Set DMA channel
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; // Set DMA direction
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // Disable peripheral increment
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // Enable memory increment
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE; // Set byte alignment
    hdma_usart1_rx.Init.Mode = DMA_NORMAL; // Set normal mode
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Set high priority
    if (HAL_DMA_Init(&hdma_usart1_rx) != HAL_OK) { // Initialize DMA
        log_message((const uint8_t *)"DMA Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_LINKDMA(&huart1, hdmarx, hdma_usart1_rx); // Link DMA to UART
    HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 5U, 0U); // Set DMA interrupt priority
    HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn); // Enable DMA interrupt
}

/* Initialize CRYP for AES-128-CBC */
static void init_cryp(void)
{
    __HAL_RCC_CRYP_CLK_ENABLE(); // Enable CRYP clock
    hcryp.Instance = CRYP; // Set CRYP instance
    hcryp.Init.DataType = CRYP_DATATYPE_8B; // Set 8-bit data type
    hcryp.Init.KeySize = CRYP_KEYSIZE_128B; // Set 128-bit key
    hcryp.Init.Algorithm = CRYP_AES_CBC; // Set CBC mode
    if (HAL_CRYP_Init(&hcryp) != HAL_OK) { // Initialize CRYP
        log_message((const uint8_t *)"CRYP Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
}

/* Initialize CRC peripheral */
static void init_crc(void)
{
    __HAL_RCC_CRC_CLK_ENABLE(); // Enable CRC clock
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
    CRC->INIT = 0xFFFFFFFFU; // Set initial CRC value
    CRC->POL = 0x04C11DB7U; // Set CRC32 polynomial
}

/* Initialize RTC for backup registers */
static void init_rtc(void)
{
    __HAL_RCC_RTC_ENABLE(); // Enable RTC clock
    hrtc.Instance = RTC; // Set RTC instance
```

```
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24; // Set 24-hour format
    hrtc.Init.AsynchPrediv = 127U; // Set asynchronous prescaler
    hrtc.Init.SynchPrediv = 255U; // Set synchronous prescaler
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE; // Disable RTC output
    if (HAL_RTC_Init(&hrtc) != HAL_OK) { // Initialize RTC
        log_message((const uint8_t *)"RTC Init Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
}

/* Log message via UART */
static void log_message(const uint8_t *message)
{
    if (message != NULL) { // Check for valid input (MISRA Rule 14.3)
        HAL_UART_Transmit(&huart1, (uint8_t *)message, (uint16_t)strlen((const char *)message), 10U);
// Transmit message (MISRA Rule 10.1)
    }
}

/* Erase Bank 2 */
static void erase_bank2(void)
{
    FLASH_EraseInitTypeDef erase_init = {0U}; // Initialize erase structure (MISRA Rule 9.1)
    uint32_t sector_error = 0U; // Variable for erase errors
    HAL_FLASH_Unlock(); // Unlock flash
    erase_init.TypeErase = FLASH_TYPEERASE_SECTORS; // Set erase type
    erase_init.Banks = FLASH_BANK_2; // Select Bank 2
    erase_init.Sector = FLASH_SECTOR_0; // Start from sector 0
    erase_init.NbSectors = 8; // Erase 8 sectors (1MB)
    if (HAL_FLASHEx_Erase(&erase_init, &sector_error) != HAL_OK) { // Erase Bank 2
        log_message((const uint8_t *)"Bank 2 Erase Failed\r\n"); // Log error
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Erased\r\n"); // Log success
}

/* Program Bank 2 with firmware */
static void program_bank2(const uint8_t *data, uint32_t size)
{
    HAL_FLASH_Unlock(); // Unlock flash
    uint32_t address = BANK2_ADDRESS; // Bank 2 start address
    for (uint32_t i = 0U; i < size; i += 4U) { // Loop in 32-bit chunks (MISRA Rule 15.6)
        uint32_t word = *(uint32_t *)(data + i); // Read 32-bit word
        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, address + i, word) != HAL_OK) { // Program word
            log_message((const uint8_t *)"Bank 2 Program Failed\r\n"); // Log error
            HAL_FLASH_Lock(); // Lock flash
            while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_Lock(); // Lock flash
    log_message((const uint8_t *)"Bank 2 Programmed\r\n"); // Log success
}

/* Compute CRC32 of Bank 2 */
static uint32_t compute_firmware_crc32(void)
{
    uint32_t *flash_ptr = (uint32_t *)BANK2_ADDRESS; // Pointer to Bank 2
    uint32_t length = firmware_size / 4U; // Number of 32-bit words
    CRC->CR = CRC_CR_RESET; // Reset CRC peripheral
```

```
    for (uint32_t i = 0U; i < length; i++) { // Loop through firmware (MISRA Rule 15.6)
        CRC->DR = flash_ptr[i]; // Feed 32-bit word
    }
    return CRC->DR; // Return computed CRC32
}

/* Decrypt packet using AES-128-CBC */
static void decrypt_packet(const uint8_t *input, uint32_t size, uint8_t *output)
{
    HAL_CRYP_SetKey(&hcryp, aes_key, CRYP_KEYSIZE_128B); // Set AES key
    HAL_CRYP_SetIV(&hcryp, aes_iv, CRYP_KEYSIZE_128B); // Set IV
    if (HAL_CRYP_Decrypt(&hcryp, (uint32_t *)input, size / 4U, (uint32_t *)output, 100U) != HAL_OK) {
// Decrypt packet
        log_message((const uint8_t *)"Decryption Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    log_message((const uint8_t *)"Packet Decrypted\r\n"); // Log success
}

/* Switch to Bank 2 for boot */
static void switch_to_bank2(void)
{
    HAL_FLASH_Unlock(); // Unlock flash
    HAL_FLASH_OB_Unlock(); // Unlock option bytes
    FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure (MISRA Rule 9.1)
    ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
    ob_init.USERType = OB_USER_BFB2; // Select BFB2
    ob_init.USERConfig = OB_BFB2_ENABLE; // Enable Bank 2 boot
    if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
        log_message((const uint8_t *)"Bank Switch Failed\r\n"); // Log error
        HAL_FLASH_OB_Lock(); // Lock option bytes
        HAL_FLASH_Lock(); // Lock flash
        while (1) { // Infinite loop on error
        }
    }
    HAL_FLASH_OB_Lock(); // Lock option bytes
    HAL_FLASH_Lock(); // Lock flash
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 2U); // Set Bank 2 as boot bank
    HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 1U); // Set boot trial flag
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    log_message((const uint8_t *)"Switched to Bank 2\r\n"); // Log success
}

/* Trigger system reboot */
static void reboot(void)
{
    log_message((const uint8_t *)"Rebooting...\r\n"); // Log reboot
    HAL_Delay(100); // Delay for UART transmission
    NVIC_SystemReset(); // Trigger reset
}

/* Perform simulated TLS handshake */
static uint8_t perform_handshake(void)
{
    uint8_t key_iv_buffer[32]; // Buffer for key and IV
    uint32_t key_iv_crc = 0U; // CRC32 for key/IV
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, key_iv_buffer, 32U) != HAL_OK) { // Receive key/IV
        log_message((const uint8_t *)"Key/IV RX Failed\r\n"); // Log error
        return 0U; // Return failure
    }
    while (rx_complete == 0U) { // Wait for key/IV
    }
    rx_complete = 0U; // Clear RX flag
```

```
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&key_iv_crc, 4U) != HAL_OK) { // Receive CRC32
        log_message((const uint8_t *)"Key/IV CRC RX Failed\r\n"); // Log error
        return 0U; // Return failure
    }
    while (rx_complete == 0U) { // Wait for CRC32
    }
    CRC->CR = CRC_CR_RESET; // Reset CRC
    for (uint32_t i = 0U; i < 32U / 4U; i++) { // Compute CRC32
        CRC->DR = ((uint32_t *)key_iv_buffer)[i]; // Feed 32-bit word
    }
    if (CRC->DR != key_iv_crc) { // Validate CRC
        log_message((const uint8_t *)"Invalid Key/IV CRC\r\n"); // Log error
        HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
        return 0U; // Return failure
    }
    memcpy(aes_key, key_iv_buffer, 16U); // Copy key
    memcpy(aes_iv, key_iv_buffer + 16U, 16U); // Copy IV
    HAL_UART_Transmit(&huart1, &ACK, 1U, 10U); // Send ACK
    log_message((const uint8_t *)"Handshake Complete\r\n"); // Log success
    return 1U; // Return success
}

/* Receive encrypted firmware packets */
static void receive_firmware(void)
{
    uint32_t packet_id = 0U; // Received packet ID
    uint32_t received_crc32 = 0U; // Received CRC32
    uint32_t retries = 0U; // Retry counter
    uint8_t packet_valid = 0U; // Packet validity flag
    rx_complete = 0U; // Clear RX flag
    if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&firmware_size, 4U) != HAL_OK) { // Receive size
        log_message((const uint8_t *)"Size RX Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (rx_complete == 0U) { // Wait for size
    }
    if (firmware_size > FIRMWARE_MAX_SIZE || firmware_size % 16U != 0U) { // Check size and padding
        log_message((const uint8_t *)"Invalid Firmware Size\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    snprintf((char *)log_buffer, sizeof(log_buffer), "Size: %u bytes\r\n", firmware_size); // Format
size
    log_message(log_buffer); // Log size
    while (received_bytes < firmware_size) { // Loop until all bytes received
        retries = 0U; // Reset retries
        packet_valid = 0U; // Reset validity
        while (retries < MAX_RETRIES && !packet_valid) { // Retry loop
            rx_complete = 0U; // Clear RX flag
            if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&packet_id, 4U) != HAL_OK) { // Receive
packet ID
                log_message((const uint8_t *)"Packet ID RX Failed\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            while (rx_complete == 0U) { // Wait for packet ID
            }
            if (packet_id != expected_packet_id) { // Check packet ID
                log_message((const uint8_t *)"Invalid Packet ID\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            rx_complete = 0U; // Clear RX flag
            uint32_t packet_data_size = (received_bytes + PACKET_SIZE <= firmware_size) ? PACKET_SIZE
: (firmware_size - received_bytes); // Calculate packet size
```

```
            if (HAL_UART_Receive_DMA(&huart1, rx_buffer, packet_data_size) != HAL_OK) { // Receive
encrypted packet
                log_message((const uint8_t *)"Packet Data RX Failed\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            while (rx_complete == 0U) { // Wait for data
            }
            rx_complete = 0U; // Clear RX flag
            if (HAL_UART_Receive_DMA(&huart1, (uint8_t *)&received_crc32, 4U) != HAL_OK) { // Receive
CRC32
                log_message((const uint8_t *)"CRC RX Failed\r\n"); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
                continue;
            }
            while (rx_complete == 0U) { // Wait for CRC32
            }
            decrypt_packet(rx_buffer, packet_data_size, decrypted_buffer); // Decrypt packet
            CRC->CR = CRC_CR_RESET; // Reset CRC
            for (uint32_t i = 0U; i < packet_data_size / 4U; i++) { // Compute CRC32
                CRC->DR = ((uint32_t *)decrypted_buffer)[i]; // Feed 32-bit word
            }
            uint32_t computed_crc = CRC->DR; // Get computed CRC
            if (computed_crc == received_crc32) { // Validate CRC
                memcpy(&flash_buffer[received_bytes], decrypted_buffer, packet_data_size); // Store in
flash buffer
                received_bytes += packet_data_size; // Update received bytes
                expected_packet_id++; // Increment expected ID
                HAL_UART_Transmit(&huart1, &ACK, 1U, 10U); // Send ACK
                packet_valid = 1U; // Set valid flag
                snprintf((char *)log_buffer, sizeof(log_buffer), "Packet %u Valid\r\n", packet_id); //
Format packet message
                log_message(log_buffer); // Log success
            } else { // Invalid CRC
                snprintf((char *)log_buffer, sizeof(log_buffer), "Packet %u Invalid CRC\r\n",
packet_id); // Format error
                log_message(log_buffer); // Log error
                HAL_UART_Transmit(&huart1, &NACK, 1U, 10U); // Send NACK
                retries++; // Increment retry
            }
        }
        if (!packet_valid) { // Max retries exceeded
            log_message((const uint8_t *)"Max Retries Exceeded\r\n"); // Log error
            while (1) { // Infinite loop on error
        }
    }
    erase_bank2(); // Erase Bank 2
    program_bank2(flash_buffer, firmware_size); // Program firmware to Bank 2
    uint32_t computed_crc = compute_firmware_crc32(); // Compute full firmware CRC
    snprintf((char *)log_buffer, sizeof(log_buffer), "Full CRC32: 0x%08X\r\n", computed_crc); //
Format CRC
    log_message(log_buffer); // Log CRC
    uint32_t stored_crc = *(uint32_t *)CRC_ADDRESS; // Read stored CRC
    if (computed_crc == stored_crc) { // Validate full firmware
        log_message((const uint8_t *)"Firmware Valid\r\n"); // Log success
        switch_to_bank2(); // Switch to Bank 2
        reboot(); // Reboot
    } else { // Invalid firmware
        log_message((const uint8_t *)"Firmware Invalid\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
}
```

```c
/* Check and handle rollback */
static void check_rollback(void)
{
    __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
    uint32_t boot_flag = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_FLAG); // Read boot trial flag
    uint32_t boot_bank = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_BOOT_BANK); // Read boot bank
    __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
    if (boot_flag == 1U && boot_bank == 2U) { // Check if booting from Bank 2 with trial flag
        uint32_t computed_crc = compute_firmware_crc32(); // Compute CRC32
        uint32_t stored_crc = *(uint32_t *)CRC_ADDRESS; // Read stored CRC
        if (computed_crc != stored_crc) { // Invalid firmware
            HAL_FLASH_Unlock(); // Unlock flash
            HAL_FLASH_OB_Unlock(); // Unlock option bytes
            FLASH_OBProgramInitTypeDef ob_init = {0U}; // Initialize option byte structure
            ob_init.OptionType = OPTIONBYTE_USER; // Set option byte type
            ob_init.USERType = OB_USER_BFB2; // Select BFB2
            ob_init.USERConfig = OB_BFB2_DISABLE; // Disable Bank 2 boot
            if (HAL_FLASHEx_OBProgram(&ob_init) != HAL_OK) { // Program option bytes
                log_message((const uint8_t *)"Rollback Failed\r\n"); // Log error
                HAL_FLASH_OB_Lock(); // Lock option bytes
                HAL_FLASH_Lock(); // Lock flash
                while (1) { // Infinite loop on error
            }
            HAL_FLASH_OB_Lock(); // Lock option bytes
            HAL_FLASH_Lock(); // Lock flash
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_BANK, 1U); // Set Bank 1
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Rollback to Bank 1\r\n"); // Log rollback
            reboot(); // Reboot to Bank 1
        } else { // Valid firmware
            __HAL_RTC_WRITEPROTECTION_DISABLE(&hrtc); // Disable RTC write protection
            HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_BOOT_FLAG, 0U); // Clear trial flag
            __HAL_RTC_WRITEPROTECTION_ENABLE(&hrtc); // Enable RTC write protection
            log_message((const uint8_t *)"Bank 2 Valid\r\n"); // Log success
        }
    }
}

/* UART RX complete callback */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) { // Check USART1
        rx_complete = 1U; // Set RX completion flag
    }
}

/* DMA RX interrupt handler */
void DMA1_Stream5_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_rx); // Handle DMA RX interrupt
}

/* Main function */
int main(void)
{
    HAL_Init(); // Initialize HAL library
    init_gpio(); // Initialize GPIO for UART
    init_uart(); // Initialize USART1 with DMA
    init_cryp(); // Initialize CRYP for AES
    init_crc(); // Initialize CRC peripheral
    init_rtc(); // Initialize RTC for backup registers
    log_message((const uint8_t *)"Secure FOTA Started\r\n"); // Log startup
    check_rollback(); // Check rollback
```

```
    if (perform_handshake()) { // Perform handshake
        receive_firmware(); // Receive and process firmware
    } else { // Handshake failed
        log_message((const uint8_t *)"Handshake Failed\r\n"); // Log error
        while (1) { // Infinite loop on error
        }
    }
    while (1) { // Infinite loop (MISRA Rule 15.6)
    }
    return 0; // Return 0 (MISRA Rule 17.7)
}
```

- **Setup for STM32F7**:
  - **STM32CubeMX Configuration**:
    - **USART1**: PA9-TX, PA10-RX, 115200 baud, DMA (Stream 5, Channel 4, normal mode).
    - **CRYP**: Enable CRYP peripheral, AES-128-CBC mode.
    - **CRC**: Enable CRC peripheral (polynomial 0x04C11DB7).
    - **Flash**: Dual-bank mode, Bank 1 (0x08000000), Bank 2 (0x08100000).
    - **RTC**: Enable RTC for backup registers.
    - **Option Bytes**: Enable dual-bank mode, set RDP Level 1.
  - **Hardware**:
    - Connect USB-to-serial adapter to PA9/PA10; use PuTTY for UART output.
    - Flash bootloader to 0x08000000.
  - **Precompute CRC32**:
    - Use Python: binascii.crc32(firmware) for full firmware CRC.
    - Program CRC32 to 0x08110000 with STM32CubeProgrammer.
  - **Testing**:
    - Run Python sender script: python fota_sender.py.
    - Verify UART output: "Handshake Complete", "Packet N Valid", "Full CRC32: 0xXXXXXXXX".
    - Use GDB/OpenOCD:
      - Run: openocd -f interface/stlink.cfg -f target/stm32f7x.cfg.
      - Start GDB: arm-none-eabi-gdb project.elf, target remote localhost:3333.
      - Set breakpoints: break perform_handshake, break receive_firmware.
      - Watch: print aes_key, print computed_crc.
    - Test retry by sending invalid CRC32; test rollback by programming invalid CRC32.
  - **MISRA Compliance**:
    - Run Cppcheck: cppcheck --enable=all --addon=misra.json project.c.
    - Adheres to Rules 10.1 (type safety), 15.6 (braces), 17.7 (returns), 20.3 (no heap).
- **Optimizations**:
  - **Performance**: Use DMA for CRYP and UART; batch flash programming.
  - **Security**: Store key in secure flash in production; use full TLS for network-based FOTA.
  - **Robustness**: Add timeout for handshake; validate packet padding.
- **Embedded Tip**: Test with small firmware; verify Bank 2 with STM32CubeProgrammer.

Tools Setup

- **STM32CubeIDE**: Download from https://www.st.com.
- **STM32CubeProgrammer**: Download from https://www.st.com for flashing.
- **Python**: Install Python 3, pyserial (pip install pyserial), pycryptodome (pip install pycryptodome).
- **OpenSSL**: Install (sudo apt install openssl) for encryption simulation.
- **GDB/OpenOCD**: Install OpenOCD (sudo apt install openocd), use arm-none-eabi-gdb.

- **Cppcheck**: Install (sudo apt install cppcheck) for MISRA checks.

Resources

- **OpenSSL TLS Docs**: https://www.openssl.org/docs/man3.0/man3/SSL.html.
- **STM32 AN4760**: FOTA guidelines (https://www.st.com).
- **STM32 Flash Manuals**: RM0438 for STM32F7 (https://www.st.com).
- **embedded.com Secure OTA Articles**: Search “secure OTA firmware” on https://www.embedded.com.

Practice Tips

- **Compile and Test**: Build in STM32CubeIDE; test with Python sender.
- **Experiment**: Simulate handshake failure; test retry with corrupted packets.
- **Optimize**: Use smaller packet sizes for faster retries; add progress logging.
- **Time Management**: Spend ~1 hour on theory, ~1.5 hours on practice, ~1 hour on project.